Test backtracking si propagarea restrictiilor

1.Ce avantaje are metoda backtracking comparativ cu generează și testează?

Select one or more:
a. Găsește mai puține soluții corecte decât metoda generează și testează
b. Metoda backtracking are, în general, o complexitate polinomială, față de cea exponențială a metodei generează și testează
c. Reduce numărul de stări explorate prin eliminarea din spațiul stărilor a acelor subarbori care nu satisfac restricțiile la pasul curent
d. Backtracking are o complexitate temporală mai redusă decât generează și testează, dar în general ambele sunt exponențiale
The correct answers are:
Reduce numărul de stări explorate prin eliminarea din spațiul stărilor a acelor subarbori care nu satisfac restricțiile la pasul curent ,
Backtracking are o complexitate temporală mai redusă decât generează și testează, dar în general ambele sunt exponențiale

2.Care este complexitatea metodei backtracking pentru un joc de Sudoku cu 43 de căsuțe libere și domeniul de la {1,..,9} pentru fiecare căsuță?

Select one or more:
a. O(43^9)
b. Θ(43^9)
c. O(9^43)
d. Θ(9^43)
The correct answer is: O(9^43)

3.Fie problema damelor: Dându-se o tablă de șah cu n linii și n coloane, găsiți toate posibilitățile de a poziționa n dame pe tabla de șah astfel încât nicio pereche de dame să nu se atace. Cum puteți modela această problemă pentru a o rezolva cu backtracking?

Select one or more:
a. Considerăm că avem n^2 variabile, fiecare cu domeniul {1, ..., n}

b. Considerăm că avem n variabile, fiecare cu domeniul {1, ..., n} Pentru fiecare damă i=1..n, considerăm că ea este pe rândul i și calculăm doar coloana j (domeniul, de la 1..n)
c. Considerăm că avem n^2 variabile, fiecare cu domeniul {0, 1}
d. Considerăm că avem n variabile, fiecare cu domeniul {1, ..., n^2}
The correct answers are: Considerăm că avem n^2 variabile, fiecare cu domeniul {0, 1}, Considerăm că avem n variabile, fiecare cu domeniul {1, ..., n^2}, Considerăm că avem n variabile, fiecare cu domeniul {1, ..., n}

4.Fie domeniul variabilei A = {1, 2, 5, 7}, iar al variabilei B = {1, 3, 4} Constrângerea pentru cele două variabile este că A+B=8 (valoarea asignată variabilei A + valoarea asignată variabilei B trebuie să fie egală cu 8). Dacă aplicăm REVISE(A, B), domeniul lui A va fi?

Select one or more:
a. {1, 2, 5, 7}
b. {5, 7}
c. {1, 2}
d. {1, 5, 7}
The correct answer is: {5, 7}

5.Putem rezolva complet un joc de Sudoku aplicând doar algoritmi de propagare a restricțiilor (fără metode similare cu backtracking)?

Select one or more:
a. În general, nu. Pentru jocuri mai complicate, va trebui să luăm decizii de asignare cu backtracking sau alți algoritmi similari.
b. Pentru jocuri foarte simple, putem avea noroc și algoritmii de propagare a restricțiilor să lase doar o valoare (sau niciuna, în caz de joc ce nu poate fi rezolvat) pentru fiecare domeniu.
c. Algoritmii de propagare a restricțiilor nu pot fi folosiți niciodată pentru a rezolva complet un joc de Sudoku.
d. Da, întotdeauna pot fi folosiți.
The correct answers are: În general, nu. Pentru jocuri mai complicate, va trebui să luăm decizii de asignare cu backtracking sau alți algoritmi similari., Pentru jocuri foarte simple, putem avea noroc și algoritmii de propagare a restricțiilor să lase doar o valoare (sau niciuna, în caz de joc ce nu poate fi rezolvat) pentru fiecare domeniu.

Test Minimax

1.Pentru jocuri cu câţi jucători se poate aplica algoritmul Minimax?

Alegeţi una sau mai multe opţiuni:
a. 3
b. 2
c. oricâţi
d. 4
Răspunsul corect este: oricâţi

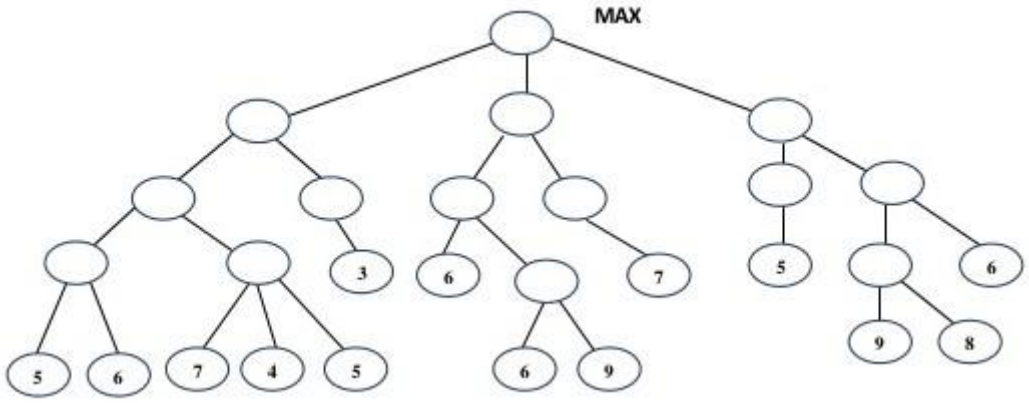
2.Ce tip de algoritm este Minimax?

Alegeţi una sau mai multe opţiuni:
a. Programare dinamică
b. Backtracking
c. Divide et impera
d. Greedy
Răspunsul corect este: Backtracking

3.În cazul algoritmului minimax limitat, ce se întâmplă dacă ajungem la nivelul limită?

Alegeţi una sau mai multe opţiuni:
a. evaluăm mutarea folosind euristica stabilită şi întoarcem cea mai bună mutare
b. câştigăm jocul
c. ne trece rândul
d. pierdem jocul
Răspunsul corect este: evaluăm mutarea folosind euristica stabilită şi întoarcem cea mai bună mutare

4.Folosind algoritmul Minimax pentru graful de mai jos, ce valoare vom avea în rădăcină?



Răspunsul corect este: 6

5. Câte noduri vor fi evaluate de algoritmul Alfa-Beta în arborele de mai sus

Răspunsul corect este: 21

Test parcurgerea grafurilor

1.Care algoritm parcurge toate nodurile unui graf?

Alegeţi una sau mai multe opţiuni:
1. Parcurgerea în adâncime
2. Sortarea topologică
3. Parcurgerea în lăţime
4. Niciuna din variante
The correct answers are: Parcurgerea în adâncime, Sortarea topologică

2.Câte culori sunt minim necesare pentru algoritmii de parcurgere (în adâncime sau lăţime) a unui graf?

Alegeţi o opţiune:
a. 3
b. 2
c. 1
d. 4
Răspunsul corect este: 2

3.Se poate detecta dacă un graf are cicluri prin prezenţa, la parcurgerea în adâncime, a arcelor:

Alegeţi o opţiune:
a. Direct, în arborele de parcurgere
b. De traversare
c. Înapoi
d. Înainte
Răspunsul corect este: Înapoi

4.Care este complexitatea pentru a itera prin toate arcele unui graf reprezentat prin liste de adiacenţă şi având n noduri şi a arce?

Alegeţi o opţiune:
a. Θ(n+a)
b. Θ(a**2)
c. Θ(n**2)
d. Θ(n*a)
Răspunsul corect este: Θ(n+a)

5.Parcurgerea în lăţime a unui graf poate fi folosită pentru:

Alegeţi una sau mai multe opţiuni:
a. Calculul numărului de componente conexe
b. Detectarea dacă un graf neorientat este conex
c. Nicio varainta din celelalte
d. Detectarea drumurilor de lungime minimă (ca număr de muchii)
The correct answers are: Detectarea drumurilor de lungime minimă (ca număr de muchii), Detectarea dacă un graf neorientat este conex

Test aplicatii DFS

1.Care este numărul maxim de sortări topologice pe care le poate avea un graf conex (dacă renunțăm la orientarea arcelor) cu n noduri?

Alegeți o opțiune:

- 1. n*n
- 2. (n-1)!
- 3. Combinări de n luate câte 2
- 4. n!

2.Care este numărul minim de arce pentru ca un graf cu n noduri să admită o sortare topologică unică?

Alegeți o opțiune:

- a. 2*n
- b. n*n/4
- c. n-1 Un exemplu simplu este un lanț, are n-1 noduri.
- d. N

3.Care dintre următoarele grafuri orientate are fiecare vârf într-o componentă tare conexă proprie cu un singur element?

Alegeți una sau mai multe opțiuni:

- [ ] a. Un graf complet
- [x] b. Un DAG (graf orientat aciclic)
- [x] c. Un lanț (v1 --> v2 --> ... --> vn)
- [ ] d. Un arbore cu sensul arcelor de la părinte spre copii

4.Pot exista grafuri conexe (cu n noduri) care să aibă cel puțin o punte și niciun punct de articulație?

Alegeți una sau mai multe opțiuni:

- [ ] a. Nu, orice punte are în capete două puncte de articulație
- [x] b. Nu, pentru n>2. În acest caz, orice punte are cel puțin un punct de articulație
- [ ] c. Da, nu există nicio legătură între punți și puncte de articulație
- [x] d. Da, dar doar pentru n=2

5.Fie un graf orientat cu n vârfuri și o singură componentă tare conexă. Dacă renunțăm la orientarea muchiilor, graful neorientat corespunzător poate avea câte puncte de articulație?

Alegeți una sau mai multe opțiuni:

- [ ] a. Întotdeauna 0
- [x] b. Minim 0
- [ ] c. Cel puțin un punct de articulație
- [x] d. Maxim n-2

Test Drumuri minime partea 1

1.Cel mai eficient mod de a determina drumurile minime de la un nod sursă la toate celelalte noduri într-un graf cu ponderi egale, folosește o structură de date de tip:

Alegeți o opțiune:

- 1. Stivă, în cadrul algoritmului DFS
- 2. Coadă, în cadrul algoritmului BFS
- 3. Heap Fibonacci, dacă graful este rar, în cadrul algoritmului Dijkstra
- 4. Heap Binar, dacă graful este rar, în cadrul algoritmului Dijkstra

2.Dându-se un graf orientat având costuri asociate, se presupune că drumurile minime de la sursa s la toate celelalte noduri au fost calculate corect folosind un algoritm de identificare a drumurilor minime de sursă unică. Care din următoarele propoziții sunt adevărate?

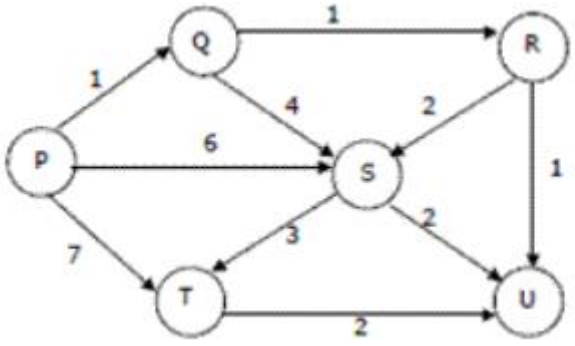
P1: Dacă se adaugă câte o unitate fiecărui arc din graf, căile calculate anterior rămân neschimbate (algoritmul va găsi exact aceleași căi).

P2: Dacă se dublează costurile tuturor arcelor din graf, căile calculate anterior rămân neschimbate (algoritmul va găsi exact aceleași căi).

Alegeți o opțiune:

- a. Ambele propoziții
- b. Doar P2
- c. Doar P1
- d. Niciuna dintre propoziții

3.Care este ordinea în care sunt parcurse nodurile din graful de mai jos, asupra căruia se aplică algoritmul Dijkstra pornind din nodul P?

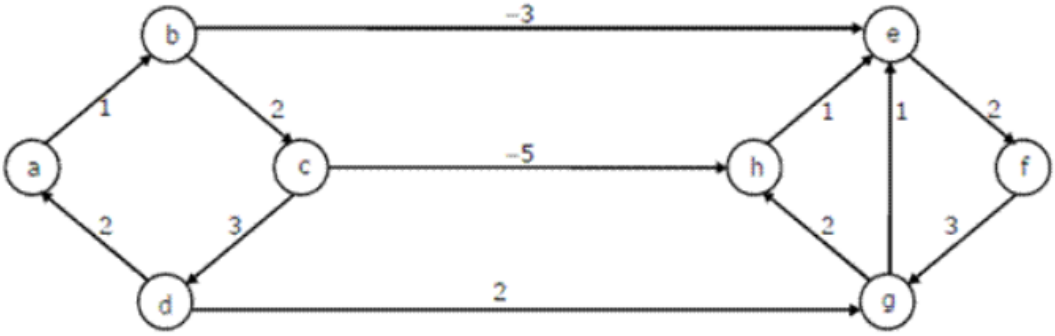


Alegeți una sau mai multe opțiuni:

- [ ] a. P, Q, T, R, U, S
- [x] b. P, Q, R, U, S, T
- [ ] c. P, Q, R, U, T, S

d. P, Q, R, S, T, U

4.Dându-se graful de mai jos, aplicarea algoritmului Dijkstra pornind din nodul a produce rezultate corecte pentru:



Alegeți una sau mai multe opțiuni:

a. Doar pentru nodul a, din cauza inițializării

b. Doar pentru nodurile a, b, c, d pentru că algoritmul ignoră arcele negative

c. Toate nodurile din graf

d. Niciun nod din graf, din cauză că graful conține costuri negative

5.Care din următoarele propoziții referitoare la algoritmul Bellman-Ford sunt adevărate?

P1: Întotdeauna identifică dacă graful conține un ciclu de cost negativ sau nu.

P2: Identifică dacă se poate ajunge la vreun ciclu de cost negativ pornind din sursă.

Alegeți una sau mai multe opțiuni:

a. Doar P2

Algoritmul nu identifică un ciclu la care nu poate să ajungă

b. Niciuna dintre propoziții

c. Ambele propoziții

d. Doar P1

Test Drumuri minime partea a 2-a

1.Drumurile de lungime minimă între toate nodurile unui graf dens se pot determina eficient folosind un algoritm de tip:

Alegeţi o opţiune:

a. Backtracking

b. Programare dinamică

c. Divide&impera

d. Greedy

e. Forţă brută

Răspunsul corect este: Programare dinamică

2.Algoritmul Floyd-Warshall nu poate fi folosit în cazul în care:

Alegeţi o opţiune:

a. Poate fi folosit în toate cazurile anterioare

b. Graful are arce negative

c. Graful are puncte de articulaţie

d. Graful este biconectat

e. Graful este tare conex

Răspunsul corect este: Poate fi folosit în toate cazurile anterioare

3.Închiderea tranzitivă a unui graf poate fi calculată eficient cu o complexitate temporală (n=număr noduri, a=număr arce):

Alegeţi o opţiune:

a. O(n**2)

b. O(a*log n +a)

c. O(n*a+log n)

d. O(n**3)

e. O(n+a)

Răspunsul corect este: O(n**3)

4.Algoritmul lui Johnson este o alternativă mai eficientă la:

Alegeți o opțiune:

a. Algoritmul lui Dijstra pentru grafuri rare cu arce negative

b. Algoritmul Bellman-Ford

c. Algoritmul Floyd-Warshall pentru grafuri rare cu arce negative

d. La niciuna din celelalte variante

e. Detectarea închiderii tranzitive a unui graf

Răspunsul corect este: Algoritmul Floyd-Warshall pentru grafuri rare cu arce negative

5.Algoritmul lui Johnson necesită:

Alegeți o opțiune:

a. Adaugarea a n (numărul de noduri al grafului) arce cu lungime zero

b. Adaugarea unui nod fictiv

c. Aplicarea algoritmului Dijkstra pentru fiecare nod

d. Aplicarea algoritmului Bellman-Ford

e. Toate celelalte răspunsuri sunt corecte

Răspunsul corect este: Toate celelalte răspunsuri sunt corecte

Test arbori minimi de acoperire

1.Care dintre următorii algoritmi ne oferă complexitatea cea mai scăzută pentru a determina arborele minim de acoperire pentru un graf **rar**?

Alegeți una sau mai multe opțiuni:

- [x] a. Algoritmul lui Prim folosind un heap binar pentru coada de priorități
- [ ] b. Algoritmul lui Prim folosind un vector pentru coada de priorități
- [x] c. Algoritmul lui Kruskal folosind mulțimi disjuncte implementate eficient
- [ ] d. Algoritmul lui Kruskal folosind un vector de culori

2.Care dintre următorii algoritmi ne oferă complexitatea cea mai scăzută pentru a determina arborele minim de acoperire pentru un graf **dens**?

Alegeți una sau mai multe opțiuni:

- [ ] a. Algoritmul lui Kruskal folosind mulțimi disjuncte implementate eficient
- [x] b. Algoritmul lui Prim folosind un vector pentru coada de priorități
- [ ] c. Algoritmul lui Kruskal folosind un vector de culori
- [ ] d. Algoritmul lui Prim folosind un heap binar pentru coada de priorități

3. Cum sunt implementate eficient mulțimile disjuncte?

Alegeți una sau mai multe opțiuni:

- [ ] a. Folosind o pădure de arbori binari
- [x] b. Folosind o pădure de arbori n-ari
- [x] c. Folosind euristica unificare după rang (union by rank)
- [x] d. Folosind euristica de compresie a căilor

4. Presupunând că avem următorul vector de culori pentru mulțimile disjuncte ale unui graf cu **n=10** noduri:

[1, 1, 3, 1, 3, 1, 7, 8, 7, 3]

Dacă alegem muchia **(2, 5)** la pasul curent al algoritmului Kruskal, cum arată vectorul de culori la terminarea pasului curent?

Alegeți una sau mai multe opțiuni:

- [ ] a. **[1, 1, 3, 1, 1, 1, 7, 8, 7, 3]**
- [x] b. **[1, 1, 1, 1, 1, 1, 7, 8, 7, 1]**
- [x] c. **[3, 3, 3, 3, 3, 3, 7, 8, 7, 3]**
- [ ] d. **[1, 3, 3, 1, 3, 1, 7, 8, 7, 3]**

5.Câți arbori minimi de acoperire diferiți poate avea un graf neorientat, cu muchii de costuri egale, cu un singur ciclu și **n** vârfuri?
Doi arbori sunt diferiți dacă conțin cel puțin o muchie diferită.

Alegeți o opțiune:

- a. Minim 1, maxim n
- b. Întotdeauna n
- c. Minim 2, maxim n
- d. Minim 3, maxim n

Test Flux Maxim

1.Ce presupune problema de identificare a fluxului maxim?

Alegeți o opțiune:

- a. Gasirea unui flux intre sursa si destinatie de valoare maxima
- b. Gasirea celei mai scurte distante intre sursa si destinatie
- c. Gasirea unui flux intre sursa si destinatie de valoare minima
- d. Gasirea unui arbore minim de acoperire

Răspunsul corect este: Gasirea unui flux intre sursa si destinatie de valoare maxima

2.Ce conditii trebuie satisfacute intr-o retea de flux?

Alegeți o opțiune:

- a. Trebuie sa se respecte capacitatile arcelor
- b. Trebuie sa se conserve fluxul
- c. Toate celelalte raspunsuri
- d. Trebuie sa se respecte simetria fluxului

Răspunsul corect este: Toate celelalte raspunsuri

3.Dandu-se o retea de flux in care f este un flux oarecare iar (A, B) o taietura a retele, atunci fluxul prin taietura (A, B)_____valoarea lui f:

Alegeți o opțiune:

- a. este mai mare ca
- b. nu are legatura cu
- c. este mai mic ca
- d. este egal cu

Răspunsul corect este: este egal cu

4.Cand se termina algoritmul Ford Fulkerson?

Alegeți o opțiune:

- a.Cand toate arcele de intrare in t sau toate arcele de iesire din s sunt saturate
- b. Cand fluxul pentru o taietura este egal cu fluxul de la drena(t)
- c. Dupa analiza fiecarei taieturi si gasirea aceleia care este de cost minim si care are capacitatea egala cu valoarea curenta a fluxului
- d. Cand nu mai poate fi gasita nicio cale de la s la t in graful rezidual

Răspunsul corect este: Cand nu mai poate fi gasita nicio cale de la s la t in graful rezidual

5.Care este complexitatea necesara gasirii unei cai reziduale (de crestere a fluxului) dintr-o retea de flux?

Alegeți o opțiune:

- a. O(|E|)
- b. O(|E|^2 log |V|)
- c. O(|E| log |V|)
- d. O(|E| ^ 2)

Răspunsul corect este: O(|E|)

Test - Algoritmul A*

1.Problema maximelor/minimelor locale nu apare la algoritmii:

Select one or more:

a. Gradientului maxim

b. Irevocabili

c. Greedy

d. A*

The correct answer is: A*

2.O problemă a algoritmului BF* („Best first”) este:

Select one or more:

a. Este neinformat

b. Este irevocabil

c. Nu este complet

d. Nu este optim

The correct answer is: Nu este optim

3.Algoritmul gradientului maxim are următoarea calitate:

Select one or more:

a. Este complet

b. Este optim

c. Are o complexitate temporală mai mică față de alți algoritmi de căutare

d. Spre deosebire de alți algoritmi de căutare, nu are nevoie să fie informat

The correct answer is: Are o complexitate temporală mai mică față de alți algoritmi de căutare

4.O euristică pozitivă este admisibilă la algoritmul A* dacă:

Select one or more:

a. Dacă este nemonotonă

b. Dacă se calculează cu o complexitate polinomială

c. Nu necesită ca algoritmul să fie informat

d. Este optimistă faţă de realitate

Feedback

Răspunsul dumneavoastră este incorect.

The correct answer is: Este optimistă faţă de realitate

5.Complexitatea unui algoritm A* este întotdeauna:

Select one or more:

a. cel mult polinomială

b. cel mult exponenţială

c. liniară

d. O(n**3)

The correct answer is: cel mult exponenţială

Test algoritmi aleatorii

1.Completati zona marcata cu ... in textul de mai jos:

Algoritmul aleatoriu de tip Las Vegas pentru identificarea liniilor repetabile intr-un fisier text este .... fata de algoritmul clasic de complexitate O(n^2)

Alegeți o opțiune:

- [ ] a. intodeauna mai lent
- [ ] b. intodeauna mai rapid cand procentul de linii repetabile este mai mare de 10% din text
- [ ] c. intodeauna mai rapid
- [x] d. cateodata mai rapid,dar poate fi si mai lent daca avem ghinion

2.Implementarea Las Vegas cu pivot aleatoriu folosita pentru implementarea sortarii rapide (QuickSort) are complexitatea?

Alegeți una sau mai multe opțiuni:

- [ ] a. ~~O(~~n^2)
- [ ] b. . ~~O(~~n*log n)
- [x] c. O(n^2)
- [x] d. O(n * log n), unde O este complexitatea unui algoritm aleatoriu

3.Care din urmatoarele afirmatii sunt adevarate pentru algoritmul Rabin-Miller pentru testarea primalitatii unui numar mare?

Alegeți una sau mai multe opțiuni:

- [x] a.Poate gresi si sa ne spuna ca un numar compus(in realitate) este de fapt prim (decis de algoritm)
- [x] b. Descopera cu siguranta daca un numar este prim doar daca il lasam sa ruleze la infinit
- [ ] c. Descopera cu siguranta daca un numar este prim
- [ ] d. Poate gresi si sa ne spuna ca un numar prim(in realitate) este de fapt compus (decis de algoritm)

4.Ce puteti spune despre urmatorul algoritm, daca vectorul A are jumatate din elemente egale cu 0 si cealalta jumatate egale cu 1?

```
find_ones(A[1...n])
        repeat
                i = random(1...n)
                x = A[i]
        until x == 1
```

Alegeți una sau mai multe opțiuni:

- [ ] a.Are complexitate O(n) – de tip algoritm aleatoriu
- [ ] b. Este un algoritm aleatoriu de tip Monte Carlo
- [x] c. Are complexitatea O(1) – de tip algoritm aleatoriu
- [x] d. Este un algoritm aleatoriu de tip Las Vegas

5.Ce puteti spune despre urmatorul algoritm, daca vectorul A are jumatate din elemente egale cu 0 si cealalta jumatate egale cu 1?

```
find_ones(A[1...n])
    k = limit(n)// stabilirea unei limite ce depinde de n, functie de complexitate theta(1)
    j = 0
    repeat
        i = random(1...n)
        x = A[i]
        j++
    until j == k or x == 1
```

Alegeți una sau mai multe opțiuni:

- [ ] a.Are complexitatea O(n) – de tip algoritm aleatoriu
- [x] b. Are complexitatea O(1) – de tip algoritm aleatoriu
- [ ] c. Este un algoritm aleatoriu de tip Las Vegas
- [x] d. Este un algoritm aleatoriu de tip Monte Carlo

1 întrebare

Corect
Marcat 1,00 din 1,00


Întrebare cu flag

Textul întrebării

Fie G un graf neorientat conex avand ponderi diferite pe muchii (fiecare muchie are alt cost). Fie e_max si e_min muchiile de cost maxim respectiv minim. Care dintre urmatoarele afirmatii este FALSA?
Alegeți o opțiune:

- a. Niciun AMA nu contine e_max
- b. Toti AMA trebuie sa contina e_min
- c. Daca e_max apartine AMA, stergerea lui conduce la pierderea conectivitatii AMA-ului
- d. G are un AMA unic

Feedback

Răspunsul corect este: Niciun AMA nu contine e_max

2 întrebare

Incorect
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care sunt domeniile variabilelor după aplicarea algoritmului AC1 ?
Alegeți o opțiune:

- a. A = {1, 2}, B = {5, 6}, C = {5, 6}
- b. A = {1, 2}, B = {5, 6, 7}, C = {5, 6, 7}
- c. A = {2}, B = {5}, C = {5}
- d. A = {1, 2, 3, 4, 5}, B = {5, 6, 7}, C = {5, 6, 7}

Feedback

Răspunsul corect este: A = {1, 2}, B = {5, 6}, C = {5, 6}

3 întrebare

Incorect
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Un algoritm A* (cu o euristică admisibilă) este întotdeauna:
Alegeți o opțiune:

- a. nici complet nici optimal
- b. complet
- c. complet și optimal
- d. optimal

Feedback

Răspunsul corect este: complet și optimal

4 întrebare

Incorect
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Dându-se un test grilă care trebuie rezolvat în 10 minute, ce conține 4 întrebări care au punctajele 56, 35, 24 și 27 și care necesită un timp de rezolvare de 8, 7, 4, 3 minute se cere să se identifice punctajul maxim ce poate fi obținut, știind că se dau punctaje parțiale pentru rezolvarea întrebărilor presupunând o rezolvare a întrebărilor uniformă în timp (ex: pentru prima întrebare se vor acorda 56/8 puncte pentru fiecare minut alocat).
Alegeți o opțiune:

- a. 56

- b. 68
- c. 62
- d. 76

Feedback

Răspunsul corect este: 76

5 întrebare

Incorect
Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Ce se schimbă la algoritmul lui Dijkstra pentru grafuri aciclice orientate?
Alegeți o opțiune:

- a. Nu se poate schimba ceva
- b. Nu se mai poate aplica
- c. Se transformă arcele orientate în neorientate
- d. Se poate eficientiza printr-o sortare topologică

Feedback

Răspunsul corect este: Se poate eficientiza printr-o sortare topologică

6 întrebare

Corect
Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Rulând algoritmul Quicksort pentru a sorta crescător vectorii v1 = [1 4 3 2 6 8 7 5] și v2 = [5 4 3 2 1] se obțin timpii t1 și t2. Ce puteți spune despre cei 2 timpi?

Alegeți o opțiune:

- a. cei 2 timpi nu pot fi comparați pentru că dimensiunea vectorului de sortat este diferită.
- b. t1 < t2
- c. t1 = t2
- d. t1 > t2

Feedback

Răspunsul corect este: t1 < t2

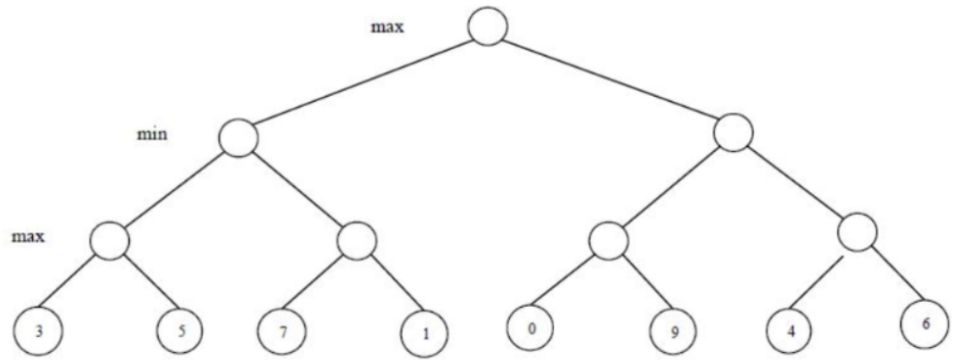
7 întrebare

Incorect
Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Pentru arborele minimax din figura următoare, menționați care sunt tăieturile alfa-beta dacă parcurgem nodurile de la dreapta spre stânga (în oglindă față de o parcurgere normală):



Alegeți o opțiune:

- a. Sunt două tăieturi: prima elimină frunza cu valoarea 0, a doua frunza cu valoarea 3
- b. Este o singură tăietură care elimină frunza cu valoarea 0
- c. Niciuna dintre celelalte variante nu este corectă
- d. Sunt trei tăieturi: prima elimină frunza cu valoarea 0, a doua frunza cu valoarea 7, a treia frunza cu valoarea 3

Feedback

Răspunsul corect este: Este o singură tăietură care elimină frunza cu valoarea 0

8 întrebare

Incorect
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Cum se modifică complexitatea și timpul de execuție pentru algoritmul de sortare prin interclasare dacă la fiecare pas impar împarte vectorul de dimensiune n în doi subvectori de dimensiune 1, respectiv n-1, iar la fiecare pas par împarte vectorul în doi subvectori de dimensiune egală:
Alegeți una sau mai multe opțiuni:

- a. Timpul de execuție crește de 2 ori
- b. Ordinul de creștere va fi mai mic
- c. Ordinul de creștere rămâne neschimbat, Θ(n * logn)
- d. Ordinul de creștere va fi mai mare

Feedback

The correct answers are: Ordinul de creștere rămâne neschimbat, Θ(n * logn), Timpul de execuție crește de 2 ori

9 întrebare

Corect
Marcat 1,00 din 1,00


Întrebare cu flag

Textul întrebării

Intr-un graf G neorientat se aplica algoritmul DFS si se obtine o padure de arbori DFS T. Fie u un nod din G si v primul nod alb vizitat de DFS dupa u. Care dintre afirmatiile de mai jos este intotdeauna corecta?
Alegeți o opțiune:

- a. (u, v) este muchie in G si v este descendent al lui u in T
- b. (u, v) este muchie in G si u este descendent al lui v in T

c. Daca (u, v) nu este muchie in G, atunci u este frunza in T

d. Daca (u, v) nu este muchie in G, atunci u si v au acelasi parinte in T

Feedback

Răspunsul corect este: Daca (u, v) nu este muchie in G, atunci u este frunza in T

10 întrebare

Corect
Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Care este complexitatea maximă a sortării a n numere folosind Quicksort Randomizat?
Alegeți o opțiune:

a. O(n^2)

b. O(n * log(n))

c. O(n!)

d. O(n)

Feedback

Răspunsul corect este: O(n^2)

11 întrebare

Incorect
Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

MINIMAX se poate aplica la jocul de șah în situația următoare:
Alegeți o opțiune:

a. Niciodată

b. Întotdeauna

c. Întotdeauna, dar folosind tăierea alfa-beta

d. Dacă se aplică pe un număr mic de niveluri și se folosesc funcții de evaluare a pozițiilor

Feedback

Răspunsul corect este: Dacă se aplică pe un număr mic de niveluri și se folosesc funcții de evaluare a pozițiilor

12 întrebare

Incorect
Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Fie G un graf ponderat avand toate ponderile mai mari ca 1 si G' același graf dar la care ponderea fiecarei muchii este patratul ponderii din primul graf. Daca T este AMA pentru G avand costul t si T' este AMA pentru G' avand costul t', care din urmatoarele afirmatii este adevarata?
Alegeți o opțiune:

a. Niciunul din celelalte raspunsuri

b. T' = T, t' = t ^ 2

c. T' = T, t' < t ^ 2

d. T' != T, t' = t ^ 2

Feedback

Răspunsul corect este: T' = T, t' < t ^ 2

13 întrebare

Incorect
Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Care este numărul maxim posibil de arbori de parcurgere în adâncime a unui graf orientat conex (dacă considerăm graful neorientat, dar nu obligatoriu tare conex) cu n noduri?
Alegeți o opțiune:

- a. n/2
- b. n-1
- c. 1
- d. n

Feedback

Răspunsul corect este: n

14 întrebare

Incorect
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Ce corespondențe sunt corecte între afirmațiile a), b), c), d) și i., ii., iii., iv.?
a) Greedy;
b) DFS;
c) Programare Dinamica;
d) Divide et Impera;

i. distanțele minime dintre oricare 2 noduri din graf;
ii. Quicksort;
iii. constructie AMA;
iv. componente conexe
Alegeți o opțiune:

- a. a-iv, b-i, c-ii, d-iii
- b. a-iii, b-iv, c-ii, d-i
- c. a-ii, b-iv, c-i, d-iii
- d. a-iii, b-iv, c-i, d-ii

Feedback

Răspunsul corect este: a-iii, b-iv, c-i, d-ii

15 întrebare

Incorect
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Tehnica programării dinamice se utilizează atunci când:
Alegeți o opțiune:

- a. problema are proprietatea substructurii optimale
- b. avem nevoie de un algoritm mai rapid decât greedy
- c. oricând avem de rezolvat o problemă de optimizare
- d. problema de rezolvat poate fi redusă la problema 3-SAT

Feedback

Răspunsul corect este: problema are proprietatea substructurii optimal

SUBIECTUL 3:

1 întrebare

Complet
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Fiind o problema de numarare ma gandesc la programare dinamica folosind o matrice. Ma gandesc ca s-ar putea pune pe linii intervalele si coloanele sa fie un range dintre minimul si

maximul din interval. Se pune 1 daca numarul de pe coloana se regaseste in intervalul respectiv. Apoi se verifica coloanele si daca pe coloana respectiva e doar un 1 se noteaza intervalul disjunct.

Feedback

Greedy: se sortează apoi se parcurg intervalele de la stânga la dreapta. O(n log n).

Comentarii

Comentariu:

2 întrebare

Complet
Marcat 0,00 din 1,00

 Întrebare cu flag

Textul întrebării

Fiind vorba de flux, as aplica algoritmul Edmonds-Karp, doar ca va fi modificat pentru a avea mai multe noduri destinatie. Vom tine un vector de bool unde vom nota daca nodul terminal respectiv este prim sau nu. Algoritmul se termina cand toate nodurile sunt bifate, sau nu se mai poate gasi o cale de transport pana la nodurile terminale.

Feedback

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comentarii

Comentariu:

3 întrebare

Complet
Marcat 0,50 din 1,00

Întrebare cu flag

Textul întrebării

Programare dinamica: o solutie obtinuta incremental astfel: gasim cea mai buna solutie folosind primele i - 1 elemente din sir, apoi incercam sa o extinde folosind elementul i. Intrucat la fiecare pas trebuie sa retinem cea mai buna slutie folosind un prefix din vectorul v, solutia va fi salvata intr-un tablou auiliar (vector).

Feedback

Greedy: se sortează descrescător și se iau primele k elemente.

Comentarii

Comentariu:

4 întrebare

Complet
Marcat 0,50 din 1,00

Întrebare cu flag

Textul întrebării

Folosim agoritmul lui Tarjan pentru gasirea punctelor de articulatie, dar si a muchiilor critice. Astfel vom gsai muchia critica care leaga cat mai multe aglomerari de orase intre ele (componente conexe) si o vom intrerupe.

Feedback

Găsim toate puntile. Gigel va alege puntea pentru care |numar noduri la stânga - numar noduri la dreapta| este minim.

Comentarii

Comentariu:

5 întrebare

Complet
Marcat 0,00 din 1,00


Întrebare cu flag

Textul întrebării

Ma gandesc sa tragem o axa verticala si orizontala pe care sa le notam cu distante si sa calculam distanta Manhattan intre puncte, distanta ce o vom retine intr-un tabel.De acolo extragem drumul care are distanta estimata (distanta Manhattan plus distanta reala) cea mai mica.

Feedback

distanța euclidiană până la destinatie din care se scade R

Comentarii

Comentariu:

6 întrebare

Complet
Marcat 1,00 din 1,00


Întrebare cu flag

Textul întrebării

Backtracking si adaugam de fiecare data un numar de muchii pentru un nod (astfel incat sa fei impar) apoi se trece mai departe si se schimba mereu numarul de muchii i

Feedback

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunt.

Comentarii

Comentariu:

Textul întrebării

Care este fluxul maxim într-un graf bipartit complet, unde toate nodurile din stânga pot fi surse, toate nodurile din dreapta consumatori, iar fiecare dintre cele n noduri din stânga este conectat la cele m noduri din dreapta prin muchii de capacitate crescătoare de la 1..m (nu contează ordinea)?

Alegeți o opțiune:

a. n*m*(m+1)

b. n*m*(m+1)/2

c. m*(m+1)/2

d. n*m

Feedback

Răspunsul dumneavoastră este corect.

Răspunsul corect este: n*m*(m+1)/2

2 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Având la dispoziție algoritmii de sortare Quicksort (cu alegerea primului element ca pivot) și MergeSort, se cere să se aleagă algoritmul cel mai eficient pentru a sorta vectorii v1 = [1 4 3 2 5] și v2 = [5 4 3 2 1]. Care dintre afirmațiile următoare este adevărată?

Alegeți o opțiune:

a. v1 se poate sorta cu oricare dintre ei, dar v2 se sortează neapărat cu Quicksort

b. v1 se poate sorta cu oricare dintre ei, dar v2 se sortează neapărat cu MergeSort

c. ambii vectori se sortează cu Quicksort

d. ambii vectori se sortează cu MergeSort

Feedback

Răspunsul corect este: v1 se poate sorta cu oricare dintre ei, dar v2 se sortează neapărat cu MergeSort

3 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Tehnica programării dinamice se utilizează atunci când:

Alegeți o opțiune:

a. oricând avem de rezolvat o problemă de optimizare

b. problema are proprietatea substructurii optimale

c. problema de rezolvat poate fi redusă la problema 3-SAT

d. avem nevoie de un algoritm mai rapid decât greedy

Feedback

Răspunsul corect este: problema are proprietatea substructurii optimale

4 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Fie G un graf ponderat avand toate ponderile mai mari ca 1 si G' același graf dar la care ponderea fiecarei muchii este patratul ponderii din primul graf. Daca T este AMA pentru G avand costul t si T' este AMA pentru G' avand costul t', care din urmatoarele afirmatii este adevarata?

Alegeți o opțiune:

a. T' = T, t' = t ^ 2

b. T' != T, t' = t ^ 2

c. T' = T, t' < t ^ 2

d. Niciunul din celelalte raspunsuri

Feedback

Răspunsul corect este: T' = T, t' < t ^ 2

Comentarii

Comentariu:

5 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

O condiție ca un algoritm A* a1 să domine alt algoritm A* a2 este ca:

Alegeți o opțiune:

a. a1 să aibă o complexitate mai mică decât a2

b. o euristică monotonă pentru a2 este mai informată decât o euristică monotonă pentru a1

c. o euristică monotonă pentru a1 este mai informată decât o euristică monotonă pentru a2

d. a1 să fie complet iar a2 nu

Feedback

Răspunsul corect este: o euristică monotonă pentru a1 este mai informată decât o euristică monotonă pentru a2

6 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Pentru care dintre următoarele distribuții de probabilitate de apariție a cheilor vom avea cel puțin doi arbori Huffman distincți?

Alegeți una sau mai multe opțiuni:

a. 1/20, 1/10, 1/10, 3/20, 7/20, 1/4

b. 1/4, 1/4, 1/4, 1/4

c. 1/16, 1/16, 1/8, 1/8, 1/4, 3/8

d. 1/16, 1/16, 1/8, 1/4, 1/2

Feedback

The correct answers are: 1/16, 1/16, 1/8, 1/8, 1/4, 3/8, 1/4, 1/4, 1/4, 1/4, 1/20, 1/10, 1/10, 3/20, 7/20, 1/4

7 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Considerand arborele obținut în urma aplicării unui BFS din nodul S într-un graf orientat fără ponderi, ce se poate spune despre acest arbore?

Alegeți o opțiune:

a. Calculează cele mai scurte drumuri între oricare două noduri din graf

b. Calculeaza cele mai scurte drumuri de la S doar pana la nodurile ce pot fi atinse din S

c. Calculeaza cele mai scurte drumuri de la S doar pana la frunzele din arbore

d. Calculeaza cele mai scurte drumuri de la S la toate nodurile din graf

Feedback

Răspunsul corect este: Calculeaza cele mai scurte drumuri de la S doar pana la nodurile ce pot fi atinse din S

8 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Care dintre următoarele afirmații sunt adevărate despre arborii optimali de căutare?

Alegeți una sau mai multe opțiuni:

a. După fiecare inserare și ștergere se re-echilibrează arborele

b. Se cunoaște probabilitatea căutării unei chei care este în arbore

c. Sunt arbori binari de căutare

d. Se cunoaște probabilitatea căutării unei chei care nu este în arbore

Feedback

The correct answers are: Sunt arbori binari de căutare, Se cunoaște probabilitatea căutării unei chei care este în arbore, Se cunoaște probabilitatea căutării unei chei care nu este în arbore

9 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Fie w ponderea minima a unei muchii dintr-un graf neorientat conex si e o muchie din graf a carei pondere este w. Care dintre urmatoarele afirmatii este FALSA?

Alegeți o opțiune:

a. Oricare AMA contine muchia e

b. Exista un AMA care contine muchia e

c. Fiecare AMA are o muchie de cost w

d. Daca e nu apartine unui AMA, atunci e face parte dintr-un ciclu in care toate muchiile au acelasi cost (w)

Feedback

Răspunsul corect este: Oricare AMA contine muchia e

10 întrebare

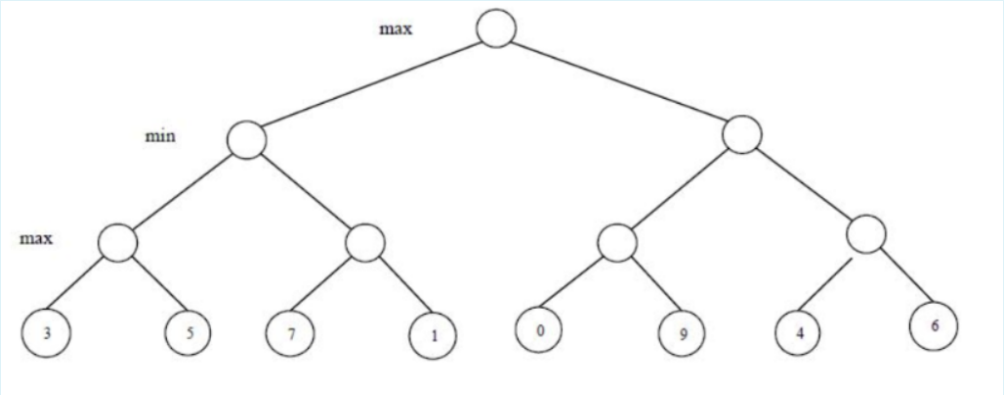
Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Pentru arborele minimax din figura următoare, menționați care sunt tăieturile alfa-beta, dacă arborele este parcurs de la stânga spre dreapta:



Alegeți o opțiune:

a. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua frunza cu valoarea 6

b. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua subarborele cu frunzele cu valorile 4 și 6

c. Niciuna dintre celelalte variante nu este corectă

d. Este o singură tăietură care elimină frunza cu valoarea 1

Feedback

Răspunsul corect este: Este o singură tăietură care elimină frunza cu valoarea 1

11 întrebare

Parțial corect

Marcat 0,50 din 1,00

Întrebare cu flag

Textul întrebării

Cum se modifică complexitatea și timpul de execuție pentru algoritmul de sortare prin interclasare dacă la fiecare pas impar împarte vectorul de dimensiune n în doi subvectori de dimensiune 1, respectiv n-1, iar la fiecare pas par împarte vectorul în doi subvectori de dimensiune egală:

Alegeți una sau mai multe opțiuni:

a. Ordinul de creștere va fi mai mic

b. Ordinul de creștere va fi mai mare

c. Ordinul de creștere rămâne neschimbat, Θ(n * logn)

d. Timpul de execuție crește de 2 ori

Feedback

The correct answers are: Ordinul de creștere rămâne neschimbat, Θ(n * logn), Timpul de execuție crește de 2 ori

12 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Este de preferat să se rezolve problema comis-voiajorului (dându-se o listă cu n locații și m străzi cu lungimi precizate, un comis-voiajor are misiunea de a livra colete la toate aceste locații, întorcându-se la final de unde a plecat, trecând prin fiecare locație doar o dată și obținând un drum de lungime minimă):

Alegeți una sau mai multe opțiuni:

a. folosind greedy, pentru că este singura metodă prin care se poate rezolva

b. folosind programare dinamică, deși soluția rămâne exponențială

c. folosind backtracking

d. folosind programare dinamică, pentru că este singura metodă prin care se poate rezolva

Feedback

The correct answers are: folosind backtracking, folosind programare dinamică, deși soluția rămâne exponențială

13 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Ce tip de relații matematice se folosesc pentru modelarea restricțiilor în scopul eficientizării rezolvării problemei Sudoku prin backtracking?

Alegeți o opțiune:

a. Incluziune

b. Sume egale

c. Sumă inegală

d. Inegalitate

Feedback

Răspunsul corect este: Inegalitate

14 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Fie G un graf neorientat conex avand ponderi diferite pe muchii (fiecare muchie are alt cost). Fie e_max si e_min muchiile de cost maxim respectiv minim. Care dintre urmatoarele afirmatii este FALSA?

Alegeți o opțiune:

a. Toti AMA trebuie sa contina e_min

b. Daca e_max apartine AMA, stergerea lui conduce la pierderea conectivitatii AMA-ului

c. G are un AMA unic

d. Niciun AMA nu contine e_max

Feedback

Răspunsul corect este: Niciun AMA nu contine e_max

15 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Care din următoarele funcții de tip euristică NU este admisibilă pentru algoritmul A* care calculează ruta de distanță minimă pe calea ferată între două

orașe, o sursă și o destinație? Pentru toate funcțiile euristică de mai jos, valoarea euristicii pentru destinație este egală cu 0, valorea fiind calculată doar pentru restul nodurilor.

Alegeți una sau mai multe opțiuni:

a. o valoare constantă pentru fiecare nod, egală cu valoarea muchiei de cost minim din graf (distanța minimă pe cale ferată între cele mai apropiate două orașe)

b. lungimea rutei deja parcurse până la fiecare nod

c. o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație

d. lungimea în linie dreaptă de la fiecare nod la destinație

Feedback

The correct answers are: lungimea rutei deja parcurse până la fiecare nod, o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație

1 întrebare

Complet

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Putem folosi backtracking. Plecand de la nodurile goale cu grad 0 adaug muchii in numar impar si adaug in solutie. Daca exista pe undeva grad par scot sau adaug o muchie si continui.

Feedback

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunt.

Comentarii

Comentariu:

2 întrebare

Complet

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Se poate face greedy prin doua BFS-uri. Primul gaseste sfarsitul celei mai lungi cai (nodul cel mai departat de nodul din care porneste) si al doilea gaseste din acest punct de fapt calea cea mai lunga ceruta de problema.

Feedback

Găsim toate puntile. Gigel va alege puntea pentru care |numar noduri la stânga - numar noduri la dreapta| este minim.

Comentarii

Comentariu:

3 întrebare

Complet

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Greedy : Se sorteaza intervalele tinand cont de sfarsitul lor. Le parcurgem si daca gasim intervale care se suprapun, il alegem pe cel cu cel mai mic sfarsit ca sa ne asiguram ca vom gasi cat mai multe. Facem acelasi lucru pentru toate intervalele incrementand un contor de fiecare data cand gasim un interval care satisface problema.

Feedback

Greedy: se sortează apoi se parcurg intervalele de la stânga la dreapta. O(n log n).

Comentarii

Comentariu:

4 întrebare

Complet

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Se scriu cazurile de baza pana la n < 4 si vectorul se construieste apoi dupa formula pt n >= 4.

Feedback

Exponențiere logaritmică pe matrice

Comentarii

Comentariu:

5 întrebare

Complet

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Se adauga o sursa unica cu arce de capacitate infinita spre sursele multiple si flux egal cu fluxul generat de sursele respective. La fel, se adauga si o destinatie unica de capacitate infinita si flux egal cu fluxul ce intra in destinatii.

Se aplica apoi Ford Fulkerson pe graful nou format

Feedback

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comentarii

Comentariu:

6 întrebare

Complet

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Tinand cont ca nu se poate stoca integral in memorie ne putem gandi la o implementare de tip programare dinamica.

sau

Putem folosi un vector sa stocam primele k elemente. Gasim minimul si il pastram . Pentru urmatoarele k elemente, comparam cu elementul minim anterior si daca sunt mai mici le pastram. Se repeta algoritmul si la sfarsit se afiseaza elementele.

Feedback

n log k- folosim un maxheap

n * k - 0.5 din punctaj

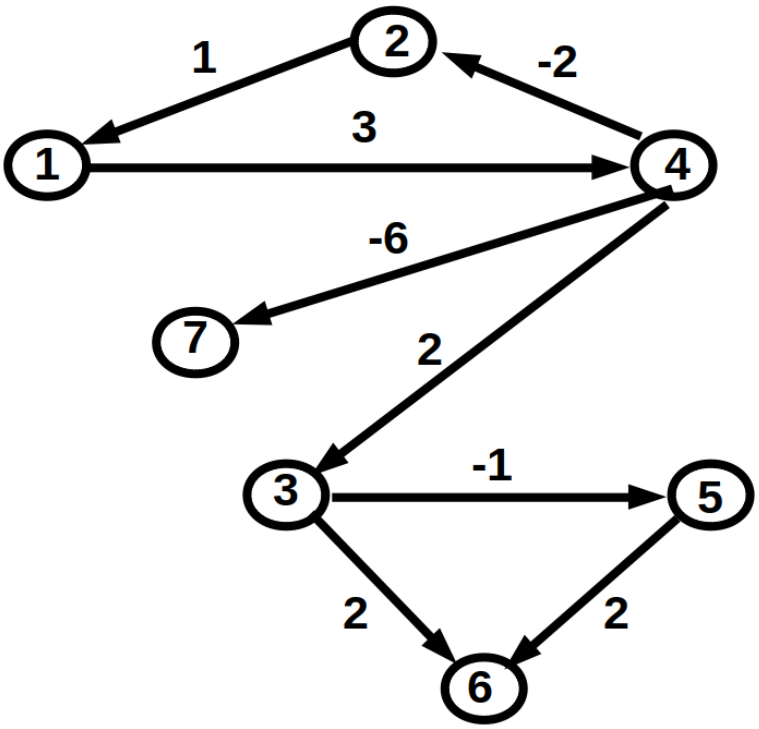
Comentarii

Comentariu:

SUBIECTUL I (10p = 5x2p)

Redactare: pe foaie, upload de poze cu toate paginile NUMEROTATE (1/n, 2/n, ..., n/n)

Timp de lucru: 40 min (inclusiv încărcarea pozelor pe moodle)



1. Considerați graful neorientat și realizați o parcurgere DFS pornind din nodul 6 în urma căreia să se evidențieze timpii de start și finish pentru fiecare nod.

2. Considerați graful orientat și realizați o parcurgere BFS pornind din nodul 4 în urma căreia să evidențiați ordinea parcurgerii nodurilor.

3. Considerând graful neorientat și având costurile din figură, găsiți un arbore parțial de cost minim, folosind algoritmul lui Prim, considerând rădacina arborelui nodul 2.

4. Considerând graful neorientat precizați care sunt componentele biconexe.

5. Pentru acest exercițiu ștergeți nodul 7 (împreună cu muchiile incidente). Considerând graful orientat și cu costurile din figură, să se aplice algoritmul A*, considerând starea inițială nodul 2 și starea finală nodul 6. Să se evidențieze, la fiecare pas, mulțimile OPEN și CLOSED, precum și modificările efectuate asupra tablourilor f și g.

Euristica h folosită este dată în următorul tabel:

node	1	2	3	4	5	6
h	5	6	1	3	2	0

Observații despre redactare

- Pentru subpunctele 1 și 2 rezolvarea se poate face folosind un desen.
- Pentru subpunctele 3 și 5 arătați la fiecare pas care este conținutul structurilor de date folosite.
- Pentru subpunctul 4 nu trebuie justificare.

SUBIECTUL 2

Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care sunt domeniile variabilelor după aplicarea algoritmului AC1 ?

Select one:

- a. A = {1, 2}, B = {5, 6, 7}, C = {5, 6, 7}
- b. A = {1, 2}, B = {5, 6}, C = {5, 6}
- c. A = {2}, B = {5}, C = {5}
- d. A = {1, 2, 3, 4, 5}, B = {5, 6, 7}, C = {5, 6, 7}

The correct answer is: A = {1, 2}, B = {5, 6}, C = {5, 6}

Cum poate fi modificat algoritmul de tip Las Vegas pentru detecția liniilor duplicate dintr-un fișier text discutat la curs într-un algoritm de tip Monte Carlo?

Select one:

- a. Algoritmii Las Vegas nu pot fi transformați în algoritmi Monte Carlo niciodată
- b. Se poate adăuga o limită de pași pentru a găsi linia duplicat, dacă nu o găsim în după limita de pași, spunem că nu există
- c. Reținem liniile alese până la momentul curent într-un hash și verificăm dacă noile linii sunt egale cu oricare din cele din hash
- d. Deși unii algoritmi de tip Las Vegas pot fi transformați în algoritmi de tip Monte Carlo, cel pentru detecția liniilor duplicate nu prezintă această proprietate

The correct answer is: Se poate adăuga o limită de pași pentru a găsi linia duplicat, dacă nu o găsim în după limita de pași, spunem că nu există

O problemă a algoritmilor greedy este că:

Select one:

- a. Se pot bloca în maxime sau minime locale
- b. Consumă multă memorie suplimentară
- c. Fac multe reveniri
- d. Au întotdeauna o complexitate mai mare decât alți algoritmi

Feedback

The correct answer is: Se pot bloca în maxime sau minime locale

Ce puteti spune daca matricea rezultata in urma aplicarii algoritmului lui Johnson contine valori negative pe diagonala principala?

Select one:

- a. Nu este nicio problema, matricea poate sa contina valori negative
- b. Acele valori nu conteaza, pentru ca reprezinta distanțe de la un nod la el insusi care nu sunt folosite nicaieri
- c. Exista o eroare in cod
- d. Nodurile corespunzatoare valorilor negative fac parte din cicluri de cost negativ

Feedback

The correct answer is: Exista o eroare in cod

În ce situații este preferabil să folosiți algoritmul Mergesort față de Quicksort care alege întotdeauna primul element ca pivot?

Select one:

a. Niciodată, deoarece Quicksort este întotdeauna mai rapid ca Mergesort

b. Întotdeauna deoarece Mergesort este mai rapid ca Quicksort

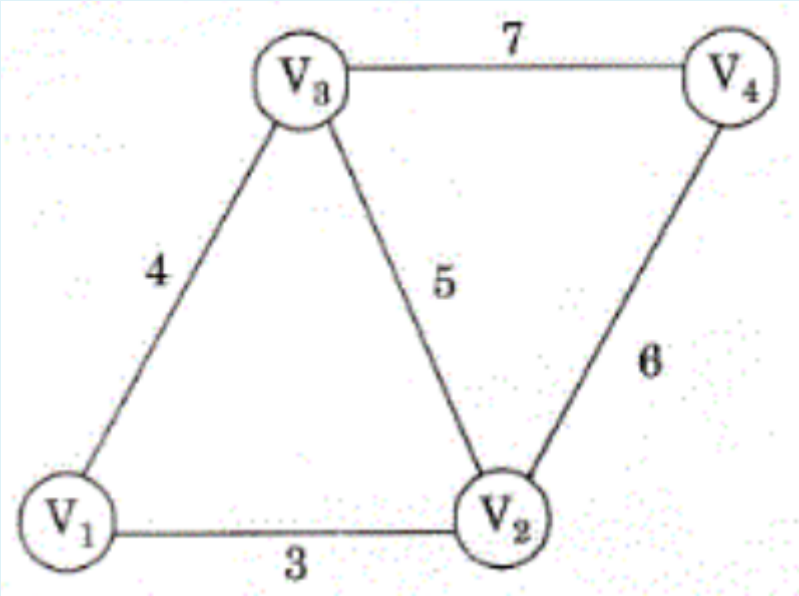
c. Când vectorul de sortat conține mai multe duplicate

d. Când vectorul este (aproape) sortat crescător sau descrescător

Feedback

The correct answer is: Când vectorul este (aproape) sortat crescător sau descrescător

Se da un graf neorientat cu n noduri {v_1, v_2 ... v_n}, n > 7, in care doua noduri v_i si v_j sunt conectate daca si numai daca 0 < |i-j| <= 2, iar in acest caz ponderea muchiei este i + j. Care este costul caii de la nodul v_5 la v_6 in AMA-ul acestui graf? Un exemplu de graf cu n=4 este prezentat in figura de mai jos:



Select one:

a. 25

b. 11

c. 31

d. 41

Feedback

The correct answer is: 31

Într-o sesiune de examene ce durează n zile trebuie date m examene (m<n). Profesorii materiilor respective impun condiționări legate de momentul maxim de timp până la care pot fi date examenele. Cum poate fi rezolvată cel mai eficient problema planificării unui număr maxim posibil de examene?

Select one:

a. folosind algoritmi aleatori, se generează diverse configurații până când se nimerește una validă

b. folosind programare dinamică, se generează un tablou care să rețină toate examenele ce pot fi date până în ziua i și apoi se verifică câte examene se pot da până în ziua n

c. folosind greedy, se sortează crescător condițiile impuse de profesori și se stabilesc zilele de examen astfel încât să se respecte condițiile

d. folosind backtracking, se generează toate variantele și se alege cea care respectă cât mai multe condiționări

Feedback

The correct answer is: folosind greedy, se sortează crescător condițiile impuse de profesori și se stabilesc zilele de examen astfel încât să se respecte condițiile

Problema „platoului" apare la algoritmul:

Select one:

a. A*

b. gradientului maxim

c. Dijkstra

d. BF*

Feedback

The correct answer is: gradientului maxim

Care este fluxul maxim într-un graf bipartit complet, unde toate nodurile din stânga pot fi surse, toate nodurile din dreapta consumatori, iar fiecare dintre cele n noduri din stânga este conectat la cele m noduri din dreapta prin muchii de capacitate crescătoare de la 1..m (nu contează ordinea)?

Select one:

a. n*m*(m+1)/2

b. n*m*(m+1)

c. m*(m+1)/2

d. n*m

Feedback

Răspunsul dumneavoastră este corect.

The correct answer is: n*m*(m+1)/2

Care este numărul maxim posibil de arbori de parcurgere în adâncime a unui graf orientat conex (dacă considerăm graful neorientat, dar nu obligatoriu tare conex) cu n noduri?

Select one:

- a. n-1
- b. n/2
- c. 1
- d. n

Feedback

The correct answer is: n

Care e avantajul principal al tăieturilor alfa-beta față de Minimax clasic?

Select one:

- a. Returnează rezultate mai bune
- b. Este mai ușor de implementat
- c. Nu oferă niciun avantaj
- d. În general, returnează rezultate mai rapid

Feedback

The correct answer is: În general, returnează rezultate mai rapid

Care dintre următoarele afirmații sunt adevărate despre arborii optimali de căutare?

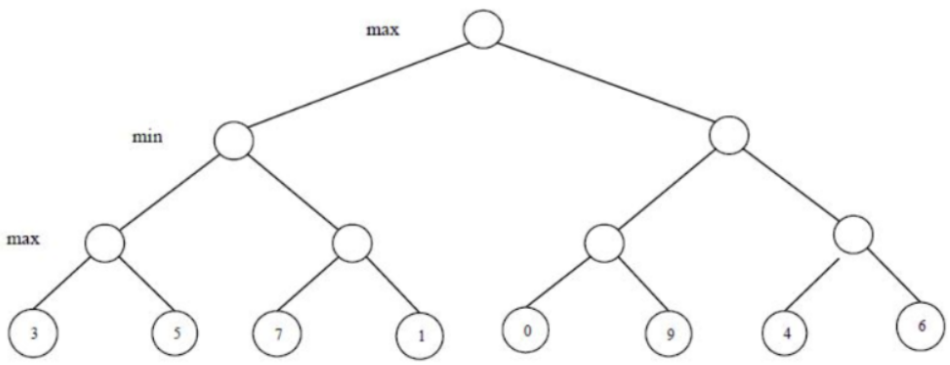
Select one or more:

- [ ] a. Mulțimea cheilor din arbore nu trebuie să fie dată în avans
- [x] b. Mulțimea cheilor din arbore trebuie cunoscută în avans
- [ ] c. Arborele optimal de căutare rezultat poate să nu fie echilibrat în anumite cazuri
- [x] d. Arborii optimali de căutare sunt întotdeauna echilibrați

Feedback

The correct answers are: Mulțimea cheilor din arbore trebuie cunoscută în avans, Arborele optimal de căutare rezultat poate să nu fie echilibrat în anumite cazuri

Pentru arborele minimax din figura următoare, menționați care sunt tăieturile alfa-beta, dacă arborele este parcurs de la stânga spre dreapta:



Select one:

- a. Niciuna dintre celelalte variante nu este corectă
- b. Este o singură tăietură care elimină frunza cu valoarea 1
- c. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua frunza cu valoarea 6
- d. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua subarborele cu frunzele cu valorile 4 și 6

The correct answer is: Este o singură tăietură care elimină frunza cu valoarea 1

Fie w ponderea minima a unei muchii dintr-un graf neorientat conex si e o muchie din graf a carei pondere este w. Care dintre urmatoarele afirmatii este FALSA?

Select one:

- a. Daca e nu apartine unui AMA, atunci e face parte dintr-un ciclu in care toate muchiile au acelasi cost (w)
- b. Oricare AMA contine muchia e
- c. Exista un AMA care contine muchia e
- d. Fiecare AMA are o muchie de cost w

Feedback

The correct answer is: Oricare AMA contine muchia e

Dându-se o serie de n sarcini, fiecare având o durată de o unitate de timp, o utilitate u[i] și un deadline (termen limită) d[i], cum se poate determina, cu o complexitate optimă, utilitatea maximă ce se poate obține prin îndeplinirea sarcinilor? Utilitatea unei sarcini u[i] este obținută doar dacă sarcina se termină înainte de deadline-ul corespunzător.

Select one:

- a. se folosește un algoritm de tip greedy sortând sarcinile în funcție de deadline și utilitate
- b. se folosește un algoritm de tip backtracking pentru a genera toate posibilitățile și se alege varianta cu utilitate maximă
- c. se folosește un algoritm de tip divide et impera în care se împarte succesiv timpul în 2 jumătăți și se identifică sarcinile ce se pot executa în intervalul respectiv
- d. Se folosește programarea dinamică pentru a stabili ce sarcini se execută și când, ținând cont de descompunerea recursivă a problemei în subprobleme

Feedback

The correct answer is: se folosește un algoritm de tip greedy sortând sarcinile în funcție de deadline și utilitate

1. Care dintre următoarele afirmații este adevărată pentru construcția unui arbore Huffman cu n chei?

Select one or more:
a. Implementarea eficientă necesită utilizarea unui heap binar
b. Complexitatea este Θ(n * logn)
c. Complexitatea este Θ(n^2)
d. Complexitatea este Θ(n)

Feedback
The correct answers are: Complexitatea este Θ(n * logn), Implementarea eficientă necesită utilizarea unui heap binar

2. Care din afirmațiile de mai jos sunt adevărate? i) Programele recursive sunt eficiente ; ii) Cea mai proastă complexitate a lui Quicksort este O(n^2) ; iii) Cea mai eficientă metodă de a implementa căutarea binara este folosind o listă înlănțuită

Select one:
a. i) si ii)
b. ii) si iii)
c. i) si iii)
d. doar ii)

Feedback
The correct answer is: doar ii)

3. Dându-se o mulțime de numere {n_1, n_2 ... n_k} și o valoare naturală S se poate detecta dacă există o submulțime de valori astfel încât suma lor să fie S folosind un algoritm de programare dinamică. Astfel, se folosește un tablou bi-dimensional dp[i, j] cu 1 <= i <= n, 0 <= j <= S care are valoarea Adevărat dacă există un un subset al {n_1, n_2 ... n_i} care să aibă suma j. Care este relația de recurență pentru calculul lui dp[i, j]?

Select one:
a. dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]
b. dp[i, j] = dp[i - 1, j] AND dp[i - 1, j - n_i]
c. dp[i, j] = dp[i - 1, j] OR dp[i, j - n_i]
d. dp[i, j] = dp[i - 1, j] AND dp[i, j - n_i]

Feedback
The correct answer is: dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]

4. Sortând o listă de n cuvinte în ordine lexicografică, fiecare de lungime n, cu ajutorul algoritmului MergeSort se poate ajunge la o complexitate maximă de:

Select one:
a. O(n^2)
b. O((n^2) * log(n))

c. O(n * log(n))
d. O((n^2) * log^2(n))

Feedback
The correct answer is: O((n^2) * log(n))

5. O euristică pentru algoritmul A* trebuie să fie:

Select one:
a. pesimistă
b. irevocabilă
c. optimistă
d. de tip gradient

Feedback
The correct answer is: optimistă

6. Care dintre afirmațiile următoare sunt adevărate referitor la algoritmul Minimax?

Select one or more:
a. Utilizarea unei funcții de evaluare a pozițiilor este esențială pentru jocuri complexe precum șahul sau go
b. Ordonarea nodurilor din arbore, în funcție de valoarea poziției, pe fiecare nivel este importantă pentru Minimax clasic, fără tăieturi alfa-beta
c. Față de Minimax, utilizarea tăieturilor alfa-beta permite o explorare mai adâncă, pe mai multe niveluri, în același interval de timp
d. Ordonarea nodurilor din arbore, în funcție de valoarea poziției, pe fiecare nivel este importantă dacă se folosesc tăieturi alfa-beta

Feedback
The correct answers are: Utilizarea unei funcții de evaluare a pozițiilor este esențială pentru jocuri complexe precum șahul sau go, Față de Minimax, utilizarea tăieturilor alfa-beta permite o explorare mai adâncă, pe mai multe niveluri, în același interval de timp, Ordonarea nodurilor din arbore, în funcție de valoarea poziției, pe fiecare nivel este importantă dacă se folosesc tăieturi alfa-beta

7. Care din următoarele funcții de tip euristică garantează optimalitatea soluției și expandează mai puține noduri decât celelalte? Pentru toate funcțiile euristice de mai jos, valoarea euristicii pentru destinație este egală cu 0, valorea fiind calculată doar pentru restul nodurilor.

Select one:
a. o valoare constantă pentru fiecare nod, egală cu valoarea muchiei de cost minim din graf (distanța minimă pe cale ferată între cele mai apropiate două orașe)
b. lungimea rutei deja parcurse până la fiecare nod
c. lungimea în linie dreaptă de la fiecare nod la destinație

d. o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație

Feedback
The correct answer is: lungimea în linie dreaptă de la fiecare nod la destinație

8. Cum se pot calcula eficient drumurile de lungime maximă care pleacă dintr-un nod pentru grafuri aciclice orientate cu ponderi w_i?

Select one:
a. Se aplică Bellman-Ford pe graful inițial
b. Nu se pot calcula
c. Se poate face calculând drumurile minime cu o sortare topologică pe graful cu arcele cu ponderi 1/w_i
d. Se aplică Dijkstra pe graful inițial

Feedback
The correct answer is: Se poate face calculând drumurile minime cu o sortare topologică pe graful cu arcele cu ponderi 1/w_i

9. Algoritmii de tip backtracking dirijiați de dependențe permit:

Select one:
a. Detectarea soluțiilor finale neoptime
b. Evitarea explorării inutile a unor zone din spațiul de căutare
c. Evitarea blocării în minime locale
d. Întotdeauna rezolvarea cu o complexitate polinomială

Feedback
The correct answer is: Evitarea explorării inutile a unor zone din spațiul de căutare

10. Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care dintre următoarele secvențe de pași sunt complete și plauzibil a fi aplicate de algoritmul AC1 ?

Select one:
a. REVISE(A, B), REVISE(B, C)
b. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)
c. REVISE(A, B), REVISE(B, C), REVISE(A, B), REVISE(B, C)
d. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B)

Feedback
The correct answer is: REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)

11. Intr-un graf neorientat G se aplica algoritmul DFS si se obtine o padure de arbori DFS - o notam cu T. Fie u un nod din G si v un succesor al lui u de culoare gri gasit de DFS. Care dintre afirmatiile de mai jos este corecta?

Select one:

a. (u, v) este muchie in G si v este un descendent, dar nu este copil direct al lui u in T
b. (u, v) este muchie in G si v este parintele lui u in T
c. (u, v) este muchie in G si u este un descendent, dar nu este copil direct al lui v in T
d. (u, v) este muchie in G si u este parintele lui v in T

Feedback
The correct answer is: (u, v) este muchie in G si u este un descendent, dar nu este copil direct al lui v in T

12. Ce puteti spune daca matricea rezultata in urma aplicarii algoritmului lui Johnson contine valori negative pe diagonala principala?

Select one:

a. Exista o eroare in cod
b. Nu este nicio problema, matricea poate sa contina valori negative
c. Acele valori nu conteaza, pentru ca reprezinta distanțe de la un nod la el insusi care nu sunt folosite nicaieri
d. Nodurile corespunzatoare valorilor negative fac parte din cicluri de cost negativ

Feedback
The correct answer is: Exista o eroare in cod

13. Care este fluxul maxim într-un graf bipartit complet, unde toate nodurile din stânga pot fi surse, toate nodurile din dreapta consumatori, iar fiecare dintre cele n noduri din stânga este conectat la cele m noduri din dreapta prin muchii de capacitate crescătoare de la 1..m (nu contează ordinea)?

Select one:

a. n*m*(m+1)/2
b. n*m
c. n*m*(m+1)
d. m*(m+1)/2

Feedback
Răspunsul dumneavoastră este corect.
The correct answer is: n*m*(m+1)/2

14. Calculul celor mai scurte distanțe de la un nod S la toate celelalte noduri din graf, într-un graf neorientat și fără ponderi se realizeaza cel mai eficient folosind:

Select one:
a. un DFS pornit din S
b. un BFS pornit din S
c. algoritmul lui Dijkstra aplicat din S
d. algoritmul lui Floyd Warshall

Feedback
The correct answer is: un BFS pornit din S

15. Dându-se un test grilă care trebuie rezolvat în 10 minute, ce conține 4 întrebări care au punctajele 56, 35, 24 și 27 și care necesită un timp de rezolvare de 8, 7, 4, 3 minute se cere să se identifice punctajul maxim ce poate fi obținut, știind că se dau punctaje parțiale pentru rezolvarea întrebărilor presupunând o rezolvare a întrebărilor uniformă în timp (ex: pentru prima întrebare se vor acorda 56/8 puncte pentru fiecare minut alocat).

Select one:
a. 62
b. 68
c. 56
d. 76

Feedback
The correct answer is: 76

Subiectul 3:

1.

Se dă un graf orientat cu *m* noduri, având capacitați numere pare pe arce. Se dau o sursă *S* și *p* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la sursă la toate destinațiile.

R: Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

2.

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

Raspunsul meu: Folosim o structura de tipul maxHeap.
Citim cate un numar si daca in heapul nostru avem mai putin de k elemente, atunci il introducem in heap.

Daca citim un numar si heapul este deja plin(are k elemente), comparam noul numar cu elementul cel mai mare din heap (operatie ce se realizeaza in O(1)), iar daca noul element este mai mic inseamna ca putem aduce o imbunatatire la top K, deci stergem elementul cel mai mare din heap si il adaugam pe cel nou citit

Complexitate : N * logK.

Raspunsul lor: n log k- folosim un maxheap
n * k - 0.5 din punctaj

3.

Cum se poate calcula eficient al *K*-lea cel mai mic număr dintr-o secvență de N numere?

Raspunsul meu: Ne putem folosi de partitionare din algoritmul quicksort.
Ideea este sa nu o ducem sortarea pana la capat si doar pana cand pivotul va fi pe pozitia k.
Astfel vom stii ca toate numerele de pe pozitiile 1...k sunt mai mici decat pivotul, deci pivotul este al k-lea cel mai mic numar.
Raspunsul lor:1p - n (folosind functia de partitionare de la qsort)
0.5p - n log k

4.

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Raspunsul meu: Pentru ca celelate fortarete din multime sa comunice in continuare, fortareata bombardata trebuie sa NU costituie un punct critic in graful format doar din nodurile din aceea multime.
Prin urmare, enuntul este similiar cu gasirea tutoror componentelor biconexe din graf.
Acest lucru poate fi realizat cu algoritmul lui Tarjan
Raspunsul lor: Gasim toate componentele biconexe.

5.

Cum puteți **afișa toate** drumurile distincte între 2 noduri dintr-un graf **orientat aciclic**?

R: Există un număr exponențial de drumuri, deci ar trebui calculate cu backtracking.

6.

Cum puteți **să calculați** numărul total de drumuri distincte între 2 noduri dintr-un graf **orientat aciclic**?

Raspunsul meu: Sortare topologica pe DAG.
Dupa care calculam raspunusul folosind o dinamica.

Raspunsul lor: Sortare topologica apoi dinamică pentru a calcula numarul de drumuri pana la un anumit nod.

1. Fie un graf G neponderat pe care se aplica un BFS pornind din nodul S. Notam cu d(S,u) si d(S,v) lungimile celor mai scurte cai gasite de BFS. Daca u este descoperit de BFS inainte de v, care din urmatoarele afirmatii este adevarata?

Alegeți o opțiune:

- a. d(S,u) >= d(S,v)
- b. d(S,u) < d(S,v)
- c. d(S,u) <= d(S,v)
- d. d(S,u) > d(S,v)

Feedback

Răspunsul corect este: d(S,u) <= d(S,v)

2. Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care dintre următoarele secvențe de pași sunt complete și plauzibil a fi aplicate de algoritmul AC1 ?

Alegeți o opțiune:

- a. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)
- b. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B)
- c. REVISE(A, B), REVISE(B, C)
- d. REVISE(A, B), REVISE(B, C), REVISE(A, B), REVISE(B, C)

Feedback

Răspunsul corect este: REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)

3. Care din următoarele funcții de tip euristică NU este admisibilă pentru algoritmul A* care calculează ruta de distanță minimă pe calea ferată între două orașe, o sursă și o destinație? Pentru toate funcțiile euristică de mai jos, valoarea euristicii pentru destinație este egală cu 0, valorea fiind calculată doar pentru restul nodurilor.

Alegeți una sau mai multe opțiuni:

- [ ] a. o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație
- [ ] b. o valoare constantă pentru fiecare nod, egală cu valoarea muchiei de cost minim din graf (distanța minimă pe cale ferată între cele mai apropiate două orașe)
- [x] c. lungimea în linie dreaptă de la fiecare nod la destinație
- [x] d. lungimea rutei deja parcurse până la fiecare nod

Feedback

The correct answers are: lungimea rutei deja parcurse până la fiecare nod, o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație

4. Pentru care dintre următoarele distribuții de probabilitate de apariție a cheilor vom avea cel puțin doi arbori Huffman distincți?

Alegeți una sau mai multe opțiuni:

- [x] a. 1/16, 1/16, 1/8, 1/4, 1/2
- [x] b. 1/16, 1/16, 1/8, 1/8, 1/4, 3/8
- [ ] c. 1/20, 1/10, 1/10, 3/20, 7/20, 1/4
- [ ] d. 1/4, 1/4, 1/4, 1/4

Feedback

The correct answers are: 1/16, 1/16, 1/8, 1/8, 1/4, 3/8, 1/4, 1/4, 1/4, 1/4, 1/20, 1/10, 1/10, 3/20, 7/20, 1/4

5. Dându-se un vector de n numere sortat crescător, fie t(n) complexitatea celui mai eficient algoritm care verifică dacă există 2 elemente în vector a căror sumă să fie mai mică de 100. Care din afirmațiile de mai jos este adevărată?

Alegeți o opțiune:

- a. n * log(n) <= t(n) < n/2
- b. t(n) apartine O(1)
- c. t(n) = n/2
- d. n <= t(n) <= n * log(n)

Feedback

Răspunsul corect este: t(n) apartine O(1)

6. Într-o sesiune de examene ce durează n zile trebuie date m examene (m<n). Profesorii materiilor respective impun condiționări legate de momentul maxim de timp până la care pot fi date examenele. Cum poate fi rezolvată cel mai eficient problema planificării unui număr maxim posibil de examene?

Alegeți o opțiune:

- a. folosind algoritmi aleatori, se generează diverse configurații până când se nimerește una validă
- b. folosind greedy, se sortează crescător condițiile impuse de profesori și se stabilesc zilele de examen astfel încât să se respecte condițiile
- c. folosind backtracking, se generează toate variantele și se alege cea care respectă cât mai multe condiționări
- d. folosind programare dinamică, se generează un tablou care să rețină toate examenele ce pot fi date până în ziua i și apoi se verifică câte examene se pot da până în ziua n

Feedback

Răspunsul corect este: folosind greedy, se sortează crescător condițiile impuse de profesori și se stabilesc zilele de examen astfel încât să se respecte condițiile

7. Un algoritm A* (cu o euristică admisibilă) este întotdeauna:

Alegeți o opțiune:

- a. nici complet nici optimal
- b. optimal
- c. complet
- d. complet și optimal

Feedback

Răspunsul corect este: complet și optimal

8. Intr-un graf G neorientat se aplica algoritmul DFS si se obtine o padure de arbori DFS T. Fie u un nod din G si v primul nod alb vizitat de DFS dupa u. Care dintre afirmatiile de mai jos este intotdeauna corecta?

Alegeți o opțiune:

- a. Daca (u, v) nu este muchie in G, atunci u si v au acelasi parinte in T
- b. (u, v) este muchie in G si u este descendent al lui v in T
- c. Daca (u, v) nu este muchie in G, atunci u este frunza in T
- d. (u, v) este muchie in G si v este descendent al lui u in T

Feedback

Răspunsul corect este: Daca (u, v) nu este muchie in G, atunci u este frunza in T

9. Care este numărul maxim posibil de arbori de parcurgere în adâncime a unui graf orientat conex (dacă considerăm graful neorientat, dar nu obligatoriu tare conex) cu n noduri?

Alegeți o opțiune:

- a. n/2
- b. n
- c. n-1
- d. 1

Feedback

Răspunsul corect este: n

10. Dandu-se un graf complet ponderat cu nodurile {v_1, v_2 ... v_n} astfel incat muchia (v_i, v_j) sa aiba costul 2 * |i-j|, calculati costul AMA al grafului.

Alegeți o opțiune:

- a. n - 1
- b. 2 * n - 2
- c. n / 2
- d. n ^ 2

Feedback

Răspunsul corect este: 2 * n - 2

11. În ce situații este preferabil să folosiți algoritmul Mergesort față de Quicksort care alege întotdeauna primul element ca pivot?

Alegeți o opțiune:

- a. Când vectorul este (aproape) sortat crescător sau descrescător
- b. Întotdeauna deoarece Mergesort este mai rapid ca Quicksort
- c. Când vectorul de sortat conține mai multe duplicate
- d. Niciodată, deoarece Quicksort este întotdeauna mai rapid ca Mergesort

Feedback

Răspunsul corect este: Când vectorul este (aproape) sortat crescător sau descrescător

12. Care e avantajul principal al tăieturilor alfa-beta față de Minimax clasic?

Alegeți o opțiune:

- a. Este mai ușor de implementat
- b. În general, returnează rezultate mai rapid
- c. Returnează rezultate mai bune
- d. Nu oferă niciun avantaj

Feedback

Răspunsul corect este: În general, returnează rezultate mai rapid

13. Dacă se folosește MergeSort pentru a sorta crescător vectorul [18, 52, 14, 8, 9, 2, 40, 25, 12, 17], atunci înaintea ultimei interclasări efectuate de către algoritm pentru obținerea vectorului sortat, ordinea elementelor este:

Alegeți o opțiune:

- a. 2, 8, 9, 14, 18, 52, 12, 17, 25, 40
- b. 14, 18, 52, 2, 8, 9, 12, 25, 40, 17
- c. 8, 9, 14, 18, 52, 2, 12, 17, 25, 40
- d. 8, 14, 18, 52, 2, 9, 25, 40, 12, 17

Feedback

Răspunsul corect este: 8, 9, 14, 18, 52, 2, 12, 17, 25, 40

14. Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care sunt domeniile variabilelor după aplicarea algoritmului AC1 ?

Alegeți o opțiune:

- a. A = {1, 2}, B = {5, 6}, C = {5, 6}
- b. A = {1, 2, 3, 4, 5}, B = {5, 6, 7}, C = {5, 6, 7}
- c. A = {1, 2}, B = {5, 6, 7}, C = {5, 6, 7}
- d. A = {2}, B = {5}, C = {5}

Feedback

Răspunsul corect este: A = {1, 2}, B = {5, 6}, C = {5, 6}

Subiectul 3

Un fotograf profesionist are *n* fotografii din *x* orașe. El dorește să genereze toate modurile posibile de combinații poze-orașe. Descrieți un algoritm pentru această problemă.

Rez mea: O prima solutie ce imi vine in minte este genererarea cu ajutorul unui algoritm backtracking a tuturor combinatiilor poze-orase. Mai exact problema este similara cu generarea tuturor combinarilor.

Rez lor:

Se foloseste backtracking pentru a genera toate perechile flori-orase care respecta

Cum se poate implementa eficient un algoritm de programare dinamică având recurența:

$$dp[n] = \begin{cases} \text{dp[n-2]} - \text{dp[n-4]}, & n \geq 4 \\ 2 * n, & n < 4 \end{cases}$$

Rez mea:

Prima idee de optimizare care imi vine in minte este exponentierea rapida pe matrici. Avem cazurile de baza:
dp[0] =0
dp[1] = 2
dp[2] = 4
dp[3] = 6
Iar apoi avem recurenta liniara dp[n] = dp[n-2] - dp[n-4]

Se construieste matricea C cu Ck-2 = 2 si Ck-4 = 4
Se foloseste ridicarea la putere in timp logaritmic si se obtine complexitate Theta(KMAX ^3 * logn), dar KMAX este 2 in cazul nostru deci Theta(logn).

Rez lor:

Exponențiere logaritmică pe matrice

Se dă un vector *v* cu *n* intervale de forma $[a_i, b_i]$. Se cere numărul maxim de intervale care se intersectează. Descrieți un algoritm eficient.

Rez mea:

Prima idee ar fi sa folsesc un algoritm de tip greedy. Pastrez intr-un vector in care tin perechie mapate, a-urile marcate cu 0, b-urile marcate cu 1.

Sortez vectorul crescator, parcurg vectorul, daca intalnesc un 0, incrementez un contor, iar daca intelesc un 1 il decrementez pentru ca am gasit un inveral ce se suprapune. Nu imi ramane decat in fiecare iteratie sa calculez un maxim dintre contor si result, unde result e initializat cu 0.

Rez lor:

Greedy: se sortează, apoi se parcurg intervalele de la stânga la dreapta. Se pornește cu un interval și cât timp se intersectează cu următorul, se aplică operația de intersecție. O(n log n).

În orașul X, rețeaua de internet este răspândita în toate cartiere, în fiecare cartier fiind amplasate mai multe routere interconectate între ele. Unele routere cedează, iar internetul pică în unul sau mai multe cartiere. Ce algoritm ați folosi pentru identificarea routerelor care duc la astfel de căderi și ce soluție sugerați pentru scăderea numărului de incidente?

Rez mea(Asta a fost de juma' de punctaj):

Ma gandesc sa folosesc Algoritmul lui Tarjan pentru a determina punctele de articulatie, iar solutia pentru scaderea numarului incidente ar fi sa elimin punctele critice.

Rez lor:

Găsim toate punctele de articulație. O soluție ar fi adăugarea de muchii redundante intre cartiere.

Se dă un graf orientat cu *m* noduri, având capacitați numere pare pe arce. Se dau o sursă *S* și *p* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la sursă la toate destinațiile.

Rez mea:

Adaug 2 noduri S'(o sursa unica) si P'(destinatie unica) cu capacitatii infinite iar apoi folosesc algoritmul Edmons-Karp pentru a determina fluxul maxim.

Rez lor:

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mari numere. Secvența este prea mare pentru a fi stocată integral in memorie.

Rez mea:

Avand in vedere ca K << N voi folosi o coada de prioritati, un minheap mai exact. Introduc in coada de prioritati primele K elemente din secventa.

Imi raman N - K elemente. Parcurg aceste elemente si daca vreunul este mai mare decat varful cozii, atunci elimin varful si introduc acest nod.

O alternativa ar fi fost bucket sort, doar ca problema nu ne permite stocarea secventei in memoriei

Rez lor:

n log k- folosim un minheap

n * k - 0.5 din punctaj

Considerand arborele obținut în urma aplicării unui BFS din nodul S într-un graf neorientat conex fără ponderi, ce se poate spune despre acest arbore?

Select one:

a. Calculeaza cele mai scurte drumuri de la S la toate nodurile din graf

b. Calculeaza cele mai scurte drumuri de la S doar pană la frunzele din arbore

c. Calculeaza cele mai lungi drumuri de la S la toate nodurile din graf

d. Calculează cele mai scurte drumuri între oricare două noduri din graf

Feedback

The correct answer is: Calculeaza cele mai scurte drumuri de la S la toate nodurile din graf

Question 2

Incorrect

Mark 0.00 out of 1.00

Not flaggedFlag question

Question text

Se da un graf neorientat cu n noduri {v_1, v_2 ... v_n}, n > 7, in care doua noduri v_i si v_j sunt conectate daca si numai daca 0 < |i-j| <= 2, iar in acest caz ponderea muchiei este i + j. Care este costul caii de la nodul v_5 la v_6 in AMA-ul acestui graf? Un exemplu de graf cu n=4 este prezentat in figura de mai jos:

Select one:

a. 41

b. 25

c. 11

d. 31

Feedback

The correct answer is: 31

Question 3

Incorrect

Mark 0.00 out of 1.00

Not flaggedFlag question

Question text

Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care sunt domeniile variabilelor după aplicarea algoritmului AC1 ?

Select one:

a. A = {2}, B = {5}, C = {5}

b. A = {1, 2}, B = {5, 6, 7}, C = {5, 6, 7}

c. A = {1, 2, 3, 4, 5}, B = {5, 6, 7}, C = {5, 6, 7}

d. A = {1, 2}, B = {5, 6}, C = {5, 6}

Feedback

The correct answer is: A = {1, 2}, B = {5, 6}, C = {5, 6}

Question 4

Correct

Mark 1.00 out of 1.00

Not flaggedFlag question

Question text

Un algoritm BF* (BEST FIRST) este întotdeauna:

Select one:

a. complet

b. optimal

c. complet și optimal

d. nici complet nici optimal

Feedback

The correct answer is: complet

Question 5

Correct

Mark 1.00 out of 1.00

Not flaggedFlag question

Question text

Cum poate fi modificat algoritmul de tip Las Vegas pentru detecția liniilor duplicate dintr-un fișier text discutat la curs într-un algoritm de tip Monte Carlo?

Select one:

a. Algoritmii Las Vegas nu pot fi transformați în algoritmi Monte Carlo niciodată

b. Reținem liniile alese până la momentul curent într-un hash și verificăm dacă noile linii sunt egale cu oricare din cele din hash

c. Se poate adăuga o limită de pași pentru a găsi linia duplicat, dacă nu o găsim în după limita de pași, spunem că nu există

d. Deși unii algoritmi de tip Las Vegas pot fi transformați în algoritmi de tip Monte Carlo, cel pentru detecția liniilor duplicate nu prezintă această proprietate

Feedback

The correct answer is: Se poate adăuga o limită de pași pentru a găsi linia duplicat, dacă nu o găsim în după limita de pași, spunem că nu există

Question 6

Partially correct

Mark 0.50 out of 1.00

Not flaggedFlag question

Question text

Care dintre următoarele afirmații este adevărată pentru construcția unui arbore Huffman cu n chei?

Select one or more:

a. Implementarea eficientă necesită utilizarea unui heap binar

b. Complexitatea este Θ(n)

c. Complexitatea este Θ(n * logn)

d. Complexitatea este Θ(n^2)

Feedback

The correct answers are: Complexitatea este Θ(n * logn), Implementarea eficientă necesită utilizarea unui heap binar

Question 7

Correct

Mark 1.00 out of 1.00

Not flaggedFlag question

Question text

Dandu-se graful de mai jos, care ordine de alegere a muchiilor nu se poate obtine folosind algoritmul lui Kruskal?

Select one:

a. (a-b), (d-f), (b-f), (d-c), (d-e)

b. (d-f), (a-b), (b-f), (d-e), (d-c)

c. (d-f), (a-b), (d-c), (b-f), (d-e)

d. (a-b), (d-f), (d-c), (b-f), (d-e)

Feedback

The correct answer is: (d-f), (a-b), (b-f), (d-e), (d-c)

Question 8

Correct

Mark 1.00 out of 1.00

Not flaggedFlag question

Question text

Dându-se o mulțime de numere {n_1, n_2...n_k} și o valoare naturală S, cum putem determina eficient dacă există o submulțime de valori astfel încât suma lor să fie S?

Select one:

a. folosim divide et impera pentru a înjumătăți suma ce trebuie găsită și numerele ce pot fi folosite pentru a ajunge la acea sumă

b. folosim programare dinamică și un tablou bidimensional pentru a memora pentru fiecare număr natural <= S dacă avem posibilitatea să obținem acea sumă folosind numerele din mulțime până la indicele i

c. se sortează mulțimea și apoi folosim un algoritm greedy în care adăugăm elementele cele mai mici din mulțime până când fie picăm pe valoarea corectă, fie depășim și atunci verificăm care obiect poate fi scos pentru a ajunge la S

d. folosim un algoritm de tip backtracking în care adăugăm obiecte până când fie ajungem la S, fie depășim această valoare

Feedback

The correct answer is: folosim programare dinamică și un tablou bidimensional pentru a memora pentru fiecare număr natural <= S dacă avem posibilitatea să obținem acea sumă folosind numerele din mulțime până la indicele i

Question 9

Partially correct

Mark 0.33 out of 1.00

Not flaggedFlag question

Question text

Pentru care dintre următoarele distribuții de probabilitate de apariție a cheilor vom avea cel puțin doi arbori Huffman distincți?

Select one or more:

a. 1/16, 1/16, 1/8, 1/8, 1/4, 3/8

b. 1/16, 1/16, 1/8, 1/4, 1/2

c. 1/20, 1/10, 1/10, 3/20, 7/20, 1/4

d. 1/4, 1/4, 1/4, 1/4

Feedback

The correct answers are: 1/16, 1/16, 1/8, 1/8, 1/4, 3/8, 1/4, 1/4, 1/4, 1/4, 1/20, 1/10, 1/10, 3/20, 7/20, 1/4

Question 10

Correct

Mark 1.00 out of 1.00

Not flaggedFlag question

Question text

Cum ați modifica algoritmul de programare dinamică pentru problema parantezării matricilor pentru a calcula toate parantezările optimale, dacă există mai multe, și nu doar una singură?

Select one:

a. Pentru fiecare subproblemă A_ij, trebuie să reținem un vector de poziții unde se poate pune optim o paranteză, în loc de o singură poziție

b. Acest lucru este imposibil de realizat plecând de la algoritmul prezentat la curs

c. Pentru fiecare subproblemă A_ij, trebuie să adăugăm o buclă suplimentară cu k de la i+1 la j, ceea ce va crește complexitatea algoritmului

d. Acest lucru este imposibil de realizat folosind programarea dinamică, trebuie rezolvat cu un algoritm de tip backtracking

Feedback

The correct answer is: Pentru fiecare subproblemă A_ij, trebuie să reținem un vector de poziții unde se poate pune optim o paranteză, în loc de o singură poziție

Question 11

Incorrect

Mark 0.00 out of 1.00

Not flaggedFlag question

Question text

Pentru arborele minimax din figura următoare, menționați care sunt tăieturile alfa-beta dacă parcurgem nodurile de la dreapta spre stânga (în oglindă față de o parcurgere normală):

Graf minimax

Select one:

a. Sunt trei tăieturi: prima elimină frunza cu valoarea 0, a doua frunza cu valoarea 7, a treia frunza cu valoarea 3

b. Niciuna dintre celelalte variante nu este corectă

c. Sunt două tăieturi: prima elimină frunza cu valoarea 0, a doua frunza cu valoarea 3

d. Este o singură tăietură care elimină frunza cu valoarea 0

Feedback

The correct answer is: Este o singură tăietură care elimină frunza cu valoarea 0

Question 12

Incorrect

Mark 0.00 out of 1.00

Not flaggedFlag question

Question text

Algoritmii greedy și de programare dinamică se aseamănă prin faptul că amândoi:

Select one:

a. se pot bloca în maxime sau minime locale

b. necesită o matrice pentru memorarea sub-soluțiilor

c. folosesc un criteriu de optim

d. au în general o complexitate exponențială

Feedback

The correct answer is: folosesc un criteriu de optim

Question 13

Incorrect

Mark 0.00 out of 1.00

Not flaggedFlag question

Question text

Intr-un graf G neorientat se aplica algoritmul DFS si se obtine o padure de arbori DFS T. Fie u un nod din G si v primul nod alb vizitat de DFS dupa u. Care dintre afirmatiile de mai jos este intotdeauna corecta?

Select one:

a. Daca (u, v) nu este muchie in G, atunci u este frunza in T

b. (u, v) este muchie in G si u este descendent al lui v in T

c. (u, v) este muchie in G si v este descendent al lui u in T

d. Daca (u, v) nu este muchie in G, atunci u si v au acelasi parinte in T

Feedback

The correct answer is: Daca (u, v) nu este muchie in G, atunci u este frunza in T

Question 14

Correct

Mark 1.00 out of 1.00

Not flaggedFlag question

Question text

Fie un graf G neponderat pe care se aplica un BFS pornind din nodul S. Notam cu d(S,u) si d(S,v) lungimile celor mai scurte cai gasite de BFS. Daca u este descoperit de BFS inainte de v, care din urmatoarele afirmatii este adevarata?

Select one:

a. d(S,u) > d(S,v)

b. d(S,u) < d(S,v)

c. d(S,u) >= d(S,v)

d. d(S,u) <= d(S,v)

Feedback

The correct answer is: d(S,u) <= d(S,v)

Question 15

Correct

Mark 1.00 out of 1.00

Not flaggedFlag question

Question text

Ce tip de relații matematice se folosesc pentru modelarea restricțiilor în scopul eficientizării rezolvării problemei Sudoku prin backtracking?

Select one:

a. Incluziune

b. Inegalitate

c. Sumă inegală

d. Sume egale

Feedback

The correct answer is: Inegalitate

Care din următoarele funcții de tip euristică NU este admisibilă pentru algoritmul A* care calculează ruta de distanță minimă pe calea ferată între două orașe, o sursă și o destinație? Pentru toate funcțiile euristică de mai jos, valoarea euristicii pentru destinație este egală cu 0, valorea fiind calculată doar pentru restul nodurilor.

Alegeți una sau mai multe opțiuni:

- a. lungimea rutei deja parcurse până la fiecare nod
- b. o valoare constantă pentru fiecare nod, egală cu valoarea muchiei de cost minim din graf (distanța minimă pe cale ferată între cele mai apropiate două orașe)
- c. o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație
- d. lungimea în linie dreaptă de la fiecare nod la destinație

Feedback

The correct answers are: lungimea rutei deja parcurse până la fiecare nod, o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație

2 întrebare

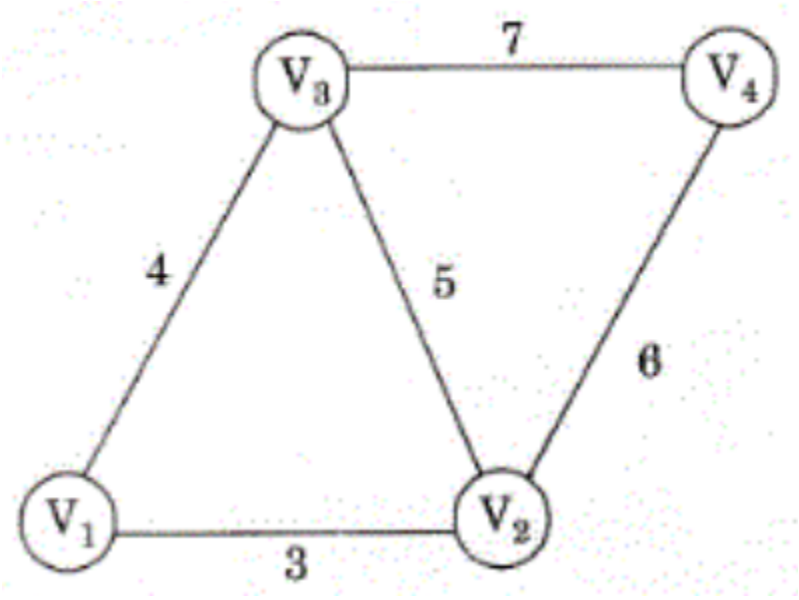
Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Se da un graf neorientat cu n noduri {v_1, v_2 ... v_n}, n > 7, in care doua noduri v_i si v_j sunt conectate daca si numai daca 0 < |i-j| <= 2, iar in acest caz ponderea muchiei este i + j. Care este costul caii de la nodul v_5 la v_6 in AMA-ul acestui graf? Un exemplu de graf cu n=4 este prezentat in figura de mai jos:



Alegeți o opțiune:

a. 25

b. 41

c. 11

d. 31

Feedback

Răspunsul corect este: 31

3 întrebare

Parțial corect

Marcat 0,50 din 1,00

Întrebare cu flag

Textul întrebării

Este de preferat să se rezolve problema comis-voiajorului (dându-se o listă cu n locații și m străzi cu lungimi precizate, un comis-voiajor are misiunea de a livra colete la toate aceste locații, întorcându-

se la final de unde a plecat, trecând prin fiecare locație doar o dată și obținând un drum de lungime minimă):

Alegeți una sau mai multe opțiuni:

- [x] a. folosind backtracking
- [ ] b. folosind programare dinamică, deși soluția rămâne exponențială
- [ ] c. folosind greedy, pentru că este singura metodă prin care se poate rezolva
- [ ] d. folosind programare dinamică, pentru că este singura metodă prin care se poate rezolva

Feedback

The correct answers are: folosind backtracking, folosind programare dinamică, deși soluția rămâne exponențială

4 întrebare

Nu a primit răspuns

Marcat din 1,00

Întrebare cu flag

Textul întrebării

Intr-un graf G neorientat se aplica algoritmul DFS si se obtine o padure de arbori DFS T. Fie u un nod din G si v primul nod alb vizitat de DFS dupa u. Care dintre afirmatiile de mai jos este intotdeauna corecta?

Alegeți o opțiune:

- a. Daca (u, v) nu este muchie in G, atunci u si v au acelasi parinte in T
- b. Daca (u, v) nu este muchie in G, atunci u este frunza in T
- c. (u, v) este muchie in G si v este descendent al lui u in T
- d. (u, v) este muchie in G si u este descendent al lui v in T

Feedback

Răspunsul corect este: Daca (u, v) nu este muchie in G, atunci u este frunza in T

5 întrebare

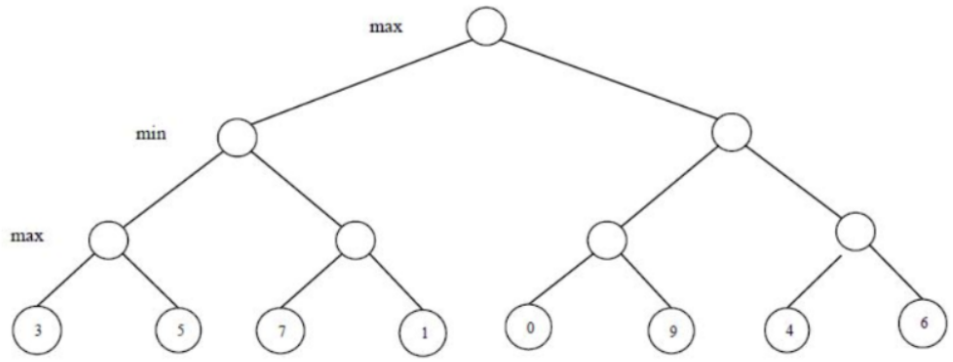
Nu a primit răspuns

Marcat din 1,00

Întrebare cu flag

Textul întrebării

Pentru arborele minimax din figura următoare, menționați care sunt tăieturile alfa-beta dacă parcurgem nodurile de la dreapta spre stânga (în oglindă față de o parcurgere normală):



Alegeți o opțiune:

a. Este o singură tăietură care elimină frunza cu valoarea 0

b. Sunt trei tăieturi: prima elimină frunza cu valoarea 0, a doua frunza cu valoarea 7, a treia frunza cu valoarea 3

c. Sunt două tăieturi: prima elimină frunza cu valoarea 0, a doua frunza cu valoarea 3

d. Niciuna dintre celelalte variante nu este corectă

Feedback

Răspunsul corect este: Este o singură tăietură care elimină frunza cu valoarea 0

6 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Dându-se un test grilă care trebuie rezolvat în 10 minute, ce conține 4 întrebări care au punctajele 56, 35, 24 și 27 și care necesită un timp de rezolvare de 8, 7, 4, 3 minute se cere să se identifice punctajul maxim ce poate fi obținut, știind că se dau punctaje parțiale pentru rezolvarea întrebărilor presupunând o rezolvare a întrebărilor uniformă în timp (ex: pentru prima întrebare se vor acorda 56/8 puncte pentru fiecare minut alocat).

Alegeți o opțiune:

a. 76

b. 68

c. 56

d. 62

Feedback

Răspunsul corect este: 76

7 întrebare

Incorect

Marcat 0,00 din 1,00

Întrebare cu flag

Textul întrebării

Fie G un graf neorientat conex avand ponderi diferite pe muchii (fiecare muchie are alt cost). Fie e_max si e_min muchiile de cost maxim respectiv minim. Care dintre urmatoarele afirmatii este FALSA?

Alegeți o opțiune:

a. Niciun AMA nu contine e_max

b. G are un AMA unic

c. Toti AMA trebuie sa contina e_min

d. Daca e_max apartine AMA, stergerea lui conduce la pierderea conectivitatii AMA-ului

Feedback

Răspunsul corect este: Niciun AMA nu contine e_max

8 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Care din algoritmii următori nu este de tip Greedy?

Alegeți o opțiune:

a. Prim

b. Bellman-Ford

c. Kruskal

d. Dijkstra

Feedback

Răspunsul corect este: Bellman-Ford

9 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

O condiție ca un algoritm A* a1 să domine alt algoritm A* a2 este ca:

Alegeți o opțiune:

- a. a1 să aibă o complexitate mai mică decât a2
- b. o euristică monotonă pentru a1 este mai informată decât o euristică monotonă pentru a2
- c. a1 să fie complet iar a2 nu
- d. o euristică monotonă pentru a2 este mai informată decât o euristică monotonă pentru a1

Feedback

Răspunsul corect este: o euristică monotonă pentru a1 este mai informată decât o euristică monotonă pentru a2

10 întrebare

Corect

Marcat 1,00 din 1,00

Întrebare cu flag

Textul întrebării

Se dă un graf G cu n noduri reprezentat prin matricea de adiacență care are 0 pe diagonala principala si 1 in rest. Pentru acest graf:

Alegeți o opțiune:

- a. Nu se poate construi un AMA al grafului G
- b. Graful G are mai mulți AMA distincți, cu costuri diferite
- c. Graful G are mai mulți AMA distincți, toți de cost n-1

d. Graful G are un singur AMA de cost n-1

Feedback

Răspunsul corect este: Graful G are mai mulți AMA distincți, toți de cost n-1

11 întrebare

Nu a primit răspuns

Marcat din 1,00

Întrebare cu flag

Textul întrebării

Dându-se o mulțime de numere {n_1, n_2 ... n_k} și o valoare naturală S se poate detecta dacă există o submulțime de valori astfel încât suma lor să fie S folosind un algoritm de programare dinamică. Astfel, se folosește un tablou bi-dimensional dp[i, j] cu 1 <= i <= n, 0 <= j <= S care are valoarea Adevărat dacă există un un subset al {n_1, n_2 ... n_i} care să aibă suma j. Care este relația de recurență pentru calculul lui dp[i, j]?

Alegeți o opțiune:

a. dp[i, j] = dp[i - 1, j] AND dp[i, j - n_i]

b. dp[i, j] = dp[i - 1, j] AND dp[i - 1, j - n_i]

c. dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]

d. dp[i, j] = dp[i - 1, j] OR dp[i, j - n_i]

Feedback

Răspunsul corect este: dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]

12 întrebare

Nu a primit răspuns

Marcat din 1,00

Întrebare cu flag

Textul întrebării

Cum se modifică complexitatea și timpul de execuție pentru algoritmul de sortare prin interclasare dacă la fiecare pas impar împarte vectorul de dimensiune n în doi subvectori de dimensiune 1, respectiv n-1, iar la fiecare pas par împarte vectorul în doi subvectori de dimensiune egală:

Alegeți una sau mai multe opțiuni:

- a. Ordinul de creștere va fi mai mic
- b. Ordinul de creștere va fi mai mare
- c. Ordinul de creștere rămâne neschimbat, Θ(n * logn)
- d. Timpul de execuție crește de 2 ori

Feedback

The correct answers are: Ordinul de creștere rămâne neschimbat, Θ(n * logn), Timpul de execuție crește de 2 ori

13 întrebare

Nu a primit răspuns

Marcat din 1,00

Întrebare cu flag

Textul întrebării

O diferență între algoritmii divide and impera și de programare dinamică este că:

Alegeți o opțiune:

- a. La programarea dinamică nu este nevoie să se păstreze soluțiile parțiale
- b. În ambii este nevoie de un criteriu de optim

c. Primii au în general o complexitate computațională mai mare

d. Primii sunt folosiți de regulă când subsoluțiile se folosesc doar o dată în calcul

Feedback

Răspunsul corect este: Primii sunt folosiți de regulă când subsoluțiile se folosesc doar o dată în calcul

14 întrebare

Nu a primit răspuns

Marcat din 1,00

Întrebare cu flag

Textul întrebării

Fie un graf G neponderat pe care se aplica un BFS pornind din nodul S. Notam cu d(S,u) si d(S,v) lungimile celor mai scurte cai gasite de BFS. Daca u este descoperit de BFS inainte de v, care din urmatoarele afirmatii este adevarata?

Alegeți o opțiune:

a. d(S,u) >= d(S,v)

b. d(S,u) <= d(S,v)

c. d(S,u) > d(S,v)

d. d(S,u) < d(S,v)

Feedback

Răspunsul corect este: d(S,u) <= d(S,v)

15 întrebare

Nu a primit răspuns

Marcat din 1,00

Întrebare cu flag

Textul întrebării

Care din următoarele probleme poate fi rezolvată eficient prin backtracking?

Alegeți o opțiune:

a. Găsirea celor mai scurte drumuri în grafuri

b. Jocul de table

c. Găsirea celor mai utile produse de cumpărat în condițiile unui buget limitat

d. Rebus

Feedback

Răspunsul corect este: Rebus

Started on	Wednesday, 3 June 2020, 12:10 PM
State	Finished
Completed on	Wednesday, 3 June 2020, 12:40 PM
Time taken	29 mins 39 secs
Marks	10.00/15.00
Grade	**13.33** out of 20.00 (**67**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Intr-un graf G orientat pe care se aplica algoritmul DFS, fie u un nod din G si v un descendent al lui u de culoare neagra gasit de DFS. Care dintre afirmatiile de mai jos este corecta?

Select one:

- a. (u, v) este arc invers in G
- b. nu se poate intalni o asemenea situatie intr-un graf orientat
- c. (u, v) este arc direct in G
- d. (u, v) este arc inainte sau transversal in G, depinzand de timpii de descoperire ai lui u si v ✔

The correct answer is: (u, v) este arc inainte sau transversal in G, depinzand de timpii de descoperire ai lui u si v

Question **2**

Correct

Mark 1.00 out of 1.00

Dacă se folosește MergeSort pentru a sorta crescător vectorul [18, 52, 14, 8, 9, 2, 40, 25, 12, 17], atunci înaintea ultimei interclasări efectuate de către algoritm pentru obținerea vectorului sortat, ordinea elementelor este:

Select one:

- a. 2, 8, 9, 14, 18, 52, 12, 17, 25, 40
- b. 8, 9, 14, 18, 52, 2, 12, 17, 25, 40 ✔
- c. 14, 18, 52, 2, 8, 9, 12, 25, 40, 17
- d. 8, 14, 18, 52, 2, 9, 25, 40, 12, 17

The correct answer is: 8, 9, 14, 18, 52, 2, 12, 17, 25, 40

Question **3**

Incorrect

Mark 0.00 out of 1.00

Dându-se o mulțime de numere {n_1, n_2 ... n_k} și o valoare naturală S se poate detecta dacă există o submulțime de valori astfel încât suma lor să fie S folosind un algoritm de programare dinamică. Astfel, se folosește un tablou bi-dimensional dp[i, j] cu 1 <= i <= n, 0 <= j <= S care are valoarea Adevărat dacă există un un subset al {n_1, n_2 ... n_i} care să aibă suma j. Care este relația de recurență pentru calculul lui dp[i, j]?

Select one:

- a. dp[i, j] = dp[i - 1, j] AND dp[i, j - n_i]
- b. dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]
- c. dp[i, j] = dp[i - 1, j] AND dp[i - 1, j - n_i] ✖
- d. dp[i, j] = dp[i - 1, j] OR dp[i, j - n_i]

The correct answer is: dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]

Question **4**

Incorrect

Mark 0.00 out of 1.00

Ce puteti spune daca matricea rezultata in urma aplicarii algoritmului lui Johnson contine valori negative pe diagonala principala?

Select one:

- a. Nu este nicio problema, matricea poate sa contina valori negative
- b. Nodurile corespunzatoare valorilor negative fac parte din cicluri de cost negativ ✖
- c. Acele valori nu conteaza, pentru ca reprezinta distanțe de la un nod la el insusi care nu sunt folosite nicaieri
- d. Exista o eroare in cod

The correct answer is: Exista o eroare in cod

Question **5**

Correct

Mark 1.00 out of 1.00

Care este relația de recurență corespunzătoare problemei determinării celei mai apropiate perechi de puncte într-un plan, folosind o implementare eficientă a algoritmului prezentat la curs?

Select one:

- a. T(n) = 2 * T(n/2) + n ✔
- b. T(n) = 2 * T(n/2) + n * logn
- c. T(n) = 2 * T(n/2) + logn
- d. T(n) = 2 * T(n/2) + n^2

The correct answer is: T(n) = 2 * T(n/2) + n

Question **6**

Incorrect

Mark 0.00 out of 1.00

Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care sunt domeniile variabilelor după aplicarea algoritmului AC1 ?

Select one:

- a. A = {1, 2}, B = {5, 6, 7}, C = {5, 6, 7} ✖
- b. A = {2}, B = {5}, C = {5}
- c. A = {1, 2, 3, 4, 5}, B = {5, 6, 7}, C = {5, 6, 7}
- d. A = {1, 2}, B = {5, 6}, C = {5, 6}

The correct answer is: A = {1, 2}, B = {5, 6}, C = {5, 6}

Question **7**

Correct

Mark 1.00 out of 1.00

Un algoritm BF* (BEST FIRST) este întotdeauna:

Select one:

- a. optimal
- b. complet și optimal
- c. nici complet nici optimal
- d. complet ✔

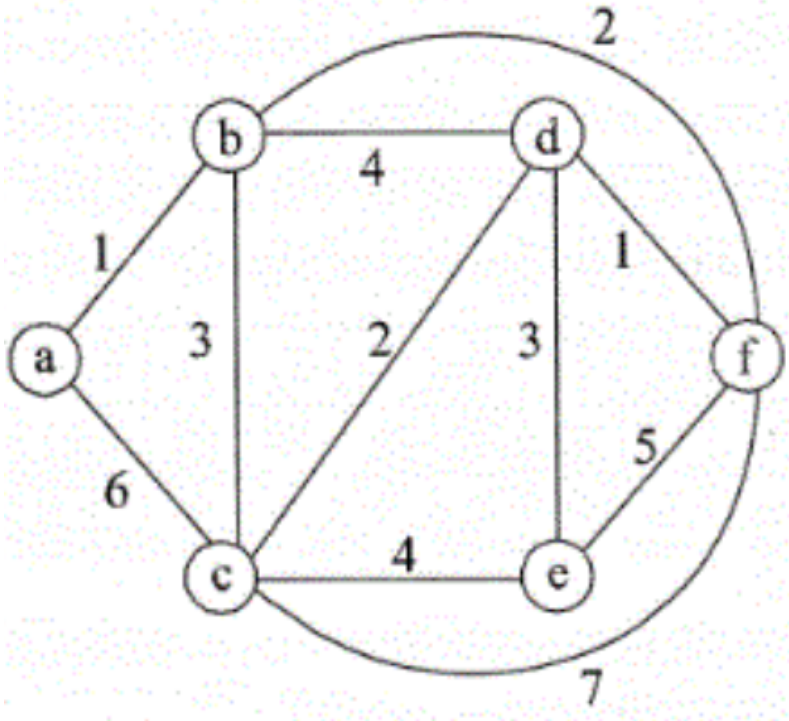
The correct answer is: complet

Question **8**

Correct

Mark 1.00 out of 1.00

Dandu-se graful de mai jos, care ordine de alegere a muchiilor nu se poate obtine folosind algoritmul lui Kruskal?



Select one:

- a. (d-f), (a-b), (b-f), (d-e), (d-c) ✔
- b. (a-b), (d-f), (d-c), (b-f), (d-e)
- c. (a-b), (d-f), (b-f), (d-c), (d-e)
- d. (d-f), (a-b), (d-c), (b-f), (d-e)

The correct answer is: (d-f), (a-b), (b-f), (d-e), (d-c)

Question **9**

Incorrect

Mark 0.00 out of 1.00

Fie G un graf ponderat avand toate ponderile mai mari ca 1 si G' același graf dar la care ponderea fiecarei muchii este patratul ponderii din primul graf. Daca T este AMA pentru G avand costul t si T' este AMA pentru G' avand costul t', care din urmatoarele afirmatii este adevarata?

Select one:

- a. Niciunul din celelalte raspunsuri ✖
- b. T' != T, t' = t ^ 2
- c. T' = T, t' < t ^ 2
- d. T' = T, t' = t ^ 2

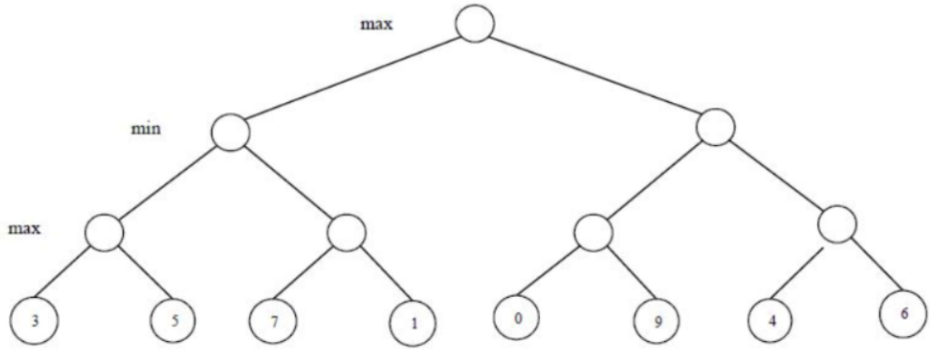
The correct answer is: T' = T, t' < t ^ 2

Question **10**

Incorrect

Mark 0.00 out of 1.00

Pentru arborele minimax din figura următoare, menționați care sunt tăieturile alfa-beta, dacă arborele este parcurs de la stânga spre dreapta:



Select one:

- a. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua subarborele cu frunzele cu valorile 4 și 6
- b. Este o singură tăietură care elimină frunza cu valoarea 1
- c. Niciuna dintre celelalte variante nu este corectă ✖
- d. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua frunza cu valoarea 6

The correct answer is: Este o singură tăietură care elimină frunza cu valoarea 1

Question **11**

Correct

Mark 1.00 out of 1.00

Care din următoarele funcții de tip euristică garantează optimalitatea soluției și expandează mai puține noduri decât celelalte? Pentru toate funcțiile euristice de mai jos, valoarea euristicii pentru destinație este egală cu 0, valorea fiind calculată doar pentru restul nodurilor.

Select one:

- a. o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație
- b. o valoare constantă pentru fiecare nod, egală cu valoarea muchiei de cost minim din graf (distanța minimă pe cale ferată între cele mai apropiate două orașe)
- c. lungimea în linie dreaptă de la fiecare nod la destinație ✔
- d. lungimea rutei deja parcurse până la fiecare nod

The correct answer is: lungimea în linie dreaptă de la fiecare nod la destinație

Question **12**

Correct

Mark 1.00 out of 1.00

Pentru a calcula eficient valoarea lui x^19, avem nevoie să știm cel puțin următoarele numere:

Select one:

- a. Toate numerele de la x^2 la x^19
- b. x^2, x^4, x^9, x^19 ✔
- c. x^2, x^5, x^19
- d. x^2, x^4, x^8, x^16, x^17, x^18, x^19

The correct answer is: x^2, x^4, x^9, x^19

Question **13**

Correct

Mark 1.00 out of 1.00

Pentru a rezolva problema Sudoku, care dintre următoarele afirmații este adevărată?

Select one or more:

- a. Un algoritm de tip propagarea restricțiilor (de ex. AC1) poate rezolva oricare joc de Sudoku
- b. Un algoritm de tip minimax poate rezolva oricare joc de Sudoku
- c. Un algoritm de tip backtracking poate rezolva oricare joc de Sudoku ✔
- d. Un algoritm de tip backtracking, combinat cu propagarea restricțiilor (de ex. AC1) la fiecare k (k >= 1) pași, poate rezolva oricare joc de Sudoku ✔

The correct answers are: Un algoritm de tip backtracking poate rezolva oricare joc de Sudoku, Un algoritm de tip backtracking, combinat cu propagarea restricțiilor (de ex. AC1) la fiecare k (k >= 1) pași, poate rezolva oricare joc de Sudoku

Question **14**

Correct

Mark 1.00 out of 1.00

Cum ați modifica algoritmul de programare dinamică pentru problema parantezării matricilor pentru a calcula toate parantezările optimale, dacă există mai multe, și nu doar una singură?

Select one:

- a. Pentru fiecare subproblemă A_ij, trebuie să adăugăm o buclă suplimentară cu k de la i+1 la j, ceea ce va crește complexitatea algoritmului
- b. Acest lucru este imposibil de realizat plecând de la algoritmul prezentat la curs
- c. Pentru fiecare subproblemă A_ij, trebuie să reținem un vector de poziții unde se poate pune optim o paranteză, în loc de o singură poziție ✔
- d. Acest lucru este imposibil de realizat folosind programarea dinamică, trebuie rezolvat cu un algoritm de tip backtracking

The correct answer is: Pentru fiecare subproblemă A_ij, trebuie să reținem un vector de poziții unde se poate pune optim o paranteză, în loc de o singură poziție

Question **15**

Correct

Mark 1.00 out of 1.00

Dandu-se un graf complet ponderat cu nodurile {v_1, v_2 ... v_n} astfel incat muchia (v_i, v_j) sa aiba costul 2 * |i-j|, calculati costul AMA al grafului.

Select one:

- a. n / 2
- b. n ^ 2
- c. 2 * n - 2 ✔
- d. n - 1

The correct answer is: 2 * n - 2

◄ Barem Subiectul I

Jump to...

Examen PA - Subiect III - 03.06.2020 ►

Started on	Wednesday, 3 June 2020, 12:50 PM
State	Finished
Completed on	Wednesday, 3 June 2020, 1:25 PM
Time taken	35 mins 1 sec
Marks	6.00/6.00
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Complete

Mark 1.00 out of 1.00

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mari numere. Secvența este prea mare pentru a fi stocată integral in memorie.

Ma gandesc sa folosesc o coada de prioritati minima, in care voi baga primele K numere. Apoi de la numarul K + 1 pana la N o sa compar minimul din coada cu elementul curent. Daca minimul din coada este mai mare decat numarul curent, scot minimul din coada si adaug noul numar in coada.

De exemplu, pt sirul 1,2,3,0,-1,5 si k = 2 adaug 1,2 in coada dupa la 3 inlocuiesc minimul cu 3, adica voi avea 2 3 in coada, la 0 dau skip, la -1, skip la 5, inlocuiesc 5 cu 2 in coada si obtin 3 5.

Algoritm de tip greedy

n log k- folosim un minheap

n * k - 0.5 din punctaj

Comment:

Question **2**

Complete

Mark 1.00 out of 1.00

Se dă un vector *v* cu *n* intervale de forma [a_i, b_i]. Se cere numărul maxim de intervale disjuncte (care nu se intersectează cu niciun interval). Descrieți un algoritm eficient.

Abordare de tip greedy

Sortez screscator intervalele [ai, bi] dupa timpul de finalizare in n log n. Iau un contor de intervale disjuncte, initializat la 0. Dupa, imi iau start stop la timpul la primul element din vectorul sortat [a0, b0]. Apoi, parcurg vectorul de intervale si daca intervalul curent este continut in intervalul [start, stop] inseamna ca el este redundant si poate fi omis. Altfel, intervalul intervalul curent nu se intersecteaza cu cel anterior si ar trebui sa cresc contor-ul de intervale disjuncte.

Greedy: se sortează apoi se parcurg intervalele de la stânga la dreapta. O(n log n).

Comment:

Question **3**

Complete

Mark 1.00 out of 1.00

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *n* noduri care au un ciclu de lungime cel puțin 2.

Cred ca as putea folosi o abordare de tip backtracking in care voi genera toate grafurile posibile si asta imi va lua cam O(2^(n^2)) (ma uit in matricea de adiacenta si iau cele n^2 elemente si le fac pe rand 0 sau 1, mai putin pe cele de pe diagonala principala, care vor ramane 0). Dupa, verific in prima faza daca elementul a[i][j] = a[j][i] in matrice, asta insemnand ca muchia este bidirectionala. Daca nu, aplic un kosaraju/tarjan si pot sa iau toate componentele conexe, dupa care verific daca am mai putin de n componente.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunț.

Comment:

Question **4**

Complete

Mark 1.00 out of 1.00

Cum puteți **să calculați** numărul total de drumuri distincte între 2 noduri dintr-un graf **orientat aciclic**?

Ma gandesc ca as putea folosi o sortare topologica a nodurilor, fiind DAG. Dupa as aplica o tehnica de programare dinamica, unde as avea un dp[i] = numarul de noduri prin care se poate ajunge din i la destinatie. Ma gandesc, ca as avea doua for-uri (pentru noduri din sortare, pentru vecinii nodului curent), asa ca ma astept la o complexitate n^2. Mai trebuie sa stabilesc dinamica exacta a problemei.

Sortare topologica apoi dinamică pentru a calcula numarul de drumuri pana la un anumit nod.

Comment:

Question **5**

Complete

Mark 1.00 out of 1.00

Se dă un graf orientat cu *x* noduri, având capacitați numere din șirul Fibonacci. Se știe că există *A* surse și *B* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la toate destinațiile.

Ma gandesc sa unesc cele A surse si cele B drene intr-un nod sursa si un nod drena. Dupa voi putea aplica algoritmul Edmonds-Karp pentru a gasi cantitatea maxima de flux dintre sursa si destinatie.

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comment:

Question **6**

Complete

Mark 1.00 out of 1.00

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Cred ca voi putea aplica algoritmul lui Tarjan pe aceasta problema, pentru a determina componentele biconexe ale fortaretei. Acest lucru se datoreaza faptului ca daca intr-o componenta biconexa, un nod este eliminat, atunci subgraful asociat componentei biconexe ramane conex. Astfel, daca o fortareata e bombardata, celelalte vor comunica in continuare. Tarjan presupune doar o parcurgere dfs, asa ca ma astept la o complexitate de O(N + M). Daca am fortarete izolate, cred ca nu le voi considera componente biconexe separate, intrucat ele nu comunica cu nimeni, dar nici pe cele cu 2 noduri, intrucat acestea vor deveni izolate. Daca am ceva similar cu un lant, atunci nu cred ca e problema, fiecare mai putin capatul stanga si dreapta fac parte dintr-o comp biconex separaa

Gasim toate componentele biconexe.

Comment:

◄ Examen PA - Subiect II - 03.06.2020

Jump to...

Examen Subiectul 1 - Late submit ►

SUBIECTUL III

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *N* noduri pentru care toate gradele de intrare sunt pare.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunt.

Se dă o matrice *n* x *n*. În *k* celule se află ponei (k mult mai mic decât n). Pentru fiecare celulă din matrice, se dorește să se știe care este distanța minimă către cel mai apropiat ponei. Descrieți un algoritm eficient pentru această problemă.

BFS pe matrice în care se pun în coadă toate celulele cu ponei speciali. Se face BFS pornind cu această coadă și se calculează distanțele. Complexitate O(n^2) unde n e dimensiunea matricei. Se punctează cu 0.5p o soluție mai ineficientă.

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze o singură fortăreață, a.î. să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea fortăreței.

1p - Găsim toate punctele de articulatie și componentele biconexe din jurul lor. Gigel va bombarda punctul de articulație care are în jurul său cele mai multe componente biconexe.

0.5p - Întoarce orice punct de articulație

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Se dă un graf neorientat conex reprezentând rețeaua de socializare Facebook (muchie x-y înseamnă *x* este prieten cu *y*). Gigel a făcut o postare pe Wall, care este văzută de prieteni. Se întreabă dacă postarea poate, ca prin mai multe share-uri, să fie văzută de Botzgorel (care nu este prieten cu Gigel). Descrieți un algoritm eficient.

BFS(Gigel) / DFS(Gigel) și dacă vizităm nodul Botzgorel, atunci răspunsul este DA. Altfel NU.

Care este complexitatea maximă a sortării a n numere folosind Quicksort Randomizat?

Alegeți o opțiune:

- a. O(n * log(n))
- b. O(n)
- c. O(n!)
- d. O(n^2)

Feedback

Răspunsul corect este: O(n^2)

Pentru a calcula eficient valoarea lui x^19, avem nevoie să știm cel puțin următoarele numere:

Alegeți o opțiune:

- a. x^2, x^5, x^19
- b. x^2, x^4, x^9, x^19
- c. Toate numerele de la x^2 la x^19
- d. x^2, x^4, x^8, x^16, x^17, x^18, x^19

Feedback

Răspunsul corect este: x^2, x^4, x^9, x^19

Fie G un graf neorientat conex avand ponderi diferite pe muchii (fiecare muchie are alt cost). Fie e_max si e_min muchiile de cost maxim respectiv minim. Care dintre urmatoarele afirmatii este FALSA?

Alegeți o opțiune:

- a. G are un AMA unic
- b. Niciun AMA nu contine e_max
- c. Daca e_max apartine AMA, stergerea lui conduce la pierderea conectivitatii AMA-ului
- d. Toti AMA trebuie sa contina e_min

Feedback

Răspunsul corect este: Niciun AMA nu contine e_max

Un algoritm de tip gradient maxim este întotdeauna:

Alegeți una sau mai multe opțiuni:

- [ ] a. optimal
- [x] b. nici complet nici optimal

- [x] c. mai rapid sau cel puțin la fel de rapid ca A*
- [ ] d. complet și optimal

Feedback

The correct answers are: mai rapid sau cel puțin la fel de rapid ca A*, nici complet nici optimal

Care este numărul maxim posibil de arbori de parcurgere în adâncime a unui graf orientat conex (dacă considerăm graful neorientat, dar nu obligatoriu tare conex) cu n noduri?

Alegeți o opțiune:

- a. n-1
- b. 1
- c. n
- d. n/2

Feedback

Răspunsul corect este: n

O diferență între algoritmii divide and impera și de programare dinamică este că:

Alegeți o opțiune:

- a. În ambii este nevoie de un criteriu de optim
- b. Primii sunt folosiți de regulă când subsoluțiile se folosesc doar o dată în calcul
- c. La programarea dinamică nu este nevoie să se păstreze soluțiile parțiale
- d. Primii au în general o complexitate computațională mai mare

Feedback

Răspunsul corect este: Primii sunt folosiți de regulă când subsoluțiile se folosesc doar o dată în calcul

Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care dintre următoarele secvențe de pași sunt complete și plauzibil a fi aplicate de algoritmul AC1 ?

Alegeți o opțiune:

- a. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B)
- b. REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)

c. REVISE(A, B), REVISE(B, C)

d. REVISE(A, B), REVISE(B, C), REVISE(A, B), REVISE(B, C)

Feedback

Răspunsul corect este: REVISE(A, B), REVISE(B, A), REVISE(B, C), REVISE(C, B), REVISE(A, B), REVISE(B, A), REVISE (B, C), REVISE(C, B)

Dându-se o mulțime de numere {n_1, n_2 ... n_k} și o valoare naturală S se poate detecta dacă există o submulțime de valori astfel încât suma lor să fie S folosind un algoritm de programare dinamică. Astfel, se folosește un tablou bi-dimensional dp[i, j] cu 1 <= i <= n, 0 <= j <= S care are valoarea Adevărat dacă există un un subset al {n_1, n_2 ... n_i} care să aibă suma j. Care este relația de recurență pentru calculul lui dp[i, j]?

Alegeți o opțiune:

a. dp[i, j] = dp[i - 1, j] AND dp[i, j - n_i]

b. dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]

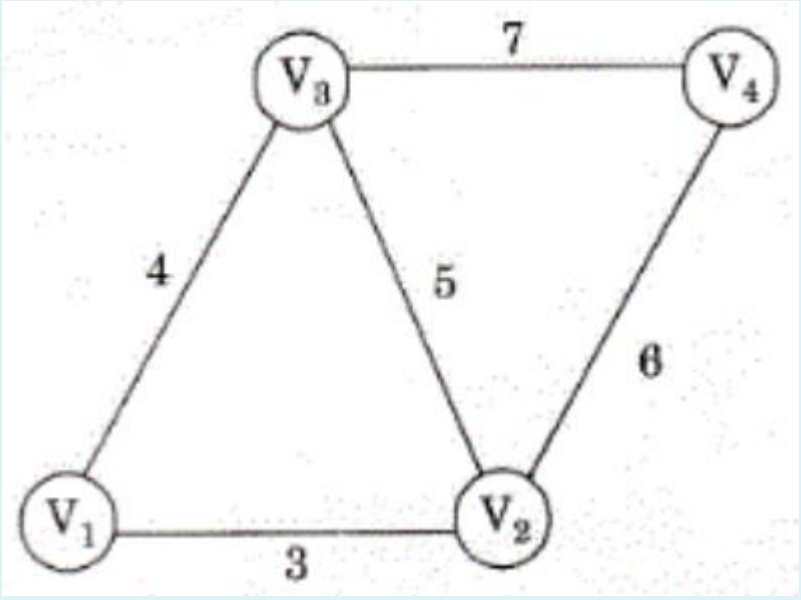
c. dp[i, j] = dp[i - 1, j] OR dp[i, j - n_i]

d. dp[i, j] = dp[i - 1, j] AND dp[i - 1, j - n_i]

Feedback

Răspunsul corect este: dp[i, j] = dp[i - 1, j] OR dp[i - 1, j - n_i]

Se da un graf neorientat cu n noduri {v_1, v_2 ... v_n}, n > 7, in care doua noduri v_i si v_j sunt conectate daca si numai daca 0 < |i-j| <= 2, iar in acest caz ponderea muchiei este i + j. Care este costul caii de la nodul v_5 la v_6 in AMA-ul acestui graf? Un exemplu de graf cu n=4 este prezentat in figura de mai jos:



Alegeți o opțiune:

- a. 31
- b. 25
- c. 11
- d. 41

Feedback

Răspunsul corect este: 31

Fie o problemă care are trei variabile cu următoarele domenii, A = {1, 2, 3, 4, 5}, B = {5, 6, 7} și C = {1, 2, 3, 4, 5, 6, 7}. Știind că între cele trei variabile sunt definite restricțiile A + B == 7 și B == C, care sunt domeniile variabilelor după aplicarea algoritmului AC1 ?

Alegeți o opțiune:

- a. A = {1, 2}, B = {5, 6, 7}, C = {5, 6, 7}
- b. A = {1, 2}, B = {5, 6}, C = {5, 6}
- c. A = {2}, B = {5}, C = {5}
- d. A = {1, 2, 3, 4, 5}, B = {5, 6, 7}, C = {5, 6, 7}

Feedback

Răspunsul corect este: A = {1, 2}, B = {5, 6}, C = {5, 6}

Fie w ponderea minima a unei muchii dintr-un graf neorientat conex si e o muchie din graf a carei pondere este w. Care dintre urmatoarele afirmatii este FALSA?

Alegeți o opțiune:

- a. Oricare AMA contine muchia e
- b. Exista un AMA care contine muchia e
- c. Daca e nu apartine unui AMA, atunci e face parte dintr-un ciclu in care toate muchiile au acelasi cost (w)
- d. Fiecare AMA are o muchie de cost w

Feedback

Răspunsul corect este: Oricare AMA contine muchia e

Dându-se o serie de n sarcini, fiecare având o durată de o unitate de timp, o utilitate u[i] și un deadline (termen limită) d[i], cum se poate determina, cu o complexitate optimă, utilitatea maximă ce se poate obține prin îndeplinirea sarcinilor? Utilitatea unei sarcini u[i] este obținută doar dacă sarcina se termină înainte de deadline-ul corespunzător.

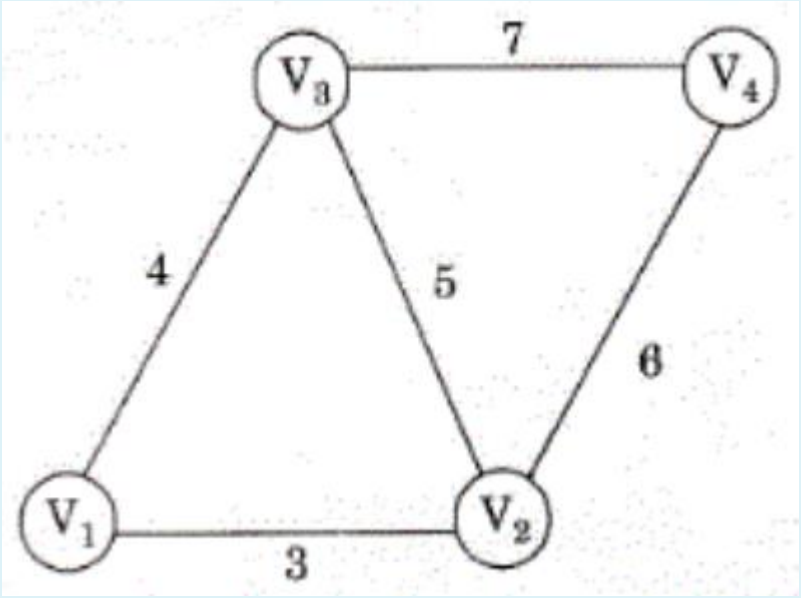
Alegeți o opțiune:

- a. Se folosește programarea dinamică pentru a stabili ce sarcini se execută și când, ținând cont de descompunerea recursivă a problemei în subprobleme
- b. se folosește un algoritm de tip backtracking pentru a genera toate posibilitățile și se alege varianta cu utilitate maximă
- c. se folosește un algoritm de tip divide et impera în care se împarte succesiv timpul în 2 jumătăți și se identifică sarcinile ce se pot executa în intervalul respectiv
- d. se folosește un algoritm de tip greedy sortând sarcinile în funcție de deadline și utilitate

Feedback

Răspunsul corect este: se folosește un algoritm de tip greedy sortând sarcinile în funcție de deadline și utilitate

Care este costul unui AMA al unui graf neorientat cu n noduri {v_1, v_2 ... v_n} in care doua noduri v_i si v_j sunt conectate daca si numai daca 0 < |i-j| <= 2, iar in acest caz ponderea muchiei este i + j? Un exemplu cu n=4 este prezentat in figura de mai jos:



Alegeți o opțiune:

- a. n ^ 2 - n + 1
- b. (11 * n ^ 2 - 5 * n)/12
- c. 2 * n + 1
- d. 6 * n - 11

Feedback

Răspunsul corect este: n ^ 2 - n + 1

Care din următoarele proprietăți nu sunt caracteristice problemelor rezolvabile prin programare dinamică?

Alegeți o opțiune:

- a. Optimalitate
- b. Memoizarea
- c. Decompozabilitatea recursivă a problemei în subprobleme
- d. Implică doi sau mai mulți jucători cu scopuri opuse

Feedback

Răspunsul corect este: Implică doi sau mai mulți jucători cu scopuri opuse

Care e avantajul principal al tăieturilor alfa-beta față de Minimax clasic?

Alegeți o opțiune:

- a. Este mai ușor de implementat
- b. În general, returnează rezultate mai rapid
- c. Nu oferă niciun avantaj

d. Returnează rezultate mai bune

Feedback

Răspunsul corect este: În general, returnează rezultate mai rapid

Pentru a calcula eficient valoarea lui x^19, avem nevoie să știm cel puțin următoarele numere:

Select one:

a. x^2, x^4, x^9, x^19

b. x^2, x^5, x^19

c. Toate numerele de la x^2 la x^19

d. x^2, x^4, x^8, x^16, x^17, x^18, x^19

Feedback

The correct answer is: x^2, x^4, x^9, x^19

Fie un graf G neponderat pe care se aplica un BFS pornind din nodul S. Notam cu d(S,u) si d(S,v) lungimile celor mai scurte cai gasite de BFS. Daca u este descoperit de BFS inainte de v, care din urmatoarele afirmatii este adevarata?

Select one:

a. d(S,u) < d(S,v)

b. d(S,u) <= d(S,v)

c. d(S,u) > d(S,v)

d. d(S,u) >= d(S,v)

Feedback

The correct answer is: d(S,u) <= d(S,v)

Cum ați modifica algoritmul de programare dinamică pentru problema parantezării matricilor pentru a calcula toate parantezările optimale, dacă există mai multe, și nu doar una singură?

Select one:

a. Acest lucru este imposibil de realizat folosind programarea dinamică, trebuie rezolvat cu un algoritm de tip backtracking

b. Pentru fiecare subproblemă A_ij, trebuie să adăugăm o buclă suplimentară cu k de la i+1 la j, ceea ce va crește complexitatea algoritmului

c. Acest lucru este imposibil de realizat plecând de la algoritmul prezentat la curs

d. Pentru fiecare subproblemă A_ij, trebuie să reținem un vector de poziții unde se poate pune optim o paranteză, în loc de o singură poziție

Feedback

The correct answer is: Pentru fiecare subproblemă A_ij, trebuie să reținem un vector de poziții unde se poate pune optim o paranteză, în loc de o singură poziție

Care este complexitatea algoritmului Bellman-Ford, dacă este rulat pe un graf complet cu n noduri?

Select one:

a. theta((n^2) * log(n))

b. theta(n^2)

c. theta(n^3)

d. theta((n^3) * log(n))

Feedback

The correct answer is: theta(n^3)

Fie o problemă pe care vrem să o rezolvăm cu algoritmi de propagarea restricțiilor. Problema are 3 variabile, A, B, respectiv C, care au valori numerice. Pentru care dintre următoarele seturi de restricții putem aplica algoritmul AC3?

Select one or more:

a. A + B == 1 , B + C == 2

b. A != B , B != C, A != C

c. A != B + C , B != A + C

d. A + B > C , A != B

Feedback

The correct answers are: A + B == 1 , B + C == 2, A != B , B != C, A != C

Pentru a identifica componentele tare conexe ale unui graf se va folosi:

Select one or more:

a. un algoritm bazat pe BFS

b. algoritmul lui Tarjan

c. oricare dintre celelalte optiuni, pentru ca toate fac parte din aceeasi clasa de complexitate

d. algoritmul lui Kosaraju

Feedback

The correct answers are: algoritmul lui Kosaraju, algoritmul lui Tarjan

Dându-se o serie de n sarcini, fiecare având o durată de o unitate de timp, o utilitate u[i] și un deadline (termen limită) d[i], cum se poate determina, cu o complexitate optimă, utilitatea maximă ce se poate obține prin îndeplinirea sarcinilor? Utilitatea unei sarcini u[i] este obținută doar dacă sarcina se termină înainte de deadline-ul corespunzător.

Select one:

a. Se folosește programarea dinamică pentru a stabili ce sarcini se execută și când, ținând cont de descompunerea recursivă a problemei în subprobleme

b. se folosește un algoritm de tip greedy sortând sarcinile în funcție de deadline și utilitate

c. se folosește un algoritm de tip divide et impera în care se împarte succesiv timpul în 2 jumătăți și se identifică sarcinile ce se pot executa în intervalul respectiv

d. se folosește un algoritm de tip backtracking pentru a genera toate posibilitățile și se alege varianta cu utilitate maximă

Feedback

The correct answer is: se folosește un algoritm de tip greedy sortând sarcinile în funcție de deadline și utilitate

Fie G un graf ponderat avand toate ponderile mai mari ca 1 si G' același graf dar la care ponderea fiecarei muchii este patratul ponderii din primul graf. Daca T este AMA pentru G avand costul t si T' este AMA pentru G' avand costul t', care din urmatoarele afirmatii este adevarata?

Select one:

a. T' = T, t' < t ^ 2

b. Niciunul din celelalte raspunsuri

c. T' = T, t' = t ^ 2

d. T' != T, t' = t ^ 2

Feedback

The correct answer is: T' = T, t' < t ^ 2

Arborii optimi (optimali) la căutare pot fi construiți eficient cu un algoritm:

Select one:

a. backtracking

b. de programare dinamică

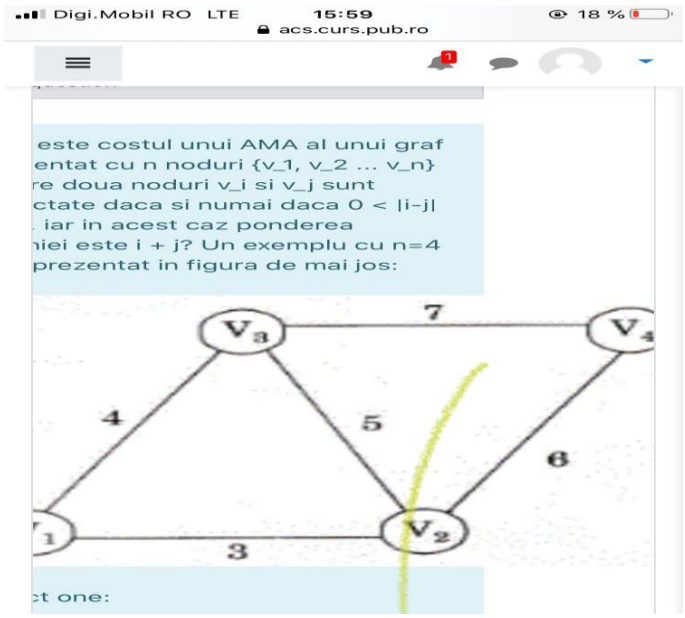
c. divide and impera

d. greedy

Feedback

The correct answer is: de programare dinamică

Care este costul unui AMA al unui graf neorientat cu n noduri {v_1, v_2 ... v_n} in care doua noduri v_i si v_j sunt conectate daca si numai daca 0 < |i-j| <= 2, iar in acest caz ponderea muchiei este i + j? Un exemplu cu n=4 este prezentat in figura de mai jos:



Select one:

a. n ^ 2 - n + 1

b. (11 * n ^ 2 - 5 * n)/12

c. 6 * n - 11

d. 2 * n + 1

Feedback

The correct answer is: n ^ 2 - n + 1

Care este relația de recurență corespunzătoare problemei determinării celei mai apropiate perechi de puncte într-un plan, folosind o implementare eficientă a algoritmului prezentat la curs?

Select one:

a. T(n) = 2 * T(n/2) + n^2

b. T(n) = 2 * T(n/2) + n

c. T(n) = 2 * T(n/2) + n * logn

d. T(n) = 2 * T(n/2) + logn

Feedback

The correct answer is: T(n) = 2 * T(n/2) + n

Un algoritm BF* (BEST FIRST) este întotdeauna:

Select one:

a. optimal

b. complet și optimal

c. nici complet nici optimal

d. complet

Feedback

The correct answer is: complet

Care din următoarele funcții de tip euristică garantează optimalitatea soluției și expandează mai puține noduri decât celelalte? Pentru toate funcțiile euristice de mai jos, valoarea euristicii pentru destinație este egală cu 0, valorea fiind calculată doar pentru restul nodurilor.

Select one:

a. lungimea rutei deja parcurse până la fiecare nod

b. lungimea în linie dreaptă de la fiecare nod la destinație

c. o valoare constantă pentru fiecare nod, egală cu valoarea muchiei de cost minim din graf (distanța minimă pe cale ferată între cele mai apropiate două orașe)

d. o valoare constantă pentru fiecare nod, egală cu distanța euclidiană între orașele sursă și destinație

Feedback

The correct answer is: lungimea în linie dreaptă de la fiecare nod la destinație

Este de preferat să se rezolve problema comis-voiajorului (dându-se o listă cu n locații și m străzi cu lungimi precizate, un comis-voiajor are misiunea de a livra colete la toate aceste locații, întorcându-se la final de unde a plecat, trecând prin fiecare locație doar o dată și obținând un drum de lungime minimă):

Select one or more:

a. folosind greedy, pentru că este singura metodă prin care se poate rezolva

b. folosind backtracking

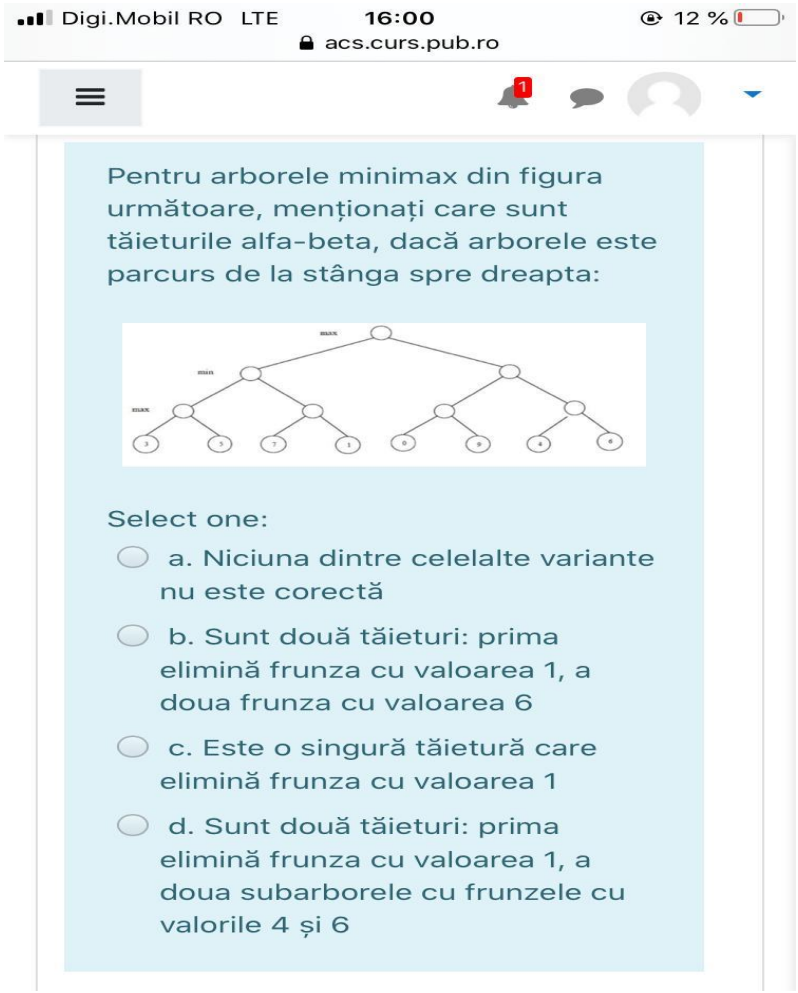
c. folosind programare dinamică, pentru că este singura metodă prin care se poate rezolva

d. folosind programare dinamică, deși soluția rămâne exponențială

Feedback

The correct answers are: folosind backtracking, folosind programare dinamică, deși soluția rămâne exponențială

Pentru arborele minimax din figura următoare, menționați care sunt tăieturile alfa-beta, dacă arborele este parcurs de la stânga spre dreapta:



Arbore mimimax

Select one:

a. Niciuna dintre celelalte variante nu este corectă

b. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua frunza cu valoarea 6

c. Este o singură tăietură care elimină frunza cu valoarea 1

d. Sunt două tăieturi: prima elimină frunza cu valoarea 1, a doua subarborele cu frunzele cu valorile 4 și 6

Feedback

The correct answer is: Este o singură tăietură care elimină frunza cu valoarea 1

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze o singură fortăreață, a.î. să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea fortăreței.

1p - Găsim toate punctele de articulatie și componentele biconexe din jurul lor. Gigel va bombarda punctul de articulație care are în jurul său cele mai multe componente biconexe.

0.5p - Întoarce orice punct de articulație

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

n log k- folosim un maxheap

n * k - 0.5 din punctaj

Se dă un graf neorientat cu *n* noduri cu costuri pozitive pe muchii și *k* surse (k mult mai mic decât n). Pentru fiecare nod din graf se cere drumul de lungime minima față de cea mai apropiata sursă. Descrieți un algoritm eficient pentru această problemă.

Sol1: Se poate adaugă o sursă fictivă cu muchie de cost 0 către toate sursele inițiale și să se facă Dijsktra din aceeastă sursă.

Sol2: Se adaugă toate sursele în heap și continuă cu Dijkstra. Complexitatea este O(M log N). Se puncteaza cu 0.5p o soluție mai ineficientă.

Cum se poate implementa eficient un algoritm de programare dinamică având recurența:

$$dp[n] = \begin{cases} \mathrm{dp}[n-2] - \mathrm{dp}[n-4], & n \geq 4 \\ 2 * n, & n < 4 \end{cases}$$

Exponențiere logaritmică pe matrice

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Carrefour Orhideea dorește să afle în câte moduri se pot aranja fructele în tăvițe. Se dau *n* tăvițe și *m* fructe (m <= n). Un fruct poate fi pus în orice tăviță. Descrieți un algoritm pentru această problemă.

Se foloseste backtracking pentru a genera toate modalitatile de aranjare pe tavite care respecta restrictiile din enunt.

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

n log k- folosim un maxheap

n * k - 0.5 din punctaj

În orașul X, rețeaua de internet este răspândita în toate cartiere, în fiecare cartier fiind amplasate mai multe routere interconectate între ele. Unele routere cedează, iar internetul pică în unul sau mai multe cartiere. Ce algoritm ați folosi pentru identificarea routerelor care duc la astfel de căderi și ce soluție sugerați pentru scăderea numărului de incidente?

Găsim toate punctele de articulație. O soluție ar fi adăugarea de muchii redundante intre cartiere.

Cum se poate calcula eficient al *K*-lea cel mai mic număr dintr-o secvență de N numere?

1p - n (folosind functia de partitionare de la qsort)

0.5p - n log k

Se dă un graf neorientat cu *n* noduri cu costuri pozitive pe muchii și *k* surse (k mult mai mic decât n). Pentru fiecare nod din graf se cere drumul de lungime minima față de cea mai apropiata sursă. Descrieți un algoritm eficient pentru această problemă.

Sol1: Se poate adaugă o sursă fictivă cu muchie de cost 0 către toate sursele inițiale și să se facă Dijsktra din aceeastă sursă.

Sol2: Se adaugă toate sursele în heap și continuă cu Dijkstra. Complexitatea este O(M log N). Se puncteaza cu 0.5p o soluție mai ineficientă.

Cum puteți **afișa toate** drumurile distincte între 2 noduri dintr-un graf **orientat aciclic**?

Există un număr exponențial de drumuri, deci ar trebui calculate cu backtracking

Question 1
Wrong
What is recurrence for worst case of QuickSort and what is the time complexity in Worst case?

Recurrence is T(n) = T(n-2) + O(n) and time complexity is O(n^2)

Recurrence is T(n) = T(n-1) + O(n) and time complexity is O(n^2)

C Recurrence is T(n) = 2T(n/2) + O(n) and time complexity is O(nLogn)

D Recurrence is T(n) = T(n/10) + T(9n/10) + O(n) and time complexity is O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 1 Explanation:
The worst case of QuickSort occurs when the picked pivot is always one of the corner elements in sorted array. In worst case, QuickSort recursively calls one subproblem with size 0 and other subproblem with size (n-1). So recurrence is T(n) = T(n-1) + T(0) + O(n) The above expression can be rewritten as T(n) = T(n-1) + O(n) void exchange(int *a, int *b) { int temp; temp = *a; *a = *b; *b = temp; } int partition(int arr[], int si, int ei) { int x = arr[ei]; int i = (si - 1); int j; for (j = si; j <= ei - 1; j++) { if(arr[j] <= x) { i++; exchange(&arr[i], &arr[j]); } } exchange (&arr[i + 1], &arr[ei]); return (i + 1); } /* Implementation of Quick Sort arr[] --> Array to be sorted si --> Starting index ei --> Ending index */ void quickSort(int arr[], int si, int ei) { int pi; /* Partitioning index */ if(si < ei) { pi = partition(arr, si, ei); quickSort(arr, si, pi - 1); quickSort(arr, pi + 1, ei); } } [/sourcecode]

Question 2
Wrong
Suppose we have a O(n) time algorithm that finds median of an unsorted array. Now consider a QuickSort implementation where we first find median using the above algorithm, then use median as pivot. What will be the worst case time complexity of this modified QuickSort.

A O(n^2 Logn)

O(n^2)

C O(n Logn Logn)

O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 2 Explanation:
If we use median as a pivot element, then the recurrence for all cases becomes T(n) = 2T(n/2) + O(n) The above recurrence can be solved using Master Method. It falls in case 2 of master method.

Question 3
Wrong
Given an unsorted array. The array has this property that every element in array is at most k distance from its position in sorted array where k is a positive integer smaller than size of array. Which sorting algorithm can be easily modified for sorting this array and what is the obtainable time complexity?

A Insertion Sort with time complexity O(kn)

Heap Sort with time complexity O(nLogk)

Quick Sort with time complexity O(kLogk)

D Merge Sort with time complexity O(kLogk)

Analysis of Algorithms **Sorting** **QuickSort** **HeapSort**
Discuss it

Question 3 Explanation:
1) to sort the array firstly create a min-heap with first k+1 elements and a separate array as resultant array. 2) because elements are at most k distance apart from original position so, it is guranteed that the smallest element will be in this K+1 elements. 3) remove the smallest element from the min-heap(extract min) and put it in the result array. 4) Now,insert another element from the unsorted array into the mean-heap, now,the second smallest element will be in this, perform extract min and continue this process until no more elements are in the unsorted array.finally, use simple heap sort for the remaining elements Time Complexity ---------- 1) O(k) to build the initial min-heap 2) O((n-k)logk) for remaining elements... 3) 0(1) for extract min so overall O(k) + O((n-k)logk) + 0(1) = O(nlogk)

Question 4
Wrong

Which of the following is not true about comparison based sorting algorithms?

The minimum possible time complexity of a comparison based sorting algorithm is O(nLogn) for a random input array

B Any comparison based sorting algorithm can be made stable by using position as a criteria when two elements are compared

C Counting Sort is not a comparison based sorting algortihm

Heap Sort is not a comparison based sorting algorithm.

Analysis of Algorithms **Sorting** **HeapSort** **CountingSort**
Discuss it

Question 4 Explanation:
See http://www.geeksforgeeks.org/lower-bound-on-comparison-based-sorting-algorithms/ for point A. See http://www.geeksforgeeks.org/stability-in-sorting-algorithms/ for B. C is true, count sort is an Integer Sorting algorithm.

Question 5
Wrong

What is time complexity of fun()?

```
int fun(int n)
{
  int count = 0;
  for (int i = n; i > 0; i /= 2)
     for (int j = 0; j < i; j++)
        count += 1;
  return count;
}
```

O(n^2)

B O(nLogn)

O(n)

D O(nLognLogn)

Analysis of Algorithms
Discuss it

Question 5 Explanation:
For a input integer n, the innermost statement of fun() is executed following times. n + n/2 + n/4 + ... 1 So time complexity T(n) can be written as T(n) = O(n + n/2 + n/4 + ... 1) = O(n)
The value of count is also n + n/2 + n/4 + .. + 1

Question 6
Correct
What is the time complexity of fun()?

```
int fun(int n)
{
  int count = 0;
  for (int i = 0; i < n; i++)
     for (int j = i; j > 0; j--)
        count = count + 1;
  return count;
}
```

A Theta (n)
Theta (n^2)
C Theta (n*Logn)
D Theta (nLognLogn)

Analysis of Algorithms
Discuss it

Question 6 Explanation:
The time complexity can be calculated by counting number of times the expression "count = count + 1;" is executed. The expression is executed 0 + 1 + 2 + 3 + 4 + ... + (n-1) times.
Time complexity = Theta(0 + 1 + 2 + 3 + .. + n-1) = Theta (n*(n-1)/2) = Theta(n^2)

Question 7
Correct
The recurrence relation capturing the optimal time of the Tower of Hanoi problem with n discs is. (GATE CS 2012)

A T(n) = 2T(n – 2) + 2
B T(n) = 2T(n – 1) + n
C T(n) = 2T(n/2) + 1
T(n) = 2T(n – 1) + 1

Analysis of Algorithms
Discuss it

Question 7 Explanation:
Following are the steps to follow to solve Tower of Hanoi problem recursively.

```
Let the three pegs be A, B and C. The goal is to move n pegs from A to C.
To move n discs from peg A to peg C:
    move n-1 discs from A to B. This leaves disc n alone on peg A
    move disc n from A to C
    move n?1 discs from B to C so they sit on disc n
```

The recurrence function T(n) for time complexity of the above recursive solution can be written as following. T(n) = 2T(n-1) + 1

Question 8
Wrong

Let w(n) and A(n) denote respectively, the worst case and average case running time of an algorithm executed on an input of size n. which of the following is ALWAYS TRUE? (GATE CS 2012)

(A)

(B)

(C)

(D)

A

B B

C

D D

Analysis of Algorithms
Discuss it

Question 8 Explanation:
The worst case time complexity is always greater than or same as the average case time complexity.

Question 9
Wrong

Which of the following is not O(n^2)?

A (15^10) * n + 12099

B n^1.98

n^3 / (sqrt(n))

(2^20) * n

Analysis of Algorithms
Discuss it

Question 9 Explanation:
The order of growth of option c is $n^{2.5}$ which is higher than n^2.

Question 10
Correct

Which of the given options provides the increasing order of asymptotic complexity of functions f1, f2, f3 and f4?

```
f1(n) = 2^n
f2(n) = n^(3/2)
f3(n) = nLogn
f4(n) = n^(Logn)
```

f3, f2, f4, f1

B f3, f2, f1, f4

C f2, f3, f1, f4

D f2, f3, f4, f1

Analysis of Algorithms
Discuss it

Question 10 Explanation:

```
f1(n) = 2^n
f2(n) = n^(3/2)
f3(n) = nLogn
f4(n) = n^(Logn)
```

Except f3, all other are exponential. So f3 is definitely first in output. Among remaining, n^(3/2) is next. One way to compare f1 and f4 is to take Log of both functions. Order of growth of Log(f1(n)) is Θ(n) and order of growth of Log(f4(n)) is Θ(Logn * Logn). Since Θ(n) has higher growth than Θ(Logn * Logn), f1(n) grows faster than f4(n). **Following is another way to compare f1 and f4.** Let us compare f4 and f1. Let us take few values to compare

```
n = 32, f1 = 2^32, f4 = 32^5 = 2^25
n = 64, f1 = 2^64, f4 = 64^6 = 2^36
..............
..............
```

Also see http://www.wolframalpha.com/input/?i=2^n+vs+n^%28log+n%29 Thanks to fella26 for suggesting the above explanation.

Question 12
Correct

What is the best time complexity of bubble sort?

A N^2

B NlogN

N

D N(logN)^2

Analysis of Algorithms **Sorting** **BubbleSort**
Discuss it

Question 12 Explanation:
The bubble sort is at its best if the input data is sorted. i.e. If the input data is sorted in the same order as expected output. This can be achieved by using one boolean variable. The boolean variable is used to check whether the values are swapped at least once in the inner loop. Consider the following code snippet: [sourcecode language="C"] int main() { int arr[] = {10, 20, 30, 40, 50}, i, j, isSwapped; int n = sizeof(arr) / sizeof(*arr); isSwapped = 1; for(i = 0; i < n - 1 && isSwapped; ++i) { isSwapped = 0; for(j = 0; j < n - i - 1; ++j) if (arr[j] > arr[j + 1]) { swap(&arr[j], &arr[j + 1]); isSwapped = 1; } } for(i = 0; i < n; ++i) printf("%d ", arr[i]); return 0; } [/sourcecode] Please observe that in the above code, the outer loop runs only once.

Question 13

What is the worst case time complexity of insertion sort where position of the data to be inserted is calculated using binary search?

A N

B NlogN

N^2

D N(logN)^2

Analysis of Algorithms **Sorting** **InsertionSort** **BinarySearch**
Discuss it

Question 13 Explanation:
Applying binary search to calculate the position of the data to be inserted doesn't reduce the time complexity of insertion sort. This is because insertion of a data at an appropriate position involves two steps: 1. Calculate the position. 2. Shift the data from the position calculated in step #1 one step right to create a gap where the data will be inserted. Using binary search reduces the time complexity in step #1 from O(N) to O(logN). But, the time complexity in step #2 still remains O(N). So, overall complexity remains O(N^2).

Question 14
Correct

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A N

B N^2

NlogN

D N(logN)^2

Analysis of Algorithms **Sorting**
Discuss it

Question 14 Explanation:
The number of comparisons that a comparison sort algorithm requires increases in proportion to Nlog(N), where N is the number of elements to sort. This bound is asymptotically tight: Given a list of distinct numbers (we can assume this because this is a worst-case analysis), there are N factorial permutations exactly one of which is the list in sorted order. The sort algorithm must gain enough information from the comparisons to identify the correct permutations. If the algorithm always completes after at most f(N) steps, it cannot distinguish more than 2^f(N) cases because the keys are distinct and each comparison has only two possible outcomes. Therefore, 2^f(N) >= N! or equivalently f(N) >= log(N!). Since log(N!) is Omega(NlogN), the answer is NlogN. For more details, read here

Question 15
Wrong

In a modified merge sort, the input array is splitted at a position one-third of the length(N) of the array. Which of the following is the tightest upper bound on time complexity of this modified Merge Sort.

N(logN base 3)

B N(logN base 2/3)

C N(logN base 1/3)

N(logN base 3/2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 15 Explanation:
The time complexity is given by: T(N) = T(N/3) + T(2N/3) + N Solving the above recurrence relation gives, T(N) = N(logN base 3/2)

Question 18
Wrong

The following statement is valid. log(n!) = θ(n log n).

True

False

Analysis of Algorithms
Discuss it

Question 18 Explanation:
Order of growth of $\log n!$ and $n \log n$ is same for large values of n, i.e., $\theta(\log n!) = \theta(n \log n)$. So time complexity of fun() is $\theta(n \log n)$. The expression $\theta(\log n!) = \theta(n \log n)$ can be easily derived from following Stirling's approximation (or Stirling's formula).

$$\log n! = n \log n - n + O(\log(n))$$

Question 19

Question
19
Wrong

What does it mean when we say that an algorithm X is asymptotically more efficient than Y?

A X will be a better choice for all inputs

X will be a better choice for all inputs except possibly small inputs

X will be a better choice for all inputs except possibly large inputs

D Y will be a better choice for small inputs

Analysis of Algorithms
Discuss it

Question 19 Explanation:
In asymptotic analysis we consider growth of algorithm in terms of input size. An algorithm X is said to be asymptotically better than Y if X takes smaller time than y for all input sizes n larger than a value n0 where n0 > 0.

Question 20
Correct

What is the time complexity of Floyd–Warshall algorithm to calculate all pair shortest path in a graph with **n** vertices?

A O(n^2logn)

B Theta(n^2logn)

C Theta(n^4)

Theta(n^3)

Analysis of Algorithms

Discuss it

Question 20 Explanation:
Floyd–Warshall algorithm uses three nested loops to calculate all pair shortest path. So, time complexity is Thete(n^3). Read here for more details.

Question 21
Wrong

A list of n string, each of length n, is sorted into lexicographic order using the merge-sort algorithm. The worst case running time of this computation is

O (n log n)

O (n^2 log n)

C O (n^2 + log n)

D O (n^2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 21 Explanation:
The recurrence tree for merge sort will have height Log(n). And O(n^2) work will be done at each level of the recurrence tree (Each level involves n comparisons and a comparison takes O(n) time in worst case). So time complexity of this Merge Sort will be $O(n^2 logn)$.

Question 22
Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n)

A A

B

C C

D D

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 22 Explanation:
The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get θ(nLogn).

Question 23
Wrong

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let T(n) be the number of comparisons required to sort n elements. Then

A T(n) <= 2T(n/5) + n

T(n) <= T(n/5) + T(4n/5) + n

C T(n) <= 2T(4n/5) + n

T(n) <= 2T(n/2) + n

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 23 Explanation:
For the case where n/5 elements are in one subset, T(n/5) comparisons are needed for the first subset with n/5 elements, T(4n/5) is for the rest 4n/5 elements, and n is for finding the pivot. If there are more than n/5 elements in one set then other set will have less than 4n/5 elements and time complexity will be less than T(n/5) + T(4n/5) + n because recursion tree will be more balanced.

Question 26
Wrong

Which of the following sorting algorithms has the lowest worst-case complexity?

Merge Sort

Bubble Sort

C Quick Sort

D Selection Sort

Analysis of Algorithms **Sorting** **SelectionSort** **MergeSort**
Discuss it

Question 26 Explanation:
Worst case complexities for the above sorting algorithms are as follows: Merge Sort — nLogn Bubble Sort — n^2 Quick Sort — n^2 Selection Sort — n^2

Question 27

Question
31
Wrong

The number of elements that can be sorted in $\Theta(logn)$ time using heap sort is

(A) $\Theta(1)$
(B) $\Theta(\sqrt{logn})$
(C) $\Theta(Logn/(LogLogn))$
(d) $\Theta(Logn)$

A

B B

C

D D

Analysis of Algorithms **Sorting** **HeapSort**
Discuss it

Question 31 Explanation:

Time complexity of Heap Sort is $\Theta(mLogm)$ for m input elements. For m = $\Theta(Logn/(LogLogn))$, the value of $\Theta(m * Logm)$ will be $\Theta([Logn/(LogLogn)] * [Log(Logn/(LogLogn))])$ which will be $\Theta([Logn/(LogLogn)] * [LogLogn - LogLogLogn])$ which is $\Theta(Logn)$

Question 32

Question 33

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? <pre> (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n) </pre>

A A

B

C C

D D

Analysis of Algorithms **Sorting** **GATE-CS-2009** **QuickSort**
Discuss it

Question 33 Explanation:

Answer(B) The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get \theta(nLogn).

Question 37

Wrong

The minimum number of comparisons required to find the minimum and the maximum of 100 numbers is _____.

148

B 147

146

D 140

Analysis of Algorithms **GATE-CS-2014-(Set-1)**
Discuss it

Question 37 Explanation:

Steps to find minimum and maximum element out of n numbers:

```
1. Pick 2 elements(a, b), compare them. (say a > b)
2. Update min by comparing (min, b)
3. Update max by comparing (max, a)
```

Therefore, we need 3 comparisons for each 2 elements, so total number of required comparisons will be (3n)/2 - 2, because we do not need to update min or max in the very first step. Recurrence relation will be:

T(n) = T(⌈n/2⌉)+T(⌊n/2⌋)+2 = 2T(n/2)+2 = ⌈3n/2⌉-2
By putting the value n=100, (3*100/2)-2 = **148** which is answer.

Question 39
Wrong

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

$O(n^2)$

B $O(nLogn)$

$Theta(nLogn)$

D $O(n^3)$

Analysis of Algorithms **Sorting** **GATE-CS-2014-(Set-3)** **QuickSort**
Discuss it

Question 39 Explanation:
For any input, there are some permutations for which worst case will be $O(n^2)$. In some case, choosing the middle element minimizes the chances of encountering $O(n^2)$, but in worst case it can go to $O(n^2)$. Whichever element we take as Pivot, either first or middle, worst case will be $O(n^2)$ since Pivot is fixed in position. While choosing a random pivot minimizes the chances of encountering worst case i.e. $O(n^2)$. Refer this article on Quick Sort.

Question 42
Wrong

Two matrices M1 and M2 are to be stored in arrays A and B respectively. Each array can be stored either in row-major or column-major order in contiguous memory locations. The time complexity of an algorithm to compute M1 × M2 will be

best if A is in row-major, and B is in column- major order

B best if both are in row-major order

C best if both are in column-major order

independent of the storage scheme

Analysis of Algorithms **GATE-CS-2004**
Discuss it

Question 42 Explanation:
This is a trick question. Note that the questions asks about time complexity, not time taken by the program. for time complexity, it doesn't matter how we store array elements, we always need to access same number of elements of M1 and M2 to multiply the matrices. It is always constant or O(1) time to do element access in arrays, the constants may differ for different schemes, but not the time complexity.

Question 46
Wrong

In a permutation a1.....an of n distinct integers, an inversion is a pair (ai, aj) such that i < j and ai > aj. What would be the worst case time complexity of the Insertion Sort algorithm, if the inputs are restricted to permutations of 1.....n with at most n inversions?

$\Theta(n^2)$

B $\Theta(n \log n)$

C $\Theta(n^{1.5})$

$\Theta(n)$

Analysis of Algorithms **Sorting** **GATE-CS-2003** **InsertionSort**
Discuss it

Question 46 Explanation:
Insertion sort runs in $\Theta(n + f(n))$ time, where f(n) denotes the number of inversion initially present in the array being sorted. Source:
http://cs.xidian.edu.cn/jpkc/Algorithm/down/Solution%20to%202-4%20Inversions.pdf

Question 47
Correct

Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort?

A O(n)

B O(n Log n)

$O(n^2)$

D O(n!)

Analysis of Algorithms **Sorting** **GATE-CS-2001** **QuickSort**
Discuss it

Question 47 Explanation:
Randomized quicksort has expected time complexity as O(nLogn), but worst case time complexity remains same. In worst case the randomized function can pick the index of corner element every time.

Question 49
Correct

Which one of the following is the recurrence equation for the worst case time complexity of the Quicksort algorithm for sorting $n(\geq 2)$ numbers? In the recurrence equations given in the options below, c is a constant.

A $T(n) = 2T(n/2) + cn$

$T(n) = T(n-1) + T(0) + cn$

C $T(n) = 2T(n-2) + cn$

D $T(n) = T(n/2) + cn$

Analysis of Algorithms **Sorting** **GATE-CS-2015 (Set 1)** **QuickSort**
Discuss it

Question 49 Explanation:
In worst case, the chosen pivot is always placed at a corner position and recursive call is made for following. a) for subarray on left of pivot which is of size n-1 in worst case. b) for subarray on right of pivot which is of size 0 in worst case.

Question 51
Correct

An unordered list contains n distinct elements. The number of comparisons to find an element

in this list that is neither maximum nor minimum is

A Θ(nlogn)

B Θ(n)

C Θ(logn)

Θ(1)

Analysis of Algorithms **GATE-CS-2015 (Set 2)**
Discuss it

Question 51 Explanation:
We only need to consider any 3 elements and compare them. So the number of comparisons is constants, that makes time complexity as Θ(1) The catch here is, we need to return any element that is neither maximum not minimum. Let us take an array {10, 20, 15, 7, 90}. Output can be 10 or 15 or 20 Pick any three elements from given liar. Let the three elements be 10, 20 and 7. Using 3 comparisons, we can find that the middle element is 10.

Question 53
Correct

In the following table, the left column contains the names of standard graph algorithms and the right column contains the time complexities of the algorithms. Match each algorithm with its time complexity.

1. Bellman-Ford algorithm 2. Kruskal's algorithm 3. Floyd-Warshall algorithm 4. Topological sorting	A : O (m log n) B : O (n^3) C : O (nm) D : O (n + m)

1→ C, 2 → A, 3 → B, 4 → D

B 1→ B, 2 → D, 3 → C, 4 → A

C 1→ C, 2 → D, 3 → A, 4 → B

D 1→ B, 2 → A, 3 → C, 4 → D

Analysis of Algorithms **Gate IT 2005**
Discuss it

Question 53 Explanation:

- **Bellman-Ford algorithm: Time complexity: O(VE)**
- **Kruskal's algorithm**:**Time Complexity:** O(ElogE) or O(ElogV). Sorting of edges takes O(ELogE) time. After sorting, we iterate through all edges and apply find-union algorithm. The find and union operations can take atmost O(LogV) time. So overall complexity is O(ELogE + ELogV) time. The value of E can be atmost V^2, so O(LogV) are O(LogE) same. Therefore, overall time complexity is O(ElogE) or O(ElogV)
- **Floyd-Warshall algorithm**: **Time Complexity**: O(V^3)
- **Topological sorting: Time Complexity:** The above algorithm is simply DFS with an extra stack. So time complexity is same as DFS which is O(V+E).

Question 55
Correct

The worst case running times of Insertion sort, Merge sort and Quick sort, respectively, are:

A $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$

B $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \text{ Log } n)$

C $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$

$\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 1)**
Discuss it

Question 55 Explanation:

- Insertion Sort takes $\Theta(n^2)$ in worst case as we need to run two loops. The outer loop is needed to one by one pick an element to be inserted at right position. Inner loop is used for two things, to find position of the element to be inserted and moving all sorted greater elements one position ahead. Therefore the worst case recursive formula is $T(n) = T(n-1) + \Theta(n)$.
- Merge Sort takes $\Theta(n \text{ Log } n)$ time in all cases. We always divide array in two halves, sort the two halves and merge them. The recursive formula is $T(n) = 2T(n/2) + \Theta(n)$.
- QuickSort takes $\Theta(n^2)$ in worst case. In QuickSort, we take an element as pivot and partition the array around it. In worst case, the picked element is always a corner element and recursive formula becomes $T(n) = T(n-1) + \Theta(n)$. An example scenario when worst case happens is, arrays is sorted and our code always picks a corner element as pivot.

Question 56
Correct

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in ascending order, which of the following are TRUE ?

I. Quicksort runs in $\Theta(n^2)$ time
II. Bubblesort runs in $\Theta(n^2)$ time
III. Mergesort runs in $\Theta(n)$ time
IV. Insertion sort runs in $\Theta(n)$ time

A I and II only

B I and III only

C II and IV only

I and IV only

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 2)**
Discuss it

Question 56 Explanation:
I. Given an array in ascending order, Recurrence relation for total number of comparisons for quicksort will be T(n) = T(n-1)+O(n) //partition algo will take O(n) comparisons in any case. = O(n^2) **II.** Bubble Sort runs in $\Theta(n^2)$ time If an array is in ascending order, we could make a small modification in Bubble Sort Inner for loop which is responsible for bubbling the kth largest element to the end in kth iteration. Whenever there is no swap after the completion of inner for loop of bubble sort in any iteration, we can declare that array is sorted in case of Bubble Sort taking O(n) time in Best Case. **III.** Merge Sort runs in $\Theta(n)$ time Merge Sort relies on Divide and Conquer paradigm to sort an array and there is no such worst or best case input for merge sort. For any sequence, Time complexity will be given by

following recurrence relation, T(n) = 2T(n/2) + Θ(n) // In-Place Merge algorithm will take Θ(n) due to copying an entire array. = Θ(nlogn) **IV.** Insertion sort runs in Θ(n) time Whenever a new element which will be greater than all the elements of the intermediate sorted sub-array (because given array is sorted) is added, there won't be any swap but a single comparison. In n-1 passes we will be having 0 swaps and n-1 comparisons. Total time complexity = O(n) // N-1 Comparisons This solution is contributed by **Pranjul Ahuja**

//// For an array already sorted in ascending order, Quicksort has a complexity $\Theta(n^2)$ [Worst Case] Bubblesort has a complexity Θ(n) [Best Case] Mergesort has a complexity Θ(n log n) [Any Case] Insertsort has a complexity Θ(n) [Best Case]

Question 57

Question 58

Correct

The characters a to h have the set of frequencies based on the first 8 Fibonacci numbers as follows

a : 1, b : 1, c : 2, d : 3, e : 5, f : 8, g : 13, h : 21/

A Huffman code is used to represent the characters. What is the sequence of characters corresponding to the following code? 110111100111010

fdheg

B ecgdf

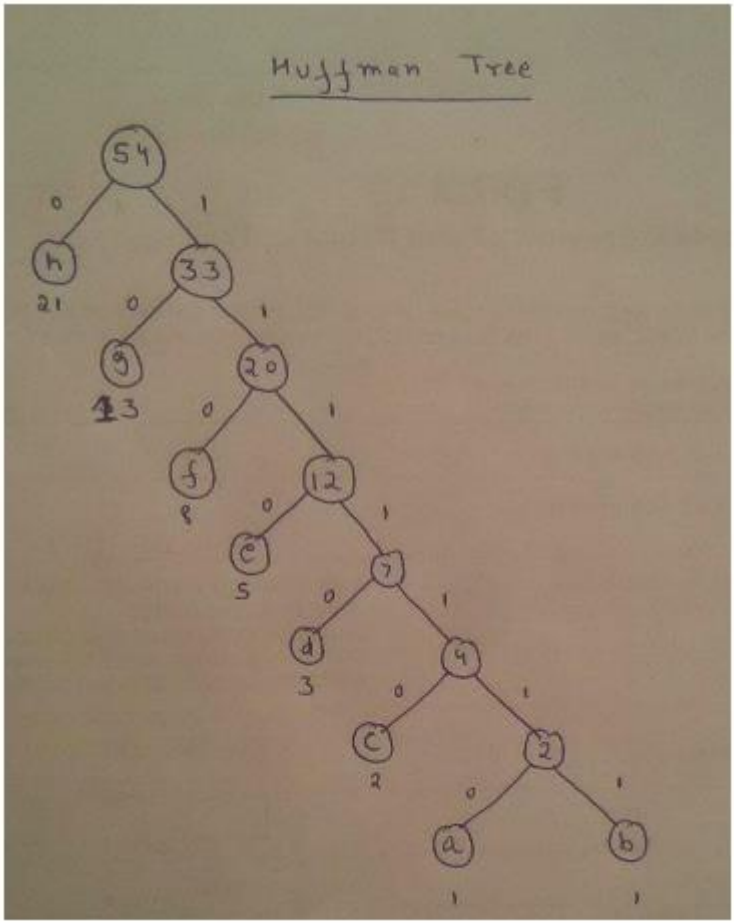
C dchfg

D fehdg

Analysis of Algorithms **GATE IT 2006**
Discuss it

Question 58 Explanation:

Background Required - Generating Prefix codes using Huffman Coding. First we apply greedy algorithm on the frequencies of the characters to generate the binary tree as shown in the Figure given below. Assigning 0 to the left edge and 1 to the right edge, prefix codes for the characters are as below.



a - 1111110 b - 1111111 c - 111110 d - 11110 e - 1110 f - 110 g - 10 h - 0 Given String can be decomposed as 110 11110 0 1110 10 f d h e g This solution is contributed by **Pranjul Ahuja** .

Question 61
Wrong

Which of the following is TRUE?

The cost of searching an AVL tree is θ (log n) but that of a binary search tree is O(n)

B The cost of searching an AVL tree is θ (log n) but that of a complete binary tree is θ (n log n)

The cost of searching a binary search tree is O (log n) but that of an AVL tree is θ(n)

D The cost of searching an AVL tree is θ (n log n) but that of a binary search tree is O(n)

Analysis of Algorithms **Balanced Binary Search Trees** **Gate IT 2008**
Discuss it

Question 61 Explanation:
AVL tree is a balanced tree.
AVL tree's time complexity of searching = θ(logn)
But a binary search tree, may be skewed tree, so in worst case BST searching time = θ(n)

Question 63
Correct

If we use Radix Sort to sort n integers in the range $(n^{k/2},n^k]$, for some k>0 which is independent of n, the time taken would be?

A $\Theta(n)$

B $\Theta(kn)$

$\Theta(nlogn)$

D $\Theta(n^2)$

Analysis of Algorithms **Sorting** **RadixSort** **Gate IT 2008**
Discuss it

Question 63 Explanation:
Radix sort time complexity = O(wn)
for n keys of word size= w
=>w = $\log(n^k)$
O(wn)=O(klogn.n)
=> kO(nlogn)

Question 64
Correct

The auxiliary space of insertion sort is O(1), what does O(1) mean ?

A The memory (space) required to process the data is not constant.

It means the amount of extra memory Insertion Sort consumes doesn't depend on the input. The algorithm should use the same amount of memory for all inputs.

C It takes only 1 kb of memory .

D It is the speed at which the elements are traversed.

Analysis of Algorithms **Sorting** **InsertionSort**
Discuss it

Question 64 Explanation:
The term O(1) states that the space required by the insertion sort is constant i.e., space required doesn't depend on input.

Question 65
Wrong

A sorting technique is called stable if:

A It takes O(nlog n)time

It maintains the relative order of occurrence of non-distinct elements

C It uses divide and conquer paradigm

It takes O(n) space

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 66
Correct

Suppose we want to arrange the ii numbers stored in an array such that all negative values occur before all positive ones. Minimum number of exchanges required in the worst case is:

A n-1

B n

C n+1

None of the above

Analysis of Algorithms **GATE CS 1999**
Discuss it

Question 66 Explanation:
When we have 'i' numbers stored in an array, we have to swap all positive numbers with negative and in worst case positive numbers will be i/2.

Question 67
Wrong

If one uses straight two-way merge sort algorithm to sort the following elements in ascending order *20, 47, 15, 8, 9, 4, 40, 30, 12, 17* then the order of these elements after the second pass of the algorithm is:

8, 9, 15, 20, 47, 4, 12, 17, 30, 40

8, 15, 20, 47, 4, 9, 30, 40, 12, 17

C 15, 20, 47, 4, 8, 9, 12, 30, 40, 17

D 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 67 Explanation:
In first pass the elements are sorted in n/4 (first 2 elements in each group) sub arrays but in second pass the elements are sorted in n/2 (first 4 elements in each group) sub arrays.

Question 68

Question
71
Wrong

Which of the following algorithm design technique is used in finding all pairs of shortest distances in a graph?

Dynamic programming

B Backtracking

Greedy

D Divide and Conquer

Analysis of Algorithms **GATE CS 1998**
Discuss it

Question 72
Wrong

Give the correct matching for the following pairs:

```
A. O(log n)      1. Selection sort
B. O(n)          2. Insertion sort
C. O(nlog n)     3. Binary search
D. O(n^2)        4. Merge sort

codes:
     A   B   C   D
a.   3   1   2   4
```

b. 3 1 4 2
c. 1 3 4 2
d. 1 4 3 2

a

b

C c

D d

Analysis of Algorithms **GATE CS 1998**
Discuss it

Question 75
Correct

The correct matching for the following pairs is

(A) All pairs shortest path	(1) Greedy
(B) Quick Sort	(2) Depth-First search
(C) Minimum weight spanning tree	(3) Dynamic Programming
(D) Connected Components	(4) Divide and and Conquer

Codes:

	A	B	C	D
a	2	4	1	3
b	3	4	1	2
c	3	4	2	1
d	4	1	2	3

A a

b

C c

D d

Analysis of Algorithms **GATE CS 1997**
Discuss it

Question 78
Correct

Quicksort is run on two inputs shown below to sort in ascending order taking first element as pivot,

(i) 1, 2, 3,....., n
(ii) n, n-1, n-2,....., 2, 1

Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

A C1 < C2

B C1 > C2

C1 = C2

D We cannot say anything for arbitrary n

Analysis of Algorithms **Sorting** **QuickSort** **GATE CS 1996**
Discuss it

Question 78 Explanation:

Quick Sort performs in worst case when input is sorted in either ascending order or descending order. So, quick sort will return equal number of comparisons for both given inputs. Option (C) is correct.

Question 83
Correct

Given A, an array of size n, comprised of an increasing sequence of numbers followed immediately by a decreasing one. What is worst case time complexity of optimal algorithm to determine if a given number x is in the array?

Θ(log n)

Θ(n)

C Θ(n^2)

D Θ(log n)^2

Analysis of Algorithms **GATE CS Mock 2018**
Discuss it

Question 83 Explanation:
This is an application of Binary search, which has time complexity Θ(log n) in worst case. Option (A) is correct.

Question 84
Wrong

Match the following:

Algorithm	Time complexity (Worst case)
A. Optimal merge pattern	1. O(m*n)
B. Power of an element	2. O(log n)
C. Prims MST algorithm	3. O(n log n)
D. Longest common subsequence	4. O(m log n)

Which of the following option is correct?

A – 3, B – 2, C – 4, D – 1

A – 3, B – 1, C – 4, D – 2

C A – 3, B – 4, C – 2, D – 1

D A – 1, B – 2, C – 4, D – 3

Analysis of Algorithms **GATE CS Mock 2018 | Set 2**
Discuss it

Question 84 Explanation:
Correct matching is A – 3, B – 2, C – 4, D – 1. Option (A) is correct.

Question 88
Wrong

If b is the branching factor and m is the maximum depth of the search tree, what is the space complexity of greedy search?

A O(b+m)

O(bm)

O(b^m)

D $O(m^m)$

Analysis of Algorithms **UGC-NET CS 2017 Nov - III**
Discuss it

Question 88 Explanation:

1. In binary tree branching factor is 2 and space complexity for height n is $O(2^n)$.
2. In ternary tree branching factor is 3 and space complexity for height n is $O(3^n)$.

   Similarly

3. If branching factor is b and height is m for search tree then space complexity of greedy search is $O(b^m)$.

So, option (C) is correct.

Question 89
Wrong

The time complexity of computing the transitive closure of a binary relation on a set of n elements is known to be

O(nlogn)

B $O(n^{3/2})$

O(n^3)

D O(n)

Analysis of Algorithms **Set Theory & Algebra** **ISRO CS 2017 - May**
Discuss it

Question 89 Explanation:
Transitive closure generally uses Floyd-Warshall Algorithm which gives a time complexity of $O(n^3)$ Refer: Transitive closure of a graph

Question 92
Wrong

Any decision tree that sorts n elements has height _____.

Ω(lg n)

B Ω(n)

Ω(n lg n)

D $\Omega(n^2)$

Analysis of Algorithms **UGC NET CS 2017 Jan - III**
Discuss it

Question 93
Correct

Dijkstra's algorithm is based on

A Divide and conquer paradigm

B Dynamic programming

Greedy Approach

D Backtracking paradigm

Analysis of Algorithms **UGC NET CS 2017 Jan - III**

Discuss it

Question 93 Explanation:

Dijkstra's algorithm is based on Greedy Approach. For more information Refer:Greedy Algorithms | Set 7 (Dijkstra's shortest path algorithm) Option (C) is correct.

Question 94

Correct

Match the following with respect to algorithm paradigms :

	List – I		**List – II**
a.	Merge sort	i.	Dynamic programming
b.	Huffman coding	ii.	Greedy approach
c.	Optimal polygon triangulation	iii.	Divide and conquer
d.	Subset sum problem	iv.	Back tracking

Codes :

	a	b	c	d
(1)	iii	i	ii	iv
(2)	ii	i	iv	iii
(3)	ii	i	iii	iv
(4)	iii	ii	i	iv

A (1)

B (2)

C (3)

(4)

Analysis of Algorithms **UGC NET CS 2017 Jan - III**

Discuss it

Question 95

Correct

What is the best method to go for the game playing problem?

A Optimal Search

B Random Search

Heuristic Search

D Stratified Search

Analysis of Algorithms **UGC NET CS 2017 Jan - III**

Discuss it

Question 97

Correct

Consider a weighted complete graph G on the vertex set $\{v_1, v_2, ... v_n\}$ such that the weight of the edge (v_i, v_j) is $4 | i – j|$. The weight of minimum cost spanning tree of G is :

A $4n^2$

B n

4n – 4

D 2n – 2

Analysis of Algorithms **UGC NET CS 2016 Aug - III**

Discuss it

Question 98

Wrong

Match the following:

a.	Prim's algorithm	i.	$O(V^2E)$
b.	Bellman-Ford algorithm	ii.	$O(VE\ lgV)$
c.	Floyd-Warshall algorithm	iii.	$O(E\ lgV)$
d.	Johnson's algorithm	iv.	$O(V^3)$

Where V is the set of nodes and E is the set of edges in the graph.

Codes :

	a	b	c	d
(1)	i	iii	iv	ii
(2)	i	iii	ii	iv
(3)	iii	i	iv	ii
(4)	iii	i	ii	iv

(1)

B (2)

(3)

D (4)

Analysis of Algorithms **UGC NET CS 2016 Aug - III**

Discuss it

Question 98 Explanation:

- Prim's algorithm takes O(E lgV) time.
- Bellman-Ford algorithm takes $O(V^2E)$ time.
- Floyd-Warshall algorithm takes $O(V^3)$ time.
- Johnson's algorithm takes O(VE lgV) time.

So, option (C) is correct.

Question 99

Correct

(a)	Huffman codes	(i)	$O(n^2)$
(b)	Optimal polygon triangulation	(ii)	$\theta(n^3)$
(c)	Activity selection problem	(iii)	O(nlgn)
(d)	Quicksort	(iv)	$\theta(n)$

Codes :

	(a)	(b)	(c)	(d)
(1)	(i)	(ii)	(iv)	(iii)
(2)	(i)	(iv)	(ii)	(iii)
(3)	(iii)	(ii)	(iv)	(i)
(4)	(iii)	(iv)	(ii)	(i)

Match the following:

A (1)

B (2)

(3)

D (4)

Analysis of Algorithms **UGC NET CS 2016 July – III**

Discuss it

Question 99 Explanation:

- Huffman codes takes O(nlgn) time.
- Optimal polygon triangulation takes $\theta(n^3)$ time

  Activity selection problem takes $\theta(n)$ time

-
- Quicksort takes $O(n^2)$ time

So, option (C) is correct.

Question 100

Correct

Given two sequences X and Y : X = < a, b, c, b, d, a, b > Y = < b, d, c, a, b, a > . The longest common subsequence of X and Y is :

A < b, c, a>

B < c, a, b>

C < b, c, a, a>

< b, c, b, a>

Analysis of Algorithms **UGC NET CS 2015 Dec – III**

Discuss it

Question 101

Wrong

Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

Greedy algorithm, $\theta (V^3)$

B Greedy algorithm, $\theta (V^2 \lg n)$

Dynamic programming, $\theta (V^3)$

D Dynamic programming, $\theta (V^2 \lg n)$

Analysis of Algorithms **UGC NET CS 2015 Dec – III**
Discuss it

Question 101 Explanation:
Floyd-Warshall algorithm utilizes dynamic programming to solve the all-pairs shortest paths problem on a directed graph in $\theta (V^3)$ time. For more information on Floyd-Warshall algorithm ReferDynamic Programming | Set 16 (Floyd Warshall Algorithm) Option (C) is correct.

Question 102
Correct

To determine the efficiency of an algorithm the time factor is measured by:

A Counting micro seconds

Counting number of key operations

C Counting number of statements

D Counting kilobytes of algorithm

Analysis of Algorithms **UGC NET CS 2015 Jun - II**
Discuss it

Question 102 Explanation:
To determine the efficiency of an algorithm the time factor is measured by counting number of key operations. It is not measured by counting of statements because less no of statement does not mean that program is more efficient. Many times we have to write large code for better efficieny. We measure space complexity in kilobytes. So, option (B) is correct.

Question 104
Wrong

An all-pairs shortest-paths problem is efficiently solved using:

Dijkstra' algorithm

B Bellman-Ford algorithm

C Kruskal algorithm

Floyd-Warshall algorithm

Analysis of Algorithms **UGC NET CS 2015 Jun - III**
Discuss it

Question 104 Explanation:
An all-pairs shortest-paths problem is efficiently solved using Floyd-Warshall algorithm. For more information on all-pairs shortest-paths problem and its solution using Floyd-Warshall algorithm Refer:Dynamic Programming | Set 16 (Floyd Warshall Algorithm) So, option (D) is correct.

Match the following:

List - I List - II

(a) Bucket sort	(i) $O(n^3 \lg n)$
(b) Matrix chain multiplication	(ii) $O(n^3)$
(c)Huffman codes	(iii) O(nlgn)
(d)All pairs shortest paths	(iv)O(n)

codes:

	(a)	(b)	(c)	(d)
(A)	(iv)	(ii)	(i)	(iii)
(B)	(ii)	(iv)	(i)	(iii)
(C)	(iv)	(ii)	(i)	(iii)
(D)	(iii)	(ii)	(iv)	(i)

(A)
B (B)
C (C)
(D)

Analysis of Algorithms **UGC NET CS 2014 Dec - III**
Discuss it

Question 110
Correct
Dijkstra algorithm, which solves the single-source shortest--paths problem, is a _____, and the Floyd-Warshall algorithm, which finds shortest paths between all pairs of vertices, is a _____

A Greedy algorithm, Divide-conquer algorithm
B Divide-conquer algorithm, Greedy algorithm
Greedy algorithm, Dynamic programming algorithm
D Dynamic programming algorithm, Greedy algorithm

Analysis of Algorithms **UGC NET CS 2014 Dec - III**
Discuss it

Question 110 Explanation:
Dijkstra algorithm, which solves the single-source shortest–paths problem, is a Greedy algorithm, and the Floyd-Warshall algorithm, which finds shortest paths between all pairs of vertices, is a Dynamic programming algorithm. For more information on Floyd-Warshall and Dijkstra algorithm Refer:Dynamic Programming | Set 16 (Floyd Warshall Algorithm) Greedy Algorithms | Set 7 (Dijkstra's shortest path algorithm) Option (C) is correct.
Question 114
Wrong

Match the following:

codes:

List - I

(a) The 8-Queen’s problem

(b) Single-Source shortest paths

(c)STRASSEN’s Matrix multiplication

(d)Optimal binary search trees

List - II

(i)Dynamic programming

(ii) Divide and conquer

(iii)Greedy approach

(iv)Backtracking

	(a)	(b)	(c)	(d)
(1)	(iv)	(i)	(iii)	(ii)
(2)	(iv)	(iii)	(i)	(ii)
(3)	(iii)	(iv)	(i)	(ii)
(4)	(iv)	(iii)	(ii)	(i)

A (1)

(2)

C (3)

(4)

Analysis of Algorithms **UGC NET CS 2018 July - II**
Discuss it

Question 114 Explanation:

- The 8-Queen’s problem is a Backtracking algorithm.
- Single-Source shortest paths is a Greedy approach
- STRASSEN’s Matrix multiplication is a Divide and conquer
- Optimal binary search trees is a Dynamic programming. So, option (D) is correct.

Question 1
Correct

What is recurrence for worst case of QuickSort and what is the time complexity in Worst case?

A Recurrence is T(n) = T(n-2) + O(n) and time complexity is O(n^2)
Recurrence is T(n) = T(n-1) + O(n) and time complexity is O(n^2)

C Recurrence is T(n) = 2T(n/2) + O(n) and time complexity is O(nLogn)

D Recurrence is T(n) = T(n/10) + T(9n/10) + O(n) and time complexity is O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 1 Explanation:

The worst case of QuickSort occurs when the picked pivot is always one of the corner elements in sorted array. In worst case, QuickSort recursively calls one subproblem with size 0 and other subproblem with size (n-1). So recurrence is T(n) = T(n-1) + T(0) + O(n) The above expression can be rewritten as T(n) = T(n-1) + O(n) void exchange(int *a, int *b) { int temp; temp = *a; *a = *b; *b = temp; } int partition(int arr[], int si, int ei) { int x = arr[ei]; int i = (si - 1); int j; for (j = si; j <= ei - 1; j++) { if(arr[j] <= x) { i++; exchange(&arr[i], &arr[j]); } } exchange (&arr[i + 1], &arr[ei]); return (i + 1); } /* Implementation of Quick Sort arr[] --> Array to be sorted si --> Starting index ei --> Ending index */ void quickSort(int arr[], int

si, int ei) { int pi; /* Partitioning index */ if(si < ei) { pi = partition(arr, si, ei); quickSort(arr, si, pi - 1); quickSort(arr, pi + 1, ei); } } [/sourcecode]

Question 2
Correct

Suppose we have a O(n) time algorithm that finds median of an unsorted array. Now consider a QuickSort implementation where we first find median using the above algorithm, then use median as pivot. What will be the worst case time complexity of this modified QuickSort.

A O(n^2 Logn)

B O(n^2)

C O(n Logn Logn)

O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 2 Explanation:
If we use median as a pivot element, then the recurrence for all cases becomes T(n) = 2T(n/2) + O(n) The above recurrence can be solved using Master Method. It falls in case 2 of master method.

Question 3
Wrong

Which of the following is not a stable sorting algorithm in its typical implementation.

Insertion Sort

B Merge Sort

Quick Sort

D Bubble Sort

Sorting **QuickSort** **InsertionSort** **MergeSort**
Discuss it

Question 3 Explanation:
See following for details. http://www.geeksforgeeks.org/stability-in-sorting-algorithms/

Question 4
Correct

Which of the following sorting algorithms in its typical implementation gives best performance when applied on an array which is sorted or almost sorted (maximum 1 or two elements are misplaced).

A Quick Sort

B Heap Sort

C Merge Sort

Insertion Sort

Sorting **QuickSort** **InsertionSort** **HeapSort**
Discuss it

Question 4 Explanation:
Insertion sort takes linear time when input array is sorted or almost sorted (maximum 1 or 2 elements are misplaced). All other sorting algorithms mentioned above will take more than lienear time in their typical implementation.

Question 5
Wrong

Given an unsorted array. The array has this property that every element in array is at most k distance from its position in sorted array where k is a positive integer smaller than size of array. Which sorting algorithm can be easily modified for sorting this array and what is the obtainable time complexity?

A Insertion Sort with time complexity O(kn)

Heap Sort with time complexity O(nLogk)

C Quick Sort with time complexity O(kLogk)

Merge Sort with time complexity O(kLogk)

Analysis of Algorithms **Sorting** **QuickSort** **HeapSort**
Discuss it

Question 5 Explanation:
1) to sort the array firstly create a min-heap with first k+1 elements and a separate array as resultant array. 2) because elements are at most k distance apart from original position so, it is guranteed that the smallest element will be in this K+1 elements. 3) remove the smallest element from the min-heap(extract min) and put it in the result array. 4) Now,insert another element from the unsorted array into the mean-heap, now,the second smallest element will be in this, perform extract min and continue this process until no more elements are in the unsorted array.finally, use simple heap sort for the remaining elements Time Complexity ---------- 1) O(k) to build the initial min-heap 2) O((n-k)logk) for remaining elements... 3) 0(1) for extract min so overall O(k) + O((n-k)logk) + 0(1) = O(nlogk)

Question 6

Question 6
Wrong

Consider a situation where swap operation is very costly. Which of the following sorting algorithms should be preferred so that the number of swap operations are minimized in general?

A Heap Sort

Selection Sort

C Insertion Sort

Merge Sort

Sorting **SelectionSort** **InsertionSort** **MergeSort**
Discuss it

Question 6 Explanation:

Selection sort makes O(n) swaps which is minimum among all sorting algorithms mentioned above.

Question 7
Wrong
Which of the following is not true about comparison based sorting algorithms?

A The minimum possible time complexity of a comparison based sorting algorithm is O(nLogn) for a random input array

Any comparison based sorting algorithm can be made stable by using position as a criteria when two elements are compared

C Counting Sort is not a comparison based sorting algortihm

Heap Sort is not a comparison based sorting algorithm.

Analysis of Algorithms **Sorting** **HeapSort** **CountingSort**
Discuss it

Question 7 Explanation:
See http://www.geeksforgeeks.org/lower-bound-on-comparison-based-sorting-algorithms/ for point A. See http://www.geeksforgeeks.org/stability-in-sorting-algorithms/ for B. C is true, count sort is an Integer Sorting algorithm.

Question 8
Wrong
Suppose we are sorting an array of eight integers using quicksort, and we have just finished the first partitioning with the array looking like this:

```
2  5  1  7  9  12  11  10
```

Which statement is correct?

The pivot could be either the 7 or the 9.

B The pivot could be the 7, but it is not the 9

C The pivot is not the 7, but it could be the 9

Neither the 7 nor the 9 is the pivot.

Sorting **QuickSort**
Discuss it

Question 8 Explanation:
7 and 9 both are at their correct positions (as in a sorted array). Also, all elements on left of 7 and 9 are smaller than 7 and 9 respectively and on right are greater than 7 and 9 respectively.

Question 9
Correct
Suppose we are sorting an array of eight integers using heapsort, and we have just finished some heapify (either maxheapify or minheapify) operations. The array now looks like this: 16 14 15 10 12 27 28 How many heapify operations have been performed on root of heap?

A 1

2

C 3 or 4

D 5 or 6

Sorting **Heap** **HeapSort**
Discuss it

Question 9 Explanation:
In Heapsort, we first build a heap, then we do following operations till the heap size becomes 1. a) Swap the root with last element b) Call heapify for root c) reduce the heap size by 1. In this question, it is given that heapify has been called few times and we see that last two elements in given array are the 2 maximum elements in array. So situation is clear, it is maxheapify whic has been called 2 times.

Question 10

Wrong

What is the best time complexity of bubble sort?

N^2

B NlogN

N

D N(logN)^2

Analysis of Algorithms **Sorting** **BubbleSort**
Discuss it

Question 10 Explanation:
The bubble sort is at its best if the input data is sorted. i.e. If the input data is sorted in the same order as expected output. This can be achieved by using one boolean variable. The boolean variable is used to check whether the values are swapped at least once in the inner loop. Consider the following code snippet: int main() { int arr[] = {10, 20, 30, 40, 50}, i, j, isSwapped; int n = sizeof(arr) / sizeof(*arr); isSwapped = 1; for(i = 0; i < n - 1 && isSwapped; ++i) { isSwapped = 0; for(j = 0; j < n - i - 1; ++j) if (arr[j] > arr[j + 1]) { swap(&arr[j], &arr[j + 1]); isSwapped = 1; } } for(i = 0; i < n; ++i) printf("%d ", arr[i]); return 0; } [/sourcecode] Please observe that in the above code, the outer loop runs only once .

Question 11

Wrong

You have to sort 1 GB of data with only 100 MB of available main memory. Which sorting technique will be most appropriate?

Heap sort

Merge sort

C Quick sort

D Insertion sort

Sorting **QuickSort** **MergeSort** **HeapSort**
Discuss it

Question 11 Explanation:
The data can be sorted using external sorting which uses merging technique. This can be done as follows: 1. Divide the data into 10 groups each of size 100. 2. Sort each group and write them to disk. 3. Load 10 items from each group into main memory. 4. Output the smallest item from the main memory to disk. Load the next item from the group whose item was chosen. 5. Loop step #4 until all items are not outputted. The step 3-5 is called as merging technique.

Question 12

Wrong

What is the worst case time complexity of insertion sort where position of the data to be inserted is calculated using binary search?

A N

NlogN

N^2

D N(logN)^2

Analysis of Algorithms **Sorting** **InsertionSort** **BinarySearch**
Discuss it

Question 12 Explanation:
Applying binary search to calculate the position of the data to be inserted doesn't reduce the time complexity of insertion sort. This is because insertion of a data at an appropriate position involves two steps: 1. Calculate the position. 2. Shift the data from the position calculated in step #1 one step right to create a gap where the data will be inserted. Using binary search reduces the time complexity in step #1 from O(N) to O(logN). But, the time complexity in step #2 still remains O(N). So, overall complexity remains O(N^2).

Question 13

Correct

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A N

B N^2

NlogN

D N(logN)^2

Analysis of Algorithms **Sorting**
Discuss it

Question 13 Explanation:
The number of comparisons that a comparison sort algorithm requires increases in proportion to Nlog(N), where N is the number of elements to sort. This bound is asymptotically tight: Given a list of distinct numbers (we can assume this because this is a worst-case analysis), there are N factorial permutations exactly one of which is the list in sorted order. The sort algorithm must gain enough information from the comparisons to identify the correct permutations. If the algorithm always completes after at most f(N) steps, it cannot distinguish more than 2^f(N) cases because the keys are distinct and each comparison has only two possible outcomes. Therefore, 2^f(N) >= N! or equivalently f(N) >= log(N!). Since log(N!) is Omega(NlogN), the answer is NlogN. For more details, read here

Question 14

Wrong

In a modified merge sort, the input array is splitted at a position one-third of the length(N) of the array. Which of the following is the tightest upper bound on time complexity of this modified Merge Sort.

A N(logN base 3)

B N(logN base 2/3)

N(logN base 1/3)

N(logN base 3/2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 14 Explanation:
The time complexity is given by: T(N) = T(N/3) + T(2N/3) + N Solving the above recurrence relation gives, T(N) = N(logN base 3/2)

Question 15

Correct

Which sorting algorithm will take least time when all elements of input array are identical? Consider typical implementations of sorting algorithms.

Insertion Sort

B Heap Sort

C Merge Sort

D Selection Sort

Sorting **SelectionSort** **InsertionSort** **MergeSort**
Discuss it

Question 15 Explanation:
The insertion sort will take θ(n) time when input array is already sorted.

Question 16

Correct

A list of n string, each of length n, is sorted into lexicographic order using the merge-sort algorithm. The worst case running time of this computation is

A O (n log n)

O (n^2 log n)

C O (n^2 + log n)

D O (n^2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 16 Explanation:
The recurrence tree for merge sort will have height Log(n). And O(n^2) work will be done at each level of the recurrence tree (Each level involves n comparisons and a comparison takes O(n) time in worst case). So time complexity of this Merge Sort will be $O(n^2 logn)$.

Question 17

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n)

A A

B

C C

D D

Analysis of Algorithms Sorting QuickSort
Discuss it

Question 17 Explanation:
The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get θ(nLogn).

Question
18

Question 18

Correct

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let T(n) be the number of comparisons required to sort n elements. Then

A T(n) <= 2T(n/5) + n

T(n) <= T(n/5) + T(4n/5) + n

C T(n) <= 2T(4n/5) + n

D T(n) <= 2T(n/2) + n

Analysis of Algorithms Sorting QuickSort
Discuss it

Question 18 Explanation:
For the case where n/5 elements are in one subset, T(n/5) comparisons are needed for the first subset with n/5 elements, T(4n/5) is for the rest 4n/5 elements, and n is for finding the pivot. If there

are more than n/5 elements in one set then other set will have less than 4n/5 elements and time complexity will be less than T(n/5) + T(4n/5) + n because recursion tree will be more balanced.

Question 19

Correct

Which of the following sorting algorithms has the lowest worst-case complexity?

Merge Sort

B Bubble Sort

C Quick Sort

D Selection Sort

Analysis of Algorithms **Sorting** **SelectionSort** **MergeSort**
Discuss it

Question 19 Explanation:
Worst case complexities for the above sorting algorithms are as follows: Merge Sort — nLogn Bubble Sort — n^2 Quick Sort — n^2 Selection Sort — n^2

Question 20

Correct

Which sorting algorithms is most efficient to sort string consisting of ASCII characters?

A Quick sort

B Heap sort

C Merge sort

Counting sort

Sorting **QuickSort** **HeapSort** **CountingSort**
Discuss it

Question 20 Explanation:
Counting sort algorithm is efficient when range of data to be sorted is fixed. In the above question, the range is from 0 to 255(ASCII range). Counting sort uses an extra constant space proportional to range of data.

Question 22

Correct

Which of the following is true about merge sort?

A Merge Sort works better than quick sort if data is accessed from slow sequential memory.

B Merge Sort is stable sort by nature

C Merge sort outperforms heap sort in most of the practical situations.

All of the above.

Sorting **QuickSort** **MergeSort** **HeapSort**
Discuss it

Question 22 Explanation:
See Merge Sort and this.

Question 23

Wrong

Given an array where numbers are in range from 1 to n^6, which sorting algorithm can be used to sort these number in linear time?

Not possible to sort in linear time

Radix Sort

C Counting Sort

D Quick Sort

Sorting **QuickSort** **RadixSort** **CountingSort**
Discuss it

Question 23 Explanation:
See Radix Sort for explanation.

Question 24

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? <pre> (A) θ(n) (B) θ (nLogn) (C) θ(n^2) (D) θ(n^2 log n) </pre>

A A

B

C C

D D

Analysis of Algorithms **Sorting** **GATE-CS-2009** **QuickSort**
Discuss it

Question 24 Explanation:
Answer(B) The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get \theta(nLogn).

Question 26

Correct

Let P be a QuickSort Program to sort numbers in ascending order using the first element as pivot. Let t1 and t2 be the number of comparisons made by P for the inputs {1, 2, 3, 4, 5} and {4, 1, 5, 3, 2} respectively. Which one of the following holds?

A t1 = 5

B t1 < t2

t1 > t2

D t1 = t2

Sorting **GATE-CS-2014-(Set-1)** **QuickSort**
Discuss it

Question 26 Explanation:
When first element or last element is chosen as pivot, Quick Sort's worst case occurs for the sorted arrays. In every step of quick sort, numbers are divided as per the following recurrence. T(n) = T(n-1) + O(n)

Question 27

Wrong

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

$O(n^2)$

O(nLogn)

C Theta(nLogn)

D $O(n^3)$

Analysis of Algorithms **Sorting** **GATE-CS-2014-(Set-3)** **QuickSort**
Discuss it

Question 27 Explanation:
For any input, there are some permutations for which worst case will be $O(n^2)$. In some case, choosing the middle element minimizes the chances of encountering $O(n^2)$, but in worst case it can go to $O(n^2)$. Whichever element we take as Pivot, either first or middle, worst case will be $O(n^2)$ since Pivot is fixed in position. While choosing a random pivot minimizes the chances of encountering worst case i.e. $O(n^2)$. Refer this article on Quick Sort.

Question
28

Question 28

Wrong

In a permutation a1.....an of n distinct integers, an inversion is a pair (ai, aj) such that i < j and ai > aj. What would be the worst case time complexity of the Insertion Sort algorithm, if the inputs are restricted to permutations of 1.....n with at most n inversions?

$\Theta(n^2)$

B $\Theta(n \log n)$

C $\Theta(n^{1.5})$

$\Theta(n)$

Analysis of Algorithms **Sorting** **GATE-CS-2003** **InsertionSort**
Discuss it

Question 28 Explanation:

Insertion sort runs in Θ(n + f(n)) time, where f(n) denotes the number of inversion initially present in the array being sorted. Source: http://cs.xidian.edu.cn/jpkc/Algorithm/down/Solution%20to%202-4%20Inversions.pdf

Question 29

Correct

Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort?

A O(n)

B O(n Log n)

$O(n^2)$

D O(n!)

Analysis of Algorithms **Sorting** **GATE-CS-2001** **QuickSort**
Discuss it

Question 29 Explanation:
Randomized quicksort has expected time complexity as O(nLogn), but worst case time complexity remains same. In worst case the randomized function can pick the index of corner element every time.

Question 30

Wrong

Which of the following changes to typical QuickSort improves its performance on average and are generally done in practice.

```
1) Randomly picking up to make worst case less
   likely to occur.
2) Calling insertion sort for small sized arrays
   to reduce recursive calls.
3) QuickSort is tail recursive, so tail call
   optimizations can be done.
4) A linear time median searching algorithm is used
   to pick the median, so that the worst case time
   reduces to O(nLogn)
```

A 1 and 2

2, 3, and 4

1, 2 and 3

D 2, 3 and 4

Sorting GATE-CS-2015 (Mock Test) QuickSort
Discuss it

Question 30 Explanation:
The 4th optimization is generally not used, it reduces the worst case time complexity to O(nLogn), but the hidden constants are very high.

Question 31

Correct

Which one of the following is the recurrence equation for the worst case time complexity of the Quicksort algorithm for sorting n(≥ 2) numbers? In the recurrence equations given in the options below, c is a constant.

A $T(n) = 2T (n/2) + cn$

$T(n) = T(n – 1) + T(0) + cn$

C $T(n) = 2T (n – 2) + cn$

D $T(n) = T(n/2) + cn$

Analysis of Algorithms Sorting GATE-CS-2015 (Set 1) QuickSort
Discuss it

Question 31 Explanation:
In worst case, the chosen pivot is always placed at a corner position and recursive call is made for following. a) for subarray on left of pivot which is of size n-1 in worst case. b) for subarray on right of pivot which is of size 0 in worst case.

Question 32

Wrong

Assume that a mergesort algorithm in the worst case takes 30 seconds for an input of size 64. Which of the following most closely approximates the maximum input size of a problem that can be solved in 6 minutes?

A 256

512

1024

D 2048

Sorting GATE-CS-2015 (Set 3) MergeSort
Discuss it

Question 32 Explanation:

```
Time complexity of merge sort is Θ(nLogn)

c*64Log64 is 30
c*64*6 is 30
c is 5/64

For time 6 minutes

5/64*nLogn = 6*60

nLogn = 72*64 = 512 * 9

n = 512.
```

Question 33

Correct

The worst case running times of Insertion sort, Merge sort and Quick sort, respectively, are:

A $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$

B $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \text{ Log } n)$

C $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$

$\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Analysis of Algorithms Sorting GATE-CS-2016 (Set 1)
Discuss it

Question 33 Explanation:

- Insertion Sort takes $\Theta(n^2)$ in worst case as we need to run two loops. The outer loop is needed to one by one pick an element to be inserted at right position. Inner loop is used for two things, to find position of the element to be inserted and moving all sorted greater elements one position ahead. Therefore the worst case recursive formula is $T(n) = T(n-1) + \Theta(n)$.
- Merge Sort takes $\Theta(n \text{ Log } n)$ time in all cases. We always divide array in two halves, sort the two halves and merge them. The recursive formula is $T(n) = 2T(n/2) + \Theta(n)$.
- QuickSort takes $\Theta(n^2)$ in worst case. In QuickSort, we take an element as pivot and partition the array around it. In worst case, the picked element is always a corner element and

recursive formula becomes T(n) = T(n-1) + Θ(n). An example scenario when worst case happens is, arrays is sorted and our code always picks a corner element as pivot.

Question 34

Correct

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in ascending order, which of the following are TRUE ?

```
I.   Quicksort runs in Θ(n^2) time
II.  Bubblesort runs in Θ(n^2) time
III. Mergesort runs in  Θ(n) time
IV.  Insertion sort runs in  Θ(n) time
```

A I and II only

B I and III only

C II and IV only

I and IV only

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 2)**
Discuss it

Question 34 Explanation:
I. Given an array in ascending order, Recurrence relation for total number of comparisons for quicksort will be T(n) = T(n-1)+O(n) //partition algo will take O(n) comparisons in any case. = O(n^2) **II.** Bubble Sort runs in Θ(n^2) time If an array is in ascending order, we could make a small modification in Bubble Sort Inner for loop which is responsible for bubbling the kth largest element to the end in kth iteration. Whenever there is no swap after the completion of inner for loop of bubble sort in any iteration, we can declare that array is sorted in case of Bubble Sort taking O(n) time in Best Case. **III.** Merge Sort runs in Θ(n) time Merge Sort relies on Divide and Conquer paradigm to sort an array and there is no such worst or best case input for merge sort. For any sequence, Time complexity will be given by following recurrence relation, T(n) = 2T(n/2) + Θ(n) // In-Place Merge algorithm will take Θ(n) due to copying an entire array. = Θ(nlogn) **IV.** Insertion sort runs in Θ(n) time Whenever a new element which will be greater than all the elements of the intermediate sorted sub-array (because given array is sorted) is added, there won't be any swap but a single comparison. In n-1 passes we will be having 0 swaps and n-1 comparisons. Total time complexity = O(n) // N-1 Comparisons This solution is contributed by **Pranjul Ahuja**

//// For an array already sorted in ascending order, Quicksort has a complexity Θ(n^2) [Worst Case] Bubblesort has a complexity Θ(n) [Best Case] Mergesort has a complexity Θ(n log n) [Any Case] Insertsort has a complexity Θ(n) [Best Case]

Question 35

Wrong

Assume that we use Bubble Sort to sort n distinct elements in ascending order. When does the best case of Bubble Sort occur?

When elements are sorted in ascending order

B When elements are sorted in descending order

C When elements are not sorted by any order

There is no best case for Bubble Sort. It always takes O(n*n) time

Sorting BubbleSort
Discuss it

Question 36

Correct

If we use Radix Sort to sort n integers in the range $(n^{k/2},n^k]$, for some k>0 which is independent of n, the time taken would be?

A Θ(n)

B Θ(kn)

Θ(nlogn)

D $\Theta(n^2)$

Analysis of Algorithms Sorting RadixSort Gate IT 2008
Discuss it

Question 36 Explanation:
Radix sort time complexity = O(wn)
for n keys of word size= w
=>w = $\log(n^k)$
O(wn)=O(klogn.n)
=> kO(nlogn)

Question 37

Correct

Consider an array of elements arr[5]= {5,4,3,2,1} , what are the steps of insertions done while doing insertion sort in the array.

4 5 3 2 1 3 4 5 2 1 2 3 4 5 1 1 2 3 4 5

B 5 4 3 1 2 5 4 1 2 3 5 1 2 3 4 1 2 3 4 5

C 4 3 2 1 5 3 2 1 5 4 2 1 5 4 3 1 5 4 3 2

D 4 5 3 2 1 2 3 4 5 1 3 4 5 2 1 1 2 3 4 5

Sorting **InsertionSort**
Discuss it

Question 37 Explanation:
In the insertion sort , just imagine that the first element is already sorted and all the right side Elements are unsorted, we need to insert all elements one by one from left to right in the sorted Array. Sorted : 5 unsorted : 4 3 2 1 Insert all elements less than 5 on the left (Considering 5 as the key) Now key value is 4 and array will look like this Sorted : 4 5 unsorted : 3 2 1 Similarly for all the cases the key will always be the newly inserted value and all the values will be compared to that key and inserted in to proper position.

Question 39

Wrong

Which of the following statements is correct with respect to insertion sort ?

```
*Online - can sort a list at runtime
*Stable - doesn't change the relative
          order of elements with equal keys.
```

Insertion sort is stable, online but not suited well for large number of elements.

Insertion sort is unstable and online

C Insertion sort is online and can be applied to more than 100 elements

D Insertion sort is stable & online and can be applied to more than 100 elements

Sorting **InsertionSort**
Discuss it

Question 39 Explanation:
Time taken by algorithm is good for small number of elements, but increases quadratically for large number of elements.

Question 40

Wrong

Consider the array A[]= {6,4,8,1,3} apply the insertion sort to sort the array . Consider the cost associated with each sort is 25 rupees , what is the total cost of the insertion sort when element 1 reaches the first position of the array ?

50

B 25

75

D 100

Sorting **C Arrays** **InsertionSort**
Discuss it

Question 40 Explanation:
When the element 1 reaches the first position of the array two comparisons are only required hence 25 * 2= 50 rupees. *step 1: 4 6 8 1 3 . *step 2: 1 4 6 8 3.

Question 42

Wrong

What is the best sorting algorithm to use for the elements in array are more than 1 million in general?

Merge sort.

B Bubble sort.

Quick sort.

D Insertion sort.

Sorting **QuickSort** **InsertionSort** **MergeSort**
Discuss it

Question 42 Explanation:
Most practical implementations of Quick Sort use randomized version. The randomized version has expected time complexity of O(nLogn). The worst case is possible in randomized version also, but worst case doesn't occur for a particular pattern (like sorted array) and randomized Quick Sort works well in practice. Quick Sort is also a cache friendly sorting algorithm as it has good locality of

reference when used for arrays. Quick Sort is also tail recursive, therefore tail call optimizations is done.

Question 43

Wrong

Which of the below given sorting techniques has highest best-case runtime complexity.

A Quick sort

Selection sort

C Insertion sort

Bubble sort

Sorting **GATE 2017 Mock**
Discuss it

Question 43 Explanation:
Quick sort best case time complexity is O(n logn)
Selection sort best case time complexity is O(n^2)
Insertion sort best case time complexity is O(n)
Bubble sort best case time complexity is O(n)

Question 45

Wrong

If one uses straight two-way merge sort algorithm to sort the following elements in ascending order *20, 47, 15, 8, 9, 4, 40, 30, 12, 17* then the order of these elements after the second pass of the algorithm is:

8, 9, 15, 20, 47, 4, 12, 17, 30, 40

8, 15, 20, 47, 4, 9, 30, 40, 12, 17

C 15, 20, 47, 4, 8, 9, 12, 30, 40, 17

D 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 45 Explanation:

In first pass the elements are sorted in n/4 (first 2 elements in each group) sub arrays but in second pass the elements are sorted in n/2 (first 4 elements in each group) sub arrays.

Question 46

Correct

Quicksort is run on two inputs shown below to sort in ascending order taking first element as pivot,

```
(i) 1, 2, 3,......., n
(ii) n, n-1, n-2,......, 2, 1
```

Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

A C1 < C2

B C1 > C2

C1 = C2

D We cannot say anything for arbitrary n

Analysis of Algorithms **Sorting** **QuickSort** **GATE CS 1996**
Discuss it

Question 46 Explanation:
Quick Sort performs in worst case when input is sorted in either ascending order or descending order. So, quick sort will return equal number of comparisons for both given inputs. Option (C) is correct.

Question 50

Wrong

You are given a sequence of n elements to sort. The input sequence consists of n/k subsequences,each containing k elements. The elements in a given subsequence are all smaller than the elements in the succeeding subsequence and larger than the elements in the preceding subsequence. Thus, all that is needed to sort the whole sequence of length n is to sort the k elements in each of the n/k subsequences. The lower bound on the number of comparisons needed to solve this variant of the sorting problem is

A Ω (n)

Ω (n/k)

Ω (nlogk)

D Ω (n/klogn/k)

Sorting UGC-NET CS 2017 Nov - III
Discuss it

C E BINE SUS

Question 51

Wrong

Which of the following is true for computation time in insertion, deletion and finding maximum and minimum element in a sorted array ?

A Insertion – 0(1), Deletion – 0(1), Maximum – 0(1), Minimum – 0(l)

B Insertion – 0(1), Deletion – 0(1), Maximum – 0(n), Minimum – 0(n)

Insertion – 0(n), Deletion – 0(n), Maximum – 0(1), Minimum – 0(1)

Insertion – 0(n), Deletion – 0(n), Maximum – 0(n), Minimum – 0(n)

Sorting UGC NET CS 2017 Jan - II
Discuss it

Question 51 Explanation:

1. In a sorted array, if we want to insert or delete then we have to traverse whole array and check where is the suitable position, so it will take O(n).
2. If array is sorted then end position will tell the maximum or minimum, so finding maximum or minimum will take O(1).

So, option (C) is correct.

Question 52

Correct

Consider the following sorting algorithms. I. Quicksort II. Heapsort III. Mergesort Which of them perform in least time in the worst case?

A I and II only

II and III only

C III only

D I, II and III

Sorting ISRO CS 2014
Discuss it

Question 52 Explanation:
Worst case complexity of Heap Sort and Merge Sort is O(n log n) whereas the worst case complexity of Quicksort is $O(n^2)$. (B) option is correct.

Question 53

Wrong

Which of the following algorithms sort n integers, having the range 0 to $(n^2 - 1)$, in ascending order in O(n) time ?

A Selection sort

B Bubble sort

Radix sort

Insertion sort

Sorting UGC NET CS 2015 Jun - II
Discuss it

Question 53 Explanation:

- Selection sort takes $O(n^2)$ time.
- Bubble sort takes $O(n^2)$ time.
- Radix sort takes O(n) time.
- Insertion sort takes $O(n^2)$ time.

So, option (C) is correct.

Question 54

Correct

Which one of the following in place sorting algorithms needs the minimum number of swaps?

A Quick sort

B Insertion sort

Selection sort

D Heap sort

Sorting **ISRO CS 2011**
Discuss it

Question 54 Explanation:
Selection sort takes minimum number of swaps to sort an array. It takes maximum of O(n) comparisons to sort an array with n elements. Refer: Selection sort

Question 56

Correct

Selection sort algorithm design technique is an example of

Greedy method

B Divide-and-conquer

C Dynamic Programming

D Backtracking

Sorting **ISRO CS 2007**
Discuss it

Question 56 Explanation:
The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array. 1) The subarray which is already sorted. 2) Remaining subarray which is unsorted. In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray. Clearly, it is a greedy approach to sort the array. Option (A) is correct.

Question 58

Wrong

You have to sort a list L, consisting of a sorted list followed by a few ‘random’ elements. Which of the following sorting method would be most suitable for such a task?

A Bubble sort

B Selection sort

Quick sort

Insertion sort

Sorting **UGC NET CS 2014 Dec - II**
Discuss it

Question 58 Explanation:
For a sorted list with few random numbers:

- Bubble sort will take $O(n^2)$ time in best case.
- Selection sort will take $O(n^2)$ time in best case.
- Quick sort will take $O(n^2)$ time in this case because it is the worst case for quick sort.
- Insertion sort will take O(n) So, option (D) is correct.

Question 1

Correct

Which of the following algorithms is NOT a divide & conquer algorithm by nature?

A Euclidean algorithm to compute the greatest common divisor

Heap Sort

C Cooley-Tukey fast Fourier transform

D Quick Sort

Divide and Conquer
Discuss it

Question 1 Explanation:
See Divide and Conquer

Question
2

Question 3

Wrong

Consider the polynomial p(x) = a0 + a1x + a2x^2 +a3x^3, where ai != 0, for all i. The minimum number of multiplications needed to evaluate p on an input x is:

3

B 4

6

D 9

Divide and Conquer
Discuss it

Question 3 Explanation:
Multiplications can be minimized using following order for evaluation of the given expression. p(x) = a0 + x(a1 + x(a2 + a3x))

Question 4

Correct

Maximum Subarray Sum problem is to find the subarray with maximum sum. For example, given an array {12, -13, -5, 25, -20, 30, 10}, the maximum subarray sum is 45. The naive solution for this problem is to calculate sum of all subarrays starting with every element and return the maximum of all. We can solve this using Divide and Conquer, what will be the worst case time complexity using Divide and Conquer.

A O(n)

O(nLogn)

C O(Logn)

D O(n^2)

Divide and Conquer
Discuss it

Question 4 Explanation:
See http://www.geeksforgeeks.org/divide-and-conquer-maximum-sum-subarray/

Question 5

Correct

Consider a situation where you don't have function to calculate power (pow() function in C) and you need to calculate x^n where x can be any number and n is a positive integer. What can be the best possible time complexity of your power function?

A O(n)

B O(nLogn)

C O(LogLogn)

O(Logn)

Divide and Conquer
Discuss it

Question 5 Explanation:
We can calculate power using divide and conquer in O(Logn) time. See http://www.geeksforgeeks.org/write-a-c-program-to-calculate-powxn/.

Question 6

Wrong

Consider the problem of searching an element x in an array 'arr[]' of size n. The problem can be solved in O(Logn) time if. 1) Array is sorted 2) Array is sorted and rotated by k. k is given to you and k <= n 3) Array is sorted and rotated by k. k is NOT given to you and k <= n 4) Array is not sorted

A 1 Only

1 & 2 only

1, 2 and 3 only

D 1, 2, 3 and 4

Divide and Conquer
Discuss it

Question 6 Explanation:
See http://www.geeksforgeeks.org/search-an-element-in-a-sorted-and-pivoted-array/

Question 9

Wrong

Consider the problem of computing min-max in an unsorted array where min and max are minimum and maximum elements of array. Algorithm A1 can compute min-max in a1 comparisons without divide and conquer. Algorithm A2 can compute min-max in a2 comparisons by scanning the array linearly. What could be the relation between a1 and a2 considering the worst case scenarios?

A a1 < a2

a1 > a2

a1 = a2

D Depends on the input

Divide and Conquer **GATE 2017 Mock**
Discuss it

Question 9 Explanation:
When Divide and Conquer is used to find the minimum-maximum element in an array, Recurrence relation for the number of comparisons is
T(n) = 2T(n/2) + 2 where 2 is for comparing the minimums as well the maximums of the left and right subarrays
On solving, T(n) = 1.5n - 2.
While doing linear scan, it would take 2*(n-1) comparisons in the worst case to find both minimum as well maximum in one pass.

Question 1

Correct

Which of the following standard algorithms is not a Greedy algorithm?

A Dijkstra's shortest path algorithm

B Prim's algorithm

C Kruskal algorithm

D Huffman Coding

Bellmen Ford Shortest path algorithm

Greedy Algorithms
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/ and http://www.geeksforgeeks.org/dynamic-programming-set-23-bellman-ford-algorithm/

Question 3

Wrong

A networking company uses a compression technique to encode the message before transmitting over the network. Suppose the message contains the following characters with their frequency:

character	Frequency
a	5
b	9
c	12
d	13
e	16
f	45

Note : Each character in input message takes 1 byte. If the compression technique used is Huffman Coding, how many bits will be saved in the message?

A 224

800

576

D 324

Greedy Algorithms
Discuss it

Question 3 Explanation:

```
Total number of characters in the message = 100.
Each character takes 1 byte. So total number of bits needed = 800.

After Huffman Coding, the characters can be represented with:
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
Total number of bits needed = 224
Hence, number of bits saved = 800 - 224 = 576
See here for complete explanation and algorithm.
```

Question 4

Correct

What is the time complexity of Huffman Coding?

A O(N)

O(NlogN)

C O(N(logN)^2)

D O(N^2)

Greedy Algorithms
Discuss it

Question 4 Explanation:
O(nlogn) where n is the number of unique characters. If there are n nodes, extractMin() is called 2*(n – 1) times. extractMin() takes O(logn) time as it calles minHeapify(). So, overall complexity is O(nlogn). See here for more details of the algorithm.

Question 5

Wrong

In question #2, which of the following represents the word *"dead"*?

A 1011111100101

B 0100000011010

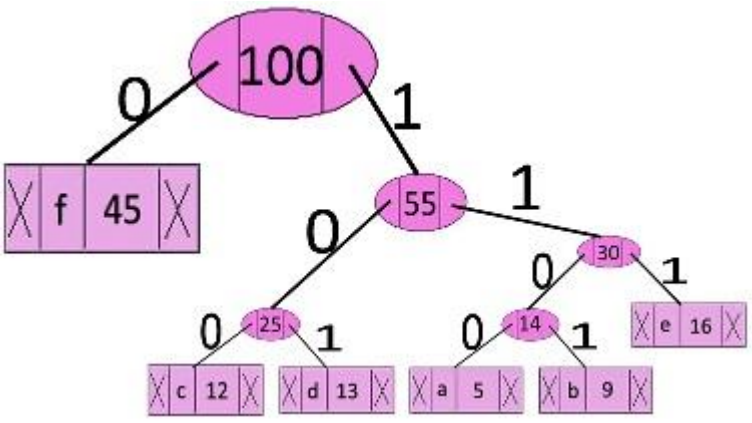
Both A and B

None of these

Greedy Algorithms
Discuss it

Question 5 Explanation:

The *Huffman Tree* generated is:



```
character   code-word
    f          0
    c          100
    d          101
    a          1100
    b          1101
    e          111
```

The word *dead* can be represented as: 101 111 1100 101 However, the alternative codeword can also be found by assigning 1 to the left edge and 0 to the right edge of the tree, i.e. *dead* can also be represented as: 010 000 0011 010 See here for more details of the algorithm.

Question 6

Wrong

Which of the following is true about Kruskal and Prim MST algorithms? Assume that Prim is implemented for adjacency list representation using Binary Heap and Kruskal is implemented using union by rank.

Worst case time complexity of both algorithms is same.

B Worst case time complexity of Kruskal is better than Prim

Worst case time complexity of Prim is better than Kruskal

Greedy Algorithms
Discuss it

Question 6 Explanation:
See following Kruskal's Minimum Spanning Tree Algorithm Prim's MST for Adjacency List Representation

Question 7

Wrong

Which of the following is true about Huffman Coding.

A Huffman coding may become lossy in some cases

B Huffman Codes may not be optimal lossless codes in some cases

In Huffman coding, no code is prefix of any other code.

All of the above

Greedy Algorithms
Discuss it

Question 7 Explanation:
Huffman coding is a lossless data compression algorithm. The codes assigned to input characters are Prefix Codes, means the codes are assigned in such a way that the code assigned to one character is not prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding.

Question 8

Correct

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. Which of the following is the Huffman code for the letter a, b, c, d, e, f?

0, 10, 110, 1110, 11110, 11111

B 11, 10, 011, 010, 001, 000

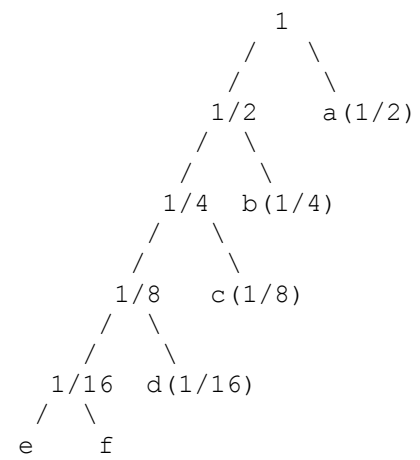
C 11, 10, 01, 001, 0001, 0000

D 110, 100, 010, 000, 001, 111

Greedy Algorithms **GATE-CS-2007**
Discuss it

Question 8 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.
```



Question

9

Question 9

Wrong

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. What is the average length of Huffman codes?

3

B 2.1875

C 2.25

1.9375

Greedy Algorithms **GATE-CS-2007**
Discuss it

Question 9 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.

                  1
                /   \
               /     \
             1/2     a(1/2)
            /  \
           /    \
         1/4   b(1/4)
        /   \
       /     \
     1/8    c(1/8)
     /  \
    /    \
  1/16   d(1/16)
  /  \
 e    f
The average length = (1*1/2 + 2*1/4 + 3*1/8 + 4*1/16 + 5*1/32 + 5*1/32)
                   = 1.9375
```

Question 11

Wrong

Consider the weights and values of items listed below. Note that there is only one unit of each item.

Item number	Weight (in Kgs)	Value (in Rupees)
1	10	60
2	7	28
3	4	20
4	2	24

The task is to pick a subset of these items such that their total weight is no more than 11 Kgs and their total value is maximized. Moreover, no item may be split. The total value of items picked by an optimal algorithm is denoted by V_{opt}. A greedy algorithm sorts the items by their value-to-weight ratios in descending order and packs them greedily, starting from the first item in the ordered list. The total value of items picked by the greedy algorithm is denoted by V_{greedy}. The value of $V_{opt} - V_{greedy}$ is _____ .

Note -This was Numerical Type question.

16

8

C 44

D 60

Greedy Algorithms Dynamic Programming GATE CS 2018

Discuss it

Question 11 Explanation:

Item No	Weight	Value	Value/Weight
1	10	60	6
2	7	28	4
3	4	20	5
4	2	24	12

After sorting :

Item No	Weight	Value	Value/Weight
4	2	24	12
1	10	60	6
3	4	20	5
2	7	28	4

First we will pick item_4 (Value weight ratio is highest). Second highest is item_1, but cannot be picked because of its weight. Now item_3 shall be picked. item_2 cannot be included because of its weight. Therefore, overall profit by V_{greedy} = 20+24 = 44 Hence, $\mathbf{V_{opt} - V_{greedy} = 60\text{-}44 = 16}$ So, answer is 16.

Question 12

Wrong

A text is made up of the characters a, b, c, d, e each occurring with the probability 0.11, 0.40, 0.16, 0.09 and 0.24 respectively. The optimal Huffman coding technique will have the average length of:

A 2.40

2.16

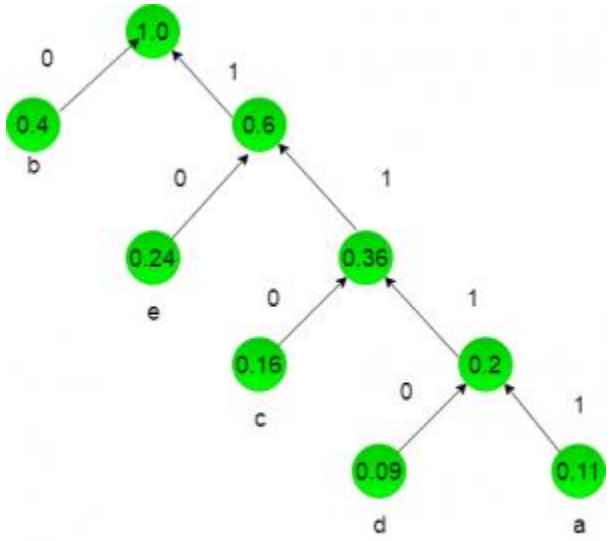
2.26

D 2.15

Greedy Algorithms UGC-NET CS 2017 Nov - III
Discuss it

Question 12 Explanation:

a = 0.11 b = 0.40 c = 0.16 d = 0.09 e = 0.24 we will draw a huffman tree:



Text now huffman coding for character:

```
 a = 1111
      b = 0
      c = 110
      d = 1111
      e = 10
lenghth for each character = no of bits * frequency of occurence:
a = 4 * 0.11
  = 0.44
b = 1 * 0.4
  =  0.4
c = 3 * 0.16
  = 0.48
d = 4 * 0.09
  =  0.36
e = 2 * 0.24
  = 0.48
Now add these lenght for average length:
 0.44 + 0.4 + 0.48 + 0.36 + 0.48 = 2.16
```

So, option (B) is correct.

Question 13

Correct

Six files F1, F2, F3, F4, F5 and F6 have 100, 200, 50, 80, 120, 150 records respectively. In what order should they be stored so as to optimize act. Assume each file is accessed with the same frequency

F3, F4, F1, F5, F6, F2

B F2, F6, F5, F1, F4, F3

C F1, F2, F3, F4, F5, F6

D Ordering is immaterial as all files are accessed with the same frequency.

Greedy Algorithms **ISRO CS 2015**
Discuss it

Question 13 Explanation:

```
This question is based on the Optimal Storage on Tape problem
which uses greedy approach to find the optimal time to retrieve them.
There are n programs of length L that are to be stored on a computer
tape. Associated with each program i is a length Li. So in order to
retrieve these programs most optimally, we need to store them in the
non-decreasing order of length Li.
So, the correct order is F3, F4, F1, F5, F6, F2
```

Question 14

Correct

Consider a job scheduling problem with 4 jobs J_1, J_2, J_3, J_4 and with corresponding deadlines: (d_1, d_2, d_3, d_4) = (4, 2, 4, 2). Which of the following is not a feasible schedule without violating any job schedule?

A J_2, J_4, J_1, J_3

J_4, J_1, J_2, J_3

C J_4, J_2, J_1, J_3

D J_4, J_2, J_3, J_1

Greedy Algorithms **ISRO CS 2007**
Discuss it

Question
15

Question 15

Correct

What is the other name of Dijkstra algorithm?

single-source shortest path problem

B multiple-source shortest path problem

C multiple-destination shortest path problem

D single-destination shortest path problem

Greedy Algorithms
Discuss it

Question 2

Correct

We use dynamic programming approach when

A We need an optimal solution

The solution has optimal substructure

C The given problem can be reduced to the 3-SAT problem

D It's faster than Greedy

Dynamic Programming
Discuss it

Question 2 Explanation:
http://www.geeksforgeeks.org/dynamic-programming-set-2-optimal-substructure-property/ Option (D) is incorrect because Greedy algorithms are generally faster than Dynamic programming. See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/

Question 3

Wrong

An algorithm to find the length of the longest monotonically increasing sequence of numbers in an array A[0 :n-1] is given below. Let Li denote the length of the longest monotonically increasing sequence starting at index i in the array.

Initialize $L_{n-1} = 1$

For all i such that $0 \le i \le n-2$

$$L_i = \begin{cases} 1 + L_{i+1} & \text{if } A[i] < A[i+1] \\ 1 & \text{Otherwise} \end{cases}$$

Finally the length of the longest monotonically increasing sequence is $\text{Max}(L_0, L_1, \ldots, L_{n-1})$.

Which of the following statements is TRUE?

The algorithm uses dynamic programming paradigm

The algorithm has a linear complexity and uses branch and bound paradigm

C The algorithm has a non-linear polynomial complexity and uses branch and bound paradigm

D The algorithm uses divide and conquer paradigm.

Dynamic Programming
Discuss it

Question 3 Explanation:
Please see http://www.geeksforgeeks.org/dynamic-programming-set-3-longest-increasing-subsequence/

Question 4

Wrong

Kadane algorithm is used to find:

Maximum sum subsequence in an array

Maximum sum subarray in an array

C Maximum product subsequence in an array

D Maximum product subarray in an array

Dynamic Programming
Discuss it

Question 4 Explanation:
Kadane algorithm is used to find the maximum sum subarray in an array. It runs in O(n) time complexity. See details of the algorithm here.

Question 5

Wrong

Four matrices M1, M2, M3 and M4 of dimensions pxq, qxr, rxs and sxt respectively can be multiplied is several ways with different number of total scalar multiplications. For example, when multiplied as ((M1 X M2) X (M3 X M4)), the total number of multiplications is pqr + rst + prt. When multiplied as (((M1 X M2) X M3) X M4), the total number of scalar multiplications is pqr + prs + pst. If p = 10, q = 100, r = 20, s = 5 and t = 80, then the number of scalar multiplications needed is

248000

B 44000

19000

D 25000

Dynamic Programming
Discuss it

Question 5 Explanation:
It is basically matrix chain multiplication problem. We get minimum number of multiplications using ((M1 X (M2 X M3)) X M4). Total number of multiplications = 100x20x5 (for M2 x M3) + 10x100x5 + 10x5x80 = 19000.

Question 6

Wrong

The subset-sum problem is defined as follows. Given a set of n positive integers, S = {a1 ,a2 ,a3 ,…,an} and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array X, with n rows and W+1 columns. X[i, j],1 <= i <= n, 0 <= j <= W, is TRUE if and only if there is a subset of {a1 ,a2 ,...,ai} whose elements sum to j. Which of the following is valid for 2 <= i <= n and ai <= j <= W?

A X[i, j] = X[i - 1, j] ∨ X[i, j -ai]

X[i, j] = X[i - 1, j] ∨ X[i - 1, j - ai]

C X[i, j] = X[i - 1, j] ∧ X[i, j - ai]

X[i, j] = X[i - 1, j] ∧ X[i -1, j - ai]

Dynamic Programming
Discuss it

Question 6 Explanation:
X[I, j] (2 <= i <= n and ai <= j <= W), is true if any of the following is true 1) Sum of weights excluding ai is equal to j, i.e., if X[i-1, j] is true. 2) Sum of weights including ai is equal to j, i.e., if X[i-1, j-ai] is true so that we get (j – ai) + ai as j See http://www.geeksforgeeks.org/dynamic-programming-subset-sum-problem/ for details.

Question 7

Wrong

In the above question, which entry of the array X, if TRUE, implies that there is a subset whose elements sum to W?

X[1, W]

B X[n ,0]

X[n, W]

D X[n -1, n]

Dynamic Programming
Discuss it

Question 7 Explanation:
If we get the entry X[n, W] as true then there is a subset of {a1, a2, .. an} that has sum as W.
Reference: http://en.wikipedia.org/wiki/Subset_sum_problem

Question 8

Wrong

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences X[m] and Y[n] of lengths m and n respectively, with indexes of X and Y starting from 0. We wish to find the length of the longest common sub-sequence(LCS) of X[m] and Y[n] as l(m,n), where an incomplete recursive definition for the function l(i,j) to compute the length of The LCS of X[m] and Y[n] is given below:

```
l(i,j) = 0, if either i=0 or j=0
       = expr1, if i,j > 0 and X[i-1] = Y[j-1]
       = expr2, if i,j > 0 and X[i-1] != Y[j-1]
```

expr1 ≡ l(i-1, j) + 1

B expr1 ≡ l(i, j-1)

expr2 ≡ max(l(i-1, j), l(i, j-1))

D expr2 ≡ max(l(i-1,j-1),l(i,j))

Dynamic Programming **GATE-CS-2009**
Discuss it

Question 8 Explanation:
In Longest common subsequence problem, there are two cases for X[0..i] and Y[0..j]

```
1) The last characters of two strings match.
   The length of lcs is length of lcs of X[0..i-1] and Y[0..j-1]
2) The last characters don't match.
   The length of lcs is max of following two lcs values
   a) LCS of X[0..i-1] and Y[0..j]
   b) LCS of X[0..i] and Y[0..j-1]
```

Question 9

Wrong

Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B. Then x + 10y = ___.

A 33

B 23

43

34

Dynamic Programming GATE-CS-2014-(Set-2)
Discuss it

Question 9 Explanation:
//The LCS is of length 4. There are 3 LCS of length 4 "qprr", "pqrr" and qpqr A subsequence is a sequence that can be derived from another sequence by selecting zero or more elements from it, without changing the order of the remaining elements. Subsequence need not be contiguous. Since the length of given strings A = “qpqrr” and B = “pqprqrp” are very small, we don’t need to build a 5x7 matrix and solve it using dynamic programming. Rather we can solve it manually just by brute force. We will first check whether there exist a subsequence of length 5 since min_length(A,B) = 5. Since there is no subsequence , we will now check for length 4. “qprr”, “pqrr” and “qpqr” are common in both strings. X = 4 and Y = 3 X + 10Y = 34 This solution is contributed by **Pranjul Ahuja**

Question 10

Correct

Let A1, A2, A3, and A4 be four matrices of dimensions 10 x 5, 5 x 20, 20 x 10, and 10 x 5, respectively. The minimum number of scalar multiplications required to find the product A1A2A3A4 using the basic matrix multiplication method is

1500

B 2000

C 500

D 100

Dynamic Programming GATE-CS-2016 (Set 2)
Discuss it

Question 10 Explanation:
We have many ways to do matrix chain multiplication because matrix multiplication is associative. In other words, no matter how we parenthesize the product, the result of the matrix chain multiplication obtained will remain the same. Here we have four matrices A1, A2, A3, and A4, we would have: ((A1A2)A3)A4 = ((A1(A2A3))A4) = (A1A2)(A3A4) = A1((A2A3)A4) = A1(A2(A3A4)). However, the order in which we parenthesize the product affects the number of simple arithmetic operations needed to compute the product, or the efficiency. Here, A1 is a 10 × 5 matrix, A2 is a 5 x 20 matrix, and A3 is a 20 x 10 matrix, and A4 is 10 x 5. If we multiply two matrices A and B of order l x m and m x n respectively,then the number of scalar multiplications in the multiplication of A and B will be lxmxn. Then, The number of scalar multiplications required in the following sequence of matrices will be : A1((A2A3)A4) = (5 x 20 x 10) + (5 x 10 x 5) + (10 x 5 x 5) = 1000 + 250 + 250 = 1500. All other parenthesized options will require number of multiplications more than 1500.

Question 12

Wrong

Consider the following two sequences :

X = < B, C, D, C, A, B, C >, and
Y = < C, A, D, B, C, B >
The length of longest common subsequence of X and Y is :

A 5

3

4

D 2

Dynamic Programming UGC-NET CS 2017 Nov - III
Discuss it

Question 13

Correct

Consider a sequence F_{00} defined as : $F_{00}(0) = 1$, $F_{00}(1) = 1$ $F_{00}(n) = 10 * F_{00}(n - 1) + 100$ $F_{00}(n - 2)$ for $n \geq 2$ Then what shall be the set of values of the sequence F_{00} ?

(1, 110, 1200)

B (1, 110, 600, 1200)

C (1, 2, 55, 110, 600, 1200)

D (1, 55, 110, 600, 1200)

Dynamic Programming **UGC NET CS 2017 Jan - II**
Discuss it

Question 13 Explanation:

```
 F00(0) = 1, F00(1) = 1
 F00(n) = 10 * F00(n - 1) + 100
 F00(2) = 10 * F00(1) + 100
 = 10 * 1 + 100
 = 10 + 100
 = 110
Similarly:
F00(3) = 10 * F00(2) + 100
 = 10 * 110 + 100
 = 1100 + 100
 = 1200
The sequence will be (1, 110, 1200).
```

So, (A) will be the answer.

Question 14

Correct

The following paradigm can be used to find the solution of the problem in minimum time: Given a set of non-negative integer, and a value K, determine if there is a subset of the given set with sum equal to K:

A Divide and Conquer

Dynamic Programming

C Greedy Algorithm

D Branch and Bound

Dynamic Programming **ISRO CS 2018**
Discuss it

Question 14 Explanation:
Given problem is Subset-sum problem in which a set of non-negative integers, and a value sum is given, to determine if there is a subset of the given set with sum equal to given sum. With recursion technique, time complexity of the above problem is exponential. We can solve the problem in Pseudo-polynomial time using Dynamic programming. Refer: Subset Sum Problem Option (B) is correct

Question 15

Wrong

What happens when a top-down approach of dynamic programming is applied to any problem?

It increases both, the time complexity and the space complexity

It increases the space complexity and decreases the time complexity.

C It increases the time complexity and decreases the space complexity

D It decreases both, the time complexity and the space complexity

Dynamic Programming
Discuss it

Question 1

Correct

Which of the following algorithms can be used to most efficiently determine the presence of a cycle in a given graph ?

Depth First Search

B Breadth First Search

C Prim's Minimum Spanning Tree Algorithm

D Kruskal' Minimum Spanning Tree Algorithm

Graph Traversals
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/applications-of-depth-first-search/ and
http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 2

Correct

Traversal of a graph is different from tree because

There can be a loop in graph so we must maintain a visited flag for every vertex

B DFS of a graph uses stack, but inorrder traversal of a tree is recursive

C BFS of a graph uses queue, but a time efficient BFS of a tree is recursive.

D All of the above

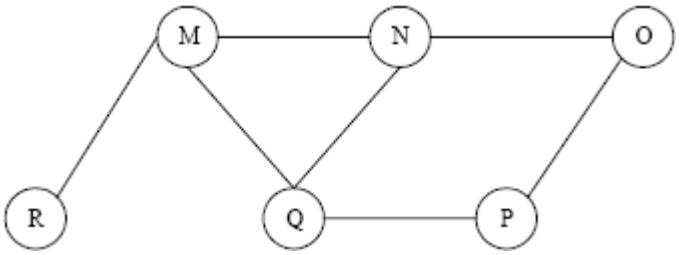
Graph Traversals
Discuss it

Question 2 Explanation:
See http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 4

Correct

The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is



A MNOPQR

B NQMPOR

QMNPRO

D QMNPOR

Graph Traversals
Discuss it

Question 4 Explanation:
Option (A) is MNOPQR. It cannot be a BFS as the traversal starts with M, but O is visited before N and Q. In BFS all adjacent must be visited before adjacent of adjacent. Option (B) is NQMPOR. It also cannot be BFS, because here, P is visited before O. (C) and (D) match up to QMNP. We see that M was added to the queue before N and P (because M comes before NP in QMNP). Because R is M's neighbor, it gets added to the queue before the neighbor of N and P (which is O). Thus, R is visited before O.

Question 5

Wrong

Let G be an undirected graph. Consider a depth-first traversal of G, and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statements is always true? (GATE CS 2000)

A {u,v} must be an edge in G, and u is a descendant of v in T

B {u,v} must be an edge in G, and v is a descendant of u in T

If {u,v} is not an edge in G then u is a leaf in T

If {u,v} is not an edge in G then u and v must have the same parent in T

Graph Traversals
Discuss it

Question 5 Explanation:

```
In DFS, if 'v' is visited
after 'u', then one of the following is true.
1) (u, v) is an edge.
      u
    /   \
   v     w
  /     / \
 x     y   z

2) 'u' is a leaf node.
      w
    /   \
   x     v
  /     / \
 u     y   z
```

In DFS, after visiting a node, we first recur for all unvisited children. If there are no unvisited children (u is leaf), then control goes back to parent and parent then visits next unvisited children.

Question 7

Correct

Make is a utility that automatically builds executable programs and libraries from source code by reading files called makefiles which specify how to derive the target program. Which of the following standard graph algorithms is used by Make.

A Strongly Connected Components

Topological Sorting

C Breadth First Search

D Dijkstra's Shortest Path

Graph Traversals
Discuss it

Question 7 Explanation:
Make can decide the order of building a software using topological sorting. Topological sorting produces the order considering all dependencies provide by makefile. See following for details.
Topological Sorting

Question 9

Correct

Which of the following condition is sufficient to detect cycle in a directed graph?

A There is an edge from currently being visited node to an already visited node.

There is an edge from currently being visited node to an ancestor of currently visited node in DFS forest.

C Every node is seen twice in DFS.

D None of the bove

Graph Traversals
Discuss it

Question 9 Explanation:
See Detect Cycle in a Graph

Question 10

Wrong

Is following statement true/false If a DFS of a directed graph contains a back edge, any other DFS of the same graph will also contain at least one back edge. Source:
http://courses.csail.mit.edu/6.006/oldquizzes/solutions/q2-s2009-sol.pdf

True

False

Graph Traversals
Discuss it

Question 10 Explanation:
A back edge means a cycle in graph. So if there is a cycle, all DFS traversals would contain at least one back edge.

Question 14

Wrong

Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time on Depth First Search of G? Assume that the graph is represented using adjacency matrix.

A O(n)

O(m+n)

O(n^2)

D O(mn)

Graph Traversals **GATE-CS-2014-(Set-1)**
Discuss it

Question 14 Explanation:
Depth First Search of a graph takes O(m+n) time when the graph is represented using adjacency list. In adjacency matrix representation, graph is represented as an "n x n" matrix. To do DFS, for every vertex, we traverse the row corresponding to that vertex to find all adjacent vertices (In adjacency list representation we traverse only the adjacent vertices of the vertex). Therefore time complexity becomes O(n^2)

Question 15

Correct

Consider the tree arcs of a BFS traversal from a source node W in an unweighted, connected, undirected graph. The tree T formed by the tree arcs is a data structure for computing.

A the shortest path between every pair of vertices.

the shortest path from W to every vertex in the graph.

C the shortest paths from W to only those nodes that are leaves of T.

D the longest path in the graph

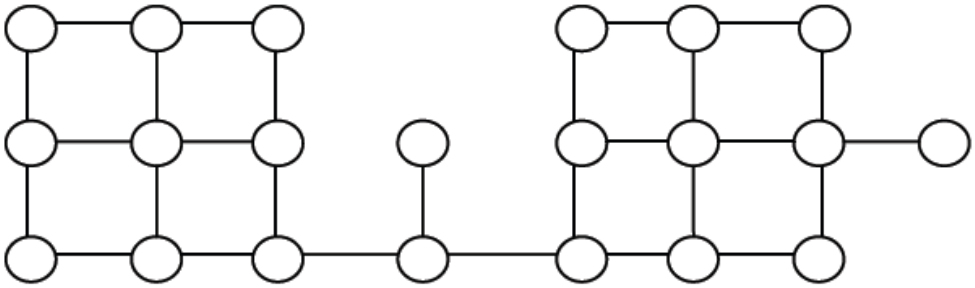
Graph Traversals GATE-CS-2014-(Set-2)
Discuss it

Question 15 Explanation:
BFS always produces shortest path from source to all other vertices in an unweighted graph.

Question 16

Wrong

Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is



_____.

A 17

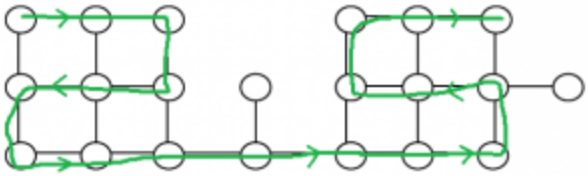
B 18

19

20

Graph Traversals GATE-CS-2014-(Set-3)
Discuss it

Question 16 Explanation:
The following diagram shows the worst case situation where the recursion tree has maximum depth.



So the recursion depth is 19 (including the first node).

Question 17

Wrong

Let T be a depth first search tree in an undirected graph G. Vertices u and n are leaves of this tree T. The degrees of both u and n in G are at least 2. which one of the following statements is true?

There must exist a vertex w adjacent to both u and n in G

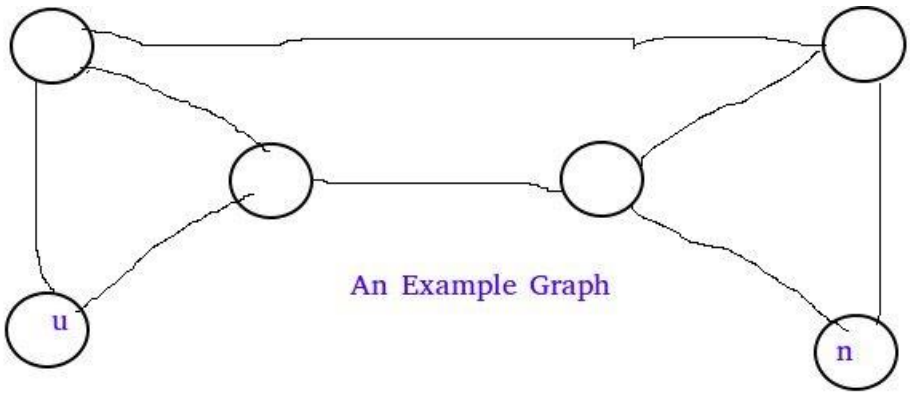
B There must exist a vertex w whose removal disconnects u and n in G

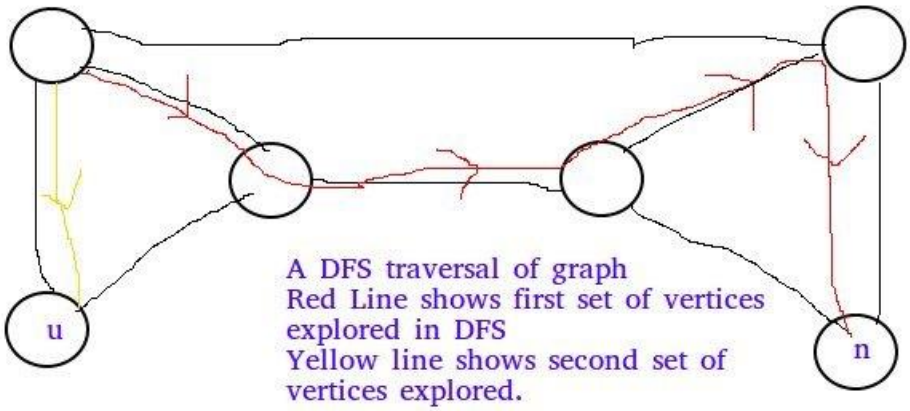
C There must exist a cycle in G containing u and n

There must exist a cycle in G containing u and all its neighbours in G.

Graph Traversals **GATE-CS-2006**
Discuss it

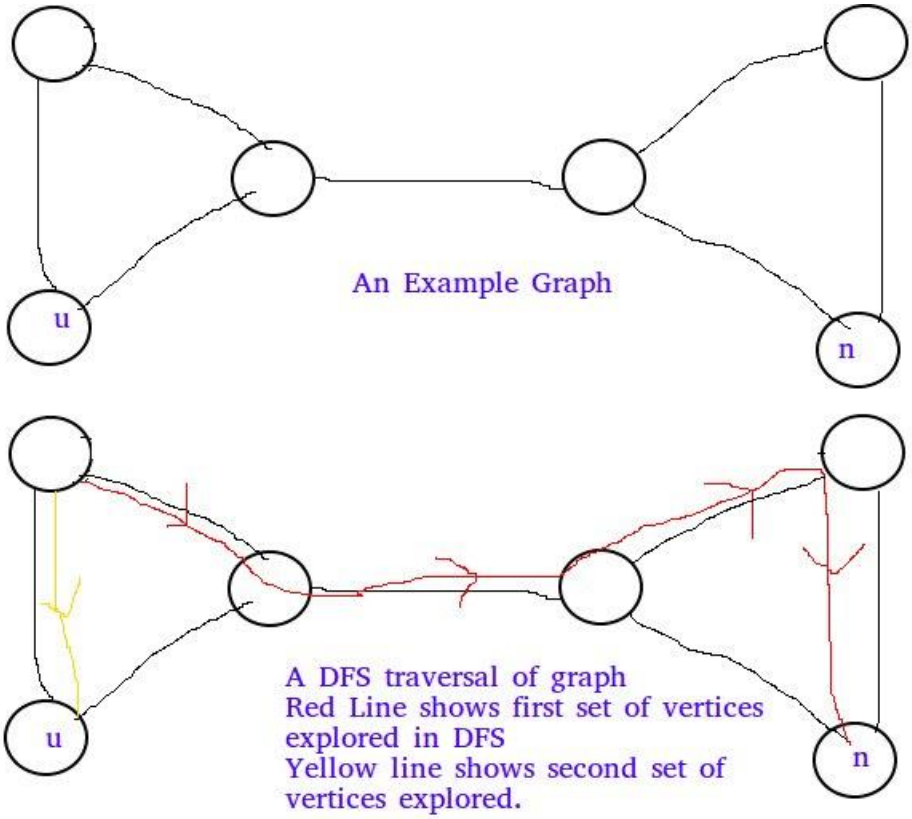
Question 17 Explanation:
Below example shows that A and B are FALSE:





Below

example shows C is false:



Question
18

Question 19

Correct

In a depth-first traversal of a graph G with n vertices, k edges are marked as tree edges. The number of connected components in G is

A k

B k + 1

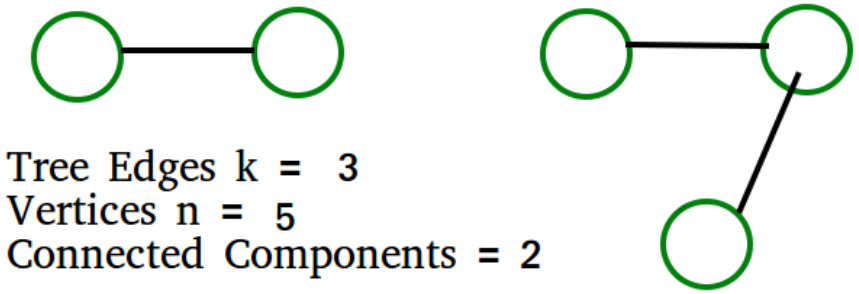
C n - k - 1

n - k

Graph Traversals **Gate IT 2005**
Discuss it

Question 19 Explanation:

Tree edges are the edges that are part of DFS tree. If there are x tree edges in a tree, then x+1 vertices in the tree. The output of DFS is a forest if the graph is disconnected. Let us see below simple example where graph is disconnected.

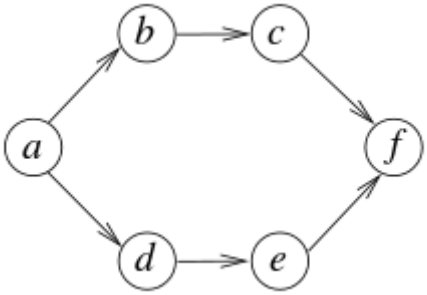


The above example matches with D option **More Examples:** 1) All vertices of Graph are connected. k must be n-1. We get number of connected components = n- k = n - (n-1) = 1 2) No vertex is connected. k must be 0. We get number of connected components = n- k = n - 0 = n

Question
20

Question 20

Wrong



Consider the following directed graph. The number of different topological orderings of the vertices of the graph is Note : This question was asked as Numerical Answer Type.

A 1

B 2

4

6

Graph Traversals GATE-CS-2016 (Set 1)
Discuss it

Question 20 Explanation:

Following are different 6 Topological Sortings

```
a-b-c-d-e-f
a-d-e-b-c-f
a-b-d-c-e-f
a-d-b-c-e-f
a-b-d-e-c-f
a-d-b-e-c-f
```

Question 21

Correct

In an adjacency list representation of an undirected simple graph G = (V, E), each edge (u, v) has two adjacency list entries: [v] in the adjacency list of u, and [u] in the adjacency list of v. These are called twins of each other. A twin pointer is a pointer from an adjacency list entry to its twin. If |E| = m and |V | = n, and the memory size is not a constraint, what is the time complexity of the most efficient algorithm to set the twin pointer in each entry in each adjacency list?

A $\Theta(n^2)$

$\Theta(m+n)$

C $\Theta(m^2)$

D $\Theta(n^4)$

Graph Traversals **GATE-CS-2016 (Set 2)**
Discuss it

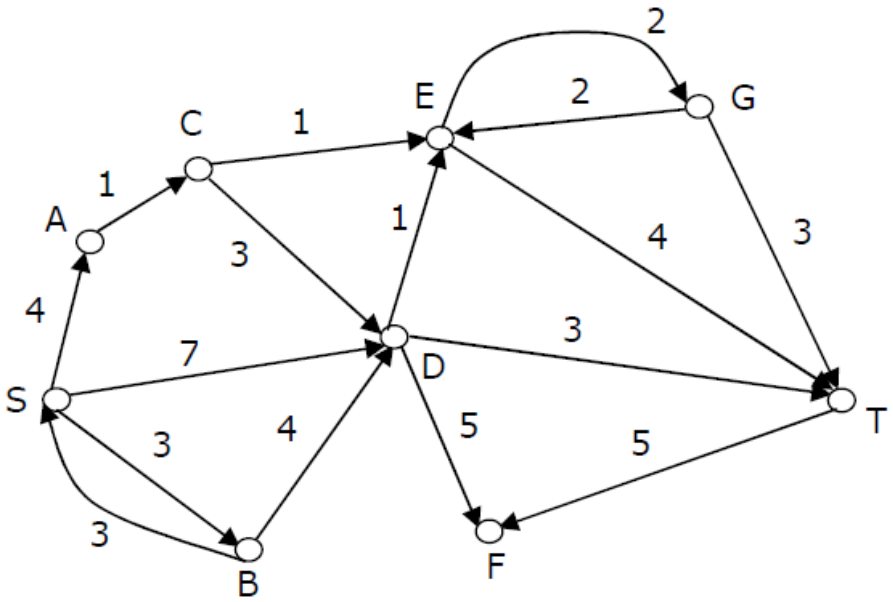
Question 21 Explanation:
First you need to find twins of each node. You can do this using **level order traversal (i.e., BFS)** once. Time complexity of BFS is Θ(m +n). And you have to use linked list for representation which is extra space (but memory size is not a constraint here). Final, time complexity is Θ(m + n) to set twin pointer. Option (B) is correct.

Question 1

Wrong

Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices S and T. Which one will be reported by Dijstra?s shortest path algorithm? Assume that, in any iteration, the shortest path to a vertex v is updated only when a strictly shorter path to v is

discovered.



A SDT

SBDT

C SACDT

SACET

Graph Shortest Paths
Discuss it

Question 1 Explanation:
See Dijkstra's shortest path algorithm When the algorithm reaches vertex 'C', the distance values of 'D' and 'E' would be 7 and 6 respectively. So the next picked vertex would be 'E'

Question 2

Wrong

To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is:

Queue

B Stack

Heap

D B-Tree

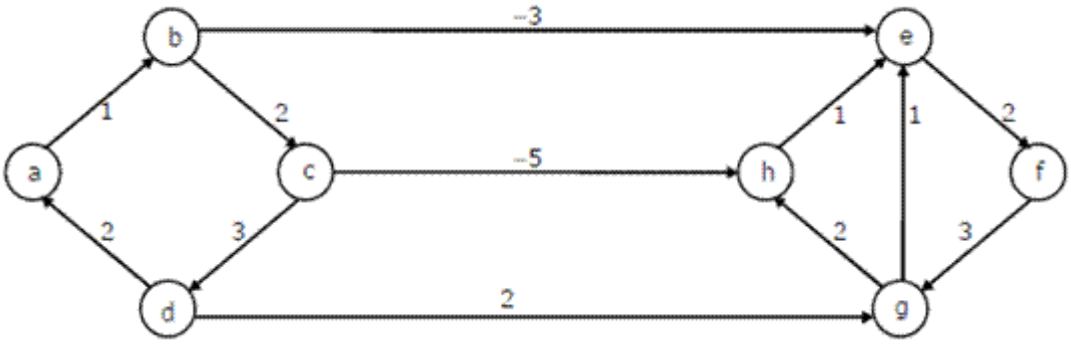
Graph Shortest Paths
Discuss it

Question 2 Explanation:
The shortest path in an un-weighted graph means the smallest number of edges that must be traversed in order to reach the destination in the graph. This is the same problem as solving the weighted version where all the weights happen to be 1. If we use Queue (FIFO) instead of Priority Queue (Min Heap), we get the shortest path in linear time O(|V| + |E|). Basically we do BFS traversal of the graph to get the shortest paths.

Question 3

Wrong

Dijkstra's single source shortest path algorithm when run from vertex a in the below graph, computes the correct shortest path distance to



A only vertex a

B only vertices a, e, f, g, h

only vertices a, b, c, d

all the vertices

Graph Shortest Paths
Discuss it

Question 3 Explanation:
Dijkstra's single source shortest path is not guaranteed to work for graphs with negative weight edges, but it works for the given graph. Let us see... Let us run the 1st pass b 1 b is minimum, so shortest distance to b is 1. After 1st pass, distances are c 3, e -2. e is minimum, so shortest distance to e is -2 After 2nd pass, distances are c 3, f 0. f is minimum, so shortest distance to f is 0 After 3rd pass, distances are c 3, g 3. Both are same, let us take g. so shortest distance to g is 3. After 4th pass,

distances are c 3, h 5 c is minimum, so shortest distance to c is 3 After 5th pass, distances are h -2 h is minimum, so shortest distance to h is -2

Question 7

Wrong

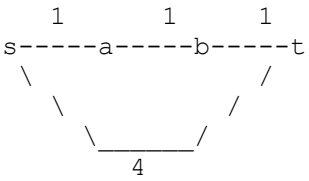
In a weighted graph, assume that the shortest path from a source 's' to a destination 't' is correctly calculated using a shortest path algorithm. Is the following statement true? If we increase weight of every edge by 1, the shortest path always remains same.

Yes

No

Graph Shortest Paths
Discuss it

Question 7 Explanation:
See the following counterexample. There are 4 edges s-a, a-b, b-t and s-t of wights 1, 1, 1 and 4 respectively. The shortest path from s to t is s-a, a-b, b-t. IF we increase weight of every edge by 1, the shortest path changes to s-t.



Question 8

Wrong

Following statement is true or false?

```
If we make following changes to  Dijkstra, then it can be used to find
the longest simple path, assume that the graph is acyclic.

1) Initialize all distances as minus infinite instead of plus infinite.

2) Modify the relax condition in  Dijkstra's algorithm to update distance
  of an adjacent v of the currently considered vertex u only
  if "dist[u]+graph[u][v] > dist[v]". In shortest path algo,
  the sign is opposite.
```

True

False

Graph Shortest Paths
Discuss it

Question 8 Explanation:
In shortest path algo, we pick the minimum distance vertex from the set of vertices for which distance is not finalized yet. And we finalize the distance of the minimum distance vertex. For maximum distance problem, we cannot finalize the distance because there can be a longer path through not yet finalized vertices.

Question 9

Wrong

Which of the following algorithm can be used to **efficiently** calculate single source shortest paths in a Directed Acyclic Graph?

Dijkstra

B Bellman-Ford

Topological Sort

D Strongly Connected Component

Graph Shortest Paths
Discuss it

Question 9 Explanation:
Using Topological Sort, we can find single source shortest paths in O(V+E) time which is the most efficient algorithm. See following for details. Shortest Path in Directed Acyclic Graph

Question 11

Wrong

Is the following statement valid about shortest paths? *Given a graph, suppose we have calculated shortest path from a source to all other vertices. If we modify the graph such that weights of all edges is becomes double of the original weight, then the shortest path remains same only the total weight of path changes.*

True

False

Graph Shortest Paths
Discuss it

Question 11 Explanation:
The shortest path remains same. It is like if we change unit of distance from meter to kilo meter, the shortest paths don't change.

Question 12

Wrong

Match the following

```
Group A                                                  Group B
a) Dijkstra's single shortest path algo                  p) Dynamic
Programming
b) Bellmen Ford's single shortest path algo              q) Backtracking
c) Floyd Warshell's all pair shortest path algo.         r) Greedy Algorithm
```

a-r, b-q, c-p

B a-p, b-p, c-p

a-r, b-p, c-p

D a-p, b-r, c-q

Graph Shortest Paths
Discuss it

Question 12 Explanation:
Dijkstra is a greedy algorithm where we pick the minimum distant vertex from not yet finalized vertices. Bellman Ford and Floyd Warshell both are Dynamic Programming algorithms where we build the shortest paths in bottom up manner.

Question
13

Question 13

Wrong

Is the following statement valid?.

Given a weighted graph where weights of all edges are unique (no two edge have same weights), there is always a unique shortest path from a source to destination in such a graph.

True

False

Graph Shortest Paths
Discuss it

Question 13 Explanation:
There can be more than one paths with same weight. Consider a path with one edge of weight 5 and another path with two edges of weights 2 and 3. Both paths have same weights.

Question
14

Question 14

Correct

Is the following statement valid?.

Given a graph where all edges have positive weights, the shortest paths produced by Dijsktra and Bellman Ford algorithm may be different but path weight would always be same.

True

B False

Graph Shortest Paths
Discuss it

Question 14 Explanation:
Dijkstra and Bellman-Ford both work fine for a graph with all positive weights, but they are different algorithms and may pick different edges for shortest paths.

Question 15

Correct

Which of the following statement(s)is / are correct regarding Bellman-Ford shortest path algorithm?

```
P: Always finds a negative weighted cycle, if one exist s.
Q: Finds whether any negative weighted cycle is reachable
   from the source.
```

A P Only

Q Only

C Both P and Q

D Neither P nor Q

Graph Shortest Paths **GATE-CS-2009**
Discuss it

Question 15 Explanation:
Bellman-Ford shortest path algorithm is a single source shortest path algorithm. So it can only find cycles which are reachable from a given source, not any negative weight cycle. Consider a disconnected where a negative weight cycle is not reachable from the source at all. If there is a negative weight cycle, then it will appear in shortest path as the negative weight cycle will always form a shorter path when iterated through the cycle again and again.

Question 16

Wrong

Let G(V, E) an undirected graph with positive edge weights. Dijkstra's single-source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

A O(| V |2)

B O (| E | + | V | log | V |)

O (| V | log | V |)

O ((| E | + | V |) log | V |)

Graph Shortest Paths **GATE-CS-2005**
Discuss it

Question 16 Explanation:
See Time Complexity Section of http://www.geeksforgeeks.org/greedy-algorithms-set-7-dijkstras-algorithm-for-adjacency-list-representation/

Question
17

Question 17

Wrong

Let G = (V, E) be an undirected graph with a subgraph G1 = (V1, El). Weights are assigned to edges of G as follows :

$$w(e) = \begin{cases} 0 \text{ if } e \in E_1 \\ 1 \text{ otherwise} \end{cases}$$

A single-source shortest path algorithm is executed on the weighted graph (V, E, w) with an arbitrary vertex v1 of V1 as the source. Which of the following can always be inferred from the path costs computed?

The number of edges in the shortest paths from v1 to all vertices of G

G1 is connected

C V1 forms a clique in G

D G1 is a tree

Graph Shortest Paths **GATE-CS-2003**
Discuss it

Question 17 Explanation:
When shortest path shortest path from v1 (one of the vertices in V1) is computed. G1 is connected if the distance from v1 to any other vertex in V1 is greater than 0, otherwise G1 is disconnected.

Question 19

Correct

Let G = (V, E) be a simple undirected graph, and s be a particular vertex in it called the source. For x ∈ V, let d(x) denote the shortest distance in G from s to x. A breadth first search (BFS) is performed starting at s. Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T, then which one of the following CANNOT be the value of d(u) – d(v)?

A -1

B 0

C 1

2

Graph Shortest Paths **GATE-CS-2015 (Set 1)**
Discuss it

Question 19 Explanation:
Note that the given graph is undirected, so an edge (u, v) also means (v, u) is also an edge. Since a shorter path can always be obtained by using edge (u, v) or (v, u), the difference between d(u) and d(v) can not be more than 1.

Question 20

Wrong

Let G be a directed graph whose vertex set is the set of numbers from 1 to 100. There is an edge from a vertex i to a vertex j iff either j = i + 1 or j = 3i. The minimum number of edges in a path in G from vertex 1 to vertex 100 is

A 4

7

23

D 99

Graph Shortest Paths **Gate IT 2005**
Discuss it

Question 20 Explanation:
The task is to find minimum number of edges in a path in G from vertex 1 to vertex 100 such that we can move to either i+1 or 3i from a vertex i.

```
Since the task is to minimize number of edges,
we would prefer to follow 3*i.

Let us follow multiple of 3.

1 => 3 => 9 => 27 => 81, now we can't follow multiple
of 3. So we will have to follow i+1. This solution gives
a long path.

What if we begin from end, and we reduce by 1 if
the value is not multiple of 3, else we divide by 3.
100 => 99 => 33 => 11 => 10 => 9 => 3 => 1

So we need total 7 edges.
```

Question 21

Wrong

Consider the weighted undirected graph with 4 vertices, where the weight of edge {i, j} g is given by the entry W_{ij} in the matrix W

$$W = \begin{bmatrix} 0 & 2 & 8 & 5 \\ 2 & 0 & 5 & 8 \\ 8 & 5 & 0 & x \\ 5 & 8 & x & 0 \end{bmatrix}$$

The largest possible integer value of x, for which at least one shortest path between some pair of

vertices will contain the edge with weight x is _____
Note : This question was asked as Numerical Answer Type.

8

12

C 10

D 11

Graph Shortest Paths **GATE-CS-2016 (Set 1)**
Discuss it

Question 21 Explanation:
Let vertices be 0, 1, 2 and 3. x directly connects 2 to 3. The shortest path (excluding x) from 2 to 3 is of weight 12 (2-1-0-3).

Question 23

Wrong

Consider a weighted undirected graph with positive edge weights and let uv be an edge in the graph. It is known that the shortest path from the source vertex s to u has weight 53 and the shortest path from s to v has weight 65. Which one of the following statements is always true?

A weight (u, v) < 12

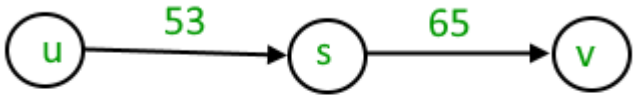
weight (u, v) ≤ 12

C weight (u, v) > 12

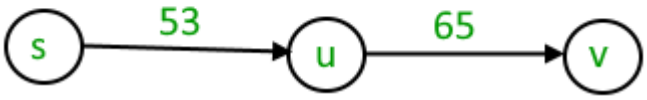
weight (u, v) ≥ 12

Graph Shortest Paths **Graph Theory** **Gate IT 2007**
Discuss it

Question 23 Explanation:



Weight(u,v) >=12



The minimum weight happens when (S,U) + (U,V) = (S,V) Else (S,U) + (U,V) >= (S,V) Given (S,U) = 53, (S,V) = 65 53 + (U,V) >= 63 (U,V) >= 12.
This solution is contributed by **Anil Saikrishna Devarasetty**

Question 27

Correct

Which of the following is application of Breath First Search on the graph?

A Finding diameter of the graph

B Finding bipartite graph

Both (a) and (b)

D None of the above

Graph Shortest Paths **ISRO CS 2018**
Discuss it

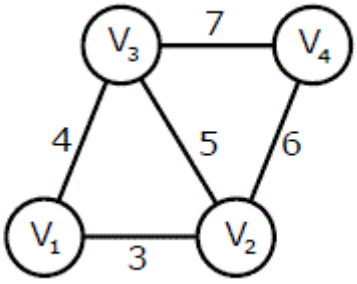
Question 27 Explanation:
BFS is used to Find the diameter of the graph and to test whether a graph is bipartite or not. BFS has many other applications also. Refer: Applications of Breadth First Traversal Option (C) is correct.

Question 1

Correct

An undirected graph G(V, E) contains n (n > 2) nodes named v1 , v2 ,….vn. Two nodes vi , vj are connected if and only if 0 < |i – j| <= 2. Each edge (vi, vj) is assigned a weight i + j. A sample graph

with n = 4 is shown below. What will be the cost of the minimum spanning tree (MST) of such a



graph with n nodes? (GATE CS 2011)

A 1/12(11n^2 – 5n)

n^2 – n + 1

C 6n – 11

D 2n + 1

Graph Minimum Spanning Tree
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/data-structures-and-algorithms-set-27/ for explanation.

Question 3

Wrong

Consider a complete undirected graph with vertex set {0, 1, 2, 3, 4}. Entry Wij in the matrix W below is the weight of the edge {i, j}. What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T? (GATE CS 2010)

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

A 7

B 8

9

10

Graph Minimum Spanning Tree
Discuss it

Question 3 Explanation:
To get the minimum spanning tree with vertex 0 as leaf, first remove 0th row and 0th column and then get the minimum spanning tree (MST) of the remaining graph. Once we have MST of the remaining graph, connect the MST to vertex 0 with the edge with minimum weight (we have two options as there are two 1s in 0th row).

Question 4

Wrong

In the graph given in above question question, what is the minimum possible weight of a path P from vertex 1 to vertex 2 in this graph such that P contains at most 3 edges?

A 7

8

C 9

10

Graph Minimum Spanning Tree
Discuss it

Question 4 Explanation:
Path: 1 -> 0 -> 4 -> 2 Weight: 1 + 4 + 3

Question 5

Correct

An undirected graph G has n nodes. Its adjacency matrix is given by an n × n square matrix whose (i) diagonal elements are 0‘s and (ii) non-diagonal elements are 1‘s. which one of the following is TRUE?

A Graph G has no minimum spanning tree (MST)

B Graph G has a unique MST of cost n-1

Graph G has multiple distinct MSTs, each of cost n-1

D Graph G has multiple spanning trees of different costs

Graph Minimum Spanning Tree
Discuss it

Question 5 Explanation:
See Question 2 of http://www.geeksforgeeks.org/data-structures-and-algorithms-set-22/

Question 7

Wrong

Let G be an undirected connected graph with distinct edge weight. Let emax be the edge with maximum weight and emin the edge with minimum weight. Which of the following statements is false? (GATE CS 2000)

A Every minimum spanning tree of G must contain emin

If emax is in a minimum spanning tree, then its removal must disconnect G

No minimum spanning tree contains emax

D G has a unique minimum spanning tree

Graph Minimum Spanning Tree
Discuss it

Question 7 Explanation:
(a) and (b) are always true. (c) is false because (b) is true. (d) is true because all edge weights are distinct for G.

Question 8

Correct

Consider a weighted complete graph G on the vertex set {v1,v2 ,v} such that the weight of the edge (v,,v) is 2|i-j|. The weight of a minimum spanning tree of G is: (GATE CS 2006)

A n — 1

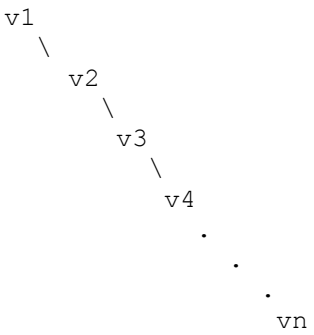
2n — 2

C nC2

D 2

Graph Minimum Spanning Tree
Discuss it

Question 8 Explanation:
Minimum spanning tree of such a graph is



Weight of the minimum spanning tree = 2|2 - 1| + 2|3 - 2| + 2|4 - 3| + 2|5 - 4| .... + 2| n - (n-1) | = 2n - 2

Question 9

Wrong

Let G be a weighted graph with edge weights greater than one and G'be the graph constructed by squaring the weights of edges in G. Let T and T' be the minimum spanning trees of G and G', respectively, with total weights t and t'. Which of the following statements is TRUE?

A T' = T with total weight t' = t^2

T' = T with total weight t' < t^2

C T' != T but total weight t' = t^2

None of the above

GATE CS 2012 **Graph Minimum Spanning Tree**
Discuss it

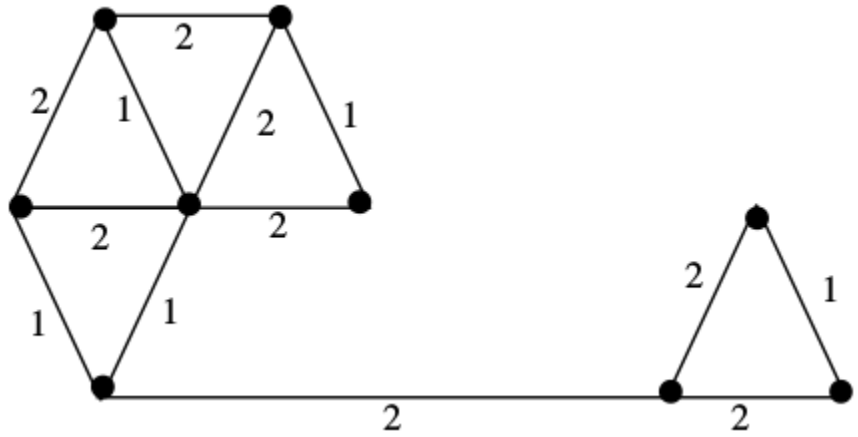
Question 9 Explanation:
Squaring the weights of the edges in a weighted graph will not change the minimum spanning tree. Assume the opposite to obtain a contradiction. If the minimum spanning tree changes then at least one edge from the old graph G in the old minimum spanning tree T must be replaced by a new edge in tree T' from the graph G' with squared edge weights. The new edge from G' must have a lower weight than the edge from G. This implies that there exists some weights C1 and C2 such that C1 < C2 and C12 >= C22. This is a contradiction.
Source: http://www.cs.nyu.edu/courses/spring06/V22.0310-001/hw3.htm Sums of squares of two or more numbers is always smaller than square of sum. Example: 2^2 + 2^2 < (4)^2 But

```
there is one counter example when the graph has only one edge.
         In that case, the two values are same.
```

Question 11

Wrong

The number of distinct minimum spanning trees for the weighted graph below is ____



4

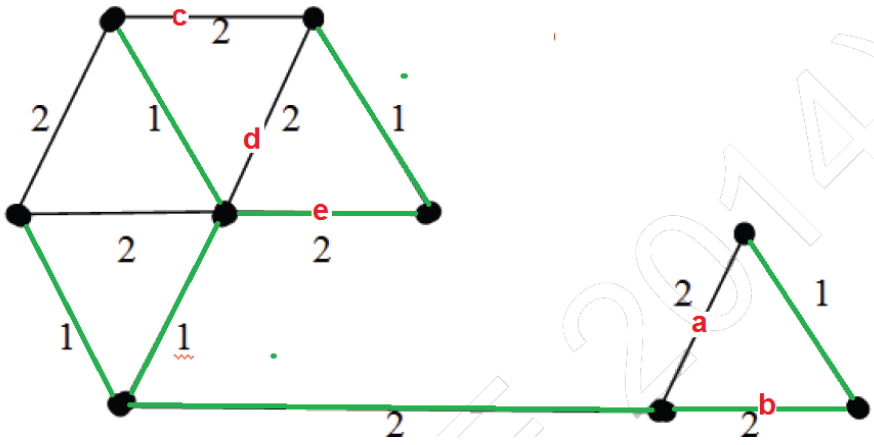
B 5

6

D 7

Graph Minimum Spanning Tree **GATE-CS-2014-(Set-2)**
Discuss it

Question 11 Explanation:
Below diagram shows a minimum spanning tree. Highlighted (in green) are the edges picked to make the MST.



In the right side of MST, we could either pick edge 'a' or 'b'. In the left side, we could either pick 'c' or 'd' or 'e' in MST. There are 2 options for one edge to be picked and 3 options for another edge to be picked. Therefore, total 2*3 possible MSTs.

Question 12

Correct

Let s and t be two vertices in a undirected graph G + (V, E) having distinct positive edge weights. Let [X, Y] be a partition of V such that s ∈ X and t ∈ Y. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y The edge e must definitely belong to:

the minimum weighted spanning tree of G

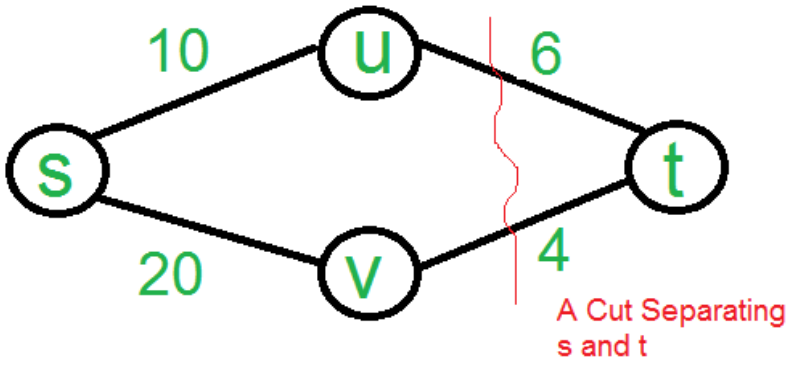
B the weighted shortest path from s to t

C each path from s to t

D the weighted longest path from s to t

Graph Minimum Spanning Tree **GATE-CS-2005**
Discuss it

Question 12 Explanation:
The minimum weight edge on any s-t cut is always part of MST. This is called Cut Property. This is the idea used in Prim's algorithm. The minimum weight cut edge is always a minimum spanning tree edge. **Why B (the weighted shortest path from s to t) is not an answer?** See below example, edge 4 (lightest in highlighted red cut from s to t) is not part of shortest path.



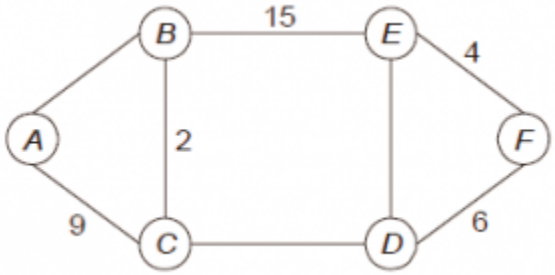
Question
13

Question 15

Wrong

The graph shown below 8 edges with distinct integer edge weights. The minimum spanning tree (MST) is of weight 36 and contains the edges: {(A, C), (B, C), (B, E), (E, F), (D, F)}. The edge weights of only those edges which are in the MST are given in the figure shown below. The minimum possible

sum of weights of all 8 edges of this graph is _____.



66

69

C 68

D 70

Graph Minimum Spanning Tree GATE-CS-2015 (Set 1)
Discuss it

Question 15 Explanation:
In every cycle, the weight of an edge that is not part of MST must by greater than or equal to weights of other edges which are part of MST. Since all edge weights are distinct, the weight must be greater. So the minimum possible weight of ED is 7, minimum possible weight of CD is 16 and minimum possible weight of AB is 10. Therefore minimum possible sum of weights is 69.

Question 16

Wrong

Let G be connected undirected graph of 100 vertices and 300 edges. The weight of a minimum spanning tree of G is 500. When the weight of each edge of G is increased by five, the weight of a minimum spanning tree becomes _____.

1000

995

C 2000

D 1995

Graph Minimum Spanning Tree GATE-CS-2015 (Set 3)
Discuss it

Question 16 Explanation:
Since there are 100 vertices, there must be 99 edges in Minimum Spanning Tree (MST). When weight of every edge is increased by 5, the increment in weight of MST is = 99 * 5 = 495 So new weight of MST is 500 + 495 which is 995

Question 17

Wrong

Let G be a weighted connected undirected graph with distinct positive edge weights. If every edge weight is increased by the same value, then which of the following statements is/are TRUE?

```
P: Minimum spanning tree of G does not change
Q: Shortest path between any pair of vertices does not change
```

P only

B Q only

C Neither P nor Q

Both P and Q

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 17 Explanation:
The shortest path may change. The reason is, there may be different number of edges in different paths from s to t. For example, let shortest path be of weight 15 and has 5 edges. Let there be another path with 2 edges and total weight 25. The weight of the shortest path is increased by 5*10 and becomes 15 + 50. Weight of the other path is increased by 2*10 and becomes 25 + 20. So the shortest path changes to the other path with weight as 45. The Minimum Spanning Tree doesn't change. Remember the Kruskal's algorithm where we sort the edges first. IF we increase all weights, then order of edges won't change.

Question
18

Question 18

Wrong

Let G be a complete undirected graph on 4 vertices, having 6 edges with weights being 1, 2, 3, 4, 5, and 6. The maximum possible weight that a minimum weight spanning tree of G can have is. [This Question was originally a Fill-in-the-Blanks question]

6

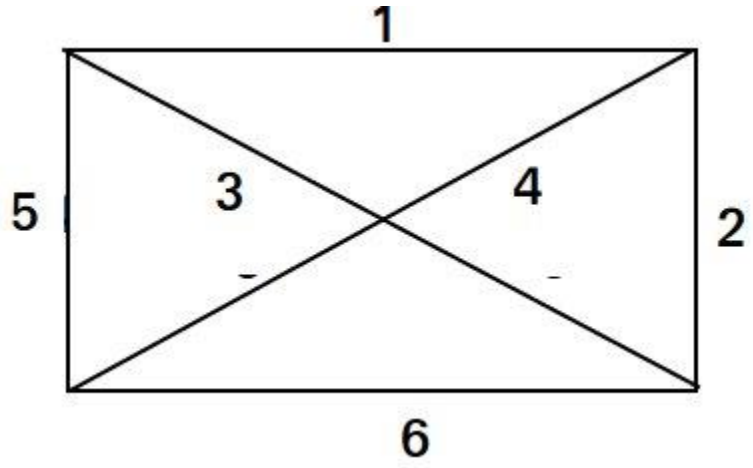
7

C 8

D 9

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 18 Explanation:
One graph that has maximum possible weight of spanning tree



Question
19

Question 19

Correct

G = (V, E) is an undirected simple graph in which each edge has a distinct weight, and e is a particular edge of G. Which of the following statements about the minimum spanning trees (MSTs) of G is/are TRUE

```
I.  If e is the lightest edge of some cycle in G,
    then every MST of G includes e
II. If e is the heaviest edge of some cycle in G,
    then every MST of G excludes e
```

A I only

II only

C both I and II

D neither I nor II

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 19 Explanation:
I is NOT true. Let G=(V, E) be a rectangular graph where V = {a, b, c, d} and E = {ab, bc, cd, da, ac}. Let the edges have weights: ab = 1, bc = 2, cd = 4, da = 5, ac = 3. Then, clearly, ac is the lightest edge of the cycle cdac, however, the MST abcd with cost 7 (= ab + bc + cd) does not include it. Let the edges have weights: ab = 6, bc - 7, cd = 4, da = 5, ac = 3. Then, again, ac is the lightest edge of the cycle cdac, and, the MST bacd with cost 13 (= ba + ac + cd) includes it. So, the MSTs of G may or may not include the lightest edge. **II is true** Let the heavies edge be e. Suppose the minimum spanning tree which contains e. If we add one more edge to the spanning tree we will create a cycle. Suppose we add edge e' to the spanning tree which generated cycle C. We can reduce the cost of the minimum spanning tree if we choose an edge other than e from C for removal which implies that e must not be in minimum spanning tree and we get a contradiction. Source:
http://www.ece.northwestern.edu/~dda902/336/hw5-sol.pdf

Question 20

Correct

What is the largest integer m such that every simple connected graph with n vertices and n edges contains at least m different spanning trees?

A 1

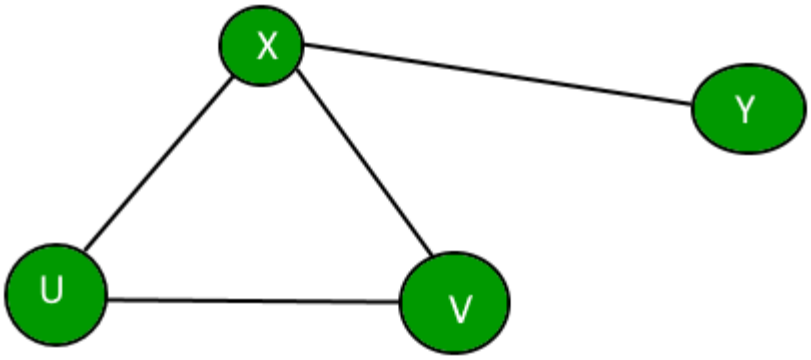
B 2

3

D n

Graph Graph Minimum Spanning Tree Gate IT 2007
Discuss it

Question 20 Explanation:
A graph is connected iff all nodes can be traversed from each node. For a graph with n nodes, there will be n-1 minimum number of edges.
Given that there are n edges, that means a cycle is there in the graph.
The simplex graph with these conditions may be:



Now we can make a different spanning tree by removing one edge from the cycle, one at a time. Minimum cycle length can be 3, So, there must be atleast 3 spanning trees in any such Graph.

Question 21

Wrong

If Kruskal's algorithm is used for finding a minimum spanning tree of a weighted graph G with n vertices and m edges and edge weights are already given in a sorted list, then, What will be the time complexity to compute the minimum cost spanning tree given that union and find operations take amortized O(1) ?

A O(m logn)

B O(n)

O(m)

O(n logm)

Graph Minimum Spanning Tree **GATE 2017 Mock**
Discuss it

Question 21 Explanation:
O(m) as you are already given edge weights in sorted order. You just have to pick the edges in the increasing order and add it to the current spanning set if its addition does not result in a cycle else throw it away.

Question 1
Wrong
What is recurrence for worst case of QuickSort and what is the time complexity in Worst case?

Recurrence is T(n) = T(n-2) + O(n) and time complexity is O(n^2)

Recurrence is T(n) = T(n-1) + O(n) and time complexity is O(n^2)

C Recurrence is T(n) = 2T(n/2) + O(n) and time complexity is O(nLogn)

D Recurrence is T(n) = T(n/10) + T(9n/10) + O(n) and time complexity is O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 1 Explanation:
The worst case of QuickSort occurs when the picked pivot is always one of the corner elements in sorted array. In worst case, QuickSort recursively calls one subproblem with size 0 and other subproblem with size (n-1). So recurrence is T(n) = T(n-1) + T(0) + O(n) The above expression can be rewritten as T(n) = T(n-1) + O(n) void exchange(int *a, int *b) { int temp; temp = *a; *a = *b; *b = temp; } int partition(int arr[], int si, int ei) { int x = arr[ei]; int i = (si - 1); int j; for (j = si; j <= ei - 1; j++) { if(arr[j] <= x) { i++; exchange(&arr[i], &arr[j]); } } exchange (&arr[i + 1], &arr[ei]); return (i + 1); } /* Implementation of Quick Sort arr[] --> Array to be sorted si --> Starting index ei --> Ending index */ void quickSort(int arr[], int si, int ei) { int pi; /* Partitioning index */ if(si < ei) { pi = partition(arr, si, ei); quickSort(arr, si, pi - 1); quickSort(arr, pi + 1, ei); } } [/sourcecode]

Question 2
Wrong
Suppose we have a O(n) time algorithm that finds median of an unsorted array. Now consider a QuickSort implementation where we first find median using the above algorithm, then use median as pivot. What will be the worst case time complexity of this modified QuickSort.

A O(n^2 Logn)

O(n^2)

C O(n Logn Logn)

O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 2 Explanation:
If we use median as a pivot element, then the recurrence for all cases becomes T(n) = 2T(n/2) + O(n) The above recurrence can be solved using Master Method. It falls in case 2 of master method.

Question 3
Wrong
Given an unsorted array. The array has this property that every element in array is at most k distance from its position in sorted array where k is a positive integer smaller than size of array. Which sorting algorithm can be easily modified for sorting this array and what is the obtainable time complexity?

A Insertion Sort with time complexity O(kn)

Heap Sort with time complexity O(nLogk)

Quick Sort with time complexity O(kLogk)

D Merge Sort with time complexity O(kLogk)

Analysis of Algorithms **Sorting** **QuickSort** **HeapSort**
Discuss it

Question 3 Explanation:
1) to sort the array firstly create a min-heap with first k+1 elements and a separate array as resultant array. 2) because elements are at most k distance apart from original position so, it is guranteed that the smallest element will be in this K+1 elements. 3) remove the smallest element from the min-heap(extract min) and put it in the result array. 4) Now,insert another element from the unsorted array into the mean-heap, now,the second smallest element will be in this, perform extract min and continue this process until no more elements are in the unsorted array.finally, use simple heap sort for the remaining elements Time Complexity ---------- 1) O(k) to build the initial min-heap 2) O((n-k)logk) for remaining elements... 3) 0(1) for extract min so overall O(k) + O((n-k)logk) + 0(1) = O(nlogk)

Question 4
Wrong

Which of the following is not true about comparison based sorting algorithms?

The minimum possible time complexity of a comparison based sorting algorithm is O(nLogn) for a random input array

B Any comparison based sorting algorithm can be made stable by using position as a criteria when two elements are compared

C Counting Sort is not a comparison based sorting algortihm

Heap Sort is not a comparison based sorting algorithm.

Analysis of Algorithms **Sorting** **HeapSort** **CountingSort**
Discuss it

Question 4 Explanation:
See http://www.geeksforgeeks.org/lower-bound-on-comparison-based-sorting-algorithms/ for point A. See http://www.geeksforgeeks.org/stability-in-sorting-algorithms/ for B. C is true, count sort is an Integer Sorting algorithm.

Question 5
Wrong

What is time complexity of fun()?

```
int fun(int n)
{
  int count = 0;
  for (int i = n; i > 0; i /= 2)
     for (int j = 0; j < i; j++)
        count += 1;
  return count;
}
```

O(n^2)

B O(nLogn)

O(n)

D O(nLognLogn)

Analysis of Algorithms
Discuss it

Question 5 Explanation:
For a input integer n, the innermost statement of fun() is executed following times. n + n/2 + n/4 + ... 1 So time complexity T(n) can be written as T(n) = O(n + n/2 + n/4 + ... 1) = O(n)
The value of count is also n + n/2 + n/4 + .. + 1

Question 6
Correct
What is the time complexity of fun()?

```
int fun(int n)
{
  int count = 0;
  for (int i = 0; i < n; i++)
     for (int j = i; j > 0; j--)
        count = count + 1;
  return count;
}
```

A Theta (n)
Theta (n^2)
C Theta (n*Logn)
D Theta (nLognLogn)

Analysis of Algorithms
Discuss it

Question 6 Explanation:
The time complexity can be calculated by counting number of times the expression "count = count + 1;" is executed. The expression is executed 0 + 1 + 2 + 3 + 4 + ... + (n-1) times.
Time complexity = Theta(0 + 1 + 2 + 3 + .. + n-1) = Theta (n*(n-1)/2) = Theta(n^2)

Question 7
Correct
The recurrence relation capturing the optimal time of the Tower of Hanoi problem with n discs is. (GATE CS 2012)

A T(n) = 2T(n – 2) + 2
B T(n) = 2T(n – 1) + n
C T(n) = 2T(n/2) + 1
T(n) = 2T(n – 1) + 1

Analysis of Algorithms
Discuss it

Question 7 Explanation:
Following are the steps to follow to solve Tower of Hanoi problem recursively.

```
Let the three pegs be A, B and C. The goal is to move n pegs from A to C.
To move n discs from peg A to peg C:
    move n-1 discs from A to B. This leaves disc n alone on peg A
    move disc n from A to C
    move n?1 discs from B to C so they sit on disc n
```

The recurrence function T(n) for time complexity of the above recursive solution can be written as following. T(n) = 2T(n-1) + 1

Question 8
Wrong

Let w(n) and A(n) denote respectively, the worst case and average case running time of an algorithm executed on an input of size n. which of the following is ALWAYS TRUE? (GATE CS 2012)

(A)

(B)

(C)

(D)

A

B B

C

D D

Analysis of Algorithms
Discuss it

Question 8 Explanation:
The worst case time complexity is always greater than or same as the average case time complexity.

Question 9
Wrong

Which of the following is not O(n^2)?

A (15^10) * n + 12099

B n^1.98

n^3 / (sqrt(n))

(2^20) * n

Analysis of Algorithms
Discuss it

Question 9 Explanation:
The order of growth of option c is $n^{2.5}$ which is higher than n^2.

Question 10
Correct

Which of the given options provides the increasing order of asymptotic complexity of functions f1, f2, f3 and f4?

```
f1(n) = 2^n
f2(n) = n^(3/2)
f3(n) = nLogn
f4(n) = n^(Logn)
```

f3, f2, f4, f1

B f3, f2, f1, f4

C f2, f3, f1, f4

D f2, f3, f4, f1

Analysis of Algorithms
Discuss it

Question 10 Explanation:

```
f1(n) = 2^n
f2(n) = n^(3/2)
f3(n) = nLogn
f4(n) = n^(Logn)
```

Except f3, all other are exponential. So f3 is definitely first in output. Among remaining, n^(3/2) is next. One way to compare f1 and f4 is to take Log of both functions. Order of growth of Log(f1(n)) is Θ(n) and order of growth of Log(f4(n)) is Θ(Logn * Logn). Since Θ(n) has higher growth than Θ(Logn * Logn), f1(n) grows faster than f4(n). **Following is another way to compare f1 and f4.** Let us compare f4 and f1. Let us take few values to compare

```
n = 32, f1 = 2^32, f4 = 32^5 = 2^25
n = 64, f1 = 2^64, f4 = 64^6 = 2^36
..............
..............
```

Also see http://www.wolframalpha.com/input/?i=2^n+vs+n^%28log+n%29 Thanks to fella26 for suggesting the above explanation.

Question 12
Correct

What is the best time complexity of bubble sort?

A N^2

B NlogN

N

D N(logN)^2

Analysis of Algorithms **Sorting** **BubbleSort**
Discuss it

Question 12 Explanation:
The bubble sort is at its best if the input data is sorted. i.e. If the input data is sorted in the same order as expected output. This can be achieved by using one boolean variable. The boolean variable is used to check whether the values are swapped at least once in the inner loop. Consider the following code snippet: [sourcecode language="C"] int main() { int arr[] = {10, 20, 30, 40, 50}, i, j, isSwapped; int n = sizeof(arr) / sizeof(*arr); isSwapped = 1; for(i = 0; i < n - 1 && isSwapped; ++i) { isSwapped = 0; for(j = 0; j < n - i - 1; ++j) if (arr[j] > arr[j + 1]) { swap(&arr[j], &arr[j + 1]); isSwapped = 1; } } for(i = 0; i < n; ++i) printf("%d ", arr[i]); return 0; } [/sourcecode] Please observe that in the above code, the outer loop runs only once.

Question 13

What is the worst case time complexity of insertion sort where position of the data to be inserted is calculated using binary search?

A N

B NlogN

N^2

D N(logN)^2

Analysis of Algorithms **Sorting** **InsertionSort** **BinarySearch**
Discuss it

Question 13 Explanation:
Applying binary search to calculate the position of the data to be inserted doesn't reduce the time complexity of insertion sort. This is because insertion of a data at an appropriate position involves two steps: 1. Calculate the position. 2. Shift the data from the position calculated in step #1 one step right to create a gap where the data will be inserted. Using binary search reduces the time complexity in step #1 from O(N) to O(logN). But, the time complexity in step #2 still remains O(N). So, overall complexity remains O(N^2).

Question 14
Correct

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A N

B N^2

NlogN

D N(logN)^2

Analysis of Algorithms **Sorting**
Discuss it

Question 14 Explanation:
The number of comparisons that a comparison sort algorithm requires increases in proportion to Nlog(N), where N is the number of elements to sort. This bound is asymptotically tight: Given a list of distinct numbers (we can assume this because this is a worst-case analysis), there are N factorial permutations exactly one of which is the list in sorted order. The sort algorithm must gain enough information from the comparisons to identify the correct permutations. If the algorithm always completes after at most f(N) steps, it cannot distinguish more than 2^f(N) cases because the keys are distinct and each comparison has only two possible outcomes. Therefore, 2^f(N) >= N! or equivalently f(N) >= log(N!). Since log(N!) is Omega(NlogN), the answer is NlogN. For more details, read here

Question 15
Wrong

In a modified merge sort, the input array is splitted at a position one-third of the length(N) of the array. Which of the following is the tightest upper bound on time complexity of this modified Merge Sort.

N(logN base 3)

B N(logN base 2/3)

C N(logN base 1/3)

N(logN base 3/2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 15 Explanation:
The time complexity is given by: T(N) = T(N/3) + T(2N/3) + N Solving the above recurrence relation gives, T(N) = N(logN base 3/2)

Question 18
Wrong

The following statement is valid. log(n!) = θ(n log n).

True

False

Analysis of Algorithms
Discuss it

Question 18 Explanation:
Order of growth of $\log n!$ and $n \log n$ is same for large values of n, i.e., $\theta(\log n!) = \theta(n \log n)$. So time complexity of fun() is $\theta(n \log n)$. The expression $\theta(\log n!) = \theta(n \log n)$ can be easily derived from following Stirling's approximation (or Stirling's formula).

$$\log n! = n \log n - n + O(\log(n))$$

Question 19

Question
19
Wrong

What does it mean when we say that an algorithm X is asymptotically more efficient than Y?

A X will be a better choice for all inputs

X will be a better choice for all inputs except possibly small inputs

X will be a better choice for all inputs except possibly large inputs

D Y will be a better choice for small inputs

Analysis of Algorithms
Discuss it

Question 19 Explanation:
In asymptotic analysis we consider growth of algorithm in terms of input size. An algorithm X is said to be asymptotically better than Y if X takes smaller time than y for all input sizes n larger than a value n0 where n0 > 0.

Question 20
Correct

What is the time complexity of Floyd–Warshall algorithm to calculate all pair shortest path in a graph with **n** vertices?

A O(n^2logn)

B Theta(n^2logn)

C Theta(n^4)

Theta(n^3)

Analysis of Algorithms

Discuss it

Question 20 Explanation:
Floyd–Warshall algorithm uses three nested loops to calculate all pair shortest path. So, time complexity is Thete(n^3). Read here for more details.

Question 21
Wrong

A list of n string, each of length n, is sorted into lexicographic order using the merge-sort algorithm. The worst case running time of this computation is

O (n log n)

O (n^2 log n)

C O (n^2 + log n)

D O (n^2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 21 Explanation:
The recurrence tree for merge sort will have height Log(n). And O(n^2) work will be done at each level of the recurrence tree (Each level involves n comparisons and a comparison takes O(n) time in worst case). So time complexity of this Merge Sort will be $O(n^2 logn)$.

Question 22
Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n)

A A

B

C C

D D

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 22 Explanation:
The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get θ(nLogn).

Question 23
Wrong

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let T(n) be the number of comparisons required to sort n elements. Then

A T(n) <= 2T(n/5) + n

T(n) <= T(n/5) + T(4n/5) + n

C T(n) <= 2T(4n/5) + n

T(n) <= 2T(n/2) + n

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 23 Explanation:
For the case where n/5 elements are in one subset, T(n/5) comparisons are needed for the first subset with n/5 elements, T(4n/5) is for the rest 4n/5 elements, and n is for finding the pivot. If there are more than n/5 elements in one set then other set will have less than 4n/5 elements and time complexity will be less than T(n/5) + T(4n/5) + n because recursion tree will be more balanced.

Question 26
Wrong

Which of the following sorting algorithms has the lowest worst-case complexity?

Merge Sort

Bubble Sort

C Quick Sort

D Selection Sort

Analysis of Algorithms **Sorting** **SelectionSort** **MergeSort**
Discuss it

Question 26 Explanation:
Worst case complexities for the above sorting algorithms are as follows: Merge Sort — nLogn Bubble Sort — n^2 Quick Sort — n^2 Selection Sort — n^2

Question 27

Question
31
Wrong

The number of elements that can be sorted in $\Theta(logn)$ time using heap sort is

(A) $\Theta(1)$
(B) $\Theta(\sqrt{logn})$
(C) $\Theta(Logn/(LogLogn))$
(d) $\Theta(Logn)$

A

B B

C

D D

Analysis of Algorithms **Sorting** **HeapSort**
Discuss it

Question 31 Explanation:

Time complexity of Heap Sort is $\Theta(mLogm)$ for m input elements. For m = $\Theta(Logn/(LogLogn))$, the value of $\Theta(m * Logm)$ will be $\Theta([Logn/(LogLogn)] * [Log(Logn/(LogLogn))])$ which will be $\Theta([Logn/(LogLogn)] * [LogLogn - LogLogLogn])$ which is $\Theta(Logn)$

Question 32

Question 33

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? <pre> (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n) </pre>

A A

B

C C

D D

Analysis of Algorithms **Sorting** **GATE-CS-2009** **QuickSort**
Discuss it

Question 33 Explanation:

Answer(B) The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get \theta(nLogn).

Question 37

Wrong

The minimum number of comparisons required to find the minimum and the maximum of 100 numbers is _____.

148

B 147

146

D 140

Analysis of Algorithms **GATE-CS-2014-(Set-1)**
Discuss it

Question 37 Explanation:

Steps to find minimum and maximum element out of n numbers:

```
1. Pick 2 elements(a, b), compare them. (say a > b)
2. Update min by comparing (min, b)
3. Update max by comparing (max, a)
```

Therefore, we need 3 comparisons for each 2 elements, so total number of required comparisons will be (3n)/2 - 2, because we do not need to update min or max in the very first step. Recurrence relation will be:

T(n) = T(⌈n/2⌉)+T(⌊n/2⌋)+2 = 2T(n/2)+2 = ⌈3n/2⌉-2
By putting the value n=100, (3*100/2)-2 = **148** which is answer.

Question 39
Wrong

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

$O(n^2)$

B $O(nLogn)$

$Theta(nLogn)$

D $O(n^3)$

Analysis of Algorithms **Sorting** **GATE-CS-2014-(Set-3)** **QuickSort**
Discuss it

Question 39 Explanation:
For any input, there are some permutations for which worst case will be $O(n^2)$. In some case, choosing the middle element minimizes the chances of encountering $O(n^2)$, but in worst case it can go to $O(n^2)$. Whichever element we take as Pivot, either first or middle, worst case will be $O(n^2)$ since Pivot is fixed in position. While choosing a random pivot minimizes the chances of encountering worst case i.e. $O(n^2)$. Refer this article on Quick Sort.

Question 42
Wrong

Two matrices M1 and M2 are to be stored in arrays A and B respectively. Each array can be stored either in row-major or column-major order in contiguous memory locations. The time complexity of an algorithm to compute M1 × M2 will be

best if A is in row-major, and B is in column- major order

B best if both are in row-major order

C best if both are in column-major order

independent of the storage scheme

Analysis of Algorithms **GATE-CS-2004**
Discuss it

Question 42 Explanation:
This is a trick question. Note that the questions asks about time complexity, not time taken by the program. for time complexity, it doesn't matter how we store array elements, we always need to access same number of elements of M1 and M2 to multiply the matrices. It is always constant or O(1) time to do element access in arrays, the constants may differ for different schemes, but not the time complexity.

Question 46
Wrong

In a permutation a1.....an of n distinct integers, an inversion is a pair (ai, aj) such that i < j and ai > aj. What would be the worst case time complexity of the Insertion Sort algorithm, if the inputs are restricted to permutations of 1.....n with at most n inversions?

$\Theta(n^2)$

B $\Theta(n \log n)$

C $\Theta(n^{1.5})$

$\Theta(n)$

Analysis of Algorithms **Sorting** **GATE-CS-2003** **InsertionSort**
Discuss it

Question 46 Explanation:
Insertion sort runs in $\Theta(n + f(n))$ time, where f(n) denotes the number of inversion initially present in the array being sorted. Source:
http://cs.xidian.edu.cn/jpkc/Algorithm/down/Solution%20to%202-4%20Inversions.pdf

Question 47
Correct

Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort?

A O(n)

B O(n Log n)

$O(n^2)$

D O(n!)

Analysis of Algorithms **Sorting** **GATE-CS-2001** **QuickSort**
Discuss it

Question 47 Explanation:
Randomized quicksort has expected time complexity as O(nLogn), but worst case time complexity remains same. In worst case the randomized function can pick the index of corner element every time.

Question 49
Correct

Which one of the following is the recurrence equation for the worst case time complexity of the Quicksort algorithm for sorting $n(\geq 2)$ numbers? In the recurrence equations given in the options below, c is a constant.

A $T(n) = 2T(n/2) + cn$

$T(n) = T(n-1) + T(0) + cn$

C $T(n) = 2T(n-2) + cn$

D $T(n) = T(n/2) + cn$

Analysis of Algorithms **Sorting** **GATE-CS-2015 (Set 1)** **QuickSort**
Discuss it

Question 49 Explanation:
In worst case, the chosen pivot is always placed at a corner position and recursive call is made for following. a) for subarray on left of pivot which is of size n-1 in worst case. b) for subarray on right of pivot which is of size 0 in worst case.

Question 51
Correct

An unordered list contains n distinct elements. The number of comparisons to find an element

in this list that is neither maximum nor minimum is

A Θ(nlogn)

B Θ(n)

C Θ(logn)

Θ(1)

Analysis of Algorithms **GATE-CS-2015 (Set 2)**
Discuss it

Question 51 Explanation:
We only need to consider any 3 elements and compare them. So the number of comparisons is constants, that makes time complexity as Θ(1) The catch here is, we need to return any element that is neither maximum not minimum. Let us take an array {10, 20, 15, 7, 90}. Output can be 10 or 15 or 20 Pick any three elements from given liar. Let the three elements be 10, 20 and 7. Using 3 comparisons, we can find that the middle element is 10.

Question 53
Correct

In the following table, the left column contains the names of standard graph algorithms and the right column contains the time complexities of the algorithms. Match each algorithm with its time complexity.

1. Bellman-Ford algorithm 2. Kruskal's algorithm 3. Floyd-Warshall algorithm 4. Topological sorting	A : O (m log n) B : O (n^3) C : O (nm) D : O (n + m)

1→ C, 2 → A, 3 → B, 4 → D

B 1→ B, 2 → D, 3 → C, 4 → A

C 1→ C, 2 → D, 3 → A, 4 → B

D 1→ B, 2 → A, 3 → C, 4 → D

Analysis of Algorithms **Gate IT 2005**
Discuss it

Question 53 Explanation:

- **Bellman-Ford algorithm: Time complexity: O(VE)**
- **Kruskal's algorithm**:**Time Complexity:** O(ElogE) or O(ElogV). Sorting of edges takes O(ELogE) time. After sorting, we iterate through all edges and apply find-union algorithm. The find and union operations can take atmost O(LogV) time. So overall complexity is O(ELogE + ELogV) time. The value of E can be atmost V^2, so O(LogV) are O(LogE) same. Therefore, overall time complexity is O(ElogE) or O(ElogV)
- **Floyd-Warshall algorithm**: **Time Complexity**: O(V^3)
- **Topological sorting: Time Complexity:** The above algorithm is simply DFS with an extra stack. So time complexity is same as DFS which is O(V+E).

Question 55
Correct

The worst case running times of Insertion sort, Merge sort and Quick sort, respectively, are:

A $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$

B $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \text{ Log } n)$

C $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$

$\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 1)**
Discuss it

Question 55 Explanation:

- Insertion Sort takes $\Theta(n^2)$ in worst case as we need to run two loops. The outer loop is needed to one by one pick an element to be inserted at right position. Inner loop is used for two things, to find position of the element to be inserted and moving all sorted greater elements one position ahead. Therefore the worst case recursive formula is $T(n) = T(n-1) + \Theta(n)$.
- Merge Sort takes $\Theta(n \text{ Log } n)$ time in all cases. We always divide array in two halves, sort the two halves and merge them. The recursive formula is $T(n) = 2T(n/2) + \Theta(n)$.
- QuickSort takes $\Theta(n^2)$ in worst case. In QuickSort, we take an element as pivot and partition the array around it. In worst case, the picked element is always a corner element and recursive formula becomes $T(n) = T(n-1) + \Theta(n)$. An example scenario when worst case happens is, arrays is sorted and our code always picks a corner element as pivot.

Question 56
Correct

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in ascending order, which of the following are TRUE ?

I. Quicksort runs in $\Theta(n^2)$ time
II. Bubblesort runs in $\Theta(n^2)$ time
III. Mergesort runs in $\Theta(n)$ time
IV. Insertion sort runs in $\Theta(n)$ time

A I and II only

B I and III only

C II and IV only

I and IV only

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 2)**
Discuss it

Question 56 Explanation:
I. Given an array in ascending order, Recurrence relation for total number of comparisons for quicksort will be T(n) = T(n-1)+O(n) //partition algo will take O(n) comparisons in any case. = O(n^2) **II.** Bubble Sort runs in $\Theta(n^2)$ time If an array is in ascending order, we could make a small modification in Bubble Sort Inner for loop which is responsible for bubbling the kth largest element to the end in kth iteration. Whenever there is no swap after the completion of inner for loop of bubble sort in any iteration, we can declare that array is sorted in case of Bubble Sort taking O(n) time in Best Case. **III.** Merge Sort runs in $\Theta(n)$ time Merge Sort relies on Divide and Conquer paradigm to sort an array and there is no such worst or best case input for merge sort. For any sequence, Time complexity will be given by

following recurrence relation, T(n) = 2T(n/2) + Θ(n) // In-Place Merge algorithm will take Θ(n) due to copying an entire array. = Θ(nlogn) **IV.** Insertion sort runs in Θ(n) time Whenever a new element which will be greater than all the elements of the intermediate sorted sub-array (because given array is sorted) is added, there won't be any swap but a single comparison. In n-1 passes we will be having 0 swaps and n-1 comparisons. Total time complexity = O(n) // N-1 Comparisons This solution is contributed by **Pranjul Ahuja**

//// For an array already sorted in ascending order, Quicksort has a complexity $\Theta(n^2)$ [Worst Case] Bubblesort has a complexity Θ(n) [Best Case] Mergesort has a complexity Θ(n log n) [Any Case] Insertsort has a complexity Θ(n) [Best Case]

Question 57

Question 58

Correct

The characters a to h have the set of frequencies based on the first 8 Fibonacci numbers as follows

a : 1, b : 1, c : 2, d : 3, e : 5, f : 8, g : 13, h : 21/

A Huffman code is used to represent the characters. What is the sequence of characters corresponding to the following code? 110111100111010

fdheg

B ecgdf

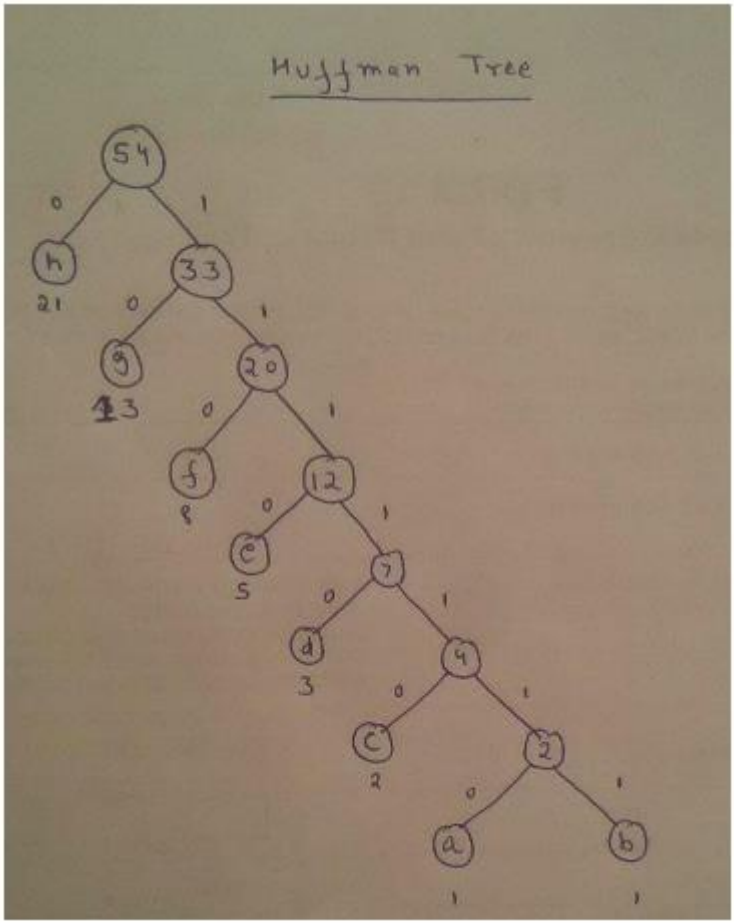
C dchfg

D fehdg

Analysis of Algorithms **GATE IT 2006**
Discuss it

Question 58 Explanation:

Background Required - Generating Prefix codes using Huffman Coding. First we apply greedy algorithm on the frequencies of the characters to generate the binary tree as shown in the Figure given below. Assigning 0 to the left edge and 1 to the right edge, prefix codes for the characters are as below.



a - 1111110 b - 1111111 c - 111110 d - 11110 e - 1110 f - 110 g - 10 h - 0 Given String can be decomposed as 110 11110 0 1110 10 f d h e g This solution is contributed by **Pranjul Ahuja** .

Question 61
Wrong

Which of the following is TRUE?

The cost of searching an AVL tree is θ (log n) but that of a binary search tree is O(n)

B The cost of searching an AVL tree is θ (log n) but that of a complete binary tree is θ (n log n)

The cost of searching a binary search tree is O (log n) but that of an AVL tree is θ(n)

D The cost of searching an AVL tree is θ (n log n) but that of a binary search tree is O(n)

Analysis of Algorithms **Balanced Binary Search Trees** **Gate IT 2008**
Discuss it

Question 61 Explanation:
AVL tree is a balanced tree.
AVL tree's time complexity of searching = θ(logn)
But a binary search tree, may be skewed tree, so in worst case BST searching time = θ(n)

Question 63
Correct

If we use Radix Sort to sort n integers in the range $(n^{k/2},n^k]$, for some k>0 which is independent of n, the time taken would be?

A $\Theta(n)$

B $\Theta(kn)$

$\Theta(nlogn)$

D $\Theta(n^2)$

Analysis of Algorithms **Sorting** **RadixSort** **Gate IT 2008**
Discuss it

Question 63 Explanation:
Radix sort time complexity = O(wn)
for n keys of word size= w
=>w = $\log(n^k)$
O(wn)=O(klogn.n)
=> kO(nlogn)

Question 64
Correct

The auxiliary space of insertion sort is O(1), what does O(1) mean ?

A The memory (space) required to process the data is not constant.

It means the amount of extra memory Insertion Sort consumes doesn't depend on the input. The algorithm should use the same amount of memory for all inputs.

C It takes only 1 kb of memory .

D It is the speed at which the elements are traversed.

Analysis of Algorithms **Sorting** **InsertionSort**
Discuss it

Question 64 Explanation:
The term O(1) states that the space required by the insertion sort is constant i.e., space required doesn't depend on input.

Question 65
Wrong

A sorting technique is called stable if:

A It takes O(nlog n)time

It maintains the relative order of occurrence of non-distinct elements

C It uses divide and conquer paradigm

It takes O(n) space

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 66
Correct

Suppose we want to arrange the ii numbers stored in an array such that all negative values occur before all positive ones. Minimum number of exchanges required in the worst case is:

A n-1

B n

C n+1

None of the above

Analysis of Algorithms **GATE CS 1999**
Discuss it

Question 66 Explanation:
When we have 'i' numbers stored in an array, we have to swap all positive numbers with negative and in worst case positive numbers will be i/2.

Question 67
Wrong

If one uses straight two-way merge sort algorithm to sort the following elements in ascending order *20, 47, 15, 8, 9, 4, 40, 30, 12, 17* then the order of these elements after the second pass of the algorithm is:

8, 9, 15, 20, 47, 4, 12, 17, 30, 40

8, 15, 20, 47, 4, 9, 30, 40, 12, 17

C 15, 20, 47, 4, 8, 9, 12, 30, 40, 17

D 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 67 Explanation:
In first pass the elements are sorted in n/4 (first 2 elements in each group) sub arrays but in second pass the elements are sorted in n/2 (first 4 elements in each group) sub arrays.

Question 68

Question
71
Wrong

Which of the following algorithm design technique is used in finding all pairs of shortest distances in a graph?

Dynamic programming

B Backtracking

Greedy

D Divide and Conquer

Analysis of Algorithms **GATE CS 1998**
Discuss it

Question 72
Wrong

Give the correct matching for the following pairs:

```
A. O(log n)      1. Selection sort
B. O(n)          2. Insertion sort
C. O(nlog n)     3. Binary search
D. O(n^2)        4. Merge sort

codes:
     A   B   C   D
a.   3   1   2   4
```

b. 3 1 4 2
c. 1 3 4 2
d. 1 4 3 2

a

b

C c

D d

Analysis of Algorithms **GATE CS 1998**
Discuss it

Question 75
Correct

The correct matching for the following pairs is

(A) All pairs shortest path	(1) Greedy
(B) Quick Sort	(2) Depth-First search
(C) Minimum weight spanning tree	(3) Dynamic Programming
(D) Connected Components	(4) Divide and and Conquer

Codes:

	A	B	C	D
a	2	4	1	3
b	3	4	1	2
c	3	4	2	1
d	4	1	2	3

A a

b

C c

D d

Analysis of Algorithms **GATE CS 1997**
Discuss it

Question 78
Correct

Quicksort is run on two inputs shown below to sort in ascending order taking first element as pivot,

(i) 1, 2, 3,....., n
(ii) n, n-1, n-2,....., 2, 1

Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

A C1 < C2

B C1 > C2

C1 = C2

D We cannot say anything for arbitrary n

Analysis of Algorithms **Sorting** **QuickSort** **GATE CS 1996**
Discuss it

Question 78 Explanation:

Quick Sort performs in worst case when input is sorted in either ascending order or descending order. So, quick sort will return equal number of comparisons for both given inputs. Option (C) is correct.

Question 83
Correct

Given A, an array of size n, comprised of an increasing sequence of numbers followed immediately by a decreasing one. What is worst case time complexity of optimal algorithm to determine if a given number x is in the array?

Θ(log n)

Θ(n)

C Θ(n^2)

D Θ(log n)^2

Analysis of Algorithms **GATE CS Mock 2018**
Discuss it

Question 83 Explanation:
This is an application of Binary search, which has time complexity Θ(log n) in worst case. Option (A) is correct.

Question 84
Wrong

Match the following:

Algorithm	Time complexity (Worst case)
A. Optimal merge pattern	1. O(m*n)
B. Power of an element	2. O(log n)
C. Prims MST algorithm	3. O(n log n)
D. Longest common subsequence	4. O(m log n)

Which of the following option is correct?

A – 3, B – 2, C – 4, D – 1

A – 3, B – 1, C – 4, D – 2

C A – 3, B – 4, C – 2, D – 1

D A – 1, B – 2, C – 4, D – 3

Analysis of Algorithms **GATE CS Mock 2018 | Set 2**
Discuss it

Question 84 Explanation:
Correct matching is A – 3, B – 2, C – 4, D – 1. Option (A) is correct.

Question 88
Wrong

If b is the branching factor and m is the maximum depth of the search tree, what is the space complexity of greedy search?

A O(b+m)

O(bm)

O(b^m)

D $O(m^m)$

Analysis of Algorithms **UGC-NET CS 2017 Nov - III**
Discuss it

Question 88 Explanation:

1. In binary tree branching factor is 2 and space complexity for height n is $O(2^n)$.
2. In ternary tree branching factor is 3 and space complexity for height n is $O(3^n)$.

   Similarly

3. If branching factor is b and height is m for search tree then space complexity of greedy search is $O(b^m)$.

So, option (C) is correct.

Question 89
Wrong

The time complexity of computing the transitive closure of a binary relation on a set of n elements is known to be

O(nlogn)

B $O(n^{3/2})$

O(n^3)

D O(n)

Analysis of Algorithms **Set Theory & Algebra** **ISRO CS 2017 - May**
Discuss it

Question 89 Explanation:
Transitive closure generally uses Floyd-Warshall Algorithm which gives a time complexity of $O(n^3)$ Refer: Transitive closure of a graph

Question 92
Wrong

Any decision tree that sorts n elements has height _____.

Ω(lg n)

B Ω(n)

Ω(n lg n)

D $\Omega(n^2)$

Analysis of Algorithms **UGC NET CS 2017 Jan - III**
Discuss it

Question 93
Correct

Dijkstra's algorithm is based on

A Divide and conquer paradigm

B Dynamic programming

Greedy Approach

D Backtracking paradigm

Analysis of Algorithms **UGC NET CS 2017 Jan - III**

Discuss it

Question 93 Explanation:

Dijkstra's algorithm is based on Greedy Approach. For more information Refer:Greedy Algorithms | Set 7 (Dijkstra's shortest path algorithm) Option (C) is correct.

Question 94

Correct

Match the following with respect to algorithm paradigms :

	List – I		**List – II**
a.	Merge sort	i.	Dynamic programming
b.	Huffman coding	ii.	Greedy approach
c.	Optimal polygon triangulation	iii.	Divide and conquer
d.	Subset sum problem	iv.	Back tracking

Codes :

	a	b	c	d
(1)	iii	i	ii	iv
(2)	ii	i	iv	iii
(3)	ii	i	iii	iv
(4)	iii	ii	i	iv

A (1)

B (2)

C (3)

(4)

Analysis of Algorithms **UGC NET CS 2017 Jan - III**

Discuss it

Question 95

Correct

What is the best method to go for the game playing problem?

A Optimal Search

B Random Search

Heuristic Search

D Stratified Search

Analysis of Algorithms **UGC NET CS 2017 Jan - III**

Discuss it

Question 97

Correct

Consider a weighted complete graph G on the vertex set $\{v_1, v_2, ... v_n\}$ such that the weight of the edge (v_i, v_j) is $4 | i – j|$. The weight of minimum cost spanning tree of G is :

A $4n^2$

B n

4n – 4

D 2n – 2

Analysis of Algorithms **UGC NET CS 2016 Aug - III**

Discuss it

Question 98

Wrong

Match the following:

a.	Prim's algorithm	i.	$O(V^2E)$
b.	Bellman-Ford algorithm	ii.	$O(VE\ lgV)$
c.	Floyd-Warshall algorithm	iii.	$O(E\ lgV)$
d.	Johnson's algorithm	iv.	$O(V^3)$

Where V is the set of nodes and E is the set of edges in the graph.

Codes :

	a	b	c	d
(1)	i	iii	iv	ii
(2)	i	iii	ii	iv
(3)	iii	i	iv	ii
(4)	iii	i	ii	iv

(1)

B (2)

(3)

D (4)

Analysis of Algorithms **UGC NET CS 2016 Aug - III**

Discuss it

Question 98 Explanation:

- Prim's algorithm takes O(E lgV) time.
- Bellman-Ford algorithm takes $O(V^2E)$ time.
- Floyd-Warshall algorithm takes $O(V^3)$ time.
- Johnson's algorithm takes O(VE lgV) time.

So, option (C) is correct.

Question 99

Correct

(a)	Huffman codes	(i)	$O(n^2)$
(b)	Optimal polygon triangulation	(ii)	$\theta(n^3)$
(c)	Activity selection problem	(iii)	O(nlgn)
(d)	Quicksort	(iv)	$\theta(n)$

Codes :

	(a)	(b)	(c)	(d)
(1)	(i)	(ii)	(iv)	(iii)
(2)	(i)	(iv)	(ii)	(iii)
(3)	(iii)	(ii)	(iv)	(i)
(4)	(iii)	(iv)	(ii)	(i)

Match the following:

A (1)

B (2)

(3)

D (4)

Analysis of Algorithms **UGC NET CS 2016 July – III**

Discuss it

Question 99 Explanation:

- Huffman codes takes O(nlgn) time.
- Optimal polygon triangulation takes $\theta(n^3)$ time

  Activity selection problem takes $\theta(n)$ time

-
- Quicksort takes $O(n^2)$ time

So, option (C) is correct.

Question 100

Correct

Given two sequences X and Y : X = < a, b, c, b, d, a, b > Y = < b, d, c, a, b, a > . The longest common subsequence of X and Y is :

A < b, c, a>

B < c, a, b>

C < b, c, a, a>

< b, c, b, a>

Analysis of Algorithms **UGC NET CS 2015 Dec – III**

Discuss it

Question 101

Wrong

Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

Greedy algorithm, $\theta (V^3)$

B Greedy algorithm, $\theta (V^2 \lg n)$

Dynamic programming, $\theta (V^3)$

D Dynamic programming, $\theta (V^2 \lg n)$

Analysis of Algorithms **UGC NET CS 2015 Dec – III**
Discuss it

Question 101 Explanation:
Floyd-Warshall algorithm utilizes dynamic programming to solve the all-pairs shortest paths problem on a directed graph in $\theta (V^3)$ time. For more information on Floyd-Warshall algorithm ReferDynamic Programming | Set 16 (Floyd Warshall Algorithm) Option (C) is correct.

Question 102
Correct

To determine the efficiency of an algorithm the time factor is measured by:

A Counting micro seconds

Counting number of key operations

C Counting number of statements

D Counting kilobytes of algorithm

Analysis of Algorithms **UGC NET CS 2015 Jun - II**
Discuss it

Question 102 Explanation:
To determine the efficiency of an algorithm the time factor is measured by counting number of key operations. It is not measured by counting of statements because less no of statement does not mean that program is more efficient. Many times we have to write large code for better efficieny. We measure space complexity in kilobytes. So, option (B) is correct.

Question 104
Wrong

An all-pairs shortest-paths problem is efficiently solved using:

Dijkstra' algorithm

B Bellman-Ford algorithm

C Kruskal algorithm

Floyd-Warshall algorithm

Analysis of Algorithms **UGC NET CS 2015 Jun - III**
Discuss it

Question 104 Explanation:
An all-pairs shortest-paths problem is efficiently solved using Floyd-Warshall algorithm. For more information on all-pairs shortest-paths problem and its solution using Floyd-Warshall algorithm Refer:Dynamic Programming | Set 16 (Floyd Warshall Algorithm) So, option (D) is correct.

Match the following:

List - I List - II

(a) Bucket sort	(i) $O(n^3 \lg n)$
(b) Matrix chain multiplication	(ii) $O(n^3)$
(c)Huffman codes	(iii) O(nlgn)
(d)All pairs shortest paths	(iv)O(n)

codes:

	(a)	(b)	(c)	(d)
(A)	(iv)	(ii)	(i)	(iii)
(B)	(ii)	(iv)	(i)	(iii)
(C)	(iv)	(ii)	(i)	(iii)
(D)	(iii)	(ii)	(iv)	(i)

(A)
B (B)
C (C)
(D)

Analysis of Algorithms **UGC NET CS 2014 Dec - III**
Discuss it

Question 110
Correct
Dijkstra algorithm, which solves the single-source shortest--paths problem, is a _____, and the Floyd-Warshall algorithm, which finds shortest paths between all pairs of vertices, is a _____

A Greedy algorithm, Divide-conquer algorithm
B Divide-conquer algorithm, Greedy algorithm
Greedy algorithm, Dynamic programming algorithm
D Dynamic programming algorithm, Greedy algorithm

Analysis of Algorithms **UGC NET CS 2014 Dec - III**
Discuss it

Question 110 Explanation:
Dijkstra algorithm, which solves the single-source shortest–paths problem, is a Greedy algorithm, and the Floyd-Warshall algorithm, which finds shortest paths between all pairs of vertices, is a Dynamic programming algorithm. For more information on Floyd-Warshall and Dijkstra algorithm Refer:Dynamic Programming | Set 16 (Floyd Warshall Algorithm) Greedy Algorithms | Set 7 (Dijkstra's shortest path algorithm) Option (C) is correct.
Question 114
Wrong

Match the following:

codes:

List - I

(a) The 8-Queen’s problem

(b) Single-Source shortest paths

(c)STRASSEN’s Matrix multiplication

(d)Optimal binary search trees

List - II

(i)Dynamic programming

(ii) Divide and conquer

(iii)Greedy approach

(iv)Backtracking

	(a)	(b)	(c)	(d)
(1)	(iv)	(i)	(iii)	(ii)
(2)	(iv)	(iii)	(i)	(ii)
(3)	(iii)	(iv)	(i)	(ii)
(4)	(iv)	(iii)	(ii)	(i)

A (1)

(2)

C (3)

(4)

Analysis of Algorithms **UGC NET CS 2018 July - II**
Discuss it

Question 114 Explanation:

- The 8-Queen’s problem is a Backtracking algorithm.
- Single-Source shortest paths is a Greedy approach
- STRASSEN’s Matrix multiplication is a Divide and conquer
- Optimal binary search trees is a Dynamic programming. So, option (D) is correct.

Question 1
Correct

What is recurrence for worst case of QuickSort and what is the time complexity in Worst case?

A Recurrence is T(n) = T(n-2) + O(n) and time complexity is O(n^2)
Recurrence is T(n) = T(n-1) + O(n) and time complexity is O(n^2)

C Recurrence is T(n) = 2T(n/2) + O(n) and time complexity is O(nLogn)

D Recurrence is T(n) = T(n/10) + T(9n/10) + O(n) and time complexity is O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 1 Explanation:

The worst case of QuickSort occurs when the picked pivot is always one of the corner elements in sorted array. In worst case, QuickSort recursively calls one subproblem with size 0 and other subproblem with size (n-1). So recurrence is T(n) = T(n-1) + T(0) + O(n) The above expression can be rewritten as T(n) = T(n-1) + O(n) void exchange(int *a, int *b) { int temp; temp = *a; *a = *b; *b = temp; } int partition(int arr[], int si, int ei) { int x = arr[ei]; int i = (si - 1); int j; for (j = si; j <= ei - 1; j++) { if(arr[j] <= x) { i++; exchange(&arr[i], &arr[j]); } } exchange (&arr[i + 1], &arr[ei]); return (i + 1); } /* Implementation of Quick Sort arr[] --> Array to be sorted si --> Starting index ei --> Ending index */ void quickSort(int arr[], int

si, int ei) { int pi; /* Partitioning index */ if(si < ei) { pi = partition(arr, si, ei); quickSort(arr, si, pi - 1); quickSort(arr, pi + 1, ei); } } [/sourcecode]

Question 2
Correct

Suppose we have a O(n) time algorithm that finds median of an unsorted array. Now consider a QuickSort implementation where we first find median using the above algorithm, then use median as pivot. What will be the worst case time complexity of this modified QuickSort.

A O(n^2 Logn)

B O(n^2)

C O(n Logn Logn)

O(nLogn)

Analysis of Algorithms **Sorting** **QuickSort**
Discuss it

Question 2 Explanation:
If we use median as a pivot element, then the recurrence for all cases becomes T(n) = 2T(n/2) + O(n) The above recurrence can be solved using Master Method. It falls in case 2 of master method.

Question 3
Wrong

Which of the following is not a stable sorting algorithm in its typical implementation.

Insertion Sort

B Merge Sort

Quick Sort

D Bubble Sort

Sorting **QuickSort** **InsertionSort** **MergeSort**
Discuss it

Question 3 Explanation:
See following for details. http://www.geeksforgeeks.org/stability-in-sorting-algorithms/

Question 4
Correct

Which of the following sorting algorithms in its typical implementation gives best performance when applied on an array which is sorted or almost sorted (maximum 1 or two elements are misplaced).

A Quick Sort

B Heap Sort

C Merge Sort

Insertion Sort

Sorting **QuickSort** **InsertionSort** **HeapSort**
Discuss it

Question 4 Explanation:
Insertion sort takes linear time when input array is sorted or almost sorted (maximum 1 or 2 elements are misplaced). All other sorting algorithms mentioned above will take more than lienear time in their typical implementation.

Question 5
Wrong

Given an unsorted array. The array has this property that every element in array is at most k distance from its position in sorted array where k is a positive integer smaller than size of array. Which sorting algorithm can be easily modified for sorting this array and what is the obtainable time complexity?

A Insertion Sort with time complexity O(kn)

Heap Sort with time complexity O(nLogk)

C Quick Sort with time complexity O(kLogk)

Merge Sort with time complexity O(kLogk)

Analysis of Algorithms **Sorting** **QuickSort** **HeapSort**
Discuss it

Question 5 Explanation:
1) to sort the array firstly create a min-heap with first k+1 elements and a separate array as resultant array. 2) because elements are at most k distance apart from original position so, it is guranteed that the smallest element will be in this K+1 elements. 3) remove the smallest element from the min-heap(extract min) and put it in the result array. 4) Now,insert another element from the unsorted array into the mean-heap, now,the second smallest element will be in this, perform extract min and continue this process until no more elements are in the unsorted array.finally, use simple heap sort for the remaining elements Time Complexity ---------- 1) O(k) to build the initial min-heap 2) O((n-k)logk) for remaining elements... 3) 0(1) for extract min so overall O(k) + O((n-k)logk) + 0(1) = O(nlogk)

Question 6

Question 6
Wrong

Consider a situation where swap operation is very costly. Which of the following sorting algorithms should be preferred so that the number of swap operations are minimized in general?

A Heap Sort

Selection Sort

C Insertion Sort

Merge Sort

Sorting **SelectionSort** **InsertionSort** **MergeSort**
Discuss it

Question 6 Explanation:

Selection sort makes O(n) swaps which is minimum among all sorting algorithms mentioned above.

Question 7
Wrong
Which of the following is not true about comparison based sorting algorithms?

A The minimum possible time complexity of a comparison based sorting algorithm is O(nLogn) for a random input array

Any comparison based sorting algorithm can be made stable by using position as a criteria when two elements are compared

C Counting Sort is not a comparison based sorting algortihm

Heap Sort is not a comparison based sorting algorithm.

Analysis of Algorithms **Sorting** **HeapSort** **CountingSort**
Discuss it

Question 7 Explanation:
See http://www.geeksforgeeks.org/lower-bound-on-comparison-based-sorting-algorithms/ for point A. See http://www.geeksforgeeks.org/stability-in-sorting-algorithms/ for B. C is true, count sort is an Integer Sorting algorithm.

Question 8
Wrong
Suppose we are sorting an array of eight integers using quicksort, and we have just finished the first partitioning with the array looking like this:

```
2  5  1  7  9  12  11  10
```

Which statement is correct?

The pivot could be either the 7 or the 9.

B The pivot could be the 7, but it is not the 9

C The pivot is not the 7, but it could be the 9

Neither the 7 nor the 9 is the pivot.

Sorting **QuickSort**
Discuss it

Question 8 Explanation:
7 and 9 both are at their correct positions (as in a sorted array). Also, all elements on left of 7 and 9 are smaller than 7 and 9 respectively and on right are greater than 7 and 9 respectively.

Question 9
Correct
Suppose we are sorting an array of eight integers using heapsort, and we have just finished some heapify (either maxheapify or minheapify) operations. The array now looks like this: 16 14 15 10 12 27 28 How many heapify operations have been performed on root of heap?

A 1

2

C 3 or 4

D 5 or 6

Sorting **Heap** **HeapSort**
Discuss it

Question 9 Explanation:
In Heapsort, we first build a heap, then we do following operations till the heap size becomes 1. a) Swap the root with last element b) Call heapify for root c) reduce the heap size by 1. In this question, it is given that heapify has been called few times and we see that last two elements in given array are the 2 maximum elements in array. So situation is clear, it is maxheapify whic has been called 2 times.

Question 10

Wrong

What is the best time complexity of bubble sort?

N^2

B NlogN

N

D N(logN)^2

Analysis of Algorithms **Sorting** **BubbleSort**
Discuss it

Question 10 Explanation:
The bubble sort is at its best if the input data is sorted. i.e. If the input data is sorted in the same order as expected output. This can be achieved by using one boolean variable. The boolean variable is used to check whether the values are swapped at least once in the inner loop. Consider the following code snippet: int main() { int arr[] = {10, 20, 30, 40, 50}, i, j, isSwapped; int n = sizeof(arr) / sizeof(*arr); isSwapped = 1; for(i = 0; i < n - 1 && isSwapped; ++i) { isSwapped = 0; for(j = 0; j < n - i - 1; ++j) if (arr[j] > arr[j + 1]) { swap(&arr[j], &arr[j + 1]); isSwapped = 1; } } for(i = 0; i < n; ++i) printf("%d ", arr[i]); return 0; } [/sourcecode] Please observe that in the above code, the outer loop runs only once .

Question 11

Wrong

You have to sort 1 GB of data with only 100 MB of available main memory. Which sorting technique will be most appropriate?

Heap sort

Merge sort

C Quick sort

D Insertion sort

Sorting **QuickSort** **MergeSort** **HeapSort**
Discuss it

Question 11 Explanation:
The data can be sorted using external sorting which uses merging technique. This can be done as follows: 1. Divide the data into 10 groups each of size 100. 2. Sort each group and write them to disk. 3. Load 10 items from each group into main memory. 4. Output the smallest item from the main memory to disk. Load the next item from the group whose item was chosen. 5. Loop step #4 until all items are not outputted. The step 3-5 is called as merging technique.

Question 12

Wrong

What is the worst case time complexity of insertion sort where position of the data to be inserted is calculated using binary search?

A N

NlogN

N^2

D N(logN)^2

Analysis of Algorithms **Sorting** **InsertionSort** **BinarySearch**
Discuss it

Question 12 Explanation:
Applying binary search to calculate the position of the data to be inserted doesn't reduce the time complexity of insertion sort. This is because insertion of a data at an appropriate position involves two steps: 1. Calculate the position. 2. Shift the data from the position calculated in step #1 one step right to create a gap where the data will be inserted. Using binary search reduces the time complexity in step #1 from O(N) to O(logN). But, the time complexity in step #2 still remains O(N). So, overall complexity remains O(N^2).

Question 13

Correct

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A N

B N^2

NlogN

D N(logN)^2

Analysis of Algorithms **Sorting**
Discuss it

Question 13 Explanation:
The number of comparisons that a comparison sort algorithm requires increases in proportion to Nlog(N), where N is the number of elements to sort. This bound is asymptotically tight: Given a list of distinct numbers (we can assume this because this is a worst-case analysis), there are N factorial permutations exactly one of which is the list in sorted order. The sort algorithm must gain enough information from the comparisons to identify the correct permutations. If the algorithm always completes after at most f(N) steps, it cannot distinguish more than 2^f(N) cases because the keys are distinct and each comparison has only two possible outcomes. Therefore, 2^f(N) >= N! or equivalently f(N) >= log(N!). Since log(N!) is Omega(NlogN), the answer is NlogN. For more details, read here

Question 14

Wrong

In a modified merge sort, the input array is splitted at a position one-third of the length(N) of the array. Which of the following is the tightest upper bound on time complexity of this modified Merge Sort.

A N(logN base 3)

B N(logN base 2/3)

N(logN base 1/3)

N(logN base 3/2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 14 Explanation:
The time complexity is given by: T(N) = T(N/3) + T(2N/3) + N Solving the above recurrence relation gives, T(N) = N(logN base 3/2)

Question 15

Correct

Which sorting algorithm will take least time when all elements of input array are identical? Consider typical implementations of sorting algorithms.

Insertion Sort

B Heap Sort

C Merge Sort

D Selection Sort

Sorting **SelectionSort** **InsertionSort** **MergeSort**
Discuss it

Question 15 Explanation:
The insertion sort will take θ(n) time when input array is already sorted.

Question 16

Correct

A list of n string, each of length n, is sorted into lexicographic order using the merge-sort algorithm. The worst case running time of this computation is

A O (n log n)

O (n^2 log n)

C O (n^2 + log n)

D O (n^2)

Analysis of Algorithms **Sorting** **MergeSort**
Discuss it

Question 16 Explanation:
The recurrence tree for merge sort will have height Log(n). And O(n^2) work will be done at each level of the recurrence tree (Each level involves n comparisons and a comparison takes O(n) time in worst case). So time complexity of this Merge Sort will be $O(n^2 logn)$.

Question 17

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? (A) θ(n) (B) θ(nLogn) (C) θ(n^2) (D) θ(n^2 log n)

A A

B

C C

D D

Analysis of Algorithms Sorting QuickSort
Discuss it

Question 17 Explanation:
The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get θ(nLogn).

Question
18

Question 18

Correct

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let T(n) be the number of comparisons required to sort n elements. Then

A T(n) <= 2T(n/5) + n

T(n) <= T(n/5) + T(4n/5) + n

C T(n) <= 2T(4n/5) + n

D T(n) <= 2T(n/2) + n

Analysis of Algorithms Sorting QuickSort
Discuss it

Question 18 Explanation:
For the case where n/5 elements are in one subset, T(n/5) comparisons are needed for the first subset with n/5 elements, T(4n/5) is for the rest 4n/5 elements, and n is for finding the pivot. If there

are more than n/5 elements in one set then other set will have less than 4n/5 elements and time complexity will be less than T(n/5) + T(4n/5) + n because recursion tree will be more balanced.

Question 19

Correct

Which of the following sorting algorithms has the lowest worst-case complexity?

Merge Sort

B Bubble Sort

C Quick Sort

D Selection Sort

Analysis of Algorithms **Sorting** **SelectionSort** **MergeSort**
Discuss it

Question 19 Explanation:
Worst case complexities for the above sorting algorithms are as follows: Merge Sort — nLogn Bubble Sort — n^2 Quick Sort — n^2 Selection Sort — n^2

Question 20

Correct

Which sorting algorithms is most efficient to sort string consisting of ASCII characters?

A Quick sort

B Heap sort

C Merge sort

Counting sort

Sorting **QuickSort** **HeapSort** **CountingSort**
Discuss it

Question 20 Explanation:
Counting sort algorithm is efficient when range of data to be sorted is fixed. In the above question, the range is from 0 to 255(ASCII range). Counting sort uses an extra constant space proportional to range of data.

Question 22

Correct

Which of the following is true about merge sort?

A Merge Sort works better than quick sort if data is accessed from slow sequential memory.

B Merge Sort is stable sort by nature

C Merge sort outperforms heap sort in most of the practical situations.

All of the above.

Sorting **QuickSort** **MergeSort** **HeapSort**
Discuss it

Question 22 Explanation:
See Merge Sort and this.

Question 23

Wrong

Given an array where numbers are in range from 1 to n^6, which sorting algorithm can be used to sort these number in linear time?

Not possible to sort in linear time

Radix Sort

C Counting Sort

D Quick Sort

Sorting **QuickSort** **RadixSort** **CountingSort**
Discuss it

Question 23 Explanation:
See Radix Sort for explanation.

Question 24

Correct

In quick sort, for sorting n elements, the (n/4)th smallest element is selected as pivot using an O(n) time algorithm. What is the worst case time complexity of the quick sort? <pre> (A) θ(n) (B) θ (nLogn) (C) θ(n^2) (D) θ(n^2 log n) </pre>

A A

B

C C

D D

Analysis of Algorithms **Sorting** **GATE-CS-2009** **QuickSort**
Discuss it

Question 24 Explanation:
Answer(B) The recursion expression becomes: T(n) = T(n/4) + T(3n/4) + cn After solving the above recursion, we get \theta(nLogn).

Question 26

Correct

Let P be a QuickSort Program to sort numbers in ascending order using the first element as pivot. Let t1 and t2 be the number of comparisons made by P for the inputs {1, 2, 3, 4, 5} and {4, 1, 5, 3, 2} respectively. Which one of the following holds?

A t1 = 5

B t1 < t2

t1 > t2

D t1 = t2

Sorting **GATE-CS-2014-(Set-1)** **QuickSort**
Discuss it

Question 26 Explanation:
When first element or last element is chosen as pivot, Quick Sort's worst case occurs for the sorted arrays. In every step of quick sort, numbers are divided as per the following recurrence. T(n) = T(n-1) + O(n)

Question 27

Wrong

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

$O(n^2)$

O(nLogn)

C Theta(nLogn)

D $O(n^3)$

Analysis of Algorithms **Sorting** **GATE-CS-2014-(Set-3)** **QuickSort**
Discuss it

Question 27 Explanation:
For any input, there are some permutations for which worst case will be $O(n^2)$. In some case, choosing the middle element minimizes the chances of encountering $O(n^2)$, but in worst case it can go to $O(n^2)$. Whichever element we take as Pivot, either first or middle, worst case will be $O(n^2)$ since Pivot is fixed in position. While choosing a random pivot minimizes the chances of encountering worst case i.e. $O(n^2)$. Refer this article on Quick Sort.

Question
28

Question 28

Wrong

In a permutation a1.....an of n distinct integers, an inversion is a pair (ai, aj) such that i < j and ai > aj. What would be the worst case time complexity of the Insertion Sort algorithm, if the inputs are restricted to permutations of 1.....n with at most n inversions?

$\Theta(n^2)$

B $\Theta(n \log n)$

C $\Theta(n^{1.5})$

$\Theta(n)$

Analysis of Algorithms **Sorting** **GATE-CS-2003** **InsertionSort**
Discuss it

Question 28 Explanation:

Insertion sort runs in Θ(n + f(n)) time, where f(n) denotes the number of inversion initially present in the array being sorted. Source: http://cs.xidian.edu.cn/jpkc/Algorithm/down/Solution%20to%202-4%20Inversions.pdf

Question 29

Correct

Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort?

A O(n)

B O(n Log n)

$O(n^2)$

D O(n!)

Analysis of Algorithms **Sorting** **GATE-CS-2001** **QuickSort**
Discuss it

Question 29 Explanation:
Randomized quicksort has expected time complexity as O(nLogn), but worst case time complexity remains same. In worst case the randomized function can pick the index of corner element every time.

Question 30

Wrong

Which of the following changes to typical QuickSort improves its performance on average and are generally done in practice.

```
1) Randomly picking up to make worst case less
   likely to occur.
2) Calling insertion sort for small sized arrays
   to reduce recursive calls.
3) QuickSort is tail recursive, so tail call
   optimizations can be done.
4) A linear time median searching algorithm is used
   to pick the median, so that the worst case time
   reduces to O(nLogn)
```

A 1 and 2

2, 3, and 4

1, 2 and 3

D 2, 3 and 4

Sorting GATE-CS-2015 (Mock Test) QuickSort
Discuss it

Question 30 Explanation:
The 4th optimization is generally not used, it reduces the worst case time complexity to O(nLogn), but the hidden constants are very high.

Question 31

Correct

Which one of the following is the recurrence equation for the worst case time complexity of the Quicksort algorithm for sorting n(≥ 2) numbers? In the recurrence equations given in the options below, c is a constant.

A $T(n) = 2T (n/2) + cn$

$T(n) = T(n – 1) + T(0) + cn$

C $T(n) = 2T (n – 2) + cn$

D $T(n) = T(n/2) + cn$

Analysis of Algorithms Sorting GATE-CS-2015 (Set 1) QuickSort
Discuss it

Question 31 Explanation:
In worst case, the chosen pivot is always placed at a corner position and recursive call is made for following. a) for subarray on left of pivot which is of size n-1 in worst case. b) for subarray on right of pivot which is of size 0 in worst case.

Question 32

Wrong

Assume that a mergesort algorithm in the worst case takes 30 seconds for an input of size 64. Which of the following most closely approximates the maximum input size of a problem that can be solved in 6 minutes?

A 256

512

1024

D 2048

Sorting GATE-CS-2015 (Set 3) MergeSort
Discuss it

Question 32 Explanation:

```
Time complexity of merge sort is Θ(nLogn)

c*64Log64 is 30
c*64*6 is 30
c is 5/64

For time 6 minutes

5/64*nLogn = 6*60

nLogn = 72*64 = 512 * 9

n = 512.
```

Question 33

Correct

The worst case running times of Insertion sort, Merge sort and Quick sort, respectively, are:

A $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$

B $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \text{ Log } n)$

C $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$

$\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Analysis of Algorithms Sorting GATE-CS-2016 (Set 1)
Discuss it

Question 33 Explanation:

- Insertion Sort takes $\Theta(n^2)$ in worst case as we need to run two loops. The outer loop is needed to one by one pick an element to be inserted at right position. Inner loop is used for two things, to find position of the element to be inserted and moving all sorted greater elements one position ahead. Therefore the worst case recursive formula is $T(n) = T(n-1) + \Theta(n)$.
- Merge Sort takes $\Theta(n \text{ Log } n)$ time in all cases. We always divide array in two halves, sort the two halves and merge them. The recursive formula is $T(n) = 2T(n/2) + \Theta(n)$.
- QuickSort takes $\Theta(n^2)$ in worst case. In QuickSort, we take an element as pivot and partition the array around it. In worst case, the picked element is always a corner element and

recursive formula becomes T(n) = T(n-1) + Θ(n). An example scenario when worst case happens is, arrays is sorted and our code always picks a corner element as pivot.

Question 34

Correct

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in ascending order, which of the following are TRUE ?

```
I.   Quicksort runs in Θ(n^2) time
II.  Bubblesort runs in Θ(n^2) time
III. Mergesort runs in  Θ(n) time
IV.  Insertion sort runs in  Θ(n) time
```

A I and II only

B I and III only

C II and IV only

I and IV only

Analysis of Algorithms **Sorting** **GATE-CS-2016 (Set 2)**
Discuss it

Question 34 Explanation:
I. Given an array in ascending order, Recurrence relation for total number of comparisons for quicksort will be T(n) = T(n-1)+O(n) //partition algo will take O(n) comparisons in any case. = O(n^2) **II.** Bubble Sort runs in Θ(n^2) time If an array is in ascending order, we could make a small modification in Bubble Sort Inner for loop which is responsible for bubbling the kth largest element to the end in kth iteration. Whenever there is no swap after the completion of inner for loop of bubble sort in any iteration, we can declare that array is sorted in case of Bubble Sort taking O(n) time in Best Case. **III.** Merge Sort runs in Θ(n) time Merge Sort relies on Divide and Conquer paradigm to sort an array and there is no such worst or best case input for merge sort. For any sequence, Time complexity will be given by following recurrence relation, T(n) = 2T(n/2) + Θ(n) // In-Place Merge algorithm will take Θ(n) due to copying an entire array. = Θ(nlogn) **IV.** Insertion sort runs in Θ(n) time Whenever a new element which will be greater than all the elements of the intermediate sorted sub-array (because given array is sorted) is added, there won't be any swap but a single comparison. In n-1 passes we will be having 0 swaps and n-1 comparisons. Total time complexity = O(n) // N-1 Comparisons This solution is contributed by **Pranjul Ahuja**

//// For an array already sorted in ascending order, Quicksort has a complexity Θ(n^2) [Worst Case] Bubblesort has a complexity Θ(n) [Best Case] Mergesort has a complexity Θ(n log n) [Any Case] Insertsort has a complexity Θ(n) [Best Case]

Question 35

Wrong

Assume that we use Bubble Sort to sort n distinct elements in ascending order. When does the best case of Bubble Sort occur?

When elements are sorted in ascending order

B When elements are sorted in descending order

C When elements are not sorted by any order

There is no best case for Bubble Sort. It always takes O(n*n) time

Sorting BubbleSort
Discuss it

Question 36

Correct

If we use Radix Sort to sort n integers in the range $(n^{k/2},n^k]$, for some k>0 which is independent of n, the time taken would be?

A Θ(n)

B Θ(kn)

Θ(nlogn)

D $\Theta(n^2)$

Analysis of Algorithms Sorting RadixSort Gate IT 2008
Discuss it

Question 36 Explanation:
Radix sort time complexity = O(wn)
for n keys of word size= w
=>w = $\log(n^k)$
O(wn)=O(klogn.n)
=> kO(nlogn)

Question 37

Correct

Consider an array of elements arr[5]= {5,4,3,2,1} , what are the steps of insertions done while doing insertion sort in the array.

4 5 3 2 1 3 4 5 2 1 2 3 4 5 1 1 2 3 4 5

B 5 4 3 1 2 5 4 1 2 3 5 1 2 3 4 1 2 3 4 5

C 4 3 2 1 5 3 2 1 5 4 2 1 5 4 3 1 5 4 3 2

D 4 5 3 2 1 2 3 4 5 1 3 4 5 2 1 1 2 3 4 5

Sorting **InsertionSort**
Discuss it

Question 37 Explanation:
In the insertion sort , just imagine that the first element is already sorted and all the right side Elements are unsorted, we need to insert all elements one by one from left to right in the sorted Array. Sorted : 5 unsorted : 4 3 2 1 Insert all elements less than 5 on the left (Considering 5 as the key) Now key value is 4 and array will look like this Sorted : 4 5 unsorted : 3 2 1 Similarly for all the cases the key will always be the newly inserted value and all the values will be compared to that key and inserted in to proper position.

Question 39

Wrong

Which of the following statements is correct with respect to insertion sort ?

```
*Online - can sort a list at runtime
*Stable - doesn't change the relative
          order of elements with equal keys.
```

Insertion sort is stable, online but not suited well for large number of elements.

Insertion sort is unstable and online

C Insertion sort is online and can be applied to more than 100 elements

D Insertion sort is stable & online and can be applied to more than 100 elements

Sorting **InsertionSort**
Discuss it

Question 39 Explanation:
Time taken by algorithm is good for small number of elements, but increases quadratically for large number of elements.

Question 40

Wrong

Consider the array A[]= {6,4,8,1,3} apply the insertion sort to sort the array . Consider the cost associated with each sort is 25 rupees , what is the total cost of the insertion sort when element 1 reaches the first position of the array ?

50

B 25

75

D 100

Sorting **C Arrays** **InsertionSort**
Discuss it

Question 40 Explanation:
When the element 1 reaches the first position of the array two comparisons are only required hence 25 * 2= 50 rupees. *step 1: 4 6 8 1 3 . *step 2: 1 4 6 8 3.

Question 42

Wrong

What is the best sorting algorithm to use for the elements in array are more than 1 million in general?

Merge sort.

B Bubble sort.

Quick sort.

D Insertion sort.

Sorting **QuickSort** **InsertionSort** **MergeSort**
Discuss it

Question 42 Explanation:
Most practical implementations of Quick Sort use randomized version. The randomized version has expected time complexity of O(nLogn). The worst case is possible in randomized version also, but worst case doesn't occur for a particular pattern (like sorted array) and randomized Quick Sort works well in practice. Quick Sort is also a cache friendly sorting algorithm as it has good locality of

reference when used for arrays. Quick Sort is also tail recursive, therefore tail call optimizations is done.

Question 43

Wrong

Which of the below given sorting techniques has highest best-case runtime complexity.

A Quick sort

Selection sort

C Insertion sort

Bubble sort

Sorting **GATE 2017 Mock**
Discuss it

Question 43 Explanation:
Quick sort best case time complexity is O(n logn)
Selection sort best case time complexity is O(n^2)
Insertion sort best case time complexity is O(n)
Bubble sort best case time complexity is O(n)

Question 45

Wrong

If one uses straight two-way merge sort algorithm to sort the following elements in ascending order *20, 47, 15, 8, 9, 4, 40, 30, 12, 17* then the order of these elements after the second pass of the algorithm is:

8, 9, 15, 20, 47, 4, 12, 17, 30, 40

8, 15, 20, 47, 4, 9, 30, 40, 12, 17

C 15, 20, 47, 4, 8, 9, 12, 30, 40, 17

D 4, 8, 9, 15, 20, 47, 12, 17, 30, 40

Analysis of Algorithms **Sorting** **GATE CS 1999**
Discuss it

Question 45 Explanation:

In first pass the elements are sorted in n/4 (first 2 elements in each group) sub arrays but in second pass the elements are sorted in n/2 (first 4 elements in each group) sub arrays.

Question 46

Correct

Quicksort is run on two inputs shown below to sort in ascending order taking first element as pivot,

```
(i) 1, 2, 3,......., n
(ii) n, n-1, n-2,......, 2, 1
```

Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

A C1 < C2

B C1 > C2

C1 = C2

D We cannot say anything for arbitrary n

Analysis of Algorithms **Sorting** **QuickSort** **GATE CS 1996**
Discuss it

Question 46 Explanation:
Quick Sort performs in worst case when input is sorted in either ascending order or descending order. So, quick sort will return equal number of comparisons for both given inputs. Option (C) is correct.

Question 50

Wrong

You are given a sequence of n elements to sort. The input sequence consists of n/k subsequences,each containing k elements. The elements in a given subsequence are all smaller than the elements in the succeeding subsequence and larger than the elements in the preceding subsequence. Thus, all that is needed to sort the whole sequence of length n is to sort the k elements in each of the n/k subsequences. The lower bound on the number of comparisons needed to solve this variant of the sorting problem is

A Ω (n)

Ω (n/k)

Ω (nlogk)

D Ω (n/klogn/k)

Sorting UGC-NET CS 2017 Nov - III
Discuss it

C E BINE SUS

Question 51

Wrong

Which of the following is true for computation time in insertion, deletion and finding maximum and minimum element in a sorted array ?

A Insertion – 0(1), Deletion – 0(1), Maximum – 0(1), Minimum – 0(l)

B Insertion – 0(1), Deletion – 0(1), Maximum – 0(n), Minimum – 0(n)

Insertion – 0(n), Deletion – 0(n), Maximum – 0(1), Minimum – 0(1)

Insertion – 0(n), Deletion – 0(n), Maximum – 0(n), Minimum – 0(n)

Sorting UGC NET CS 2017 Jan - II
Discuss it

Question 51 Explanation:

1. In a sorted array, if we want to insert or delete then we have to traverse whole array and check where is the suitable position, so it will take O(n).
2. If array is sorted then end position will tell the maximum or minimum, so finding maximum or minimum will take O(1).

So, option (C) is correct.

Question 52

Correct

Consider the following sorting algorithms. I. Quicksort II. Heapsort III. Mergesort Which of them perform in least time in the worst case?

A I and II only

II and III only

C III only

D I, II and III

Sorting ISRO CS 2014
Discuss it

Question 52 Explanation:
Worst case complexity of Heap Sort and Merge Sort is O(n log n) whereas the worst case complexity of Quicksort is $O(n^2)$. (B) option is correct.

Question 53

Wrong

Which of the following algorithms sort n integers, having the range 0 to $(n^2 - 1)$, in ascending order in O(n) time ?

A Selection sort

B Bubble sort

Radix sort

Insertion sort

Sorting UGC NET CS 2015 Jun - II
Discuss it

Question 53 Explanation:

- Selection sort takes $O(n^2)$ time.
- Bubble sort takes $O(n^2)$ time.
- Radix sort takes O(n) time.
- Insertion sort takes $O(n^2)$ time.

So, option (C) is correct.

Question 54

Correct

Which one of the following in place sorting algorithms needs the minimum number of swaps?

A Quick sort

B Insertion sort

Selection sort

D Heap sort

Sorting **ISRO CS 2011**
Discuss it

Question 54 Explanation:
Selection sort takes minimum number of swaps to sort an array. It takes maximum of O(n) comparisons to sort an array with n elements. Refer: Selection sort

Question 56

Correct

Selection sort algorithm design technique is an example of

Greedy method

B Divide-and-conquer

C Dynamic Programming

D Backtracking

Sorting **ISRO CS 2007**
Discuss it

Question 56 Explanation:
The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array. 1) The subarray which is already sorted. 2) Remaining subarray which is unsorted. In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray. Clearly, it is a greedy approach to sort the array. Option (A) is correct.

Question 58

Wrong

You have to sort a list L, consisting of a sorted list followed by a few ‘random’ elements. Which of the following sorting method would be most suitable for such a task?

A Bubble sort

B Selection sort

Quick sort

Insertion sort

Sorting **UGC NET CS 2014 Dec - II**
Discuss it

Question 58 Explanation:
For a sorted list with few random numbers:

- Bubble sort will take $O(n^2)$ time in best case.
- Selection sort will take $O(n^2)$ time in best case.
- Quick sort will take $O(n^2)$ time in this case because it is the worst case for quick sort.
- Insertion sort will take O(n) So, option (D) is correct.

Question 1

Correct

Which of the following algorithms is NOT a divide & conquer algorithm by nature?

A Euclidean algorithm to compute the greatest common divisor

Heap Sort

C Cooley-Tukey fast Fourier transform

D Quick Sort

Divide and Conquer
Discuss it

Question 1 Explanation:
See Divide and Conquer

Question
2

Question 3

Wrong

Consider the polynomial p(x) = a0 + a1x + a2x^2 +a3x^3, where ai != 0, for all i. The minimum number of multiplications needed to evaluate p on an input x is:

3

B 4

6

D 9

Divide and Conquer
Discuss it

Question 3 Explanation:
Multiplications can be minimized using following order for evaluation of the given expression. p(x) = a0 + x(a1 + x(a2 + a3x))

Question 4

Correct

Maximum Subarray Sum problem is to find the subarray with maximum sum. For example, given an array {12, -13, -5, 25, -20, 30, 10}, the maximum subarray sum is 45. The naive solution for this problem is to calculate sum of all subarrays starting with every element and return the maximum of all. We can solve this using Divide and Conquer, what will be the worst case time complexity using Divide and Conquer.

A O(n)

O(nLogn)

C O(Logn)

D O(n^2)

Divide and Conquer
Discuss it

Question 4 Explanation:
See http://www.geeksforgeeks.org/divide-and-conquer-maximum-sum-subarray/

Question 5

Correct

Consider a situation where you don't have function to calculate power (pow() function in C) and you need to calculate x^n where x can be any number and n is a positive integer. What can be the best possible time complexity of your power function?

A O(n)

B O(nLogn)

C O(LogLogn)

O(Logn)

Divide and Conquer
Discuss it

Question 5 Explanation:
We can calculate power using divide and conquer in O(Logn) time. See http://www.geeksforgeeks.org/write-a-c-program-to-calculate-powxn/.

Question 6

Wrong

Consider the problem of searching an element x in an array 'arr[]' of size n. The problem can be solved in O(Logn) time if. 1) Array is sorted 2) Array is sorted and rotated by k. k is given to you and k <= n 3) Array is sorted and rotated by k. k is NOT given to you and k <= n 4) Array is not sorted

A 1 Only

1 & 2 only

1, 2 and 3 only

D 1, 2, 3 and 4

Divide and Conquer
Discuss it

Question 6 Explanation:
See http://www.geeksforgeeks.org/search-an-element-in-a-sorted-and-pivoted-array/

Question 9

Wrong

Consider the problem of computing min-max in an unsorted array where min and max are minimum and maximum elements of array. Algorithm A1 can compute min-max in a1 comparisons without divide and conquer. Algorithm A2 can compute min-max in a2 comparisons by scanning the array linearly. What could be the relation between a1 and a2 considering the worst case scenarios?

A a1 < a2

a1 > a2

a1 = a2

D Depends on the input

Divide and Conquer **GATE 2017 Mock**
Discuss it

Question 9 Explanation:
When Divide and Conquer is used to find the minimum-maximum element in an array, Recurrence relation for the number of comparisons is
T(n) = 2T(n/2) + 2 where 2 is for comparing the minimums as well the maximums of the left and right subarrays
On solving, T(n) = 1.5n - 2.
While doing linear scan, it would take 2*(n-1) comparisons in the worst case to find both minimum as well maximum in one pass.

Question 1

Correct

Which of the following standard algorithms is not a Greedy algorithm?

A Dijkstra's shortest path algorithm

B Prim's algorithm

C Kruskal algorithm

D Huffman Coding

Bellmen Ford Shortest path algorithm

Greedy Algorithms
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/ and http://www.geeksforgeeks.org/dynamic-programming-set-23-bellman-ford-algorithm/

Question 3

Wrong

A networking company uses a compression technique to encode the message before transmitting over the network. Suppose the message contains the following characters with their frequency:

character	Frequency
a	5
b	9
c	12
d	13
e	16
f	45

Note : Each character in input message takes 1 byte. If the compression technique used is Huffman Coding, how many bits will be saved in the message?

A 224

800

576

D 324

Greedy Algorithms
Discuss it

Question 3 Explanation:

```
Total number of characters in the message = 100.
Each character takes 1 byte. So total number of bits needed = 800.

After Huffman Coding, the characters can be represented with:
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
Total number of bits needed = 224
Hence, number of bits saved = 800 - 224 = 576
See here for complete explanation and algorithm.
```

Question 4

Correct

What is the time complexity of Huffman Coding?

A O(N)

O(NlogN)

C O(N(logN)^2)

D O(N^2)

Greedy Algorithms
Discuss it

Question 4 Explanation:
O(nlogn) where n is the number of unique characters. If there are n nodes, extractMin() is called 2*(n – 1) times. extractMin() takes O(logn) time as it calles minHeapify(). So, overall complexity is O(nlogn). See here for more details of the algorithm.

Question 5

Wrong

In question #2, which of the following represents the word *"dead"*?

A 1011111100101

B 0100000011010

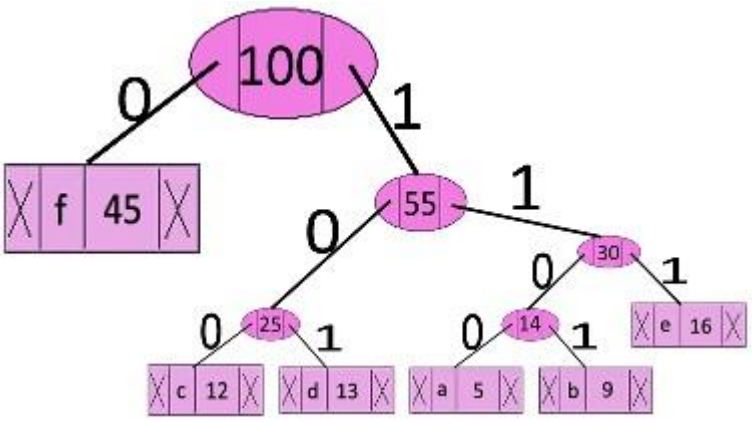
Both A and B

None of these

Greedy Algorithms
Discuss it

Question 5 Explanation:

The *Huffman Tree* generated is:



```
character   code-word
    f          0
    c          100
    d          101
    a          1100
    b          1101
    e          111
```

The word *dead* can be represented as: 101 111 1100 101 However, the alternative codeword can also be found by assigning 1 to the left edge and 0 to the right edge of the tree, i.e. *dead* can also be represented as: 010 000 0011 010 See here for more details of the algorithm.

Question 6

Wrong

Which of the following is true about Kruskal and Prim MST algorithms? Assume that Prim is implemented for adjacency list representation using Binary Heap and Kruskal is implemented using union by rank.

Worst case time complexity of both algorithms is same.

B Worst case time complexity of Kruskal is better than Prim

Worst case time complexity of Prim is better than Kruskal

Greedy Algorithms
Discuss it

Question 6 Explanation:
See following Kruskal's Minimum Spanning Tree Algorithm Prim's MST for Adjacency List Representation

Question 7

Wrong

Which of the following is true about Huffman Coding.

A Huffman coding may become lossy in some cases

B Huffman Codes may not be optimal lossless codes in some cases

In Huffman coding, no code is prefix of any other code.

All of the above

Greedy Algorithms
Discuss it

Question 7 Explanation:
Huffman coding is a lossless data compression algorithm. The codes assigned to input characters are Prefix Codes, means the codes are assigned in such a way that the code assigned to one character is not prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding.

Question 8

Correct

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. Which of the following is the Huffman code for the letter a, b, c, d, e, f?

0, 10, 110, 1110, 11110, 11111

B 11, 10, 011, 010, 001, 000

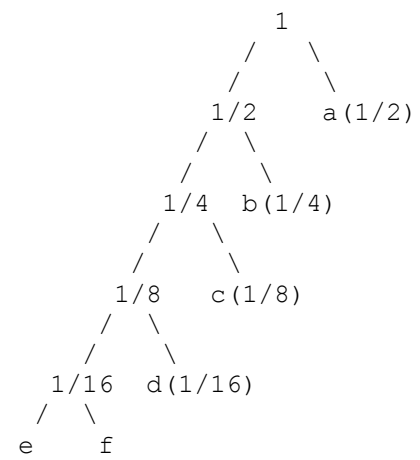
C 11, 10, 01, 001, 0001, 0000

D 110, 100, 010, 000, 001, 111

Greedy Algorithms **GATE-CS-2007**
Discuss it

Question 8 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.
```



Question

9

Question 9

Wrong

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. What is the average length of Huffman codes?

3

B 2.1875

C 2.25

1.9375

Greedy Algorithms **GATE-CS-2007**
Discuss it

Question 9 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.

                  1
                /   \
               /     \
             1/2     a(1/2)
            /  \
           /    \
         1/4   b(1/4)
        /   \
       /     \
     1/8    c(1/8)
     /  \
    /    \
  1/16   d(1/16)
  /  \
 e    f
The average length = (1*1/2 + 2*1/4 + 3*1/8 + 4*1/16 + 5*1/32 + 5*1/32)
                   = 1.9375
```

Question 11

Wrong

Consider the weights and values of items listed below. Note that there is only one unit of each item.

Item number	Weight (in Kgs)	Value (in Rupees)
1	10	60
2	7	28
3	4	20
4	2	24

The task is to pick a subset of these items such that their total weight is no more than 11 Kgs and their total value is maximized. Moreover, no item may be split. The total value of items picked by an optimal algorithm is denoted by V_{opt}. A greedy algorithm sorts the items by their value-to-weight ratios in descending order and packs them greedily, starting from the first item in the ordered list. The total value of items picked by the greedy algorithm is denoted by V_{greedy}. The value of $V_{opt} - V_{greedy}$ is _____ .

Note -This was Numerical Type question.

16

8

C 44

D 60

Greedy Algorithms Dynamic Programming GATE CS 2018

Discuss it

Question 11 Explanation:

Item No	Weight	Value	Value/Weight
1	10	60	6
2	7	28	4
3	4	20	5
4	2	24	12

After sorting :

Item No	Weight	Value	Value/Weight
4	2	24	12
1	10	60	6
3	4	20	5
2	7	28	4

First we will pick item_4 (Value weight ratio is highest). Second highest is item_1, but cannot be picked because of its weight. Now item_3 shall be picked. item_2 cannot be included because of its weight. Therefore, overall profit by V_{greedy} = 20+24 = 44 Hence, $\mathbf{V_{opt} - V_{greedy} = 60\text{-}44 = 16}$ So, answer is 16.

Question 12

Wrong

A text is made up of the characters a, b, c, d, e each occurring with the probability 0.11, 0.40, 0.16, 0.09 and 0.24 respectively. The optimal Huffman coding technique will have the average length of:

A 2.40

2.16

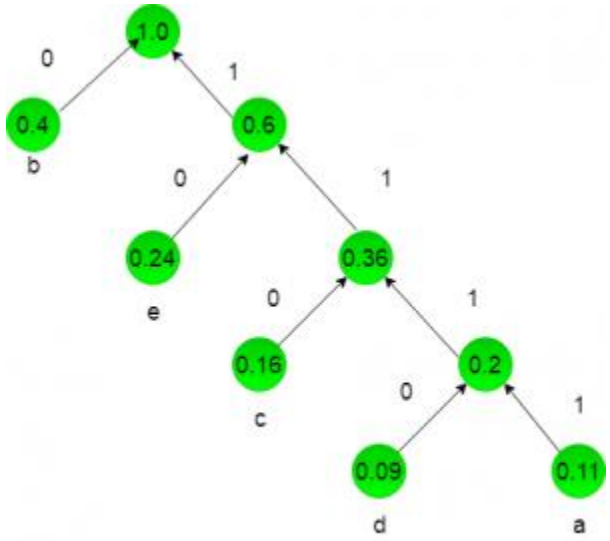
2.26

D 2.15

Greedy Algorithms UGC-NET CS 2017 Nov - III
Discuss it

Question 12 Explanation:

a = 0.11 b = 0.40 c = 0.16 d = 0.09 e = 0.24 we will draw a huffman tree:



Text now huffman coding for character:

```
 a = 1111
      b = 0
      c = 110
      d = 1111
      e = 10
lenghth for each character = no of bits * frequency of occurence:
a = 4 * 0.11
  = 0.44
b = 1 * 0.4
  =  0.4
c = 3 * 0.16
  = 0.48
d = 4 * 0.09
  =  0.36
e = 2 * 0.24
  = 0.48
Now add these lenght for average length:
 0.44 + 0.4 + 0.48 + 0.36 + 0.48 = 2.16
```

So, option (B) is correct.

Question 13

Correct

Six files F1, F2, F3, F4, F5 and F6 have 100, 200, 50, 80, 120, 150 records respectively. In what order should they be stored so as to optimize act. Assume each file is accessed with the same frequency

F3, F4, F1, F5, F6, F2

B F2, F6, F5, F1, F4, F3

C F1, F2, F3, F4, F5, F6

D Ordering is immaterial as all files are accessed with the same frequency.

Greedy Algorithms **ISRO CS 2015**
Discuss it

Question 13 Explanation:

```
This question is based on the Optimal Storage on Tape problem
which uses greedy approach to find the optimal time to retrieve them.
There are n programs of length L that are to be stored on a computer
tape. Associated with each program i is a length Li. So in order to
retrieve these programs most optimally, we need to store them in the
non-decreasing order of length Li.
So, the correct order is F3, F4, F1, F5, F6, F2
```

Question 14

Correct

Consider a job scheduling problem with 4 jobs J_1, J_2, J_3, J_4 and with corresponding deadlines: (d_1, d_2, d_3, d_4) = (4, 2, 4, 2). Which of the following is not a feasible schedule without violating any job schedule?

A J_2, J_4, J_1, J_3

J_4, J_1, J_2, J_3

C J_4, J_2, J_1, J_3

D J_4, J_2, J_3, J_1

Greedy Algorithms **ISRO CS 2007**
Discuss it

Question
15

Question 15

Correct

What is the other name of Dijkstra algorithm?

single-source shortest path problem

B multiple-source shortest path problem

C multiple-destination shortest path problem

D single-destination shortest path problem

Greedy Algorithms
Discuss it

Question 2

Correct

We use dynamic programming approach when

A We need an optimal solution

The solution has optimal substructure

C The given problem can be reduced to the 3-SAT problem

D It's faster than Greedy

Dynamic Programming
Discuss it

Question 2 Explanation:
http://www.geeksforgeeks.org/dynamic-programming-set-2-optimal-substructure-property/ Option (D) is incorrect because Greedy algorithms are generally faster than Dynamic programming. See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/

Question 3

Wrong

An algorithm to find the length of the longest monotonically increasing sequence of numbers in an array A[0 :n-1] is given below. Let Li denote the length of the longest monotonically increasing sequence starting at index i in the array.

Initialize $L_{n-1} = 1$

For all i such that $0 \le i \le n-2$

$$L_i = \begin{cases} 1 + L_{i+1} & \text{if } A[i] < A[i+1] \\ 1 & \text{Otherwise} \end{cases}$$

Finally the length of the longest monotonically increasing sequence is $\text{Max}(L_0, L_1, \ldots, L_{n-1})$.

Which of the following statements is TRUE?

The algorithm uses dynamic programming paradigm

The algorithm has a linear complexity and uses branch and bound paradigm

C The algorithm has a non-linear polynomial complexity and uses branch and bound paradigm

D The algorithm uses divide and conquer paradigm.

Dynamic Programming
Discuss it

Question 3 Explanation:
Please see http://www.geeksforgeeks.org/dynamic-programming-set-3-longest-increasing-subsequence/

Question 4

Wrong

Kadane algorithm is used to find:

Maximum sum subsequence in an array

Maximum sum subarray in an array

C Maximum product subsequence in an array

D Maximum product subarray in an array

Dynamic Programming
Discuss it

Question 4 Explanation:
Kadane algorithm is used to find the maximum sum subarray in an array. It runs in O(n) time complexity. See details of the algorithm here.

Question 5

Wrong

Four matrices M1, M2, M3 and M4 of dimensions pxq, qxr, rxs and sxt respectively can be multiplied is several ways with different number of total scalar multiplications. For example, when multiplied as ((M1 X M2) X (M3 X M4)), the total number of multiplications is pqr + rst + prt. When multiplied as (((M1 X M2) X M3) X M4), the total number of scalar multiplications is pqr + prs + pst. If p = 10, q = 100, r = 20, s = 5 and t = 80, then the number of scalar multiplications needed is

248000

B 44000

19000

D 25000

Dynamic Programming
Discuss it

Question 5 Explanation:
It is basically matrix chain multiplication problem. We get minimum number of multiplications using ((M1 X (M2 X M3)) X M4). Total number of multiplications = 100x20x5 (for M2 x M3) + 10x100x5 + 10x5x80 = 19000.

Question 6

Wrong

The subset-sum problem is defined as follows. Given a set of n positive integers, S = {a1 ,a2 ,a3 ,…,an} and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array X, with n rows and W+1 columns. X[i, j],1 <= i <= n, 0 <= j <= W, is TRUE if and only if there is a subset of {a1 ,a2 ,...,ai} whose elements sum to j. Which of the following is valid for 2 <= i <= n and ai <= j <= W?

A X[i, j] = X[i - 1, j] ∨ X[i, j -ai]

X[i, j] = X[i - 1, j] ∨ X[i - 1, j - ai]

C X[i, j] = X[i - 1, j] ∧ X[i, j - ai]

X[i, j] = X[i - 1, j] ∧ X[i -1, j - ai]

Dynamic Programming
Discuss it

Question 6 Explanation:
X[I, j] (2 <= i <= n and ai <= j <= W), is true if any of the following is true 1) Sum of weights excluding ai is equal to j, i.e., if X[i-1, j] is true. 2) Sum of weights including ai is equal to j, i.e., if X[i-1, j-ai] is true so that we get (j – ai) + ai as j See http://www.geeksforgeeks.org/dynamic-programming-subset-sum-problem/ for details.

Question 7

Wrong

In the above question, which entry of the array X, if TRUE, implies that there is a subset whose elements sum to W?

X[1, W]

B X[n ,0]

X[n, W]

D X[n -1, n]

Dynamic Programming
Discuss it

Question 7 Explanation:
If we get the entry X[n, W] as true then there is a subset of {a1, a2, .. an} that has sum as W.
Reference: http://en.wikipedia.org/wiki/Subset_sum_problem

Question 8

Wrong

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences X[m] and Y[n] of lengths m and n respectively, with indexes of X and Y starting from 0. We wish to find the length of the longest common sub-sequence(LCS) of X[m] and Y[n] as l(m,n), where an incomplete recursive definition for the function l(i,j) to compute the length of The LCS of X[m] and Y[n] is given below:

```
l(i,j) = 0, if either i=0 or j=0
       = expr1, if i,j > 0 and X[i-1] = Y[j-1]
       = expr2, if i,j > 0 and X[i-1] != Y[j-1]
```

expr1 ≡ l(i-1, j) + 1

B expr1 ≡ l(i, j-1)

expr2 ≡ max(l(i-1, j), l(i, j-1))

D expr2 ≡ max(l(i-1,j-1),l(i,j))

Dynamic Programming **GATE-CS-2009**
Discuss it

Question 8 Explanation:
In Longest common subsequence problem, there are two cases for X[0..i] and Y[0..j]

```
1) The last characters of two strings match.
   The length of lcs is length of lcs of X[0..i-1] and Y[0..j-1]
2) The last characters don't match.
   The length of lcs is max of following two lcs values
   a) LCS of X[0..i-1] and Y[0..j]
   b) LCS of X[0..i] and Y[0..j-1]
```

Question 9

Wrong

Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B. Then x + 10y = ___.

A 33

B 23

43

34

Dynamic Programming GATE-CS-2014-(Set-2)
Discuss it

Question 9 Explanation:
//The LCS is of length 4. There are 3 LCS of length 4 "qprr", "pqrr" and qpqr A subsequence is a sequence that can be derived from another sequence by selecting zero or more elements from it, without changing the order of the remaining elements. Subsequence need not be contiguous. Since the length of given strings A = “qpqrr” and B = “pqprqrp” are very small, we don’t need to build a 5x7 matrix and solve it using dynamic programming. Rather we can solve it manually just by brute force. We will first check whether there exist a subsequence of length 5 since min_length(A,B) = 5. Since there is no subsequence , we will now check for length 4. “qprr”, “pqrr” and “qpqr” are common in both strings. X = 4 and Y = 3 X + 10Y = 34 This solution is contributed by **Pranjul Ahuja**

Question 10

Correct

Let A1, A2, A3, and A4 be four matrices of dimensions 10 x 5, 5 x 20, 20 x 10, and 10 x 5, respectively. The minimum number of scalar multiplications required to find the product A1A2A3A4 using the basic matrix multiplication method is

1500

B 2000

C 500

D 100

Dynamic Programming GATE-CS-2016 (Set 2)
Discuss it

Question 10 Explanation:
We have many ways to do matrix chain multiplication because matrix multiplication is associative. In other words, no matter how we parenthesize the product, the result of the matrix chain multiplication obtained will remain the same. Here we have four matrices A1, A2, A3, and A4, we would have: ((A1A2)A3)A4 = ((A1(A2A3))A4) = (A1A2)(A3A4) = A1((A2A3)A4) = A1(A2(A3A4)). However, the order in which we parenthesize the product affects the number of simple arithmetic operations needed to compute the product, or the efficiency. Here, A1 is a 10 × 5 matrix, A2 is a 5 x 20 matrix, and A3 is a 20 x 10 matrix, and A4 is 10 x 5. If we multiply two matrices A and B of order l x m and m x n respectively,then the number of scalar multiplications in the multiplication of A and B will be lxmxn. Then, The number of scalar multiplications required in the following sequence of matrices will be : A1((A2A3)A4) = (5 x 20 x 10) + (5 x 10 x 5) + (10 x 5 x 5) = 1000 + 250 + 250 = 1500. All other parenthesized options will require number of multiplications more than 1500.

Question 12

Wrong

Consider the following two sequences :

X = < B, C, D, C, A, B, C >, and
Y = < C, A, D, B, C, B >
The length of longest common subsequence of X and Y is :

A 5

3

4

D 2

Dynamic Programming UGC-NET CS 2017 Nov - III
Discuss it

Question 13

Correct

Consider a sequence F_{00} defined as : $F_{00}(0) = 1$, $F_{00}(1) = 1$ $F_{00}(n) = 10 * F_{00}(n - 1) + 100$ $F_{00}(n - 2)$ for $n \geq 2$ Then what shall be the set of values of the sequence F_{00} ?

(1, 110, 1200)

B (1, 110, 600, 1200)

C (1, 2, 55, 110, 600, 1200)

D (1, 55, 110, 600, 1200)

Dynamic Programming **UGC NET CS 2017 Jan - II**
Discuss it

Question 13 Explanation:

```
 F00(0) = 1, F00(1) = 1
 F00(n) = 10 * F00(n - 1) + 100
 F00(2) = 10 * F00(1) + 100
 = 10 * 1 + 100
 = 10 + 100
 = 110
Similarly:
F00(3) = 10 * F00(2) + 100
 = 10 * 110 + 100
 = 1100 + 100
 = 1200
The sequence will be (1, 110, 1200).
```

So, (A) will be the answer.

Question 14

Correct

The following paradigm can be used to find the solution of the problem in minimum time: Given a set of non-negative integer, and a value K, determine if there is a subset of the given set with sum equal to K:

A Divide and Conquer

Dynamic Programming

C Greedy Algorithm

D Branch and Bound

Dynamic Programming **ISRO CS 2018**
Discuss it

Question 14 Explanation:
Given problem is Subset-sum problem in which a set of non-negative integers, and a value sum is given, to determine if there is a subset of the given set with sum equal to given sum. With recursion technique, time complexity of the above problem is exponential. We can solve the problem in Pseudo-polynomial time using Dynamic programming. Refer: Subset Sum Problem Option (B) is correct

Question 15

Wrong

What happens when a top-down approach of dynamic programming is applied to any problem?

It increases both, the time complexity and the space complexity

It increases the space complexity and decreases the time complexity.

C It increases the time complexity and decreases the space complexity

D It decreases both, the time complexity and the space complexity

Dynamic Programming
Discuss it

Question 1

Correct

Which of the following algorithms can be used to most efficiently determine the presence of a cycle in a given graph ?

Depth First Search

B Breadth First Search

C Prim's Minimum Spanning Tree Algorithm

D Kruskal' Minimum Spanning Tree Algorithm

Graph Traversals
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/applications-of-depth-first-search/ and
http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 2

Correct

Traversal of a graph is different from tree because

There can be a loop in graph so we must maintain a visited flag for every vertex

B DFS of a graph uses stack, but inorrder traversal of a tree is recursive

C BFS of a graph uses queue, but a time efficient BFS of a tree is recursive.

D All of the above

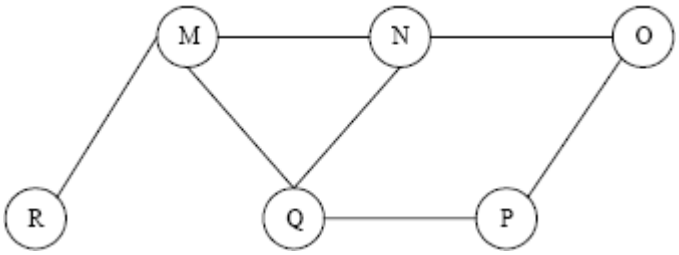
Graph Traversals
Discuss it

Question 2 Explanation:
See http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 4

Correct

The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is



A MNOPQR

B NQMPOR

QMNPRO

D QMNPOR

Graph Traversals
Discuss it

Question 4 Explanation:
Option (A) is MNOPQR. It cannot be a BFS as the traversal starts with M, but O is visited before N and Q. In BFS all adjacent must be visited before adjacent of adjacent. Option (B) is NQMPOR. It also cannot be BFS, because here, P is visited before O. (C) and (D) match up to QMNP. We see that M was added to the queue before N and P (because M comes before NP in QMNP). Because R is M's neighbor, it gets added to the queue before the neighbor of N and P (which is O). Thus, R is visited before O.

Question 5

Wrong

Let G be an undirected graph. Consider a depth-first traversal of G, and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statements is always true? (GATE CS 2000)

A {u,v} must be an edge in G, and u is a descendant of v in T

B {u,v} must be an edge in G, and v is a descendant of u in T

If {u,v} is not an edge in G then u is a leaf in T

If {u,v} is not an edge in G then u and v must have the same parent in T

Graph Traversals
Discuss it

Question 5 Explanation:

```
In DFS, if 'v' is visited
after 'u', then one of the following is true.
1) (u, v) is an edge.
      u
    /   \
   v     w
  /     / \
 x     y   z

2) 'u' is a leaf node.
      w
    /   \
   x     v
  /     / \
 u     y   z
```

In DFS, after visiting a node, we first recur for all unvisited children. If there are no unvisited children (u is leaf), then control goes back to parent and parent then visits next unvisited children.

Question 7

Correct

Make is a utility that automatically builds executable programs and libraries from source code by reading files called makefiles which specify how to derive the target program. Which of the following standard graph algorithms is used by Make.

A Strongly Connected Components

Topological Sorting

C Breadth First Search

D Dijkstra's Shortest Path

Graph Traversals
Discuss it

Question 7 Explanation:
Make can decide the order of building a software using topological sorting. Topological sorting produces the order considering all dependencies provide by makefile. See following for details.
Topological Sorting

Question 9

Correct

Which of the following condition is sufficient to detect cycle in a directed graph?

A There is an edge from currently being visited node to an already visited node.

There is an edge from currently being visited node to an ancestor of currently visited node in DFS forest.

C Every node is seen twice in DFS.

D None of the bove

Graph Traversals
Discuss it

Question 9 Explanation:
See Detect Cycle in a Graph

Question 10

Wrong

Is following statement true/false If a DFS of a directed graph contains a back edge, any other DFS of the same graph will also contain at least one back edge. Source:
http://courses.csail.mit.edu/6.006/oldquizzes/solutions/q2-s2009-sol.pdf

True

False

Graph Traversals
Discuss it

Question 10 Explanation:
A back edge means a cycle in graph. So if there is a cycle, all DFS traversals would contain at least one back edge.

Question 14

Wrong

Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time on Depth First Search of G? Assume that the graph is represented using adjacency matrix.

A O(n)

O(m+n)

O(n^2)

D O(mn)

Graph Traversals **GATE-CS-2014-(Set-1)**
Discuss it

Question 14 Explanation:
Depth First Search of a graph takes O(m+n) time when the graph is represented using adjacency list. In adjacency matrix representation, graph is represented as an "n x n" matrix. To do DFS, for every vertex, we traverse the row corresponding to that vertex to find all adjacent vertices (In adjacency list representation we traverse only the adjacent vertices of the vertex). Therefore time complexity becomes O(n^2)

Question 15

Correct

Consider the tree arcs of a BFS traversal from a source node W in an unweighted, connected, undirected graph. The tree T formed by the tree arcs is a data structure for computing.

A the shortest path between every pair of vertices.

the shortest path from W to every vertex in the graph.

C the shortest paths from W to only those nodes that are leaves of T.

D the longest path in the graph

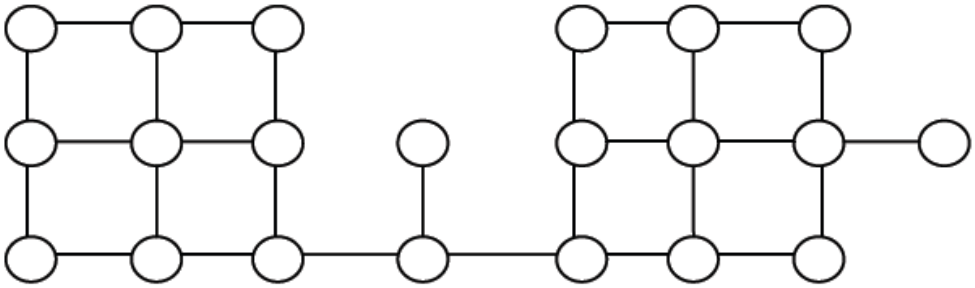
Graph Traversals GATE-CS-2014-(Set-2)
Discuss it

Question 15 Explanation:
BFS always produces shortest path from source to all other vertices in an unweighted graph.

Question 16

Wrong

Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is



_____.

A 17

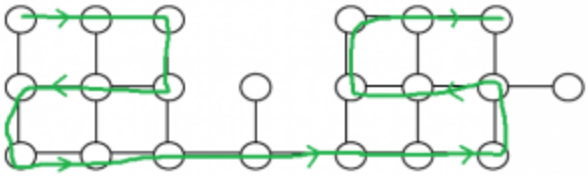
B 18

19

20

Graph Traversals GATE-CS-2014-(Set-3)
Discuss it

Question 16 Explanation:
The following diagram shows the worst case situation where the recursion tree has maximum depth.



So the recursion depth is 19 (including the first node).

Question 17

Wrong

Let T be a depth first search tree in an undirected graph G. Vertices u and n are leaves of this tree T. The degrees of both u and n in G are at least 2. which one of the following statements is true?

There must exist a vertex w adjacent to both u and n in G

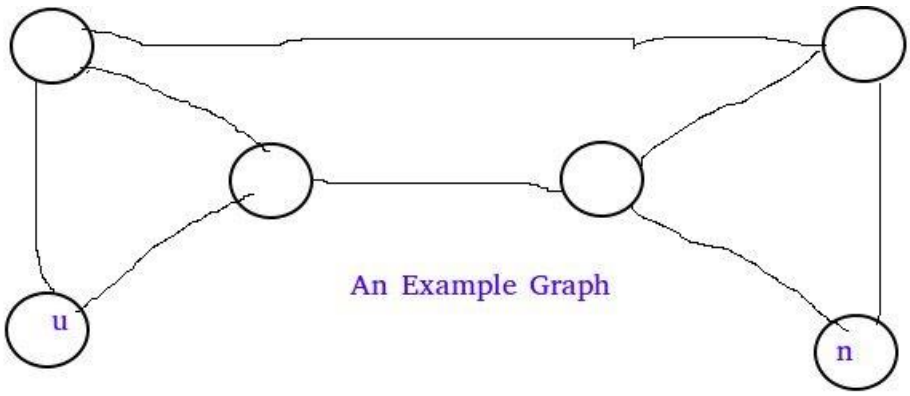
B There must exist a vertex w whose removal disconnects u and n in G

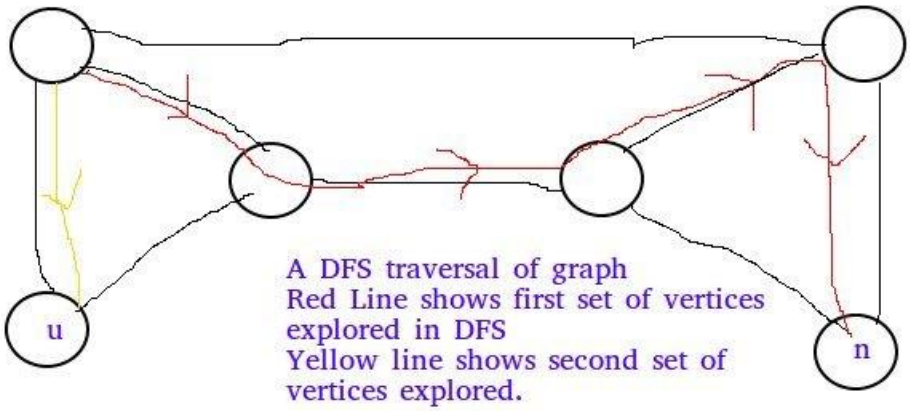
C There must exist a cycle in G containing u and n

There must exist a cycle in G containing u and all its neighbours in G.

Graph Traversals **GATE-CS-2006**
Discuss it

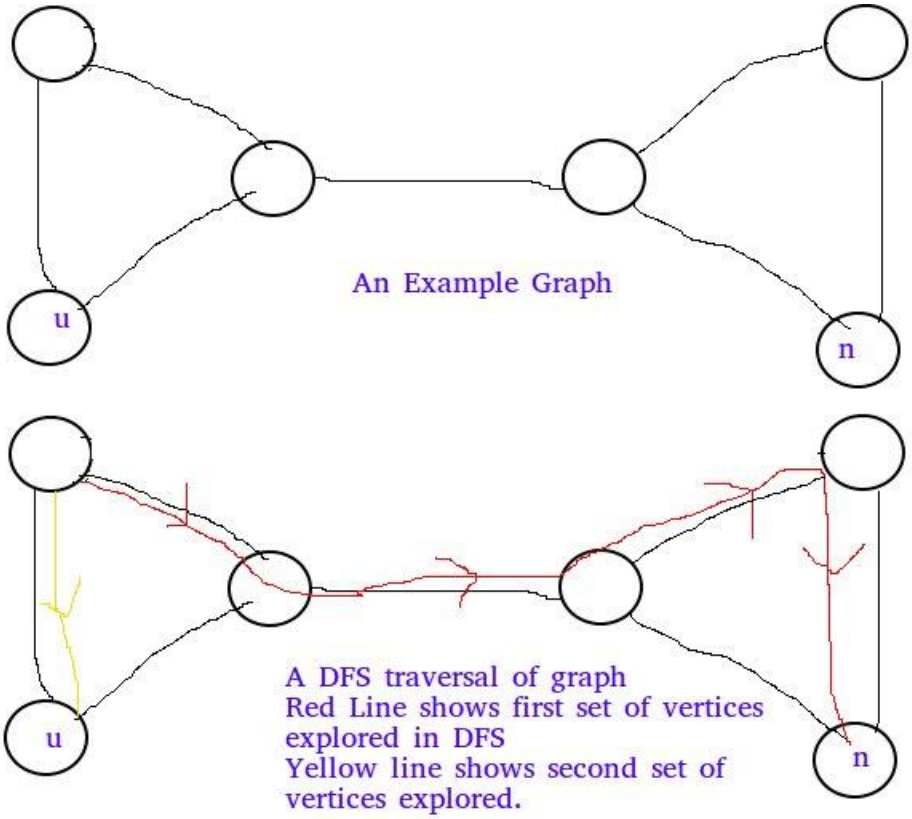
Question 17 Explanation:
Below example shows that A and B are FALSE:





Below

example shows C is false:



Question
18

Question 19

Correct

In a depth-first traversal of a graph G with n vertices, k edges are marked as tree edges. The number of connected components in G is

A k

B k + 1

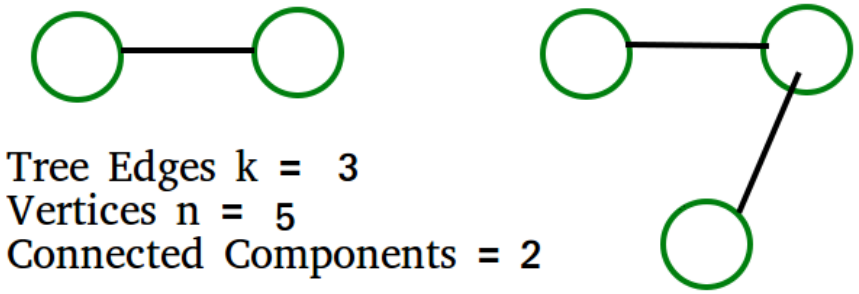
C n - k - 1

n - k

Graph Traversals **Gate IT 2005**
Discuss it

Question 19 Explanation:

Tree edges are the edges that are part of DFS tree. If there are x tree edges in a tree, then x+1 vertices in the tree. The output of DFS is a forest if the graph is disconnected. Let us see below simple example where graph is disconnected.

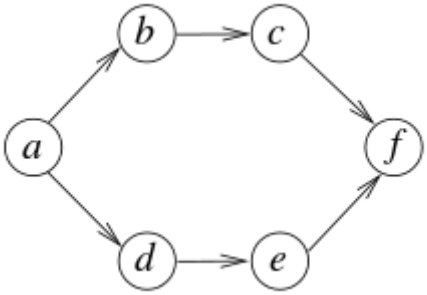


The above example matches with D option **More Examples:** 1) All vertices of Graph are connected. k must be n-1. We get number of connected components = n- k = n - (n-1) = 1 2) No vertex is connected. k must be 0. We get number of connected components = n- k = n - 0 = n

Question
20

Question 20

Wrong



Consider the following directed graph. The number of different topological orderings of the vertices of the graph is Note : This question was asked as Numerical Answer Type.

A 1

B 2

4

6

Graph Traversals GATE-CS-2016 (Set 1)
Discuss it

Question 20 Explanation:

Following are different 6 Topological Sortings

```
a-b-c-d-e-f
a-d-e-b-c-f
a-b-d-c-e-f
a-d-b-c-e-f
a-b-d-e-c-f
a-d-b-e-c-f
```

Question 21

Correct

In an adjacency list representation of an undirected simple graph G = (V, E), each edge (u, v) has two adjacency list entries: [v] in the adjacency list of u, and [u] in the adjacency list of v. These are called twins of each other. A twin pointer is a pointer from an adjacency list entry to its twin. If |E| = m and |V | = n, and the memory size is not a constraint, what is the time complexity of the most efficient algorithm to set the twin pointer in each entry in each adjacency list?

A $\Theta(n^2)$

$\Theta(m+n)$

C $\Theta(m^2)$

D $\Theta(n^4)$

Graph Traversals **GATE-CS-2016 (Set 2)**
Discuss it

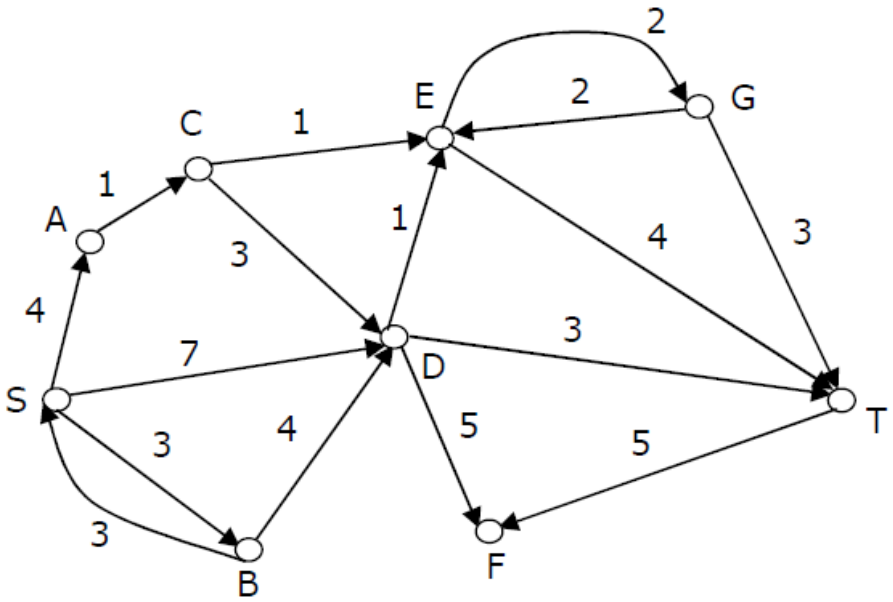
Question 21 Explanation:
First you need to find twins of each node. You can do this using **level order traversal (i.e., BFS)** once. Time complexity of BFS is Θ(m +n). And you have to use linked list for representation which is extra space (but memory size is not a constraint here). Final, time complexity is Θ(m + n) to set twin pointer. Option (B) is correct.

Question 1

Wrong

Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices S and T. Which one will be reported by Dijstra?s shortest path algorithm? Assume that, in any iteration, the shortest path to a vertex v is updated only when a strictly shorter path to v is

discovered.



A SDT

SBDT

C SACDT

SACET

Graph Shortest Paths
Discuss it

Question 1 Explanation:
See Dijkstra's shortest path algorithm When the algorithm reaches vertex 'C', the distance values of 'D' and 'E' would be 7 and 6 respectively. So the next picked vertex would be 'E'

Question 2

Wrong

To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is:

Queue

B Stack

Heap

D B-Tree

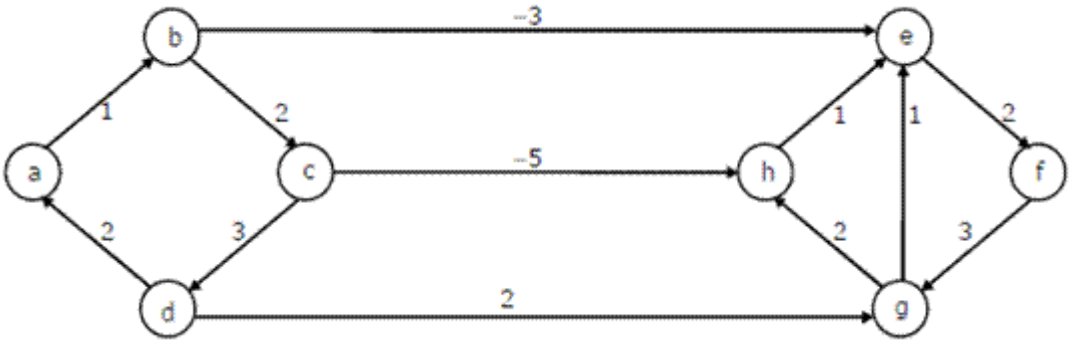
Graph Shortest Paths
Discuss it

Question 2 Explanation:
The shortest path in an un-weighted graph means the smallest number of edges that must be traversed in order to reach the destination in the graph. This is the same problem as solving the weighted version where all the weights happen to be 1. If we use Queue (FIFO) instead of Priority Queue (Min Heap), we get the shortest path in linear time O(|V| + |E|). Basically we do BFS traversal of the graph to get the shortest paths.

Question 3

Wrong

Dijkstra's single source shortest path algorithm when run from vertex a in the below graph, computes the correct shortest path distance to



A only vertex a

B only vertices a, e, f, g, h

only vertices a, b, c, d

all the vertices

Graph Shortest Paths
Discuss it

Question 3 Explanation:
Dijkstra's single source shortest path is not guaranteed to work for graphs with negative weight edges, but it works for the given graph. Let us see... Let us run the 1st pass b 1 b is minimum, so shortest distance to b is 1. After 1st pass, distances are c 3, e -2. e is minimum, so shortest distance to e is -2 After 2nd pass, distances are c 3, f 0. f is minimum, so shortest distance to f is 0 After 3rd pass, distances are c 3, g 3. Both are same, let us take g. so shortest distance to g is 3. After 4th pass,

distances are c 3, h 5 c is minimum, so shortest distance to c is 3 After 5th pass, distances are h -2 h is minimum, so shortest distance to h is -2

Question 7

Wrong

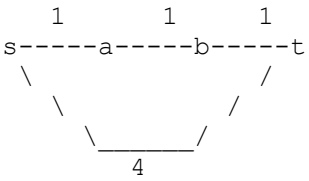
In a weighted graph, assume that the shortest path from a source 's' to a destination 't' is correctly calculated using a shortest path algorithm. Is the following statement true? If we increase weight of every edge by 1, the shortest path always remains same.

Yes

No

Graph Shortest Paths
Discuss it

Question 7 Explanation:
See the following counterexample. There are 4 edges s-a, a-b, b-t and s-t of wights 1, 1, 1 and 4 respectively. The shortest path from s to t is s-a, a-b, b-t. IF we increase weight of every edge by 1, the shortest path changes to s-t.



Question 8

Wrong

Following statement is true or false?

```
If we make following changes to  Dijkstra, then it can be used to find
the longest simple path, assume that the graph is acyclic.

1) Initialize all distances as minus infinite instead of plus infinite.

2) Modify the relax condition in  Dijkstra's algorithm to update distance
  of an adjacent v of the currently considered vertex u only
  if "dist[u]+graph[u][v] > dist[v]". In shortest path algo,
  the sign is opposite.
```

True

False

Graph Shortest Paths
Discuss it

Question 8 Explanation:
In shortest path algo, we pick the minimum distance vertex from the set of vertices for which distance is not finalized yet. And we finalize the distance of the minimum distance vertex. For maximum distance problem, we cannot finalize the distance because there can be a longer path through not yet finalized vertices.

Question 9

Wrong

Which of the following algorithm can be used to **efficiently** calculate single source shortest paths in a Directed Acyclic Graph?

Dijkstra

B Bellman-Ford

Topological Sort

D Strongly Connected Component

Graph Shortest Paths
Discuss it

Question 9 Explanation:
Using Topological Sort, we can find single source shortest paths in O(V+E) time which is the most efficient algorithm. See following for details. Shortest Path in Directed Acyclic Graph

Question 11

Wrong

Is the following statement valid about shortest paths? *Given a graph, suppose we have calculated shortest path from a source to all other vertices. If we modify the graph such that weights of all edges is becomes double of the original weight, then the shortest path remains same only the total weight of path changes.*

True

False

Graph Shortest Paths
Discuss it

Question 11 Explanation:
The shortest path remains same. It is like if we change unit of distance from meter to kilo meter, the shortest paths don't change.

Question 12

Wrong

Match the following

```
Group A                                                  Group B
a) Dijkstra's single shortest path algo                  p) Dynamic
Programming
b) Bellmen Ford's single shortest path algo              q) Backtracking
c) Floyd Warshell's all pair shortest path algo.         r) Greedy Algorithm
```

a-r, b-q, c-p

B a-p, b-p, c-p

a-r, b-p, c-p

D a-p, b-r, c-q

Graph Shortest Paths
Discuss it

Question 12 Explanation:
Dijkstra is a greedy algorithm where we pick the minimum distant vertex from not yet finalized vertices. Bellman Ford and Floyd Warshell both are Dynamic Programming algorithms where we build the shortest paths in bottom up manner.

Question
13

Question 13

Wrong

Is the following statement valid?.

Given a weighted graph where weights of all edges are unique (no two edge have same weights), there is always a unique shortest path from a source to destination in such a graph.

True

False

Graph Shortest Paths
Discuss it

Question 13 Explanation:
There can be more than one paths with same weight. Consider a path with one edge of weight 5 and another path with two edges of weights 2 and 3. Both paths have same weights.

Question
14

Question 14

Correct

Is the following statement valid?.

Given a graph where all edges have positive weights, the shortest paths produced by Dijsktra and Bellman Ford algorithm may be different but path weight would always be same.

True

B False

Graph Shortest Paths
Discuss it

Question 14 Explanation:
Dijkstra and Bellman-Ford both work fine for a graph with all positive weights, but they are different algorithms and may pick different edges for shortest paths.

Question 15

Correct

Which of the following statement(s)is / are correct regarding Bellman-Ford shortest path algorithm?

```
P: Always finds a negative weighted cycle, if one exist s.
Q: Finds whether any negative weighted cycle is reachable
   from the source.
```

A P Only

Q Only

C Both P and Q

D Neither P nor Q

Graph Shortest Paths **GATE-CS-2009**
Discuss it

Question 15 Explanation:
Bellman-Ford shortest path algorithm is a single source shortest path algorithm. So it can only find cycles which are reachable from a given source, not any negative weight cycle. Consider a disconnected where a negative weight cycle is not reachable from the source at all. If there is a negative weight cycle, then it will appear in shortest path as the negative weight cycle will always form a shorter path when iterated through the cycle again and again.

Question 16

Wrong

Let G(V, E) an undirected graph with positive edge weights. Dijkstra's single-source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

A O(| V |2)

B O (| E | + | V | log | V |)

O (| V | log | V |)

O ((| E | + | V |) log | V |)

Graph Shortest Paths **GATE-CS-2005**
Discuss it

Question 16 Explanation:
See Time Complexity Section of http://www.geeksforgeeks.org/greedy-algorithms-set-7-dijkstras-algorithm-for-adjacency-list-representation/

Question
17

Question 17

Wrong

Let G = (V, E) be an undirected graph with a subgraph G1 = (V1, El). Weights are assigned to edges of G as follows :

$$w(e) = \begin{cases} 0 \text{ if } e \in E_1 \\ 1 \text{ otherwise} \end{cases}$$

A single-source shortest path algorithm is executed on the weighted graph (V, E, w) with an arbitrary vertex v1 of V1 as the source. Which of the following can always be inferred from the path costs computed?

The number of edges in the shortest paths from v1 to all vertices of G

G1 is connected

C V1 forms a clique in G

D G1 is a tree

Graph Shortest Paths **GATE-CS-2003**
Discuss it

Question 17 Explanation:
When shortest path shortest path from v1 (one of the vertices in V1) is computed. G1 is connected if the distance from v1 to any other vertex in V1 is greater than 0, otherwise G1 is disconnected.

Question 19

Correct

Let G = (V, E) be a simple undirected graph, and s be a particular vertex in it called the source. For x ∈ V, let d(x) denote the shortest distance in G from s to x. A breadth first search (BFS) is performed starting at s. Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T, then which one of the following CANNOT be the value of d(u) – d(v)?

A -1

B 0

C 1

2

Graph Shortest Paths **GATE-CS-2015 (Set 1)**
Discuss it

Question 19 Explanation:
Note that the given graph is undirected, so an edge (u, v) also means (v, u) is also an edge. Since a shorter path can always be obtained by using edge (u, v) or (v, u), the difference between d(u) and d(v) can not be more than 1.

Question 20

Wrong

Let G be a directed graph whose vertex set is the set of numbers from 1 to 100. There is an edge from a vertex i to a vertex j iff either j = i + 1 or j = 3i. The minimum number of edges in a path in G from vertex 1 to vertex 100 is

A 4

7

23

D 99

Graph Shortest Paths **Gate IT 2005**
Discuss it

Question 20 Explanation:
The task is to find minimum number of edges in a path in G from vertex 1 to vertex 100 such that we can move to either i+1 or 3i from a vertex i.

```
Since the task is to minimize number of edges,
we would prefer to follow 3*i.

Let us follow multiple of 3.

1 => 3 => 9 => 27 => 81, now we can't follow multiple
of 3. So we will have to follow i+1. This solution gives
a long path.

What if we begin from end, and we reduce by 1 if
the value is not multiple of 3, else we divide by 3.
100 => 99 => 33 => 11 => 10 => 9 => 3 => 1

So we need total 7 edges.
```

Question 21

Wrong

Consider the weighted undirected graph with 4 vertices, where the weight of edge {i, j} g is given by the entry W_{ij} in the matrix W

$$W = \begin{bmatrix} 0 & 2 & 8 & 5 \\ 2 & 0 & 5 & 8 \\ 8 & 5 & 0 & x \\ 5 & 8 & x & 0 \end{bmatrix}$$

The largest possible integer value of x, for which at least one shortest path between some pair of

vertices will contain the edge with weight x is _____
Note : This question was asked as Numerical Answer Type.

8

12

C 10

D 11

Graph Shortest Paths **GATE-CS-2016 (Set 1)**
Discuss it

Question 21 Explanation:
Let vertices be 0, 1, 2 and 3. x directly connects 2 to 3. The shortest path (excluding x) from 2 to 3 is of weight 12 (2-1-0-3).

Question 23

Wrong

Consider a weighted undirected graph with positive edge weights and let uv be an edge in the graph. It is known that the shortest path from the source vertex s to u has weight 53 and the shortest path from s to v has weight 65. Which one of the following statements is always true?

A weight (u, v) < 12

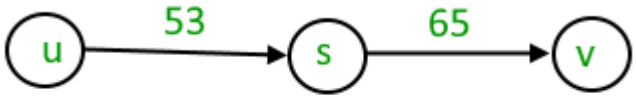
weight (u, v) ≤ 12

C weight (u, v) > 12

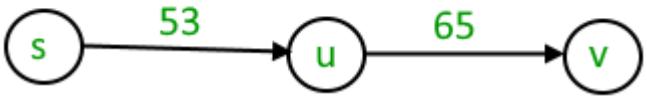
weight (u, v) ≥ 12

Graph Shortest Paths **Graph Theory** **Gate IT 2007**
Discuss it

Question 23 Explanation:



Weight(u,v) >=12



The minimum weight happens when (S,U) + (U,V) = (S,V) Else (S,U) + (U,V) >= (S,V) Given (S,U) = 53, (S,V) = 65 53 + (U,V) >= 63 (U,V) >= 12.
This solution is contributed by **Anil Saikrishna Devarasetty**

Question 27

Correct

Which of the following is application of Breath First Search on the graph?

A Finding diameter of the graph

B Finding bipartite graph

Both (a) and (b)

D None of the above

Graph Shortest Paths **ISRO CS 2018**
Discuss it

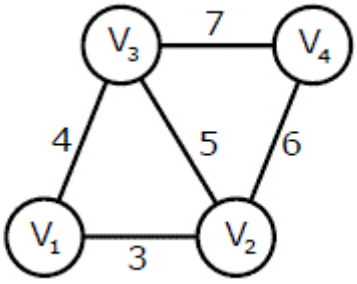
Question 27 Explanation:
BFS is used to Find the diameter of the graph and to test whether a graph is bipartite or not. BFS has many other applications also. Refer: Applications of Breadth First Traversal Option (C) is correct.

Question 1

Correct

An undirected graph G(V, E) contains n (n > 2) nodes named v1 , v2 ,….vn. Two nodes vi , vj are connected if and only if 0 < |i – j| <= 2. Each edge (vi, vj) is assigned a weight i + j. A sample graph

with n = 4 is shown below. What will be the cost of the minimum spanning tree (MST) of such a



graph with n nodes? (GATE CS 2011)

A 1/12(11n^2 – 5n)

n^2 – n + 1

C 6n – 11

D 2n + 1

Graph Minimum Spanning Tree
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/data-structures-and-algorithms-set-27/ for explanation.

Question 3

Wrong

Consider a complete undirected graph with vertex set {0, 1, 2, 3, 4}. Entry Wij in the matrix W below is the weight of the edge {i, j}. What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T? (GATE CS 2010)

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

A 7

B 8

9

10

Graph Minimum Spanning Tree
Discuss it

Question 3 Explanation:
To get the minimum spanning tree with vertex 0 as leaf, first remove 0th row and 0th column and then get the minimum spanning tree (MST) of the remaining graph. Once we have MST of the remaining graph, connect the MST to vertex 0 with the edge with minimum weight (we have two options as there are two 1s in 0th row).

Question 4

Wrong

In the graph given in above question question, what is the minimum possible weight of a path P from vertex 1 to vertex 2 in this graph such that P contains at most 3 edges?

A 7

8

C 9

10

Graph Minimum Spanning Tree
Discuss it

Question 4 Explanation:
Path: 1 -> 0 -> 4 -> 2 Weight: 1 + 4 + 3

Question 5

Correct

An undirected graph G has n nodes. Its adjacency matrix is given by an n × n square matrix whose (i) diagonal elements are 0‘s and (ii) non-diagonal elements are 1‘s. which one of the following is TRUE?

A Graph G has no minimum spanning tree (MST)

B Graph G has a unique MST of cost n-1

Graph G has multiple distinct MSTs, each of cost n-1

D Graph G has multiple spanning trees of different costs

Graph Minimum Spanning Tree
Discuss it

Question 5 Explanation:
See Question 2 of http://www.geeksforgeeks.org/data-structures-and-algorithms-set-22/

Question 7

Wrong

Let G be an undirected connected graph with distinct edge weight. Let emax be the edge with maximum weight and emin the edge with minimum weight. Which of the following statements is false? (GATE CS 2000)

A Every minimum spanning tree of G must contain emin

If emax is in a minimum spanning tree, then its removal must disconnect G

No minimum spanning tree contains emax

D G has a unique minimum spanning tree

Graph Minimum Spanning Tree
Discuss it

Question 7 Explanation:
(a) and (b) are always true. (c) is false because (b) is true. (d) is true because all edge weights are distinct for G.

Question 8

Correct

Consider a weighted complete graph G on the vertex set {v1,v2 ,v} such that the weight of the edge (v,,v) is 2|i-j|. The weight of a minimum spanning tree of G is: (GATE CS 2006)

A n — 1

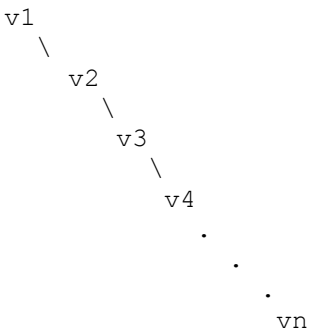
2n — 2

C nC2

D 2

Graph Minimum Spanning Tree
Discuss it

Question 8 Explanation:
Minimum spanning tree of such a graph is



Weight of the minimum spanning tree = 2|2 - 1| + 2|3 - 2| + 2|4 - 3| + 2|5 - 4| .... + 2| n - (n-1) | = 2n - 2

Question 9

Wrong

Let G be a weighted graph with edge weights greater than one and G'be the graph constructed by squaring the weights of edges in G. Let T and T' be the minimum spanning trees of G and G', respectively, with total weights t and t'. Which of the following statements is TRUE?

A T' = T with total weight t' = t^2

T' = T with total weight t' < t^2

C T' != T but total weight t' = t^2

None of the above

GATE CS 2012 **Graph Minimum Spanning Tree**
Discuss it

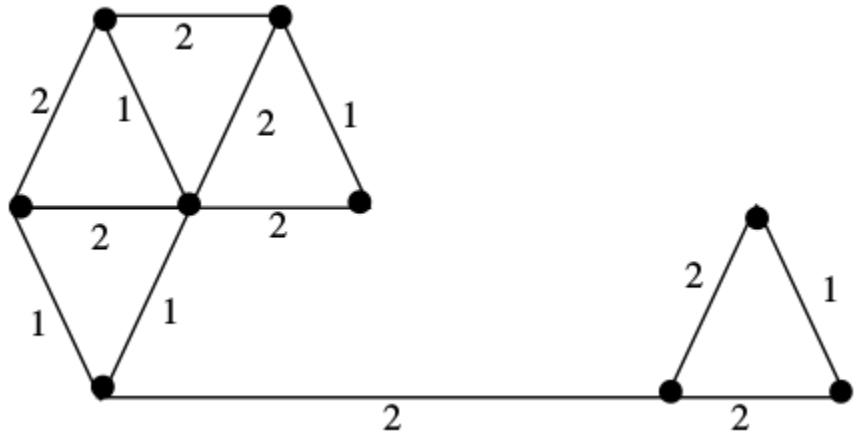
Question 9 Explanation:
Squaring the weights of the edges in a weighted graph will not change the minimum spanning tree. Assume the opposite to obtain a contradiction. If the minimum spanning tree changes then at least one edge from the old graph G in the old minimum spanning tree T must be replaced by a new edge in tree T' from the graph G' with squared edge weights. The new edge from G' must have a lower weight than the edge from G. This implies that there exists some weights C1 and C2 such that C1 < C2 and C12 >= C22. This is a contradiction.
Source: http://www.cs.nyu.edu/courses/spring06/V22.0310-001/hw3.htm Sums of squares of two or more numbers is always smaller than square of sum. Example: 2^2 + 2^2 < (4)^2 But

```
there is one counter example when the graph has only one edge.
         In that case, the two values are same.
```

Question 11

Wrong

The number of distinct minimum spanning trees for the weighted graph below is ____



4

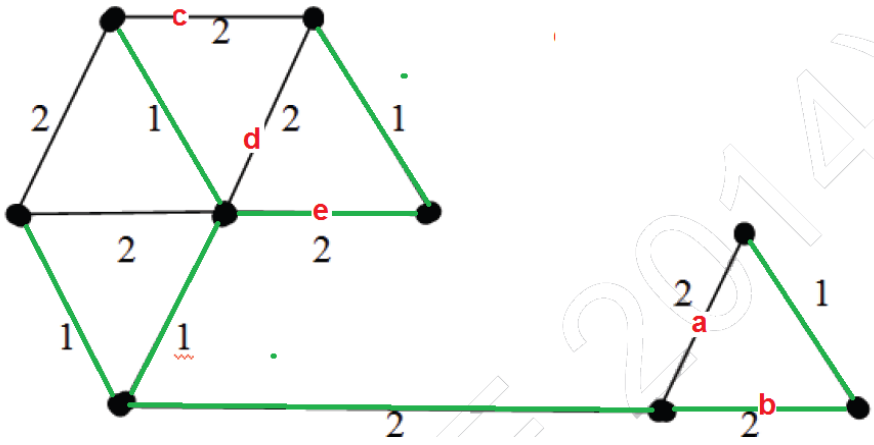
B 5

6

D 7

Graph Minimum Spanning Tree **GATE-CS-2014-(Set-2)**
Discuss it

Question 11 Explanation:
Below diagram shows a minimum spanning tree. Highlighted (in green) are the edges picked to make the MST.



In the right side of MST, we could either pick edge 'a' or 'b'. In the left side, we could either pick 'c' or 'd' or 'e' in MST. There are 2 options for one edge to be picked and 3 options for another edge to be picked. Therefore, total 2*3 possible MSTs.

Question 12

Correct

Let s and t be two vertices in a undirected graph G + (V, E) having distinct positive edge weights. Let [X, Y] be a partition of V such that s ∈ X and t ∈ Y. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y The edge e must definitely belong to:

the minimum weighted spanning tree of G

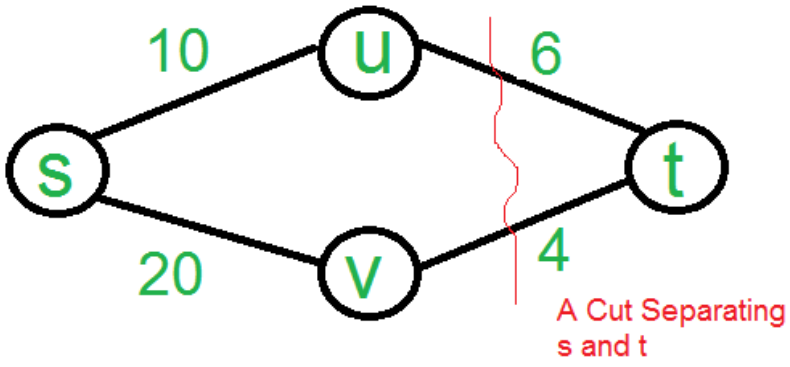
B the weighted shortest path from s to t

C each path from s to t

D the weighted longest path from s to t

Graph Minimum Spanning Tree **GATE-CS-2005**
Discuss it

Question 12 Explanation:
The minimum weight edge on any s-t cut is always part of MST. This is called Cut Property. This is the idea used in Prim's algorithm. The minimum weight cut edge is always a minimum spanning tree edge. **Why B (the weighted shortest path from s to t) is not an answer?** See below example, edge 4 (lightest in highlighted red cut from s to t) is not part of shortest path.



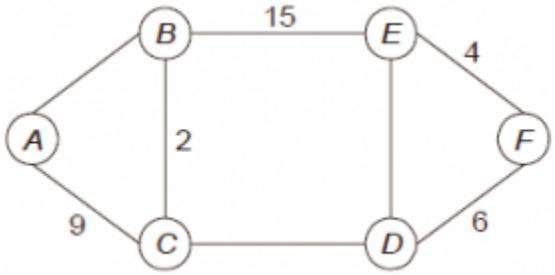
Question
13

Question 15

Wrong

The graph shown below 8 edges with distinct integer edge weights. The minimum spanning tree (MST) is of weight 36 and contains the edges: {(A, C), (B, C), (B, E), (E, F), (D, F)}. The edge weights of only those edges which are in the MST are given in the figure shown below. The minimum possible

sum of weights of all 8 edges of this graph is _____.



66

69

C 68

D 70

Graph Minimum Spanning Tree GATE-CS-2015 (Set 1)
Discuss it

Question 15 Explanation:
In every cycle, the weight of an edge that is not part of MST must by greater than or equal to weights of other edges which are part of MST. Since all edge weights are distinct, the weight must be greater. So the minimum possible weight of ED is 7, minimum possible weight of CD is 16 and minimum possible weight of AB is 10. Therefore minimum possible sum of weights is 69.

Question 16

Wrong

Let G be connected undirected graph of 100 vertices and 300 edges. The weight of a minimum spanning tree of G is 500. When the weight of each edge of G is increased by five, the weight of a minimum spanning tree becomes _____.

1000

995

C 2000

D 1995

Graph Minimum Spanning Tree GATE-CS-2015 (Set 3)
Discuss it

Question 16 Explanation:
Since there are 100 vertices, there must be 99 edges in Minimum Spanning Tree (MST). When weight of every edge is increased by 5, the increment in weight of MST is = 99 * 5 = 495 So new weight of MST is 500 + 495 which is 995

Question 17

Wrong

Let G be a weighted connected undirected graph with distinct positive edge weights. If every edge weight is increased by the same value, then which of the following statements is/are TRUE?

```
P: Minimum spanning tree of G does not change
Q: Shortest path between any pair of vertices does not change
```

P only

B Q only

C Neither P nor Q

Both P and Q

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 17 Explanation:
The shortest path may change. The reason is, there may be different number of edges in different paths from s to t. For example, let shortest path be of weight 15 and has 5 edges. Let there be another path with 2 edges and total weight 25. The weight of the shortest path is increased by 5*10 and becomes 15 + 50. Weight of the other path is increased by 2*10 and becomes 25 + 20. So the shortest path changes to the other path with weight as 45. The Minimum Spanning Tree doesn't change. Remember the Kruskal's algorithm where we sort the edges first. IF we increase all weights, then order of edges won't change.

Question
18

Question 18

Wrong

Let G be a complete undirected graph on 4 vertices, having 6 edges with weights being 1, 2, 3, 4, 5, and 6. The maximum possible weight that a minimum weight spanning tree of G can have is. [This Question was originally a Fill-in-the-Blanks question]

6

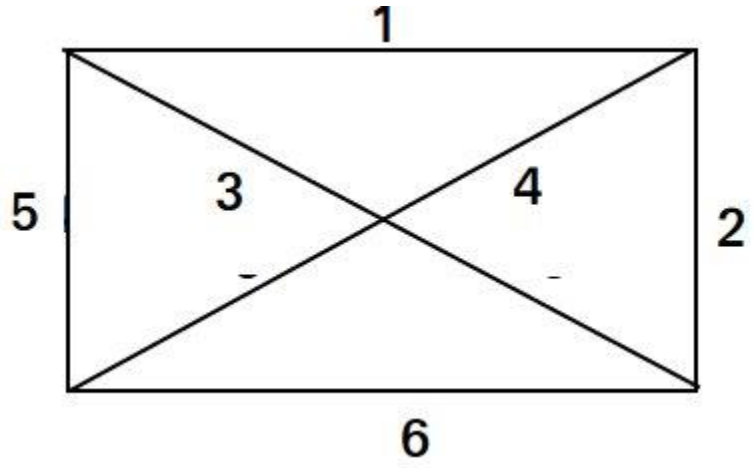
7

C 8

D 9

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 18 Explanation:
One graph that has maximum possible weight of spanning tree



Question
19

Question 19

Correct

G = (V, E) is an undirected simple graph in which each edge has a distinct weight, and e is a particular edge of G. Which of the following statements about the minimum spanning trees (MSTs) of G is/are TRUE

```
I.  If e is the lightest edge of some cycle in G,
    then every MST of G includes e
II. If e is the heaviest edge of some cycle in G,
    then every MST of G excludes e
```

A I only

II only

C both I and II

D neither I nor II

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 19 Explanation:
I is NOT true. Let G=(V, E) be a rectangular graph where V = {a, b, c, d} and E = {ab, bc, cd, da, ac}. Let the edges have weights: ab = 1, bc = 2, cd = 4, da = 5, ac = 3. Then, clearly, ac is the lightest edge of the cycle cdac, however, the MST abcd with cost 7 (= ab + bc + cd) does not include it. Let the edges have weights: ab = 6, bc - 7, cd = 4, da = 5, ac = 3. Then, again, ac is the lightest edge of the cycle cdac, and, the MST bacd with cost 13 (= ba + ac + cd) includes it. So, the MSTs of G may or may not include the lightest edge. **II is true** Let the heavies edge be e. Suppose the minimum spanning tree which contains e. If we add one more edge to the spanning tree we will create a cycle. Suppose we add edge e' to the spanning tree which generated cycle C. We can reduce the cost of the minimum spanning tree if we choose an edge other than e from C for removal which implies that e must not be in minimum spanning tree and we get a contradiction. Source:
http://www.ece.northwestern.edu/~dda902/336/hw5-sol.pdf

Question 20

Correct

What is the largest integer m such that every simple connected graph with n vertices and n edges contains at least m different spanning trees?

A 1

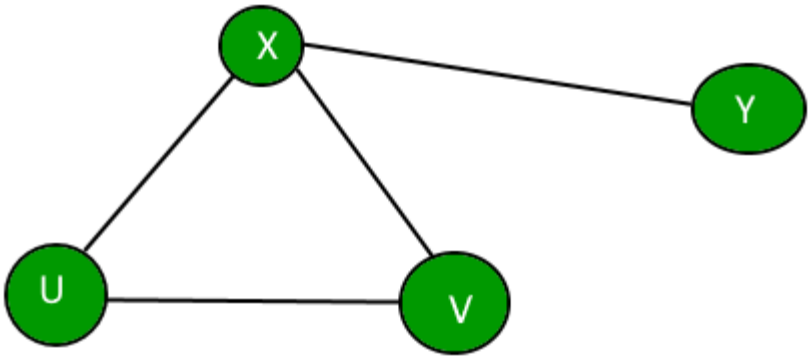
B 2

3

D n

Graph Graph Minimum Spanning Tree Gate IT 2007
Discuss it

Question 20 Explanation:
A graph is connected iff all nodes can be traversed from each node. For a graph with n nodes, there will be n-1 minimum number of edges.
Given that there are n edges, that means a cycle is there in the graph.
The simplex graph with these conditions may be:



Now we can make a different spanning tree by removing one edge from the cycle, one at a time. Minimum cycle length can be 3, So, there must be atleast 3 spanning trees in any such Graph.

Question 21

Wrong

If Kruskal's algorithm is used for finding a minimum spanning tree of a weighted graph G with n vertices and m edges and edge weights are already given in a sorted list, then, What will be the time complexity to compute the minimum cost spanning tree given that union and find operations take amortized O(1) ?

A O(m logn)

B O(n)

O(m)

O(n logm)

Graph Minimum Spanning Tree **GATE 2017 Mock**
Discuss it

Question 21 Explanation:
O(m) as you are already given edge weights in sorted order. You just have to pick the edges in the increasing order and add it to the current spanning set if its addition does not result in a cycle else throw it away.

Divide and Conquer

1

Question 1

CORRECT

Which of the following algorithms is NOT a divide & conquer algorithm by nature?

A Euclidean algorithm to compute the greatest common divisor

Heap Sort

C Cooley-Tukey fast Fourier transform

D Quick Sort

Divide and Conquer
Discuss it

Question 1 Explanation:
See Divide and Conquer

Question 2

CORRECT

Consider the following C program

```
int main()
{
   int x, y, m, n;
   scanf ("%d %d", &x, &y);
   /* x > 0 and y > 0 */
   m = x; n = y;
   while (m != n)
   {
      if(m>n)
         m = m - n;
      else
         n = n - m;
   }
   printf("%d", n);
}
```

Run on IDE

What does the program compute? (GATE CS 2004)

A x + y using repeated subtraction

B x mod y using repeated subtraction

the greatest common divisor of x and y

D the least common multiple of x and y

Divide and Conquer
Discuss it

Question 2 Explanation:
This is an implementation of Euclid's algorithm to find GCD

Question 3

CORRECT

Consider the polynomial p(x) = a0 + a1x + a2x^2 +a3x^3, where ai != 0, for all i. The minimum number of multiplications needed to evaluate p on an input x is:

3

B 4

C 6

D 9

Divide and Conquer
Discuss it

Question 3 Explanation:
Multiplications can be minimized using following order for evaluation of the given expression. p(x) = a0 + x(a1 + x(a2 + a3x))

Question 4

CORRECT

Maximum Subarray Sum problem is to find the subarray with maximum sum. For example, given an array {12, -13, -5, 25, -20, 30, 10}, the maximum subarray sum is 45. The naive solution for this problem is to calculate sum of all subarrays starting with every element and return the maximum of all. We can solve this using Divide and Conquer, what will be the worst case time complexity using Divide and Conquer.

A O(n)

O(nLogn)

C O(Logn)

D O(n^2)

Divide and Conquer
Discuss it

Question 4 Explanation:
See http://www.geeksforgeeks.org/divide-and-conquer-maximum-sum-subarray/

Question 5

CORRECT

Consider a situation where you don't have function to calculate power (pow() function in C) and you need to calculate x^n where x can be any number and n is a positive integer. What can be the best possible time complexity of your power function?

A O(n)

B O(nLogn)

C O(LogLogn)

O(Logn)

Divide and Conquer
Discuss it

Question 5 Explanation:
We can calculate power using divide and conquer in O(Logn) time.
See http://www.geeksforgeeks.org/write-a-c-program-to-calculate-powxn/.

Question 6

WRONG

Consider the problem of searching an element x in an array 'arr[]' of size n. The problem can be solved in O(Logn) time if. 1) Array is sorted 2) Array is sorted and rotated by k. k is given to you and k <= n 3) Array is sorted and rotated by k. k is NOT given to you and k <= n 4) Array is not sorted

1 Only

B 1 & 2 only

1, 2 and 3 only

D 1, 2, 3 and 4

Divide and Conquer
Discuss it

Question 6 Explanation:
See http://www.geeksforgeeks.org/search-an-element-in-a-sorted-and-pivoted-array/

Question 7

CORRECT

The secant method is used to find the root of an equation f(x) = 0. It is started from two distinct estimates x_a and x_b for the root. It is an iterative procedure involving linear interpolation to a root. The iteration stops if $f(x_b)$ is very small and then x_b is the solution. The procedure is given below. Observe that there is an expression which is missing and is marked by? Which is the suitable expression that is to be put in place of? So that it follows all steps of the secant method? **Secant**

```
Initialize: xa, xb, ε, N     // ε = convergence indicator
fb = f(xb) i = 0
while (i < N and |fb| > ε) do
   i = i + 1                  // update counter
   xt = ?                     // missing expression for
                              // intermediate value
   xa = xb                    // reset xa
   xb = xt                    // reset xb
   fb = f(xb)                 // function value at new xb
end while
if |fb| > ε
  then // loop is terminated with i = N
  write "Non-convergence"
else
  write "return xb"
end if
```

A $x_b - (f_b - f(x_a))\, f_b / (x_b - x_a)$

B $x_a - (f_a - f(x_a))\, f_a / (x_b - x_a)$

C $x_b - (f_b - x_a)\, f_b / (x_b - f_b(x_a)$

$x_a - (x_b - x_a)\, f_a / (f_b - f(x_a))$

Divide and Conquer GATE-CS-2015 (Set 2)
Discuss it

Question 7 Explanation:
ttp://en.wikipedia.org/wiki/Secant_method

Question 8

CORRECT

Suppose you are provided with the following function declaration in the C programming language.

```
int partition (int a[], int n);
```

The function treats the first element of a[] as a pivot, and rearranges the array so that all elements less than or equal to the pivot is in the left part of the array, and all elements greater than the pivot is in the right part. In addition, it moves the pivot so that the pivot is the last element of the left part. The return value is the number of elements in the left part. The following partially given function in the C programming language is used to find the kth smallest element in an array a[ ] of size n using the partition function. We assume k ≤ n

```
int kth_smallest (int a[], int n, int k)
{
   int left_end = partition (a, n);
   if (left_end+1==k)
   {
       return a [left_end];
   }
   if (left_end+1 > k)
   {
      return kth_smallest (_____________________);
   }
   else
   {
      return kth_smallest (_____________________);
    }
}
```

Run on IDE

The missing argument lists are respectively

(a, left_end, k) and (a+left_end+1, n–left_end–1, k–left_end–1)

B (a, left_end, k) and (a, n–left_end–1, k–left_end–1)

C (a, left_end+1, N–left_end–1, K–left_end–1) and(a, left_end, k)

D (a, n–left_end–1, k–left_end–1) and (a, left_end, k)

Divide and Conquer GATE-CS-2015 (Set 2)

Discuss it

Question 8 Explanation:

See method 4 of http://www.geeksforgeeks.org/kth-smallestlargest-element-unsorted-array/

Question 9

WRONG

Consider the problem of computing min-max in an unsorted array where min and max are minimum and maximum elements of array. Algorithm A1 can compute min-

max in a1 comparisons without divide and conquer. Algorithm A2 can compute min-max in a2 comparisons by scanning the array linearly. What could be the relation between a1 and a2 considering the worst case scenarios?

A a1 < a2

a1 > a2

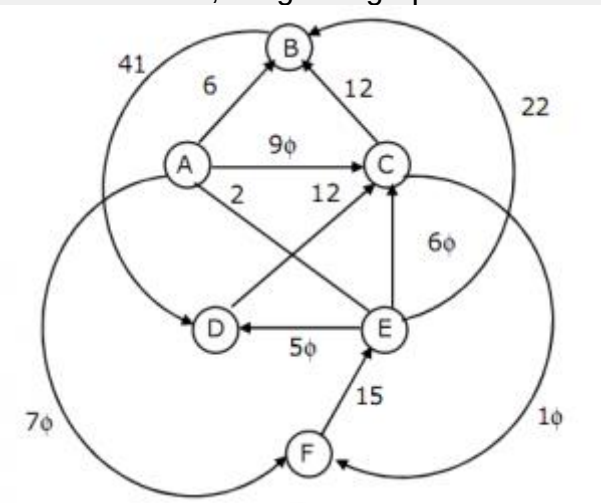
C a1 = a2

Depends on the input

Divide and Conquer GATE 2017 Mock
Discuss it

Question 9 Explanation:
When Divide and Conquer is used to find the minimum-maximum element in an array, Recurrence relation for the number of comparisons is
T(n) = 2T(n/2) + 2 where 2 is for comparing the minimums as well the maximums of the left and right subarrays
On solving, T(n) = 1.5n - 2.
While doing linear scan, it would take 2*(n-1) comparisons in the worst case to find both minimum as well maximum in one pass.

Question 10

Let *G* be the directed, weighted graph shown in below figure 

We are interested in the shortest paths from A. (a) Output the sequence of vertices identified by the Dijkstra's algorithm for single source shortest path when the algorithm is started at node A. (b) Write down sequence of vertices in the shortest path from A to E. (c) What is the cost of the shortest path from A to E?

Divide and Conquer Graph GATE CS 1996
Discuss it

You have completed 9/10 questions .
Your accuracy is 78%.

1

Greedy Algorithms

12

Question 1

CORRECT

Which of the following standard algorithms is not a Greedy algorithm?

A Dijkstra's shortest path algorithm

B Prim's algorithm

C Kruskal algorithm

D Huffman Coding

Bellmen Ford Shortest path algorithm

Greedy Algorithms
Discuss it

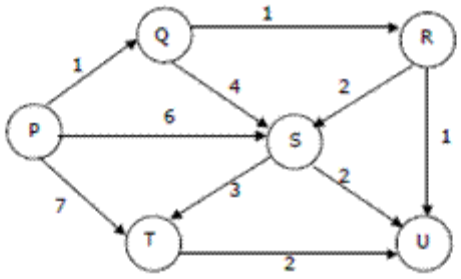
Question 1 Explanation:
See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/ and http://www.geeksforgeeks.org/dynamic-programming-set-23-bellman-ford-algorithm/

Question 2

CORRECT

Suppose we run Dijkstra’s single source shortest-path algorithm on the following edge weighted directed graph with vertex P as the source. In what order do the nodes get included

into the set of vertices for which the shortest path distances are finalized? (GATE CS 2004)



A P, Q, R, S, T, U

P, Q, R, U, S, T

C P, Q, R, U, T, S

D P, Q, T, R, U, S

Greedy Algorithms
Discuss it

Question 2 Explanation:
Refer http://www.geeksforgeeks.org/greedy-algorithms-set-6-dijkstras-shortest-path-algorithm/

Question 3

WRONG

A networking company uses a compression technique to encode the message before transmitting over the network. Suppose the message contains the following characters with their frequency:

```
character   Frequency
    a           5
    b           9
    c           12
    d           13
    e           16
    f           45
```

Run on IDE

Note : Each character in input message takes 1 byte. If the compression technique used is Huffman Coding, how many bits will be saved in the message?

A 224

B 800

576

324

Greedy Algorithms
Discuss it

Question 3 Explanation:

```
Total number of characters in the message = 100.

Each character takes 1 byte. So total number of bits needed = 800.


After Huffman Coding, the characters can be represented with:
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
Total number of bits needed = 224
Hence, number of bits saved = 800 - 224 = 576
See here for complete explanation and algorithm.
```

Question 4

WRONG

What is the time complexity of Huffman Coding?

A O(N)

O(NlogN)

O(N(logN)^2)

D O(N^2)

Greedy Algorithms
Discuss it

Question 4 Explanation:
O(nlogn) where n is the number of unique characters. If there are n nodes, extractMin() is called 2*(n – 1) times. extractMin() takes O(logn) time as it calles minHeapify(). So, overall complexity is O(nlogn). See here for more details of the algorithm.

Question 5

WRONG

In question #2, which of the following represents the word *"dead"*?

1011111100101

B 0100000011010

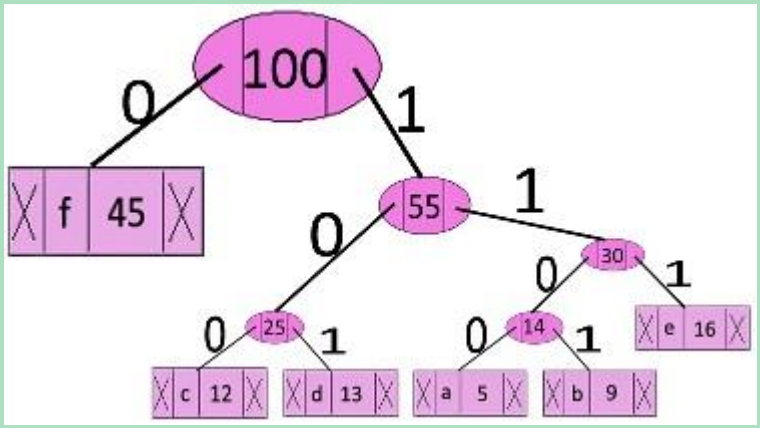
Both A and B

D None of these

Greedy Algorithms
Discuss it

Question 5 Explanation:

The *Huffman Tree* generated is:




```
character   code-word
    f          0
    c          100
    d          101
    a          1100
```

```
b          1101
e          111
```

The word *dead* can be represented as: 101 111 1100 101 However, the alternative codeword can also be found by assigning 1 to the left edge and 0 to the right edge of the tree, i.e. *dead* can also be represented as: 010 000 0011 010 See here for more details of the algorithm.

Question 6

CORRECT

Which of the following is true about Kruskal and Prim MST algorithms? Assume that Prim is implemented for adjacency list representation using Binary Heap and Kruskal is implemented using union by rank.

Worst case time complexity of both algorithms is same.

B Worst case time complexity of Kruskal is better than Prim

C Worst case time complexity of Prim is better than Kruskal

Greedy Algorithms
Discuss it

Question 6 Explanation:
See following Kruskal's Minimum Spanning Tree Algorithm Prim's MST for Adjacency List Representation

Question 7

WRONG

Which of the following is true about Huffman Coding.

A Huffman coding may become lossy in some cases

Huffman Codes may not be optimal lossless codes in some cases

In Huffman coding, no code is prefix of any other code.

D All of the above

Greedy Algorithms
Discuss it

Question 7 Explanation:
Huffman coding is a lossless data compression algorithm. The codes assigned to input characters are Prefix Codes, means the codes are assigned in such a way that the code assigned to one character is not prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding.

Question 8

CORRECT

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. Which of the following is the Huffman code for the letter a, b, c, d, e, f?

0, 10, 110, 1110, 11110, 11111

B 11, 10, 011, 010, 001, 000

C 11, 10, 01, 001, 0001, 0000

D 110, 100, 010, 000, 001, 111

Greedy Algorithms GATE-CS-2007
Discuss it

Question 8 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.


                1
              /   \
             /     \
           1/2     a(1/2)
          /  \
         /    \
       1/4   b(1/4)
```

```
        /   \
       /     \
      1/8   c(1/8)
     /  \
    /    \
  1/16  d(1/16)
 /  \
e    f
```

Question 9

WRONG

Suppose the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively. What is the average length of Huffman codes?

3

B 2.1875

C 2.25

1.9375

Greedy Algorithms GATE-CS-2007

Discuss it

Question 9 Explanation:
We get the following Huffman Tree after applying Huffman Coding Algorithm. The idea is to keep the least probable characters as low as possible by picking them first.

```
The letters a, b, c, d, e, f have probabilities
1/2, 1/4, 1/8, 1/16, 1/32, 1/32 respectively.

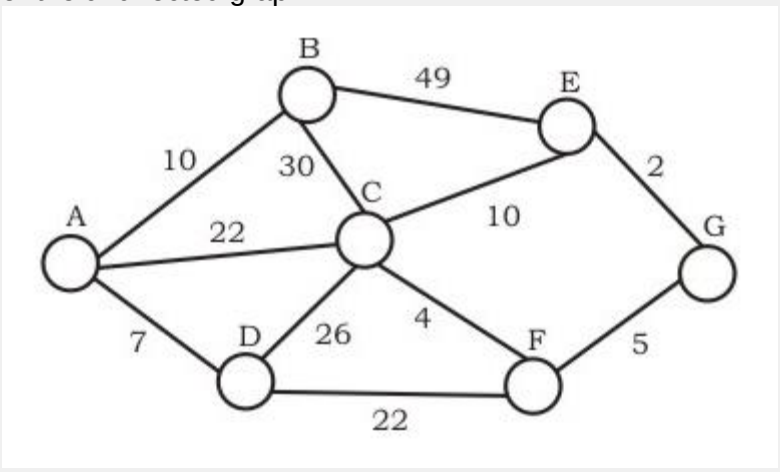

              1
             /   \
            /     \
           1/2    a(1/2)
          /  \
```

```
          /    \
         1/4  b(1/4)
        /   \
       /     \
      1/8   c(1/8)
     /  \
    /    \
  1/16  d(1/16)
 /  \
e    f
The average length = (1*1/2 + 2*1/4 + 3*1/8 + 4*1/16 + 5*1/32 + 5*1/32)
                   = 1.9375
```

Question 10

CORRECT

Consider the undirected graph



below: Using Prim's algorithm to construct a minimum spanning tree starting with node A, which one of the following sequences of edges represents a possible order in which the edges would be added to construct the minimum spanning tree?

A (E, G), (C, F), (F, G), (A, D), (A, B), (A, C)

B (A, D), (A, B), (A, C), (C, F), (G, E), (F, G)

C (A, B), (A, D), (D, F), (F, G), (G, E), (F, C)

(A, D), (A, B), (D, F), (F, C), (F, G), (G, E)

Greedy Algorithms GATE-IT-2004
Discuss it

Question 10 Explanation:
A and B are **False** : The idea behind Prim's algorithm is to construct a spanning tree - **means all vertices must be connected** but here vertices are disconnected
C. **False.** Prim's is a **greedy algorithm** and At every step, it considers all the edges that connect the two sets, and picks the minimum weight edge from these edges. In this option weight of AB<AD so must be picked up first. D.**TRUE.** It represents a possible order in which the edges would be added to construct the minimum spanning tree. Prim's algorithm is also a Greedy algorithm. It starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, the other set contains the vertices not yet included. At every step, it considers all the edges that connect the two sets, and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST. Read more at: http://www.geeksforgeeks.org/greedy-algorithms-set-5-prims-minimum-spanning-tree-mst-2/

You have completed 10/15 questions .
Your accuracy is 50%.

12

Greedy Algorithms

12

Question 11

WRONG

Consider the weights and values of items listed below. Note that there is only one unit of each item.

Item number	Weight (in Kgs)	Value (in Rupees)
1	10	60
2	7	28
3	4	20
4	2	24

The task is to pick a subset of these items such that their total weight is no more than 11 Kgs and their total value is maximized. Moreover, no item may be split. The total

value of items picked by an optimal algorithm is denoted by V_{opt}. A greedy algorithm sorts the items by their value-to-weight ratios in descending order and packs them greedily, starting from the first item in the ordered list. The total value of items picked by the greedy algorithm is denoted by V_{greedy}. The value of $V_{opt} - V_{greedy}$ is _____ .
Note -This was Numerical Type question.

16

8

C 44

D 60

Greedy Algorithms **Dynamic Programming** **GATE CS 2018**
Discuss it

Question 11 Explanation:

Item No	Weight	Value	Value/Weight
1	10	60	6
2	7	28	4
3	4	20	5
4	2	24	12

After sorting :

Item No	Weight	Value	Value/Weight
4	2	24	12
1	10	60	6
3	4	20	5
2	7	28	4

First we will pick item_4 (Value weight ratio is highest). Second highest is item_1, but cannot be picked because of its weight. Now item_3 shall be picked. item_2 cannot be included because of its weight. Therefore, overall profit by V_{greedy} = 20+24 = 44 Hence, **V_{opt} - V_{greedy} = 60-44 = 16** So, answer is 16.

Question 12

WRONG

A text is made up of the characters a, b, c, d, e each occurring with the probability 0.11, 0.40, 0.16, 0.09 and 0.24 respectively. The optimal Huffman coding technique will have the average length of:

2.40

2.16

C 2.26

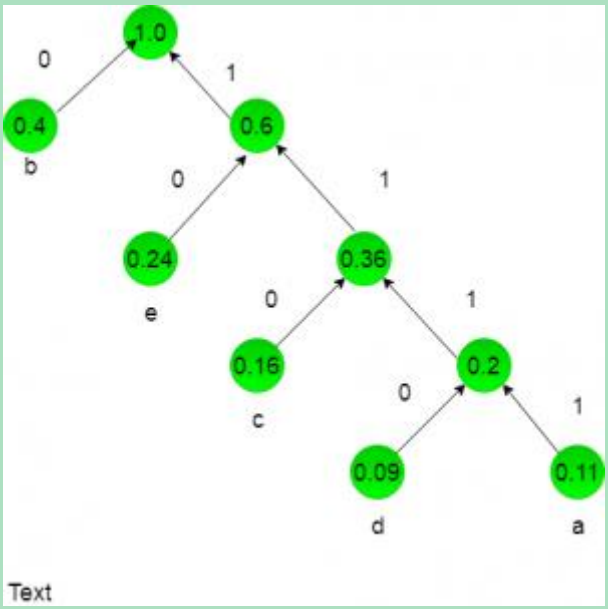
D 2.15

Greedy Algorithms UGC-NET CS 2017 Nov - III

Discuss it

Question 12 Explanation:

a = 0.11 b = 0.40 c = 0.16 d = 0.09 e = 0.24 we will draw a huffman tree:



now huffman coding for character:

```
a = 1111
    b = 0
    c = 110
    d = 1111
    e = 10
lenghth for each character = no of bits * frequency of occurence:
a = 4 * 0.11
  = 0.44
b = 1 * 0.4
```

```
  =  0.4
c = 3 * 0.16
  = 0.48
d = 4 * 0.09
  =  0.36
e = 2 * 0.24
  = 0.48
Now add these lenght for average length:
 0.44 + 0.4 + 0.48 + 0.36 + 0.48 = 2.16
```

So, option (B) is correct.

Question 13

WRONG

Six files F1, F2, F3, F4, F5 and F6 have 100, 200, 50, 80, 120, 150 records respectively. In what order should they be stored so as to optimize act. Assume each file is accessed with the same frequency

F3, F4, F1, F5, F6, F2

B F2, F6, F5, F1, F4, F3

C F1, F2, F3, F4, F5, F6

Ordering is immaterial as all files are accessed with the same frequency.

Greedy Algorithms ISRO CS 2015
Discuss it

Question 13 Explanation:

```
This question is based on the Optimal Storage on Tape problem
which uses greedy approach to find the optimal time to retrieve them.
There are n programs of length L that are to be stored on a computer
tape. Associated with each program i is a length Li. So in order to
retrieve these programs most optimally, we need to store them in the
non-decreasing order of length Li.
So, the correct order is F3, F4, F1, F5, F6, F2
```

Question 14

WRONG

Consider a job scheduling problem with 4 jobs J_1, J_2, J_3, J_4 and with corresponding deadlines: (d_1, d_2, d_3, d_4) = (4, 2, 4, 2). Which of the following is not a feasible schedule without violating any job schedule?

A J_2, J_4, J_1, J_3

J_4, J_1, J_2, J_3

J_4, J_2, J_1, J_3

D J_4, J_2, J_3, J_1

Greedy Algorithms ISRO CS 2007
Discuss it

Question 15

CORRECT

What is the other name of Dijkstra algorithm?

single-source shortest path problem

B multiple-source shortest path problem

C multiple-destination shortest path problem

D single-destination shortest path problem

Greedy Algorithms
Discuss it

You have completed 15/15 questions .
Your accuracy is 40%.

Dynamic Programming

12

Question 1

CORRECT

Which of the following standard algorithms is not Dynamic Programming based.

A Bellman–Ford Algorithm for single source shortest path

B Floyd Warshall Algorithm for all pairs shortest paths

C 0-1 Knapsack problem

Prim's Minimum Spanning Tree

Dynamic Programming
Discuss it

Question 1 Explanation:
Prim's Minimum Spanning Tree is a Greedy Algorithm. All other are dynamic programming based.

Question 2

WRONG

We use dynamic programming approach when

We need an optimal solution

The solution has optimal substructure

C The given problem can be reduced to the 3-SAT problem

D It's faster than Greedy

Dynamic Programming
Discuss it

Question 2 Explanation:
http://www.geeksforgeeks.org/dynamic-programming-set-2-optimal-substructure-property/ Option (D) is incorrect because Greedy algorithms are generally faster than Dynamic programming. See http://www.geeksforgeeks.org/greedy-algorithms-set-1-activity-selection-problem/

Question 3

CORRECT

An algorithm to find the length of the longest monotonically increasing sequence of numbers in an array A[0 :n-1] is given below. Let Li denote the length of the longest monotonically increasing sequence starting at index i in the array.

Initialize $L_{n-1} = 1$

For all i such that $0 \le i \le n-2$

$$L_i = \begin{cases} 1 + L_{i+1} & \text{if } A[i] < A[i+1] \\ 1 & \text{Otherwise} \end{cases}$$

Finally the length of the longest monotonically increasing sequence is $\text{Max}(L_0, L_1, \ldots, L_{n-1})$.

Which of the following statements is TRUE?

The algorithm uses dynamic programming paradigm

B The algorithm has a linear complexity and uses branch and bound paradigm

The algorithm has a non-linear polynomial complexity and uses branch and bound

C paradigm

D The algorithm uses divide and conquer paradigm.

Dynamic Programming
Discuss it

Question 3 Explanation:
Please see http://www.geeksforgeeks.org/dynamic-programming-set-3-longest-increasing-subsequence/

Question 4

WRONG

Kadane algorithm is used to find:

A Maximum sum subsequence in an array

Maximum sum subarray in an array

Maximum product subsequence in an array

D Maximum product subarray in an array

Dynamic Programming
Discuss it

Question 4 Explanation:
Kadane algorithm is used to find the maximum sum subarray in an array. It runs in O(n) time complexity. See details of the algorithm here.

Question 5

WRONG

Four matrices M1, M2, M3 and M4 of dimensions pxq, qxr, rxs and sxt respectively can be multiplied is several ways with different number of total scalar multiplications. For example, when multiplied as ((M1 X M2) X (M3 X M4)), the total number of multiplications is pqr + rst + prt. When multiplied as (((M1 X M2) X M3) X M4), the total number of scalar multiplications is pqr + prs + pst. If p = 10, q = 100, r = 20, s = 5 and t = 80, then the number of scalar multiplications needed is

248000

B 44000

19000

D 25000

Dynamic Programming

Discuss it

Question 5 Explanation:

It is basically matrix chain multiplication problem. We get minimum number of multiplications using ((M1 X (M2 X M3)) X M4). Total number of multiplications = 100x20x5 (for M2 x M3) + 10x100x5 + 10x5x80 = 19000.

Question 6

WRONG

The subset-sum problem is defined as follows. Given a set of n positive integers, S = {a1 ,a2 ,a3 ,…,an} and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array X, with n rows and W+1 columns. X[i, j],1 <= i <= n, 0 <= j <= W, is TRUE if and only if there is a subset of {a1 ,a2 ,...,ai} whose elements sum to j. Which of the following is valid for 2 <= i <= n and ai <= j <= W?

A $X[i, j] = X[i - 1, j] \vee X[i, j - ai]$

$X[i, j] = X[i - 1, j] \vee X[i - 1, j - ai]$

$X[i, j] = X[i - 1, j] \wedge X[i, j - ai]$

D $X[i, j] = X[i - 1, j] \wedge X[i - 1, j - ai]$

Dynamic Programming

Discuss it

Question 6 Explanation:

X[I, j] (2 <= i <= n and ai <= j <= W), is true if any of the following is true 1) Sum of weights excluding ai is equal to j, i.e., if X[i-1, j] is true. 2) Sum of weights including ai is equal to j, i.e., if X[i-1, j-ai] is true so that we get (j – ai) + ai as j
See http://www.geeksforgeeks.org/dynamic-programming-subset-sum-problem/ for details.

Question 7

CORRECT

In the above question, which entry of the array X, if TRUE, implies that there is a subset whose elements sum to W?

A X[1, W]

B X[n ,0]

X[n, W]

D X[n -1, n]

Dynamic Programming
Discuss it

Question 7 Explanation:
If we get the entry X[n, W] as true then there is a subset of {a1, a2, .. an} that has sum as W. Reference: http://en.wikipedia.org/wiki/Subset_sum_problem

Question 8

WRONG

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences X[m] and Y[n] of lengths m and n respectively, with indexes of X and Y starting from 0. We wish to find the length of the longest common sub-sequence(LCS) of X[m] and Y[n] as l(m,n), where an incomplete recursive definition for the function l(i,j) to compute the length of The LCS of X[m] and Y[n] is given below:

```
l(i,j) = 0, if either i=0 or j=0

       = expr1, if i,j > 0 and X[i-1] = Y[j-1]
       = expr2, if i,j > 0 and X[i-1] != Y[j-1]
```

A expr1 ≡ l(i-1, j) + 1

expr1 ≡ l(i, j-1)

expr2 ≡ max(l(i-1, j), l(i, j-1))

D expr2 ≡ max(l(i-1,j-1),l(i,j))

Dynamic Programming GATE-CS-2009
Discuss it

Question 8 Explanation:
In Longest common subsequence problem, there are two cases for X[0..i] and Y[0..j]

```
1) The last characters of two strings match.

   The length of lcs is length of lcs of X[0..i-1] and Y[0..j-1]

2) The last characters don't match.

   The length of lcs is max of following two lcs values

   a) LCS of X[0..i-1] and Y[0..j]

   b) LCS of X[0..i] and Y[0..j-1]
```

Question 9

WRONG

Consider two strings A = "qpqrr" and B = "pqprqrp". Let x be the length of the longest common subsequence (not necessarily contiguous) between A and B and let y be the number of such longest common subsequences between A and B. Then x + 10y = ___.

A 33

23

C 43

34

Dynamic Programming GATE-CS-2014-(Set-2)
Discuss it

Question 9 Explanation:
//The LCS is of length 4. There are 3 LCS of length 4 "qprr", "pqrr" and qpqr A subsequence is a sequence that can be derived from another sequence by selecting zero or more elements from it, without changing the order of the remaining elements. Subsequence need not be contiguous. Since the length of given strings A = “qpqrr” and B = “pqprqrp” are very small, we don’t need to build a 5x7 matrix and solve it using dynamic programming. Rather we can solve it manually just by brute force. We will first check whether there exist a subsequence of length 5 since min_length(A,B) = 5. Since there is no subsequence , we will now check for length 4. “qprr”, “pqrr” and “qpqr” are common in both strings. X = 4 and Y = 3 X + 10Y = 34 This solution is contributed by **Pranjul Ahuja**

Question 10

WRONG

Let A1, A2, A3, and A4 be four matrices of dimensions 10 x 5, 5 x 20, 20 x 10, and 10 x 5, respectively. The minimum number of scalar multiplications required to find the product A1A2A3A4 using the basic matrix multiplication method is

1500

B 2000

500

D 100

Dynamic Programming GATE-CS-2016 (Set 2)

Discuss it

Question 10 Explanation:

We have many ways to do matrix chain multiplication because matrix multiplication is associative. In other words, no matter how we parenthesize the product, the result of the matrix chain multiplication obtained will remain the same. Here we have four matrices A1, A2, A3, and A4, we would have: ((A1A2)A3)A4 = ((A1(A2A3))A4) = (A1A2)(A3A4) = A1((A2A3)A4) = A1(A2(A3A4)). However, the order in which we parenthesize the product affects the number of simple arithmetic operations needed to compute the product, or the efficiency. Here, A1 is a 10 × 5 matrix, A2 is a 5 x 20 matrix, and A3 is a 20 x 10 matrix, and A4 is 10 x 5. If we multiply two matrices A and B of order l x m and m x n respectively,then the number of scalar multiplications in the multiplication of A and B will be lxmxn. Then, The number of scalar multiplications required in the following sequence of matrices will be : A1((A2A3)A4) = (5 x 20 x 10) + (5 x 10 x 5) + (10 x 5 x 5) = 1000 + 250 + 250 = 1500. All other parenthesized options will require number of multiplications more than 1500.

You have completed 10/15 questions .

@geeksforgeeks, Some rights reserved

Dynamic Programming

12

Question 11

CORRECT

Consider the weights and values of items listed below. Note that there is only one unit of each item.

Item number	Weight (in Kgs)	Value (in Rupees)
1	10	60
2	7	28
3	4	20
4	2	24

The task is to pick a subset of these items such that their total weight is no more than 11 Kgs and their total value is maximized. Moreover, no item may be split. The total value of items picked by an optimal algorithm is denoted by V_{opt}. A greedy algorithm sorts the items by their value-to-weight ratios in descending order and packs them greedily, starting from the first item in the ordered list. The total value of items picked by the greedy algorithm is denoted by V_{greedy}. The value of $V_{opt} - V_{greedy}$ is _____ .
Note -This was Numerical Type question.

16

B 8

C 44

D 60

Greedy Algorithms **Dynamic Programming** **GATE CS 2018**
Discuss it

Question 11 Explanation:

Item No	Weight	Value	Value/Weight
1	10	60	6
2	7	28	4
3	4	20	5
4	2	24	12

After sorting :

Item No	Weight	Value	Value/Weight
4	2	24	12
1	10	60	6
3	4	20	5
2	7	28	4

First we will pick item_4 (Value weight ratio is highest). Second highest is item_1, but cannot be picked because of its weight. Now item_3 shall be picked. item_2 cannot be included because of its weight. Therefore, overall profit by V_{greedy} = 20+24 = 44 Hence, **V_{opt} - V_{greedy} = 60-44 = 16** So, answer is 16.

Question 12

WRONG

Consider the following two sequences :

X = < B, C, D, C, A, B, C >, and

Y = < C, A, D, B, C, B >

The length of longest common subsequence of X and Y is :

A 5

B 3

4

2

Dynamic Programming UGC-NET CS 2017 Nov - III

Discuss it

Question 13

WRONG

Consider a sequence F_{00} defined as : $F_{00}(0) = 1$, $F_{00}(1) = 1$ $F_{00}(n) = 10 * F_{00}(n - 1) + 100$ $F_{00}(n - 2)$ for $n \geq 2$ Then what shall be the set of values of the sequence F_{00} ?

(1, 110, 1200)

B (1, 110, 600, 1200)

(1, 2, 55, 110, 600, 1200)

D (1, 55, 110, 600, 1200)

Dynamic Programming UGC NET CS 2017 Jan - II

Discuss it

Question 13 Explanation:

$F_{00}(0) = 1$, $F_{00}(1) = 1$
$F_{00}(n) = 10 * F_{00}(n - 1) + 100$
$F_{00}(2) = 10 * F_{00}(1) + 100$
$= 10 * 1 + 100$
$= 10 + 100$
$= 110$
Similarly:
$F_{00}(3) = 10 * F_{00}(2) + 100$
$= 10 * 110 + 100$
$= 1100 + 100$
$= 1200$
The sequence will be (1, 110, 1200).
So, (A) will be the answer.

Question 14

CORRECT

The following paradigm can be used to find the solution of the problem in minimum time: Given a set of non-negative integer, and a value K, determine if there is a subset of the given set with sum equal to K:

A Divide and Conquer

Dynamic Programming

C Greedy Algorithm

D Branch and Bound

Dynamic Programming ISRO CS 2018
Discuss it

Question 14 Explanation:
Given problem is Subset-sum problem in which a set of non-negative integers, and a value sum is given, to determine if there is a subset of the given set with sum equal to given sum. With recursion technique, time complexity of the above problem is exponential. We can solve the problem in Pseudo-polynomial time using Dynamic programming. Refer: Subset Sum Problem Option (B) is correct

Question 15

WRONG

What happens when a top-down approach of dynamic programming is applied to any problem?

A It increases both, the time complexity and the space complexity

It increases the space complexity and decreases the time complexity.

C It increases the time complexity and decreases the space complexity

It decreases both, the time complexity and the space complexity

Dynamic Programming
Discuss it

Question 15 Explanation:
As the mentioned approach uses the memoization technique it always stores the previously calculated values. Due to this, the time complexity is decreased but the space complexity is increased.

You have completed 15/15 questions .
Your accuracy is 33%.

12

@geeksforgeeks, Some rights reserved

Which of the following is not a backtracking algorithm?

A Knight tour problem

B N queen problem

Tower of hanoi

M coloring problem

Graph Traversals

123

Question 1

CORRECT

Which of the following algorithms can be used to most efficiently determine the presence of a cycle in a given graph ?

Depth First Search

B Breadth First Search

C Prim's Minimum Spanning Tree Algorithm

D Kruskal' Minimum Spanning Tree Algorithm

Graph Traversals
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/applications-of-depth-first-search/ and http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 2

CORRECT

Traversal of a graph is different from tree because

There can be a loop in graph so we must maintain a visited flag for every vertex

B DFS of a graph uses stack, but inorrder traversal of a tree is recursive

C BFS of a graph uses queue, but a time efficient BFS of a tree is recursive.

D All of the above

Graph Traversals
Discuss it

Question 2 Explanation:
See http://www.geeksforgeeks.org/depth-first-traversal-for-a-graph/

Question 3

WRONG

What are the appropriate data structures for following algorithms?

```
1) Breadth First Search

2) Depth First Search

3) Prim's Minimum Spanning Tree

4) Kruskal' Minimum Spanning Tree
```

```
1) Stack

2) Queue

3) Priority Queue

4) Union Find
```

```
1) Queue

2) Stack

3) Priority Queue

4) Union Find
```

C

```
1) Stack

2) Queue

3) Union Find

4) Priority Queue
```

D

```
1) Priority Queue

2) Queue

3) Stack

4) Union Find
```

Graph Traversals

Discuss it

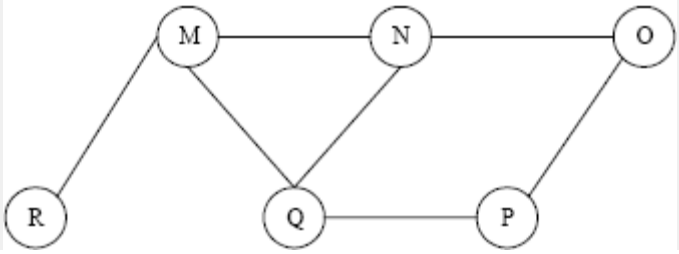
Question 3 Explanation:

1) Breadth First Search uses Queue 2) Depth First Search uses Stack 3) Prim's Minimum Spanning Tree uses Priority Queue. 4) Kruskal' Minimum Spanning Tree uses Union Find.

Question 4

WRONG

The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is



A MNOPQR

B NQMPOR

QMNPRO

QMNPOR

Graph Traversals
Discuss it

Question 4 Explanation:
Option (A) is MNOPQR. It cannot be a BFS as the traversal starts with M, but O is visited before N and Q. In BFS all adjacent must be visited before adjacent of adjacent. Option (B) is NQMPOR. It also cannot be BFS, because here, P is visited before O. (C) and (D) match up to QMNP. We see that M was added to the queue before N and P (because M comes before NP in QMNP). Because R is M's neighbor, it gets added to the queue before the neighbor of N and P (which is O). Thus, R is visited before O.

Question 5

WRONG

Let G be an undirected graph. Consider a depth-first traversal of G, and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statements is always true? (GATE CS 2000)

A {u,v} must be an edge in G, and u is a descendant of v in T

{u,v} must be an edge in G, and v is a descendant of u in T

If {u,v} is not an edge in G then u is a leaf in T

D If {u,v} is not an edge in G then u and v must have the same parent in T

Graph Traversals
Discuss it

Question 5 Explanation:

```
In DFS, if 'v' is visited
after 'u', then one of the following is true.
1) (u, v) is an edge.
     u
   /   \
  v     w
 /     / \
x     y   z

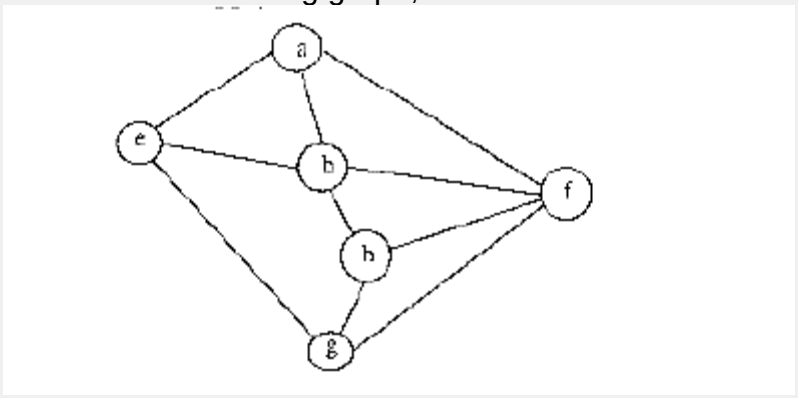
2) 'u' is a leaf node.
     w
   /   \
  x     v
 /     / \
u     y   z
```

In DFS, after visiting a node, we first recur for all unvisited children. If there are no unvisited children (u is leaf), then control goes back to parent and parent then visits next unvisited children.

Question 6

CORRECT

Consider the following graph,



Among the following sequences:

```
(I) a b e g h f

(II) a b f e h g

(III) a b f h g e

(IV) a f g h b e
```

Which are depth first traversals of the above graph?

A I, II and IV only

B I and IV only

C II, III and IV only

I, III and IV only

Graph Traversals
Discuss it

Question 6 Explanation:
We can check all DFSs for following properties.

```
In DFS, if a vertex 'v' is visited
after 'u', then one of the following is true.
1) (u, v) is an edge.
     u
   /   \
  v     w
 /     / \
x     y   z

2) 'u' is a leaf node.
     w
   /   \
  x     v
 /     / \
u     y   z
```

In DFS, after visiting a node, we first recur for all unvisited children. If there are no unvisited children (u is leaf), then control goes back to parent and parent then visits next unvisited children.

Question 7

CORRECT

Make is a utility that automatically builds executable programs and libraries from source code by reading files called makefiles which specify how to derive the target program. Which of the following standard graph algorithms is used by Make.

A Strongly Connected Components

Topological Sorting

C Breadth First Search

D Dijkstra's Shortest Path

Graph Traversals
Discuss it

Question 7 Explanation:
Make can decide the order of building a software using topological sorting. Topological sorting produces the order considering all dependencies provide by makefile. See following for details. Topological Sorting

Question 8

WRONG

Given two vertices in a graph s and t, which of the two traversals (BFS and DFS) can be used to find if there is path from s to t?

A Only BFS

Only DFS

Both BFS and DFS

D Neither BFS nor DFS

Graph Traversals
Discuss it

Question 8 Explanation:
We can use both traversals to find if there is a path from s to t.

Question 9

CORRECT

Which of the following condition is sufficient to detect cycle in a directed graph?

A There is an edge from currently being visited node to an already visited node.

There is an edge from currently being visited node to an ancestor of currently visited

node in DFS forest.

C Every node is seen twice in DFS.

D None of the bove

Graph Traversals
Discuss it

Question 9 Explanation:
See Detect Cycle in a Graph

Question 10

CORRECT

Is following statement true/false If a DFS of a directed graph contains a back edge, any other DFS of the same graph will also contain at least one back edge.
Source: http://courses.csail.mit.edu/6.006/oldquizzes/solutions/q2-s2009-sol.pdf

True

B False

Graph Traversals
Discuss it

Question 10 Explanation:
A back edge means a cycle in graph. So if there is a cycle, all DFS traversals would contain at least one back edge.

Graph Traversals

123
Question 11

WRONG

Is following statement true/false? A DFS of a directed graph always produces the same number of tree edges, i.e., independent of the order in which vertices are considered for DFS.
(Source http://courses.csail.mit.edu/6.006/oldquizzes/solutions/q2-f2008-sol.pdf)

True

False

Graph Traversals
Discuss it

Question 11 Explanation:
Consider the following graph. If we start from 'a', then there is one tree edge. If we start from 'b', then there is no tree edge. a---->b

Question 12

WRONG

If the DFS finishing time f[u] > f[v] for two vertices u and v in a directed graph G, and u and v are in the same DFS tree in the DFS forest, then u is an ancestor of v in the depth first tree. (Source http://courses.csail.mit.edu/6.006/oldquizzes/solutions/q2-f2007-sol.pdf)

True

False

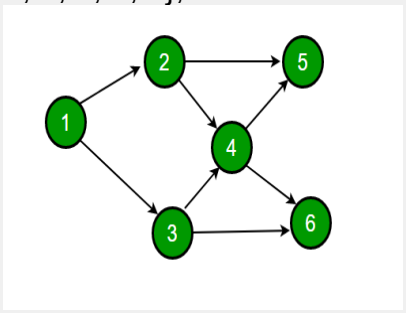
Graph Traversals
Discuss it

Question 12 Explanation:
In a graph with three nodes, r u and v, with edges (r; u) and (r; v), and r is the starting point for the DFS, u and v are siblings in the DFS tree, neither as the ancestor of the other.

Question 13

CORRECT

Consider the DAG with Consider V = {1, 2, 3, 4, 5, 6}, shown below. Which of the following is NOT a topological ordering?



A 1 2 3 4 5 6

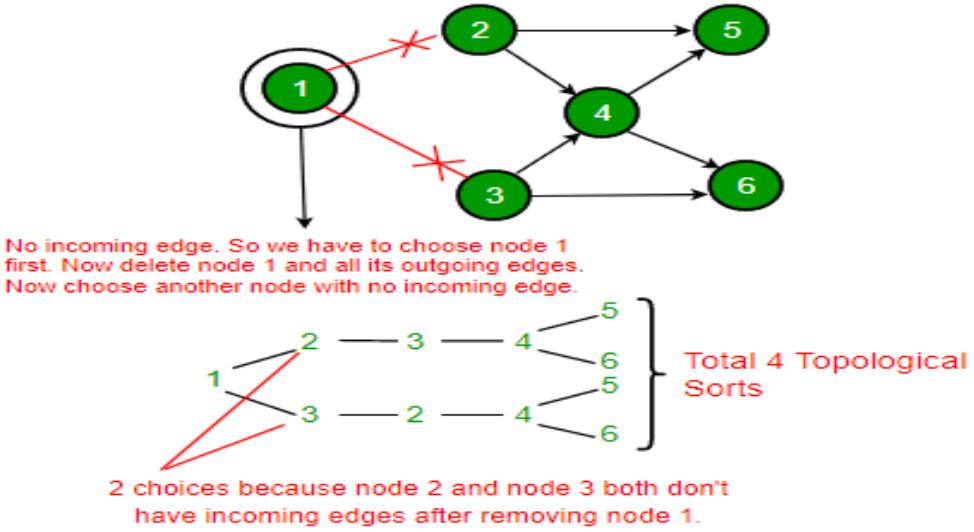
B 1 3 2 4 5 6

C 1 3 2 4 6 5

3 2 4 1 6 5

Graph Traversals GATE-CS-2007
Discuss it

Question 13 Explanation:
In option D, 1 appears after 2 and 3 which is not possible in Topological Sorting. In the given DAG it is directly visible that there is an outgoing edge from vertex 1 to vertex 2 and 3 hence 2 and 3 cannot come before vertex 1 so clearly option D is incorrect topological sort. But for questions in which it is not directly visible we should know how to find topological sort of a
DAG.



This solution is contributed by **Parul sharma.**

Question 14

WRONG

Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time on Depth First Search of G? Assume that the graph is represented using adjacency matrix.

A $O(n)$

$O(m+n)$

$O(n^2)$

D $O(mn)$

Graph Traversals **GATE-CS-2014-(Set-1)**
Discuss it

Question 14 Explanation:
Depth First Search of a graph takes O(m+n) time when the graph is represented using adjacency list. In adjacency matrix representation, graph is represented as an "n x n" matrix. To do DFS, for every vertex, we traverse the row corresponding to that vertex to find all adjacent vertices (In adjacency list representation we traverse only the adjacent vertices of the vertex). Therefore time complexity becomes $O(n^2)$

Question 15

CORRECT

Consider the tree arcs of a BFS traversal from a source node W in an unweighted, connected, undirected graph. The tree T formed by the tree arcs is a data structure for computing.

A the shortest path between every pair of vertices.

the shortest path from W to every vertex in the graph.

C the shortest paths from W to only those nodes that are leaves of T.

D the longest path in the graph

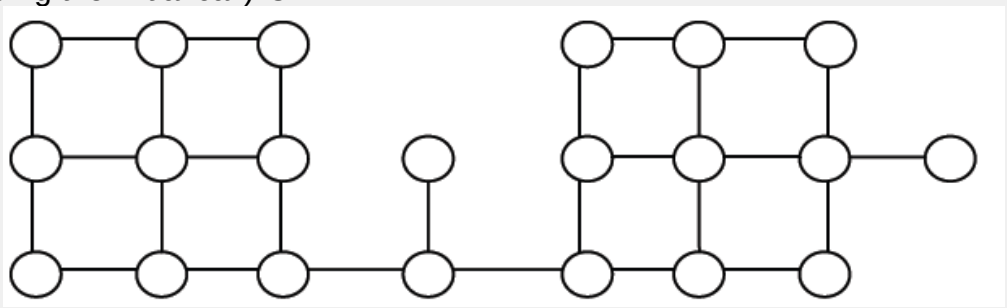
Graph Traversals GATE-CS-2014-(Set-2)
Discuss it

Question 15 Explanation:
BFS always produces shortest path from source to all other vertices in an unweighted graph.

Question 16

WRONG

Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is



_____.

17

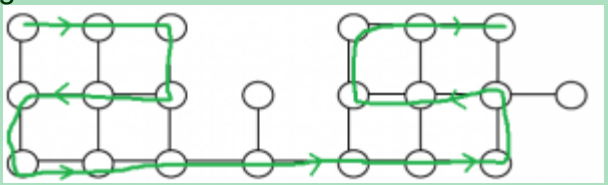
B 18

19

D 20

Graph Traversals GATE-CS-2014-(Set-3)
Discuss it

Question 16 Explanation:

The following diagram shows the worst case situation where the recursion tree has maximum depth.  So the recursion depth is 19 (including the first node).

Question 17

WRONG

Let T be a depth first search tree in an undirected graph G. Vertices u and n are leaves of this tree T. The degrees of both u and n in G are at least 2. which one of the following statements is true?

A There must exist a vertex w adjacent to both u and n in G

B There must exist a vertex w whose removal disconnects u and n in G

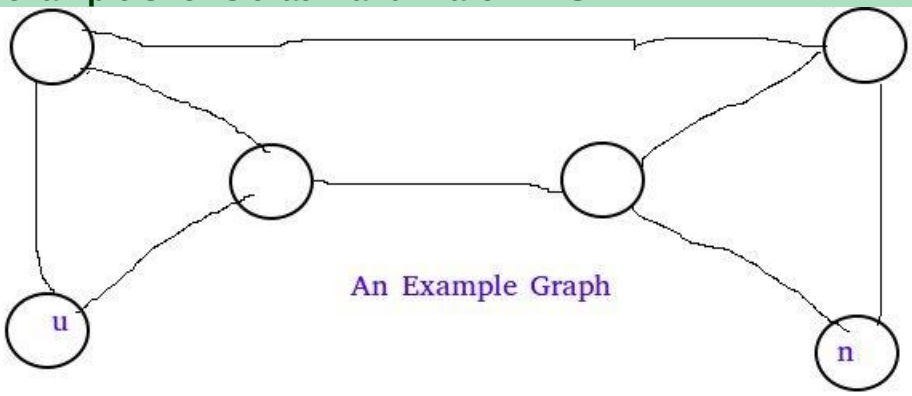
There must exist a cycle in G containing u and n

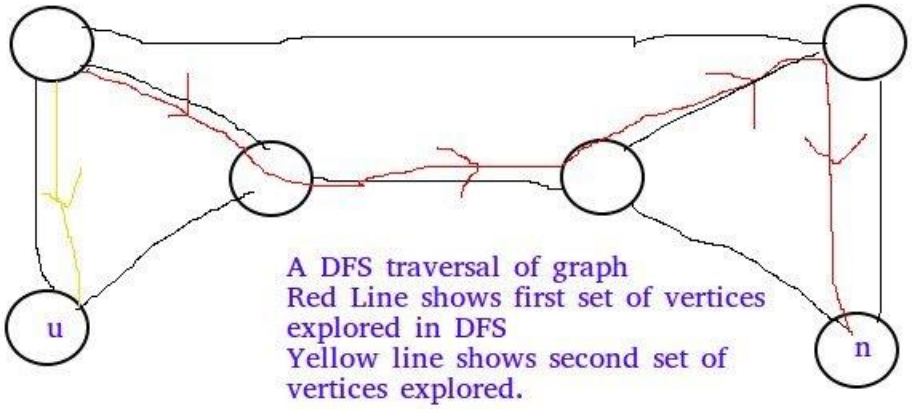
There must exist a cycle in G containing u and all its neighbours in G.

Graph Traversals GATE-CS-2006
Discuss it

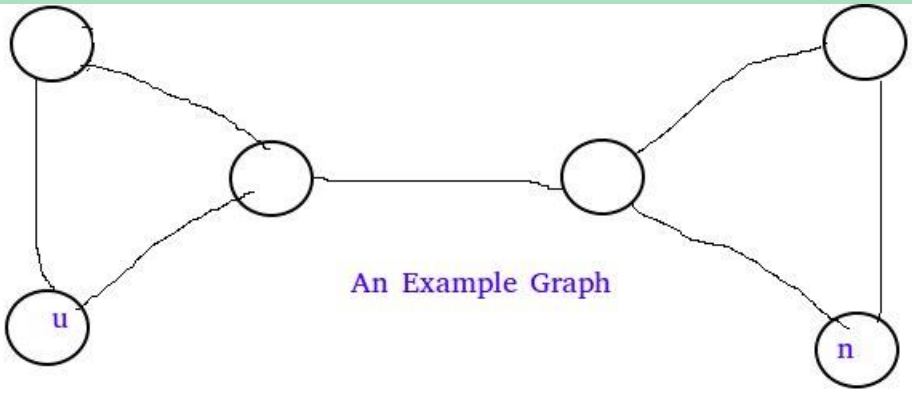
Question 17 Explanation:

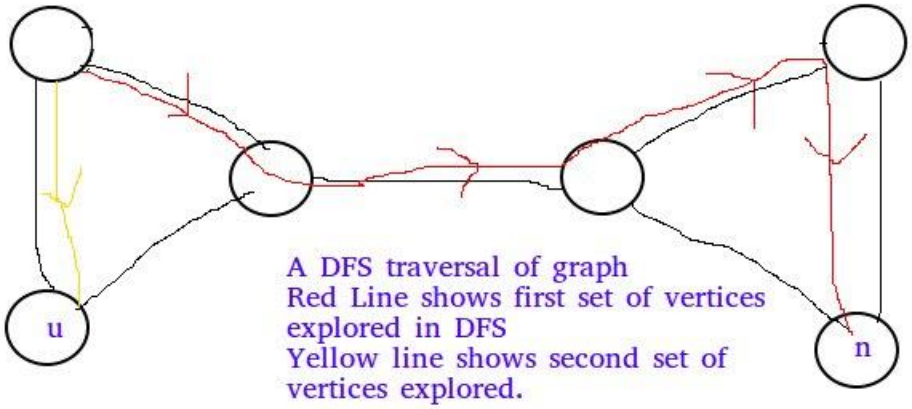
Below example shows that A and B are FALSE:





Below example shows C is false:





Question 18

CORRECT

Let G be an undirected graph. Consider a depth-first traversal of G, and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statements is always true?

A {u,v} must be an edge in G, and u is a descendant of v in T

B {u,v} must be an edge in G, and v is a descendant of u in T

If {u,v} is not an edge in G then u is a leaf in T

D If {u,v} is not an edge in G then u and v must have the same parent in T

Graph Traversals GATE-CS-2000
Discuss it

Question 18 Explanation:

```
In DFS, if 'v' is visited
after 'u', then one of the following is true.
1) (u, v) is an edge.
     u
   /   \
  v     w
 /     / \
x     y   z

2) 'u' is a leaf node.
     w
   /   \
  x     v
 /     / \
u     y   z
```

In DFS, after visiting a node, we first recur for all unvisited children. If there are no unvisited children (u is leaf), then control goes back to parent and parent then visits next unvisited children.

Question 19

WRONG

In a depth-first traversal of a graph G with n vertices, k edges are marked as tree edges. The number of connected components in G is

k

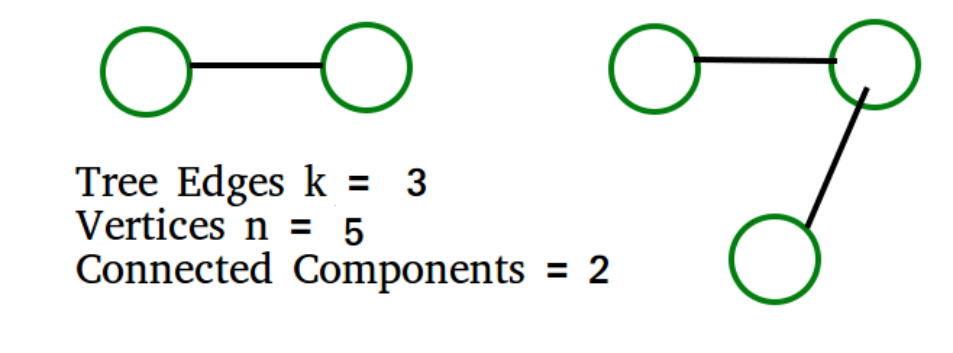
B k + 1

C n - k - 1

n - k

Graph Traversals Gate IT 2005
Discuss it

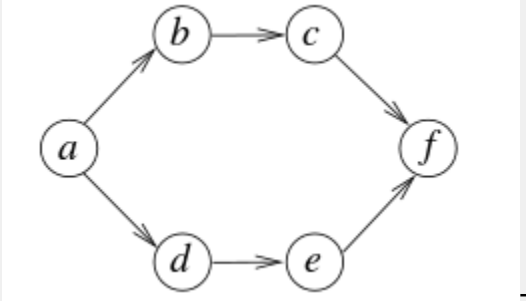
Question 19 Explanation:
Tree edges are the edges that are part of DFS tree. If there are x tree edges in a tree, then x+1 vertices in the tree. The output of DFS is a forest if the graph is disconnected. Let us see below simple example where graph is



disconnected. The above example matches with D option **More Examples:** 1) All vertices of Graph are connected. k must be n-1. We get number of connected components = n- k = n - (n-1) = 1 2) No vertex is connected. k must be 0. We get number of connected components = n- k = n - 0 = n

Question 20

WRONG

Consider the following directed graph. 

The number of different topological orderings of the vertices of the graph is Note : This question was asked as Numerical Answer Type.

1

B 2

C 4

6

Graph Traversals GATE-CS-2016 (Set 1)
Discuss it

Question 20 Explanation:
Following are different 6 Topological Sortings

```
a-b-c-d-e-f

a-d-e-b-c-f

a-b-d-c-e-f

a-d-b-c-e-f

a-b-d-e-c-f

a-d-b-e-c-f
```

You have completed 20/21 questions .

@geeksforgeeks, Some rights reserved

In an adjacency list representation of an undirected simple graph G = (V, E), each edge (u, v) has two adjacency list entries: [v] in the adjacency list of u, and [u] in the adjacency list of v. These are called twins of each other. A twin pointer is a pointer from an adjacency list entry to its twin. If |E| = m and |V | = n, and the memory size is not a constraint, what is the time complexity of the most efficient algorithm to set the twin pointer in each entry in each adjacency list?

A $\Theta(n^2)$

$\Theta(m+n)$

C $\Theta(m^2)$

D $\Theta(n^4)$

Graph Traversals GATE-CS-2016 (Set 2)
Discuss it

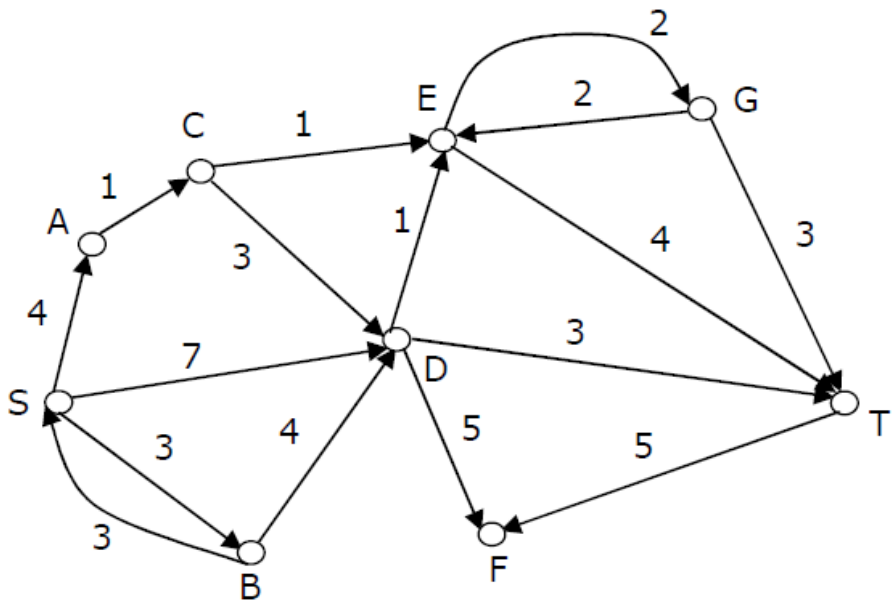
Question 21 Explanation:
First you need to find twins of each node. You can do this using **level order traversal (i.e., BFS)** once. Time complexity of BFS is Θ(m +n). And you have to use linked list for representation which is extra space (but memory size is not a constraint here). Final, time complexity is Θ(m + n) to set twin pointer. Option (B) is correct.

Graph Shortest Paths

Question 1

CORRECT

Consider the directed graph shown in the figure below. There are multiple shortest paths between vertices S and T. Which one will be reported by Dijstra?s shortest path algorithm? Assume that, in any iteration, the shortest path to a vertex v is updated only when a strictly shorter path to v is discovered.



A SDT

B SBDT

C SACDT

SACET

Graph Shortest Paths
Discuss it

Question 1 Explanation:
See Dijkstra's shortest path algorithm When the algorithm reaches vertex 'C', the distance values of 'D' and 'E' would be 7 and 6 respectively. So the next picked vertex would be 'E'

Question 2

CORRECT

To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is:

Queue

B Stack

C Heap

D B-Tree

Graph Shortest Paths

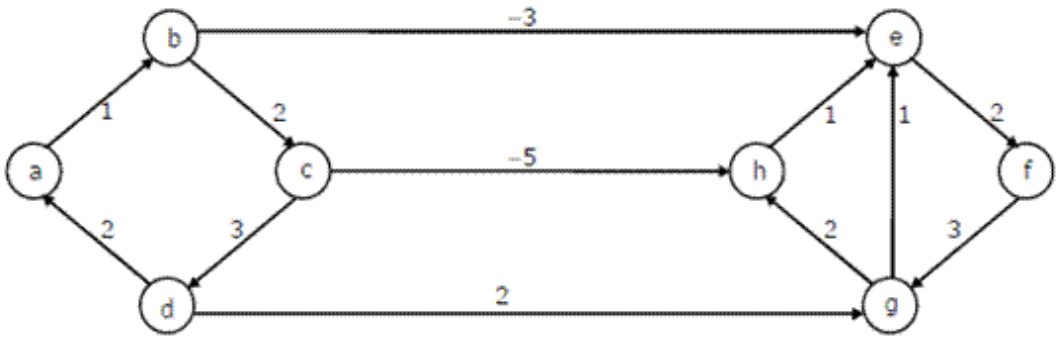
Discuss it

Question 2 Explanation:
The shortest path in an un-weighted graph means the smallest number of edges that must be traversed in order to reach the destination in the graph. This is the same problem as solving the weighted version where all the weights happen to be 1. If we use Queue (FIFO) instead of Priority Queue (Min Heap), we get the shortest path in linear time O(|V| + |E|). Basically we do BFS traversal of the graph to get the shortest paths.

Question 3

CORRECT

Dijkstra's single source shortest path algorithm when run from vertex a in the below graph, computes the correct shortest path distance
to



A only vertex a

B only vertices a, e, f, g, h

C only vertices a, b, c, d

all the vertices

Graph Shortest Paths

Discuss it

Question 3 Explanation:
Dijkstra's single source shortest path is not guaranteed to work for graphs with negative weight edges, but it works for the given graph. Let us see... Let us run the 1st pass b 1 b is minimum, so shortest distance to b is 1. After 1st pass, distances are c 3, e -2. e is minimum, so shortest distance to e is -2 After 2nd pass, distances are c 3, f 0. f is minimum, so shortest distance to f is 0 After 3rd pass, distances are c 3, g 3. Both are same, let us take g. so shortest distance to g is 3. After 4th pass, distances are c 3, h 5 c is minimum, so shortest distance to c is 3 After 5th pass, distances are h -2 h is minimum, so shortest distance to h is -2

Question 4

WRONG

In an unweighted, undirected connected graph, the shortest path from a node S to every other node is computed most efficiently, in terms of time complexity by

Dijkstra's algorithm starting from S.

B Warshall's algorithm

C Performing a DFS starting from S.

Performing a BFS starting from S.

Graph Shortest Paths
Discuss it

Question 4 Explanation:

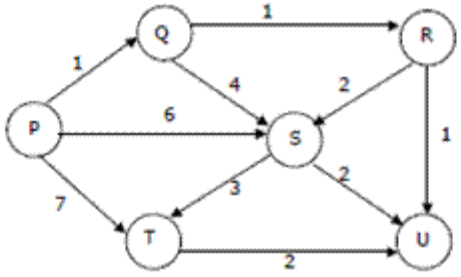
```
 * Time Comlexity of the Dijkstra's algorithm is O(|V|^2 + E)
 * Time Comlexity of the Warshall's algorithm is O(|V|^3)
 * DFS cannot be used for finding shortest paths
 * BFS can be used for unweighted graphs. Time Complexity for BFS is
O(|E| + |V|)
```

Question 5

CORRECT

Suppose we run Dijkstra's single source shortest-path algorithm on the following edge weighted directed graph with vertex P as the source. In what order do the nodes get included into the set of vertices for which the shortest path distances are

finalized? (GATE CS 2004)



A P, Q, R, S, T, U

P, Q, R, U, S, T

C P, Q, R, U, T, S

D P, Q, T, R, U, S

Graph Shortest Paths
Discuss it

Question 5 Explanation:
Refer http://www.geeksforgeeks.org/greedy-algorithms-set-6-dijkstras-shortest-path-algorithm/

Question 6

CORRECT

What is the time complexity of Bellman-Ford single-source shortest path algorithm on a complete graph of n vertices?

A [Tex]\Theta(n^2)[/Tex]

B [Tex]\Theta(n^2 Logn)[/Tex]

[Tex]\Theta(n^3)[/Tex]

D [Tex]\Theta(n^3 Logn)[/Tex]

Graph Shortest Paths
Discuss it

Question 6 Explanation:

Time complexity of Bellman-Ford algorithm is where V is number of vertices and E is number edges (See this). If the graph is complete, the value of E

becomes . So overall time complexity becomes

Question 7

WRONG

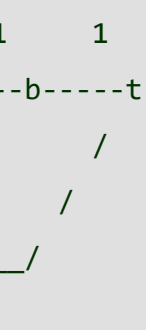
In a weighted graph, assume that the shortest path from a source 's' to a destination 't' is correctly calculated using a shortest path algorithm. Is the following statement true? If we increase weight of every edge by 1, the shortest path always remains same.

Yes

No

Graph Shortest Paths
Discuss it

Question 7 Explanation:
See the following counterexample. There are 4 edges s-a, a-b, b-t and s-t of wights 1, 1, 1 and 4 respectively. The shortest path from s to t is s-a, a-b, b-t. IF we increase weight of every edge by 1, the shortest path changes to s-t.



Question 8

WRONG

Following statement is true or false?

If we make following changes to Dijkstra, then it can be used to find the longest simple path, assume that the graph is acyclic.

1) Initialize all distances as minus infinite instead of plus infinite.

2) Modify the relax condition in Dijkstra's algorithm to update distance
of an adjacent v of the currently considered vertex u only
if "dist[u]+graph[u][v] > dist[v]". In shortest path algo,
the sign is opposite.

True

False

Graph Shortest Paths
Discuss it

Question 8 Explanation:
In shortest path algo, we pick the minimum distance vertex from the set of vertices for which distance is not finalized yet. And we finalize the distance of the minimum distance vertex. For maximum distance problem, we cannot finalize the distance because there can be a longer path through not yet finalized vertices.

Question 9

CORRECT

Which of the following algorithm can be used to **efficiently** calculate single source shortest paths in a Directed Acyclic Graph?

A Dijkstra

B Bellman-Ford

Topological Sort

D Strongly Connected Component

Graph Shortest Paths
Discuss it

Question 9 Explanation:
Using Topological Sort, we can find single source shortest paths in O(V+E) time which is the most efficient algorithm. See following for details. Shortest Path in Directed Acyclic Graph

Question 10

CORRECT

Given a directed graph where weight of every edge is same, we can efficiently find shortest path from a given source to destination using?

Breadth First Traversal

B Dijkstra's Shortest Path Algorithm

C Neither Breadth First Traversal nor Dijkstra's algorithm can be used

D Depth First Search

Graph Shortest Paths
Discuss it

Question 10 Explanation:
With BFS, we first find explore vertices at one edge distance, then all vertices at 2 edge distance, and so on.

You have completed 10/27 questions .

@geeksforgeeks, Some rights reserved

Graph Shortest Paths

123

Question 11

WRONG

Is the following statement valid about shortest paths? *Given a graph, suppose we have calculated shortest path from a source to all other vertices. If we modify the graph such that weights of all edges is becomes double of the original weight, then the shortest path remains same only the total weight of path changes.*

True

False

Graph Shortest Paths
Discuss it

Question 11 Explanation:
The shortest path remains same. It is like if we change unit of distance from meter to kilo meter, the shortest paths don't change.

Question 12

CORRECT

Match the following

Group A	Group B

```
a) Dijkstra's single shortest path algo                p) Dynamic
Programming

b) Bellmen Ford's single shortest path algo             q) Backtracking

c) Floyd Warshell's all pair shortest path algo.       r) Greedy
Algorithm
```

A a-r, b-q, c-p

B a-p, b-p, c-p

a-r, b-p, c-p

D a-p, b-r, c-q

Graph Shortest Paths
Discuss it

Question 12 Explanation:
Dijkstra is a greedy algorithm where we pick the minimum distant vertex from not yet finalized vertices. Bellman Ford and Floyd Warshell both are Dynamic Programming algorithms where we build the shortest paths in bottom up manner.

Question 13

WRONG

Is the following statement valid?.

Given a weighted graph where weights of all edges are unique (no two edge have same weights), there is always a unique shortest path from a source to destination in such a graph.

True

False

Graph Shortest Paths
Discuss it

Question 13 Explanation:
There can be more than one paths with same weight. Consider a path with one edge of weight 5 and another path with two edges of weights 2 and 3. Both paths have same weights.

Question 14

CORRECT

Is the following statement valid?.

Given a graph where all edges have positive weights, the shortest paths produced by Dijsktra and Bellman Ford algorithm may be different but path weight would always be same.

True

B False

Graph Shortest Paths
Discuss it

Question 14 Explanation:
Dijkstra and Bellman-Ford both work fine for a graph with all positive weights, but they are different algorithms and may pick different edges for shortest paths.

Question 15

WRONG

Which of the following statement(s)is / are correct regarding Bellman-Ford shortest path algorithm?

```
P: Always finds a negative weighted cycle, if one exist s.

Q: Finds whether any negative weighted cycle is reachable

   from the source.
```

P Only

Q Only

C Both P and Q

D Neither P nor Q

Graph Shortest Paths **GATE-CS-2009**
Discuss it

Question 15 Explanation:
Bellman-Ford shortest path algorithm is a single source shortest path algorithm. So it can only find cycles which are reachable from a given source, not any negative weight cycle. Consider a disconnected where a negative weight cycle is not reachable from the source at all. If there is a negative weight cycle, then it will appear in shortest path as the negative weight cycle will always form a shorter path when iterated through the cycle again and again.

Question 16

WRONG

Let G(V, E) an undirected graph with positive edge weights. Dijkstra's single-source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

A O(| V |2)

B O (| E | + | V | log | V |)

O (| V | log | V |)

O ((| E | + | V |) log | V |)

Graph Shortest Paths GATE-CS-2005
Discuss it

Question 16 Explanation:
See Time Complexity Section of http://www.geeksforgeeks.org/greedy-algorithms-set-7-dijkstras-algorithm-for-adjacency-list-representation/

Question 17

WRONG

Let G = (V, E) be an undirected graph with a subgraph G1 = (V1, El). Weights are assigned to edges of G as follows :

$$w(e) = \begin{cases} 0 \text{ if } e \in E_1 \\ 1 \text{ otherwise} \end{cases}$$

A single-source shortest path algorithm is executed on the weighted graph (V, E, w) with an arbitrary vertex v1 of V1 as the source. Which of the following can always be inferred from the path costs computed?

The number of edges in the shortest paths from v1 to all vertices of G

G1 is connected

C V1 forms a clique in G

D G1 is a tree

Graph Shortest Paths GATE-CS-2003
Discuss it

Question 17 Explanation:
When shortest path shortest path from v1 (one of the vertices in V1) is computed. G1 is connected if the distance from v1 to any other vertex in V1 is greater than 0, otherwise G1 is disconnected.

Question 18

WRONG

Let G (V, E) be a directed graph with n vertices. A path from v_i to v_j in G is sequence of vertices (v_i, v_{i+1}, ..., v_j) such that (vk, v_{k+1}) ∈ E for all k in i through j - 1. A simple path is a path in which no vertex appears more than once. Let A be an n x n array initialized as follow

$$A[j,k] = \begin{cases} 1 \text{ if } (j,k) \in E \\ 0 \text{ otherwise} \end{cases}$$

Consider the following algorithm.

```
for i = 1 to n
   for j = 1 to n
      for k = 1 to n
         A [j , k] = max (A[j, k] (A[j, i] + A [i, k]);
```

Which of the following statements is necessarily true for all j and k after terminal of the above algorithm ?

A $A[j, k] \leq n$

B If $A[j, k] \geq n - 1$, then G has a Hamiltonian cycle

If there exists a path from j to k, A[j, k] contains the longest path lens from j to k

If there exists a path from j to k, every simple path from j to k contain most A[j, k] edges

Graph Shortest Paths GATE-CS-2003

Discuss it

Question 18 Explanation:

```
In the original input matrix,  A[j , k] is 1 if there
is an edge from j to k, else 0.


Below expression is important to note:


A[j , k] = max(A[j, k] (A[j, i] + A [i, k]);


This expression puts the count of maximum edges on a path from
j to k.  In this expression, we consider every vertex k that can
become an intermediate vertex and can give longer path.
```

Question 19

WRONG

Let G = (V, E) be a simple undirected graph, and s be a particular vertex in it called the source. For x ∈ V, let d(x) denote the shortest distance in G from s to x. A breadth first search (BFS) is performed starting at s. Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T, then which one of the following CANNOT be the value of d(u) – d(v)?

-1

B 0

C 1

2

Graph Shortest Paths GATE-CS-2015 (Set 1)
Discuss it

Question 19 Explanation:
Note that the given graph is undirected, so an edge (u, v) also means (v, u) is also an edge. Since a shorter path can always be obtained by using edge (u, v) or (v, u), the difference between d(u) and d(v) can not be more than 1.

Question 20

WRONG

Let G be a directed graph whose vertex set is the set of numbers from 1 to 100. There is an edge from a vertex i to a vertex j iff either j = i + 1 or j = 3i. The minimum number of edges in a path in G from vertex 1 to vertex 100 is

A 4

7

C 23

99

Graph Shortest Paths Gate IT 2005
Discuss it

Question 20 Explanation:
The task is to find minimum number of edges in a path in G from vertex 1 to vertex 100 such that we can move to either i+1 or 3i from a vertex i.

```
Since the task is to minimize number of edges,
```

```
we would prefer to follow 3*i.

Let us follow multiple of 3.

1 => 3 => 9 => 27 => 81, now we can't follow multiple
of 3. So we will have to follow i+1. This solution gives
a long path.

What if we begin from end, and we reduce by 1 if
the value is not multiple of 3, else we divide by 3.
100 => 99 => 33 => 11 => 10 => 9 => 3 => 1

So we need total 7 edges.
```

@geeksforgeeks, Some rights reserved

Graph Shortest Paths

123

Question 21

CORRECT

Consider the weighted undirected graph with 4 vertices, where the weight of edge {i, j} g is given by the entry W_{ij} in the matrix W

$$W = \begin{bmatrix} 0 & 2 & 8 & 5 \\ 2 & 0 & 5 & 8 \\ 8 & 5 & 0 & x \\ 5 & 8 & x & 0 \end{bmatrix}$$

The largest possible integer value of x, for which at least one shortest path between some pair of vertices will contain the edge with weight x is _____

Note : This question was asked as Numerical Answer Type.

A 8

12

C 10

D 11

Graph Shortest Paths GATE-CS-2016 (Set 1)

Discuss it

Question 21 Explanation:
Let vertices be 0, 1, 2 and 3. x directly connects 2 to 3. The shortest path (excluding x) from 2 to 3 is of weight 12 (2-1-0-3).

Question 22

CORRECT

The Floyd-Warshall algorithm for all-pair shortest paths computation is based on:

A Greedy paradigm.

B Divide-and-Conquer paradigm.

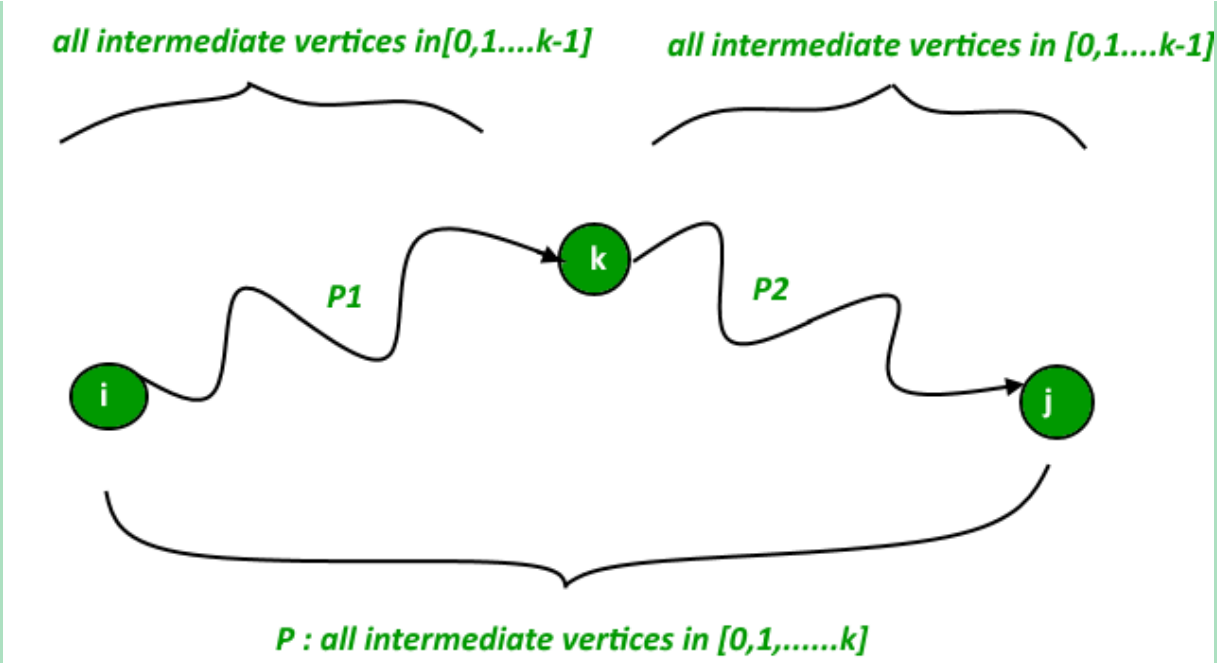
Dynamic Programming paradigm.

D neither Greedy nor Divide-and-Conquer nor Dynamic Programming paradigm.

Graph Shortest Paths GATE-CS-2016 (Set 2)

Discuss it

Question 22 Explanation:
Floyd Warshall Algorithm is a Dynamic Programming based algorithm. It finds all pairs shortest paths using following recursive nature of problem. For every pair (i, j) of source and destination vertices respectively, there are two possible cases. 1) k is not an intermediate vertex in shortest path from i to j. We keep the value of dist[i][j] as it is. 2) k is an intermediate vertex in shortest path from i to j. We update the value of dist[i][j] as dist[i][k] + dist[k][j]. The following figure is taken from the Cormen book. It shows the above optimal substructure property in the all-pairs shortest path problem.



Since there are overlapping subproblems in recursion, it uses dynamic programming.

Question 23

WRONG

Consider a weighted undirected graph with positive edge weights and let uv be an edge in the graph. It is known that the shortest path from the source vertex s to u has weight 53 and the shortest path from s to v has weight 65. Which one of the following statements is always true?

A weight (u, v) < 12

B weight (u, v) ≤ 12

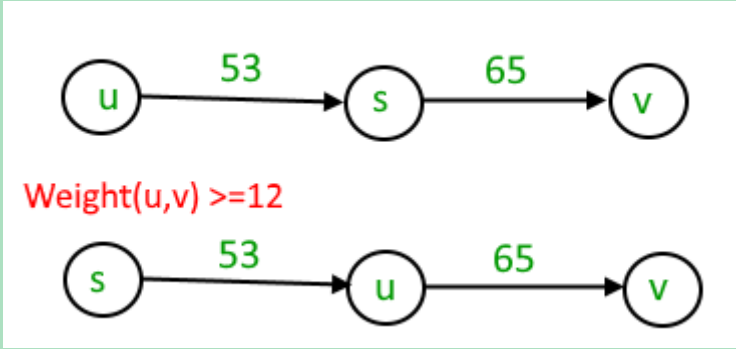
weight (u, v) > 12

weight (u, v) ≥ 12

Graph Shortest Paths Graph Theory Gate IT 2007

Discuss it

Question 23 Explanation:



The minimum weight happens when (S,U) + (U,V) = (S,V) Else (S,U) + (U,V) >= (S,V) Given (S,U) = 53, (S,V) = 65 53 + (U,V) >= 63 (U,V) >= 12. This solution is contributed by **Anil Saikrishna Devarasetty**

Question 24

CORRECT

Which of the following data structure is useful in traversing a given graph by breadth first search?

A Stack

B List

Queue

D None of the above.

Graph Graph Shortest Paths ISRO CS 2017 - May

Discuss it

Question 24 Explanation:
BFS performs level-order traversal which can be fairly done using a queue. A queue uses FIFO ordering and the nodes that we enqueue first are explored first maintaining the order of traversal.

Question 25

CORRECT

Which of the following algorithm solves the all-pair shortest path problem?

A Prim’s algorithm

B Dijikstra’s algorithm

C Bellman-Ford’s algorithm

Floyd-Warshall’s algorithm

Graph Shortest Paths ISRO CS 2017 - May
Discuss it

Question 25 Explanation:
Prim's Algorithm is used to find MST of a given graph. Refer: Prim's Minimum Spanning Tree (MST) Dijikstra's Algorithm is used to find the shortest path from source to all the other nodes in a weighted graph. Refer: Dijkstra's Algorithm BellmanFord's Algorithm is used to find the shortest distance from source to all other nodes in a graph that can contain negetive weight edges. Refer:Bellman–Ford Algorithm Floyd-Warshall's Algorithm: It is an All-pair-shortest path algorithm, used to find shortest distance between every pair of vertices. Refer: Floyd Warshall Algorithm So, option (D) is correct.

Question 26

CORRECT

Which one of the following algorithm design techniques is used in finding all pairs of shortest distances in a graph?

Dynamic programming

B Backtracking

C Greedy

D Divide and Conquer

Graph Shortest Paths ISRO CS 2008
Discuss it

Question 26 Explanation:
Floyd Warshall Algorithm is the All Pairs Shortest Path problem which uses Dynamic Programming to find shortest distances between every pair of vertices in a given edge weighted directed Graph. Option (A) is correct.

Question 27

CORRECT

Which of the following is application of Breath First Search on the graph?

A Finding diameter of the graph

B Finding bipartite graph

Both (a) and (b)

D None of the above

Graph Shortest Paths ISRO CS 2018
Discuss it

Question 27 Explanation:
BFS is used to Find the diameter of the graph and to test whether a graph is bipartite or not. BFS has many other applications also. Refer: Applications of Breadth First Traversal Option (C) is correct.

You have completed 27/27 questions .
Your accuracy is 56%.

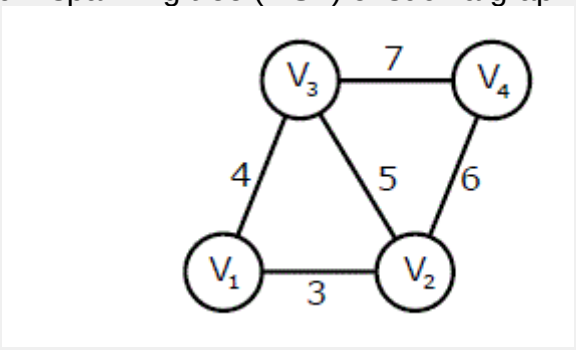
@geeksforgeeks, Some rights reserved

Graph Minimum Spanning Tree

123
Question 1

WRONG

An undirected graph G(V, E) contains n (n > 2) nodes named v1 , v2 ,….vn. Two nodes vi , vj are connected if and only if 0 < |i – j| <= 2. Each edge (vi, vj) is assigned a weight i + j. A sample graph with n = 4 is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes? (GATE CS 2011)



1/12(11n^2 – 5n)

n^2 – n + 1

C 6n – 11

D 2n + 1

Graph Minimum Spanning Tree
Discuss it

Question 1 Explanation:
See http://www.geeksforgeeks.org/data-structures-and-algorithms-set-27/ for explanation.

Question 2

WRONG

The length of the path from v5 to v6 in the MST of previous question with n = 10 is

A 11

25

31

D 41

Graph Minimum Spanning Tree
Discuss it

Question 2 Explanation:
See question 2 of http://www.geeksforgeeks.org/data-structures-and-algorithms-set-27/ for explanation.

Question 3

WRONG

Consider a complete undirected graph with vertex set {0, 1, 2, 3, 4}. Entry Wij in the matrix W below is the weight of the edge {i, j}. What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T? (GATE CS 2010)

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

A 7

8

C 9

10

Graph Minimum Spanning Tree
Discuss it

Question 3 Explanation:
To get the minimum spanning tree with vertex 0 as leaf, first remove 0th row and 0th column and then get the minimum spanning tree (MST) of the remaining graph. Once we have MST of the remaining graph, connect the MST to vertex 0 with the edge with minimum weight (we have two options as there are two 1s in 0th row).

Question 4

WRONG

In the graph given in above question question, what is the minimum possible weight of a path P from vertex 1 to vertex 2 in this graph such that P contains at most 3 edges?

A 7

8

9

D 10

Graph Minimum Spanning Tree
Discuss it

Question 4 Explanation:
Path: 1 -> 0 -> 4 -> 2 Weight: 1 + 4 + 3

Question 5

CORRECT

An undirected graph G has n nodes. Its adjacency matrix is given by an n × n square matrix whose (i) diagonal elements are 0‘s and (ii) non-diagonal elements are 1‘s. which one of the following is TRUE?

A Graph G has no minimum spanning tree (MST)

B Graph G has a unique MST of cost n-1

Graph G has multiple distinct MSTs, each of cost n-1

D Graph G has multiple spanning trees of different costs

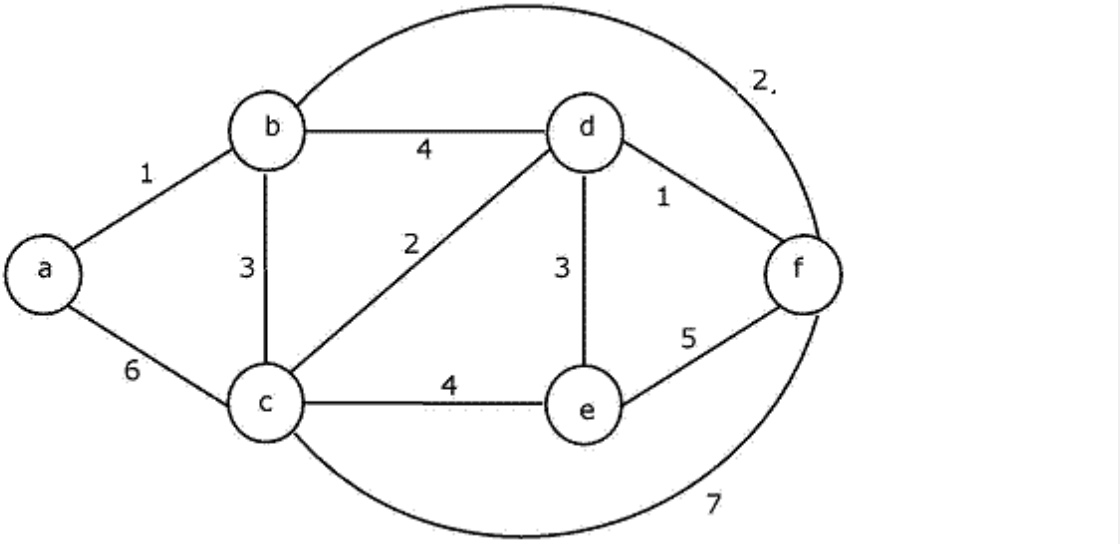
Graph Minimum Spanning Tree
Discuss it

Question 5 Explanation:
See Question 2 of http://www.geeksforgeeks.org/data-structures-and-algorithms-set-22/

Question 6

WRONG

Consider the following graph:



Which one of the following cannot be the sequence of edges added, in that order, to a minimum spanning tree using Kruskal's algorithm?

A (a—b),(d—f),(b—f),(d—c),(d—e)

B (a—b),(d—f),(d—c),(b—f),(d—e)

(d—f),(a—b),(d—c),(b—f),(d—e)

(d—f),(a—b),(b—f),(d—e),(d—c)

Graph Minimum Spanning Tree
Discuss it

Question 6 Explanation:
The edge (d-e) cannot be considered before (d-c) in Kruskal's minimum spanning tree algorithm because Kruskal's algorithm picks the edge with minimum weight from the current set of edges at each step.

Question 7

CORRECT

Let G be an undirected connected graph with distinct edge weight. Let emax be the edge with maximum weight and emin the edge with minimum weight. Which of the following statements is false? (GATE CS 2000)

A Every minimum spanning tree of G must contain emin

B If emax is in a minimum spanning tree, then its removal must disconnect G

No minimum spanning tree contains emax

D G has a unique minimum spanning tree

Graph Minimum Spanning Tree
Discuss it

Question 7 Explanation:
(a) and (b) are always true. (c) is false because (b) is true. (d) is true because all edge weights are distinct for G.

Question 8

WRONG

Consider a weighted complete graph G on the vertex set {v1,v2 ,v} such that the weight of the edge (v,,v) is 2|i-j|. The weight of a minimum spanning tree of G is: (GATE CS 2006)

A n — 1

2n — 2

nC2

D 2

Graph Minimum Spanning Tree
Discuss it

Question 8 Explanation:
Minimum spanning tree of such a graph is

```
v1
  \
    v2
      \
        v3
```

```
          \
           v4
             .
               .
                 .
                   vn
```

Weight of the minimum spanning tree = 2|2 - 1| + 2|3 - 2| + 2|4 - 3| + 2|5 - 4| .... + 2| n - (n-1) | = 2n - 2

Question 9

WRONG

Let G be a weighted graph with edge weights greater than one and G'be the graph constructed by squaring the weights of edges in G. Let T and T' be the minimum spanning trees of G and G', respectively, with total weights t and t'. Which of the following statements is TRUE?

A T' = T with total weight t' = t^2

B T' = T with total weight t' < t^2

T' != T but total weight t' = t^2

None of the above

GATE CS 2012 Graph Minimum Spanning Tree
Discuss it

Question 9 Explanation:
Squaring the weights of the edges in a weighted graph will not change the minimum spanning tree. Assume the opposite to obtain a contradiction. If the minimum spanning tree changes then at least one edge from the old graph G in the old minimum spanning tree T must be replaced by a new edge in tree T' from the graph G' with squared edge weights. The new edge from G' must have a lower weight than the edge from G. This implies that there exists some weights C1 and C2 such that C1 < C2 and C12 >= C22. This is a contradiction.
Source: http://www.cs.nyu.edu/courses/spring06/V22.0310-001/hw3.htm Sums of squares of two or more numbers is always smaller than square of sum. Example: 2^2 + 2^2 < (4)^2 But

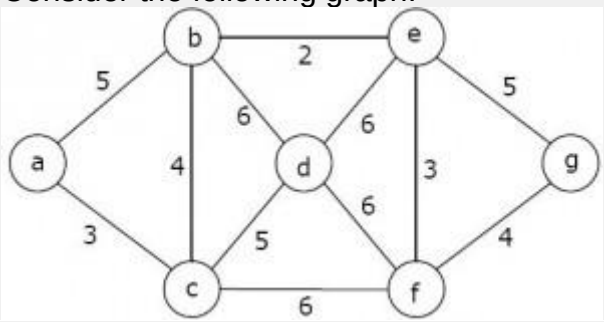
```
there is one counter example when the graph has only one edge.

         In that case, the two values are same.
```

Question 10

CORRECT

Consider the following graph:



Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?

A (b,e)(e,f)(a,c)(b,c)(f,g)(c,d)

B (b,e)(e,f)(a,c)(f,g)(b,c)(c,d)

C (b,e)(a,c)(e,f)(b,c)(f,g)(c,d)

(b,e)(e,f)(b,c)(a,c)(f,g)(c,d)

Graph Minimum Spanning Tree GATE-CS-2009

Discuss it

Question 10 Explanation:

In the sequence (b, e) (e, f) (b, c) (a, c) (f, g) (c, d) given option D, the edge (a, c) of weight 4 comes after (b, c) of weight 3. In Kruskal's Minimum Spanning Tree Algorithm, we first sort all edges, then consider edges in sorted order, so a higher weight edge cannot come before a lower weight edge.

You have completed 10/21 questions .

@geeksforgeeks, Some rights reserved

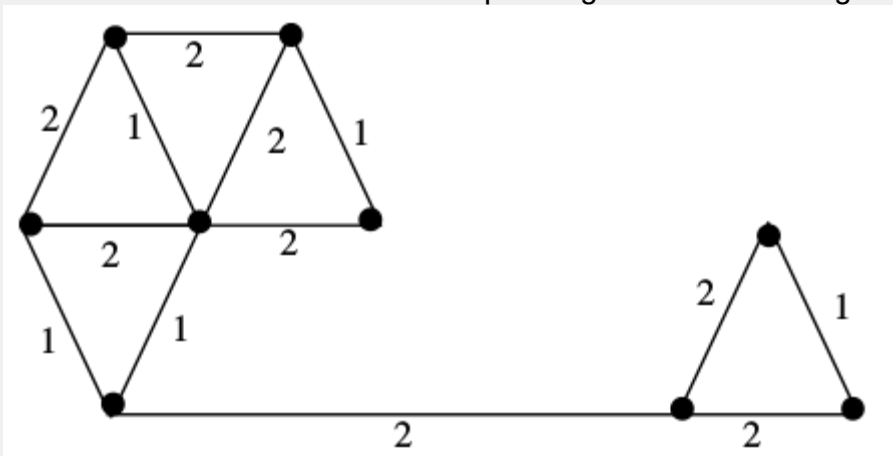
Graph Minimum Spanning Tree

123

Question 11

WRONG

The number of distinct minimum spanning trees for the weighted graph below is ____



4

B 5

6

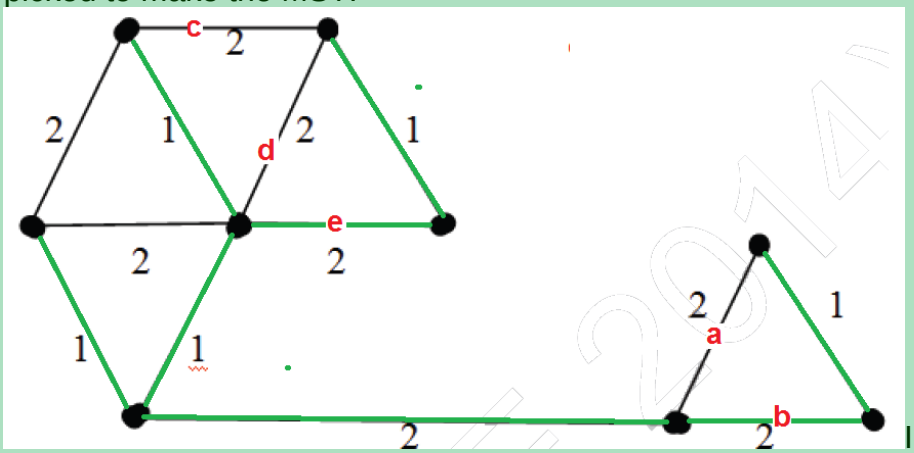
D 7

Graph Minimum Spanning Tree GATE-CS-2014-(Set-2)

Discuss it

Question 11 Explanation:

Below diagram shows a minimum spanning tree. Highlighted (in green) are the edges picked to make the MST.



In the right side of MST, we could either pick edge ‘a’ or ‘b’. In the left side, we could either pick ‘c’ or ‘d’ or ‘e’ in MST. There are 2 options for one edge to be picked and 3 options for another edge to be picked. Therefore, total 2*3 possible MSTs.

Question 12

WRONG

Let s and t be two vertices in a undirected graph G + (V, E) having distinct positive edge weights. Let [X, Y] be a partition of V such that s ∈ X and t ∈ Y. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y The edge e must definitely belong to:

the minimum weighted spanning tree of G

B the weighted shortest path from s to t

each path from s to t

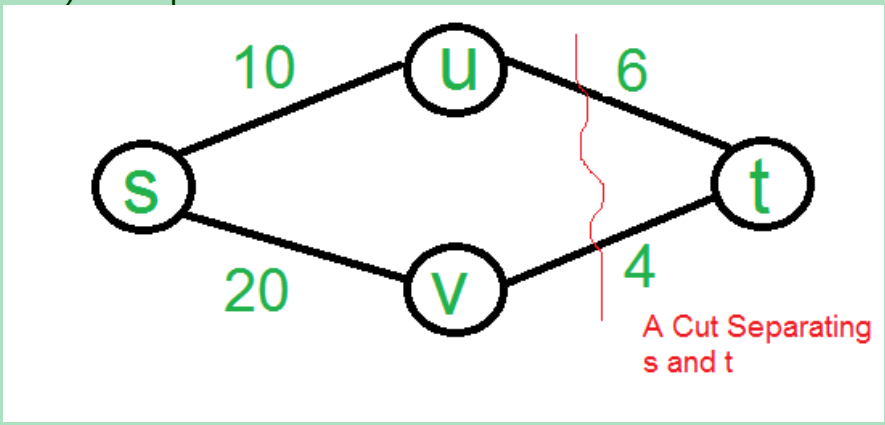
D the weighted longest path from s to t

Graph Minimum Spanning Tree GATE-CS-2005

Discuss it

Question 12 Explanation:

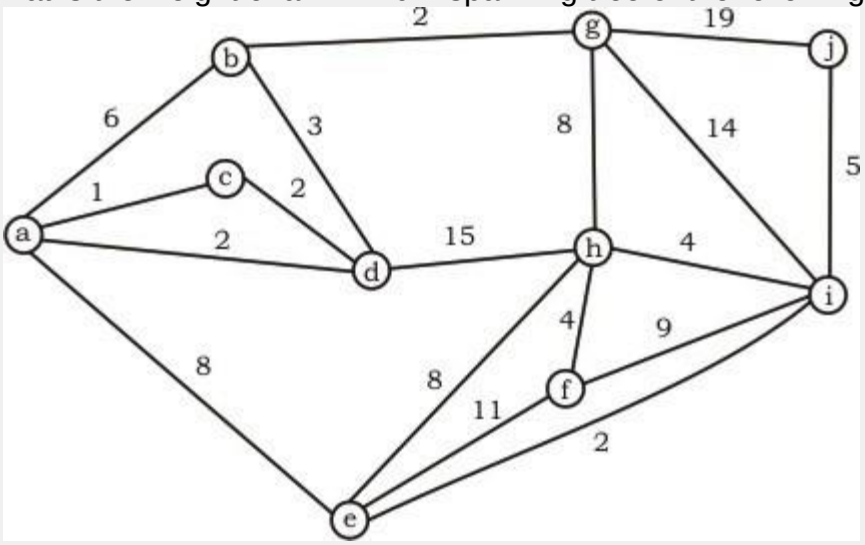
The minimum weight edge on any s-t cut is always part of MST. This is called Cut Property. This is the idea used in Prim's algorithm. The minimum weight cut edge is always a minimum spanning tree edge. **Why B (the weighted shortest path from s to t) is not an answer?** See below example, edge 4 (lightest in highlighted red cut from s to t) is not part of shortest path.



Question 13

WRONG

What is the weight of a minimum spanning tree of the following graph ?



29

31

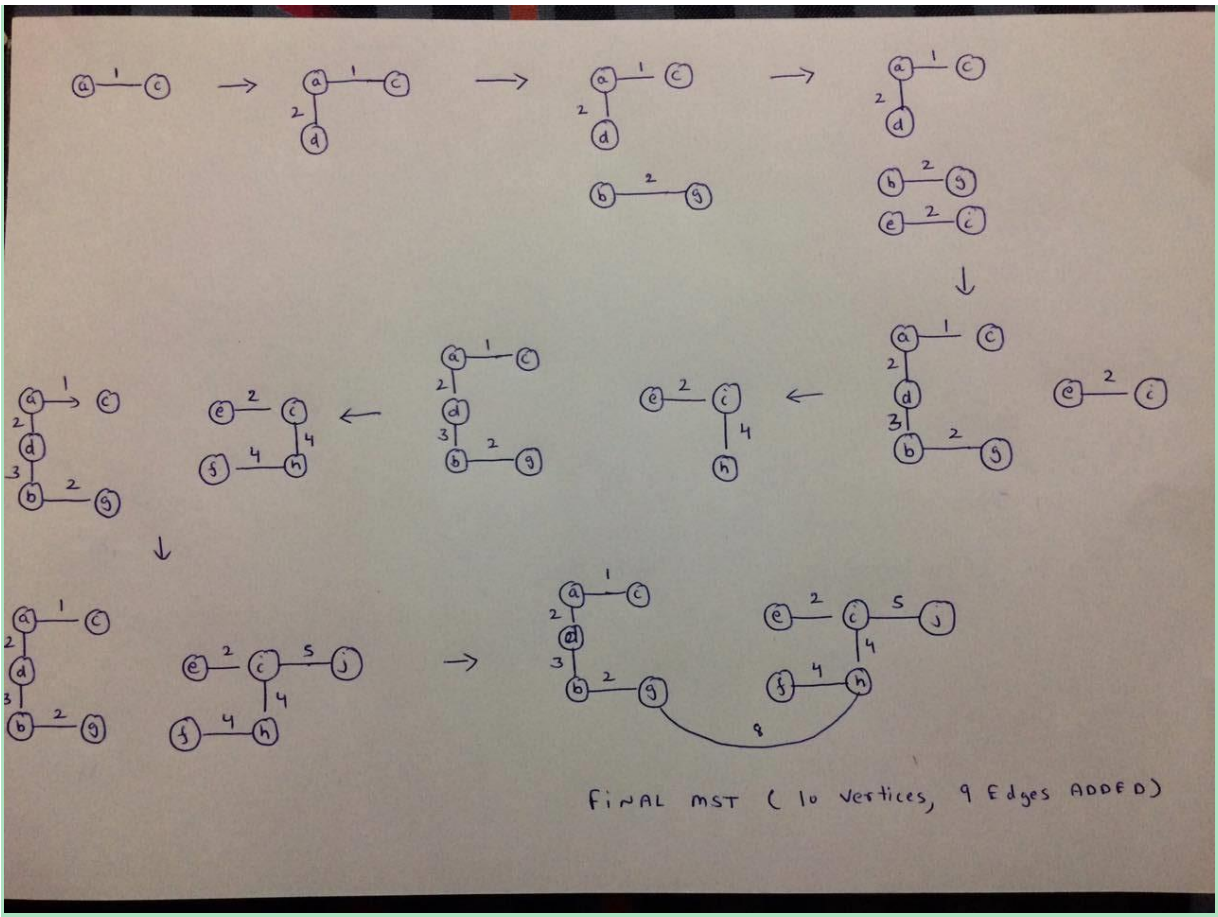
C 38

D 41

Graph Minimum Spanning Tree GATE-CS-2003

Discuss it

Question 13 Explanation:
(a,c), (a,d), (d,b), (b,g), (g,h), (h,f), (h,i), (i,j), (i,e) = 31 Background required - Minimum Spanning Tree (Prims / Kruskal) In these type of questions, always go for kruskal's algorithm to find out the the minimum spanning tree as it is easy and there are less chances of doing silly mistakes. **Algorithm:** Always pick the minimum edge weight and try to add to current forest (Collection of Trees) if no cycle is formed else discard. As soon as u have added n-1 edges to the forest, stop and you have got your minimum spanning tree. See the below image for construction of MST of this question.



Weight of minimum spanning tree = Sum of all the edges in Minimum Spanning tree = 31 This explanation has been provided by **Pranjul Ahuja. Visit the following links to learn more:** http://www.ics.uci.edu/~eppstein/161/960206.html https://en.wikipedia.org/wiki/Minimum_spanning_tree

Question 14

WRONG

Consider an undirected unweighted graph G. Let a breadth-first traversal of G be done starting from a node r. Let d(r,u) and d(r,v) be the lengths of the shortest paths from r to u and v respectively in G. If u is visited before v during the breadth-first traversal, which of the following statements is correct?

d(r, u) < d(r, v)

B d(r,u) > d(r,v)

d(r,u) <= (r,v)

D None of the above

Graph Minimum Spanning Tree **GATE-CS-2001**

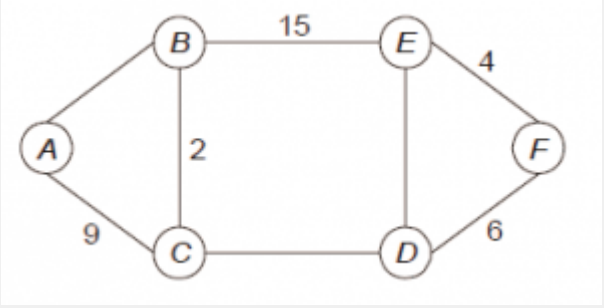
Discuss it

Question 14 Explanation:
See question 4 of http://www.geeksforgeeks.org/data-structures-and-algorithms-set-8/

Question 15

WRONG

The graph shown below 8 edges with distinct integer edge weights. The minimum spanning tree (MST) is of weight 36 and contains the edges: {(A, C), (B, C), (B, E), (E, F), (D, F)}. The edge weights of only those edges which are in the MST are given in the figure shown below. The minimum possible sum of weights of all 8 edges of this graph is _____.



A 66

69

68

D 70

Graph Minimum Spanning Tree GATE-CS-2015 (Set 1)
Discuss it

Question 15 Explanation:
In every cycle, the weight of an edge that is not part of MST must by greater than or equal to weights of other edges which are part of MST. Since all edge weights are distinct, the weight must be greater. So the minimum possible weight of ED is 7, minimum possible weight of CD is 16 and minimum possible weight of AB is 10. Therefore minimum possible sum of weights is 69.

Question 16

WRONG

Let G be connected undirected graph of 100 vertices and 300 edges. The weight of a minimum spanning tree of G is 500. When the weight of each edge of G is increased by five, the weight of a minimum spanning tree becomes _____.

A 1000

995

2000

D 1995

Graph Minimum Spanning Tree GATE-CS-2015 (Set 3)
Discuss it

Question 16 Explanation:
Since there are 100 vertices, there must be 99 edges in Minimum Spanning Tree (MST). When weight of every edge is increased by 5, the increment in weight of MST is = 99 * 5 = 495 So new weight of MST is 500 + 495 which is 995

Question 17

WRONG

Let G be a weighted connected undirected graph with distinct positive edge weights. If every edge weight is increased by the same value, then which of the following statements is/are TRUE?

```
P: Minimum spanning tree of G does not change

Q: Shortest path between any pair of vertices does not change
```

P only

Q only

C Neither P nor Q

D Both P and Q

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 17 Explanation:
The shortest path may change. The reason is, there may be different number of edges in different paths from s to t. For example, let shortest path be of weight 15 and has 5 edges. Let there be another path with 2 edges and total weight 25. The weight of the shortest path is increased by 5*10 and becomes 15 + 50. Weight of the other path is increased by 2*10 and becomes 25 + 20. So the shortest path changes to the other path with weight as 45. The Minimum Spanning Tree doesn't change. Remember the Kruskal's algorithm where we sort the edges first. IF we increase all weights, then order of edges won't change.

Question 18

WRONG

Let G be a complete undirected graph on 4 vertices, having 6 edges with weights being 1, 2, 3, 4, 5, and 6. The maximum possible weight that a minimum weight spanning tree of G can have is. [This Question was originally a Fill-in-the-Blanks question]

6

7

C 8

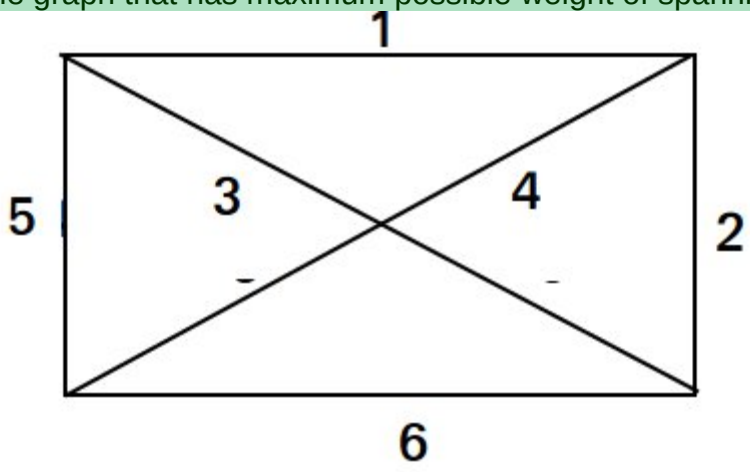
D 9

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)

Discuss it

Question 18 Explanation:

One graph that has maximum possible weight of spanning tree



Question 19

WRONG

G = (V, E) is an undirected simple graph in which each edge has a distinct weight, and e is a particular edge of G. Which of the following statements about the minimum spanning trees (MSTs) of G is/are TRUE

```
I.  If e is the lightest edge of some cycle in G,
    then every MST of G includes e
II. If e is the heaviest edge of some cycle in G,
    then every MST of G excludes e
```

A I only

II only

both I and II

D neither I nor II

Graph Minimum Spanning Tree GATE-CS-2016 (Set 1)
Discuss it

Question 19 Explanation:
I is NOT true. Let G=(V, E) be a rectangular graph where V = {a, b, c, d} and E = {ab, bc, cd, da, ac}. Let the edges have weights: ab = 1, bc = 2, cd = 4, da = 5, ac = 3. Then, clearly, ac is the lightest edge of the cycle cdac, however, the MST abcd with cost 7 (= ab + bc + cd) does not include it. Let the edges have weights: ab = 6, bc - 7, cd = 4, da = 5, ac = 3. Then, again, ac is the lightest edge of the cycle cdac, and, the MST bacd with cost 13 (= ba + ac + cd) includes it. So, the MSTs of G may or may not include the lightest edge. **II is true** Let the heavies edge be e. Suppose the minimum spanning tree which contains e. If we add one more edge to the spanning tree we will create a cycle. Suppose we add edge e' to the spanning tree which generated cycle C. We can reduce the cost of the minimum spanning tree if we choose an edge other than e from C for removal which implies that e must not be in minimum spanning tree and we get a contradiction.
Source: http://www.ece.northwestern.edu/~dda902/336/hw5-sol.pdf

Question 20

CORRECT

What is the largest integer m such that every simple connected graph with n vertices and n edges contains at least m different spanning trees?

A 1

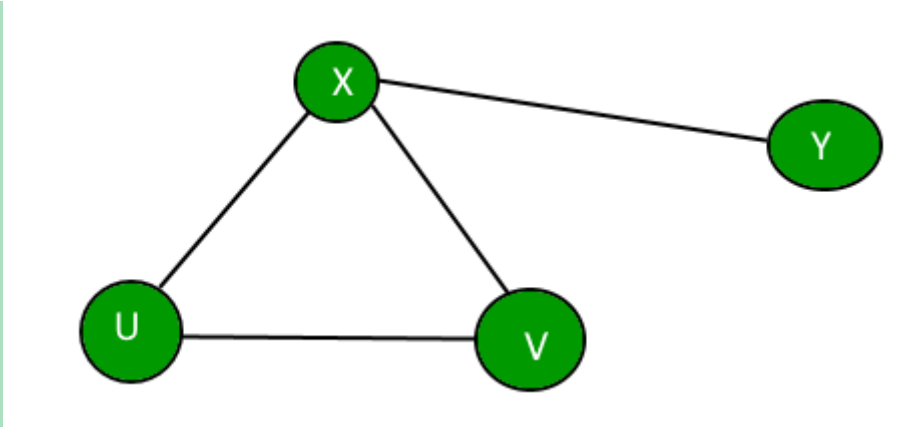
B 2

3

D n

Graph Graph Minimum Spanning Tree Gate IT 2007
Discuss it

Question 20 Explanation:
A graph is connected iff all nodes can be traversed from each node. For a graph with n nodes, there will be n-1 minimum number of edges.
Given that there are n edges, that means a cycle is there in the graph.
The simplex graph with these conditions may be:



Now we can make a different spanning tree by removing one edge from the cycle, one at a time.
Minimum cycle length can be 3, So, there must be atleast 3 spanning trees in any such Graph.

You have completed 20/21 questions .

@geeksforgeeks, Some rights reserved

If Kruskal's algorithm is used for finding a minimum spanning tree of a weighted graph G with n vertices and m edges and edge weights are already given in a sorted list, then, What will be the time complexity to compute the minimum cost spanning tree given that union and find operations take amortized O(1) ?

A O(m logn)

B O(n)

O(m)

O(n logm)

Graph Minimum Spanning Tree **GATE 2017 Mock**
Discuss it

Question 21 Explanation:
O(m) as you are already given edge weights in sorted order. You just have to pick the edges in the increasing order and add it to the current spanning set if its addition does not result in a cycle else throw it away.

Început la	miercuri, 3 iunie 2020, 12:52
Status	Terminat
Completat la	miercuri, 3 iunie 2020, 13:27
Timp luat	34 min 33 secs
Puncte	4,00/6,00
Notează	**6,67** din maxim 10,00 (**67**%) posibil

1 întrebare

Complet

Marcat 0,50 din 1,00

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

Se calculeaza cel mai bun drum, apoi se elimina pe rand nodurile pana se gaseste al doilea cel mai bun drum.

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Comentariu:

2 întrebare

Complet

Marcat 0,50 din 1,00

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *n* noduri care au un ciclu de lungime cel puțin 2.

Se va folosi un algoritm de tip backtracking in care in care voi incerca sa completez matricea de adiacenta. Daca la un pas se adauga arcul (i, j) atunci voi adauga si arcul (j, i) marcand cu 1 pozitiile corespunzatoare in matrice.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunț.

Comentariu:

3 întrebare

Complet

Marcat 0,00 din 1,00

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mari numere. Secvența este prea mare pentru a fi stocată integral in memorie.

Folosesc un max heap pentru a retine cele mai mici k elemente. Cand un element este citit verific daca se poate adauga in heap. Daca da, il adaug si elimin cel mai mare element.

n log k- folosim un minheap

n * k - 0.5 din punctaj

Comentariu:

4 întrebare

Complet

Marcat 1,00 din 1,00

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Problema necesita determinare componentelor biconexe, ele fiind submultimile cerute. Folosesc algoritmul lui Tarjan.

Gasim toate componentele biconexe.

Comentariu:

5 întrebare

Complet

Marcat 1,00 din 1,00

Se dă un graf orientat cu *x* noduri, având capacitați numere din șirul Fibonacci. Se știe că există *A* surse și *B* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la toate destinațiile.

Se adauga un nod nou sursa care se leaga de cele A surse cu capacitati infinit si un nou nod destinatie care se leaga de cele B noduri tot cu capacitate infinit. Se aplica orice algoritm de flux cunoscut, spre exemplu Ford Fulkerson cu implementarea lui Edmonds-Karp.

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comentariu:

6 întrebare

Complet

Marcat 1,00 din 1,00

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

Calculez distanta minima in grafurile X si Y de la nodurile de start/finish pana la nodul de legatura. Deoarece costurile sunt egale putem folosi 2 parcurgeri BFS, cate una pentru pentru fiecare graf. Costul total este suma celor 2 drumuri + costul de traversare (adica Z).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Comentariu:

◄ Barem Subiectul I

Salt la...

Examen Subiectul 1 - Late submit ►

Început la	miercuri, 3 iunie 2020, 12:52
Status	Terminat
Completat la	miercuri, 3 iunie 2020, 13:27
Timp luat	34 min 33 secs
Puncte	4,00/6,00
Notează	**6,67** din maxim 10,00 (**67**%) posibil

1 întrebare

Complet

Marcat 0,50 din 1,00

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

Se calculeaza cel mai bun drum, apoi se elimina pe rand nodurile pana se gaseste al doilea cel mai bun drum.

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Comentariu:

2 întrebare

Complet

Marcat 0,50 din 1,00

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *n* noduri care au un ciclu de lungime cel puțin 2.

Se va folosi un algoritm de tip backtracking in care in care voi incerca sa completez matricea de adiacenta. Daca la un pas se adauga arcul (i, j) atunci voi adauga si arcul (j, i) marcand cu 1 pozitiile corespunzatoare in matrice.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunț.

Comentariu:

3 întrebare

Complet

Marcat 0,00 din 1,00

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mari numere. Secvența este prea mare pentru a fi stocată integral in memorie.

Folosesc un max heap pentru a retine cele mai mici k elemente. Cand un element este citit verific daca se poate adauga in heap. Daca da, il adaug si elimin cel mai mare element.

n log k- folosim un minheap

n * k - 0.5 din punctaj

Comentariu:

4 întrebare

Complet

Marcat 1,00 din 1,00

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Problema necesita determinare componentelor biconexe, ele fiind submultimile cerute. Folosesc algoritmul lui Tarjan.

Gasim toate componentele biconexe.

Comentariu:

5 întrebare

Complet

Marcat 1,00 din 1,00

Se dă un graf orientat cu *x* noduri, având capacitați numere din șirul Fibonacci. Se știe că există *A* surse și *B* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la toate destinațiile.

Se adauga un nod nou sursa care se leaga de cele A surse cu capacitati infinit si un nou nod destinatie care se leaga de cele B noduri tot cu capacitate infinit. Se aplica orice algoritm de flux cunoscut, spre exemplu Ford Fulkerson cu implementarea lui Edmonds-Karp.

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comentariu:

6 întrebare

Complet

Marcat 1,00 din 1,00

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

Calculez distanta minima in grafurile X si Y de la nodurile de start/finish pana la nodul de legatura. Deoarece costurile sunt egale putem folosi 2 parcurgeri BFS, cate una pentru pentru fiecare graf. Costul total este suma celor 2 drumuri + costul de traversare (adica Z).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Comentariu:

◄ Barem Subiectul I

Salt la...

Examen Subiectul 1 - Late submit ►

SUBIECTUL III

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *N* noduri pentru care toate gradele de intrare sunt pare.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunt.

Se dă o matrice *n* x *n*. În *k* celule se află ponei (k mult mai mic decât n). Pentru fiecare celulă din matrice, se dorește să se știe care este distanța minimă către cel mai apropiat ponei. Descrieți un algoritm eficient pentru această problemă.

BFS pe matrice în care se pun în coadă toate celulele cu ponei speciali. Se face BFS pornind cu această coadă și se calculează distanțele. Complexitate O(n^2) unde n e dimensiunea matricei. Se punctează cu 0.5p o soluție mai ineficientă.

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze o singură fortăreață, a.î. să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea fortăreței.

1p - Găsim toate punctele de articulatie și componentele biconexe din jurul lor. Gigel va bombarda punctul de articulație care are în jurul său cele mai multe componente biconexe.

0.5p - Întoarce orice punct de articulație

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Se dă un graf neorientat conex reprezentând rețeaua de socializare Facebook (muchie x-y înseamnă *x* este prieten cu *y*). Gigel a făcut o postare pe Wall, care este văzută de prieteni. Se întreabă dacă postarea poate, ca prin mai multe share-uri, să fie văzută de Botzgorel (care nu este prieten cu Gigel). Descrieți un algoritm eficient.

BFS(Gigel) / DFS(Gigel) și dacă vizităm nodul Botzgorel, atunci răspunsul este DA. Altfel NU.

1) DIVIDE ET IMPERA

PROBLEME PROPUSE

1. Înmulțirea matricelor folosind algoritmul lui Strassen

Să se scrie un algoritm care calculează produsul a două matrice (C = A * B), folosind metoda divide-et-impera. Se va folosi următoarea abordare: o matrice n x n se poate scrie ca o matrice de dimensiune 2x2 ce conține ca elemente submatrice de dimensiuni n/2 x n/2. Pentru a înmulți două matrice 2x2 se folosesc doar 7 înmulțiri de submatrice, după cum este definit în continuare.

$$\begin{pmatrix} C11 & C12 \\ C21 & C22 \end{pmatrix} = \begin{pmatrix} A11 & A12 \\ A21 & A22 \end{pmatrix} * \begin{pmatrix} B11 & B12 \\ B21 & B22 \end{pmatrix}$$

Definim următoarele rezultate intermediare obținute prin înmulțirea submatricelor:

```
M1 = (A11 + A22)*(B11 + B22)
M2 = (A21 + A22)*B11
M3 = A11*(B12 - B22)
M4 = A22*(B21 - B11)

M5 = (A11 + A12)*B22
M6 = (A21 - A11)*(B11 + B12)
M7 = (A12 - A22)*(B21 + B22)
```

Folosind rezultatele intermediare putem obține submatricele care alcătuiesc rezultatul:

```
C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6
```

2. Elementul median dintr-un vector

Fie o mulțime de numere reale cu n elemente reprezentată cu un vector v nesortat. Scrieți un algoritm de complexitate O(n) care să determine elementul median al mulțimii (echivalentul elementului de pe poziția n/2 din vectorul sortat, sau al k-lea (k =n/2) cel mai mic element din vector).

3. Subsecvența de sumă maximă

Fie un șir S = (s 1 , s 2 , ..., s N) de lungime n cu numere reale. O subsecvență a șirului este de forma: (s i , s i+1 , ..., s j) cu 1 <= i <= j <= n, iar suma subsecvenței este definită ca S ij = s i + s i+1 + ... + s j . Se cere să se determine subsecvența de sumă maximă folosind un algoritm divide-et-impera.

Exemplu: Pentru n = 7 și secvența: (5 -6 3 4 -2 3 -3), subsecvența de sumă maximă este:

(3 4 -2 3) a cărei sumă este 8.
Obs: Varianta cerută folosind o abordare de tip Divide et Impera are o complexitate O(n * log n), mai mare față de alternativa Greedy care este O(n).

INDICATII DE REZOLVARE

1. Înmulțirea matricelor folosind algoritmul lui Strassen

Inițial se vor adăuga linii și coloane cu elemente nule, până când cele două matrice vor fi pătratice, de dimensiune putere a lui 2, n = 2 k . Se partiționează matricele în blocuri de dimensiuni egale, ca în desenul de mai sus. Pentru a calcula cele 7 matrice intermediare M 1-7 , se apelează recursiv procedura de înmulțire (fiecare matrice M este produsul a altor două matrice de dimensiune mai mică n/2 = 2 k /2 = 2 k-1).
Complexitate: relația de recurență va fi T(n) = 7 * T(n/2) + n 2
Aplicând teorema Master, vom obține complexitatea algoritmului lui Strassen:
T(N) = O(n log 7) ≈ O(n 2 . 8)

2. Elementul median dintr-un vector

Ne propunem să rezolvăm generalizarea problemei – identificarea celui de-al k-lea cel mai mic element din vectorul inițial, urmând ca apelul final să folosească k = n/2). Această problemă se mai numește și problema selecției celui de-al k-lea element sau statistici de ordine.
Astfel, se pornește de la funcția de partiționare a algoritmului Quicksort. Fie q indexul returnat de această funcție. Dacă k = q atunci înseamnă că s-a găsit elementul căutat (v[k]). Dacă q > k, se va apela recursiv funcția de selecție pentru v[p...q-1], în caz contrar, se apelează pentru v[q+1...r], unde p și r reprezintă capetele șirului inițial.
Complexitate: O(n), dacă se folosește partiționare aleatoare.

3. Subsecvența de sumă maximă

Se divide șirul inițial în două subșiruri de lungime egală. Se calculează subsecvența de sumă maximă pentru fiecare din cele două subșiruri. Pentru a combina soluțiile rezultate, trebuie sa aflăm și subsecvența de sumă maximă ce are capetele în fiecare din cele două subșiruri. Astfel, se va alipi sufixul de sumă maximă din primul subșir cu prefixul de sumă maximă din cel de-al doilea subșir. Această operație de combinare a subsoluțiilor are complexitatea O(n) în cel mai defavorabil caz.
Complexitate: O(n * log n)

2) GREEDY

PROBLEME PROPUSE

1. Problema rucsacului (varianta continuă)

Un camion poate transporta T tone de material. Există n tipuri de materiale disponibile, fiecare caracterizat de greutatea G[i] disponibilă și de valoarea V[i] adusă de transportarea sa ($1 \leq i \leq n$). Să se decidă ce cantități din fiecare material vor fi transportate pentru a maximiza valoarea totală a produselor transportate de către camion.

2. Planificarea temelor de către studenți

Pe parcursul unui semestru, un student are de rezolvat n teme. Se cunoaște enunțul tuturor celor n teme de la începutul semestrului. Timpul de rezolvare pentru oricare dintre teme este de o săptămână și nu se poate lucra la mai multe teme în același timp. Pentru fiecare temă, se cunoaște și un termen limită d[i] (exprimat în săptămâni) și un punctaj w[i]. Nicio fracțiune din punctaj nu se mai poate obține după expirarea termenului limită al unei teme. Să se definească o planificare de realizare a temelor, în așa fel încât punctajul obținut la final să fie maxim.

3. Numere de tip palindrom

Se dorește construirea a două numere de tip palindrom (prin inversare – citirea de la dreapta la stânga – numărul rămâne la fel), primul de lungime $L > 0$, al doilea de lungime $l > 0$ astfel încât suma lor să fie minimă. Cele două numere de tip palindrom nu pot să înceapă cu cifra 0. Pentru construirea lor, se dau numărul L și numărul de apariții al fiecărei cifre în cele două numere (adică c[i] = nr. de apariții al cifrei i în primul numărul + nr. de apariții al cifrei i în al doilea număr, $0 \leq i \leq 9$).

INDICATII REZOLVARE

1. Problema rucsacului (varianta continuă)

Se calculează profitul obținut per unitatea de material tranportată V[i]/G[i] ($1 \leq i \leq n$) și se încarcă material în ordinea acestor valori. În plus, se poate observa că se obține o îmbunătățire a complexității – de la $O(n^2)$ la $O(n * \log n)$ – dacă valorile sunt sortate descrescător înainte de a face alegerile. Folosirea unei preprocesări simple (de exemplu, o sortare) în etapa inițială pentru a reduce complexitatea algoritmului este o proprietate a mai multor algoritmi construiți folosind tehnica Greedy.
Complexitate: $O(n * \log n) + O(n) = O(n * \log n)$

2. Planificarea temelor de către studenți

Se folosește un heap cu minim în vârf pentru a prioritiza temele în funcție de punctaj și de proximitatea termenului limită. Alternativ, pentru valori întregi și mici, se pot sorta temele după punctaje, în ordine descrescătoare și, în aceasta ordine, li se caută un slot de rezolvare, pornind de la termenul limită și căutând spre stânga.
Dacă în loc de maximizarea punctajului total obținut, se dorește maximizarea numărului de teme ce pot fi rezolvate, se sortează lista de intervale de rezolvare după termenul limită și se alege întotdeauna tema care se termină cel mai curând, în așa fel încât sa nu se suprapună cu nicio altă tema aleasă în prealabil.
Complexitate: $O(n * \log n)$

3. Numere de tip palindrom

Se observă că cifrele mai mici trebuie să ocupe poziții cât mai semnificative, ceea ce asigura că suma celor două numere să fie minimă. Pentru a realiza acest lucru se vor completa în paralel

cele două numere cu cifrele parcurse în ordine crescătoare stabilind în care din cele două numere se vor poziționa. De asemenea, trebuie avut în vedere ca niciunul din cele două numere să nu înceapă cu cifra 0.
Complexitate: O(L + l)

3) PROGRAMARE DINAMICA

PROBLEME PROPUSE

1. Problema rucsacului (varianta discretă)

Un camion poate transporta T tone de mobilă. Există n piese de mobilă, fiecare caracterizată de greutatea G[i] și de valoarea V[i] adusă de transportarea sa. Să se decidă ce piese de mobilă vor fi transportate pentru a aduce o valoare maximă (soluție optimă), știind ca nu avem voie să luăm bucăți dintr-o piesă de mobilă.

2. Aranjarea cărților în bibliotecă

Se dau n dulapuri aflate într-o bibliotecă, precum și numărul cărților din fiecare dulap. Se dorește mutarea dulapurilor (cu toate cărțile aferente) într-o nouă bibliotecă astfel încât primul dulap este mutat intotdeauna, iar celelalte dulapuri se mută doar dacă nu sunt așezate unul lângă celalalt în biblioteca inițială. De asemenea, se dorește aflarea soluției care maximizează numărul cărților din noua bibliotecă.

3. Parantezarea unei expresii booleene

Se dă o expresie booleană exprimată prin stringurile “true”, “false”, “and”, “or”, “xor”. Numărați modurile în care se pot așeza parantezele astfel încât rezultatul expresiei parantezate să fie “true”.
De exemplu, pentru expresia:
true and false xor true
există două modalitati de parantezare astfel încât rezultatul să fie “true”:
((true and false) xor true)
(true and (false xor true))

4. Înmulțirea optimă a unui lanț de matrici

Întrucât înmulțirea matricilor este asociativă, determinați modul optim de realizare al înmulțirilor unui șir A 1 ... A n de n matrici astfel încât numărul total de înmulțiri scalare să fie minimă.
Exemplu: Fie A 1 o matrice 10 × 30; A 2 - 30 × 5; A 3 - 5 × 60:
(A 1 * A 2) * A 3 = (10 × 30 × 5) + (10 × 5 × 60) = 1500 + 3000 = 4500 operații de înmulțire scalari
A 1 * (A 2 * A 3) = (30 × 5× 60) + (10 × 30 × 60) = 9000 + 18000 = 27000 operații de înmulțire scalari

INDICATII REZOLVARE

1. Problema rucsacului (varianta discretă)

Se folosește o matrice care se completează pe linii, iar fiecare linie modelează o lume în care camionul poate transporta 1 tonă, 2 tone, ... n tone. Putem modela o subproblemă ca fiind best[i][j] = valoarea maximă pe care o poate transporta un camion de tonaj maxim i folosind doar primele j piese de mobilă.
Relația de recurență este:
best[i][j] = max{best[i][j-1], best[i-G[j]] + V[j]}, dacă i - G[j] ≥ 0
best[i][j] = best[i][j-1] , dacă i - G[j] < 0
best[0][j] = 0; best[i][0] = 0
Soluția problemei va fi oferită de către best[T][n]. O observație interesantă este că această soluție poate fi aplicată doar dacă greutățile obiectelor sunt numere naturale, întrucât acestea intră în

definirea relației de recurență (vezi i - G[j]). În cazul general, problema rucsacului este NP-completă și nu acceptă o soluție polinomială.
Complexitate: O(n * T)

2. Aranjarea cărților în bibliotecă

Fie c_max[i] = numărul maxim de cărți ce pot fi transferate utilizând cărțile existente pe dulapurile de la 1 la i-2, astfel încât să fie selectat și dulapul i. Formula de recurență obținută este c_max(i) = carti(i) + maxim{c_max(k) unde k = 1, i-2}, ceea ce duce la o complexitate O(i) pentru pas și O(n 2) în total.
Totuși, relația de recurență de mai sus se poate optimiza observând că, la fiecare pas i, trebuie ales maximul dintre c_max(i-2) și c_max(i-3). Răspunsul este dat de maximul din vectorul c_max[0...n].
Complexitate: O(n)

3. Parantezarea unei expresii booleene

Se folosește o matrice pentru a stoca soluții parțiale pentru subprobleme, cu semnificația A[i][j] este numărul de parantezări asupra sub-expresiei formată din termenii pornind de la i și terminand cu j, astfel încât rezultatul sub-expresiei să fie true. Pozițiile se pot referi doar la elementele operanzi (de pe pozițiile pare: 0, 2, 4 etc.), nu și la operatori. Pentru simplitate, se poate utiliza o matrice separată care sa stocheze numărul de parantezări ale sub-expresiilor în asa fel încât rezultatul sa fie false. Cele două matrice vor fi simetrice; prin urmare, acestea pot fi completate pe subdiagonale, doar deasupra diagonalei principale, caz în care rezultatul final se va regăsi în colțul din dreapta-sus al matricei true.
Complexitate: O(n 2)

4. Înmulțirea optimă a unui lanț de matrici

Se folosește o matrice care se completează iterativ, peste diagonala principală, în care fiecare celulă modelează înmulțirea optimă a matricilor între indecșii i și j, precum și o structură auxiliară în care se memorează dimensiunea matricii rezultate din înmulțirea tuturor matricilor A i ... A j .
Ulterior, putem modela o subproblemă ca fiind best[i][j] = numărul minim de operații necesare pentru a înmulții matricilor A i ... A j . Relația de recurență este:
best[i][j] = min{best[i][k] + best[k+1][j] + costul înmulțirii matricii A i...k cu A k+1...j }
best[i][i] = 0;
Se poate observa ușor că aplicând relația de recurență pentru subproblema de dimensiune 1 (best[i][i+1]) obținem:
best[i][i+1] = costul înmulțirii matricii A i cu A i+1 = nr_linii(A i) * nr_coloane(A i) * nr_coloane(A i+1)
Soluția problemei inițiale va fi oferită de către best[0][n].
Complexitate: O(n 3)

4) BACKTRACKING SI OPTIMIZARI

PROBLEME PROPUSE

Sudoku

Jocul clasic rezolvat prin BKT la nivelul căruia aplicăm diverse optimizări este Sudoku. Astfel, avem o matrice 9 X 9 subdivizată în 9 sub-matrice identice de dimensiuni 3 X 3, denumite regiuni. Regula jocului este simplă: fiecare rând, coloană sau regiune nu trebuie să conţină decât o dată cifrele de la 1 la 9. Formulat altfel, fiecare ansamblu trebuie să conţină cifrele de la 1 la 9 o singură dată. Se cere să se modeleze această problemă folosind următoarele abordări:

- Backtracking simplu;
- BKT și o metodă prospectivă (preferabil predicție parțială sau completă);
- BKT și o euristică la alegere.

Pentru fiecare dintre soluțiile implementate se va analiza impactul la nivelul timpului total de execuție (care pot fi influențați și de alte procese din sistemul de operare, respectiv alocări și procese interne ale JVM, etc.), precum și al numărului de intrări în recursivitate.

INDICATII REZOLVARE

Varianta simplă de backtracking presupune introducerea tuturor valorilor posibile {1...9}, pe rând, în fiecare celulă liberă și verificarea consistenței atribuirii la nivel de linie, coloană și regiune. În cazul încălcării oricărei reguli, se încearcă o nouă valoare sau se revine în recursivitate, dacă nu mai există nicio variantă posibilă pentru variabila curentă. Prima optimizare presupune introducerea unei metode prospective care va filtra, la fiecare iterație, numărul de valori potențiale care pot fi introduse în anumite celule. Astfel, vor fi identificate și eliminate în prealabil potențialele inconsistențe la nivel de linie, coloană sau regiune (valori care nu pot fi utilizate în iterațiile ulterioare), iar dacă domeniul de valori al unei celule libere devine vid, este necesară revenirea din recursivitate. Suplimentar, se poate introduce și euristica de a reordona variabilele pornind de la cele mai restricționate (cu cele mai puține valori pontențial a fi introduse într-o anumită celulă), reducând astfel și mai mult numărul de intrări inutile în recursivitate.

5) MINIMAX

PROBLEME PROPUSE

X și O

Aplicația presupune aplicarea unor algoritmi de tip Minimax în scopul rezolvării jocului de X și O (tic-tac-toe). Astfel, se dorește implementarea algoritmul Mini/Nega-max astfel încât calculatorul să poată juca împotriva unui jucător uman. Adițional, trebuie extins algoritmul anterior prin utilizarea optimizării de tip tăietură alpha-beta (alpha-beta pruning).
Notă: acest joc, desi rezolvabil și prin metode mai puțin costisitoare din punct de vedere al efortului de calcul, însă este ales în scop didactic. Mai mult, varianta sa generalizată este mult mai complicată și exercițiul poate fi extins si pentru aceasta.

INDICATII REZOLVARE

Întrucât spațiul stărilor este limitat și numărul de mutări posibile scade pentru fiecare nivel deoarece matricea de stări este completată succesiv, implementarea poate fi utiliza adâncimea maximă (9), atât în cazul mini/nega-max normal, cât și în cazul aplicării tăierii alpha-beta. Funcția de evaluare va lua 3 valori posibile: +1 dacă jocul este câștigat de jucătorul curent, -1 dacă jocul este câștigat de oponent și 0 pentru toate stările în care nu se poate determina un câștigător. Pentru simplitate, se poate conveni din start dacă jucătorul uman este „X” sau „O”, și dacă acesta efectuează prima mutare.

6) PARCURGERE GRAFURI. SORTARE TOPOLOGICA.

PROBLEME PROPUSE

1. Trebuie să evadez! Nu mai rezist!

Se dă un tablou bidimensional reprezentând suprafața unei camere. Camera poate să conțină obstacole, reprezentate în matrice printr-o celulă de valoare 1, prin care nu se poate circula. Celulele cu valoarea 0 reprezintă elementele libere ale camerei prin care se poate trece. Dându-se un punct interior de start, să se decidă care este cel mai scurt drum până la o iesire. Ieșirile pot să fie doar pe marginea tabloului, și sunt simbolizate prin cifra 2. Să se afiseze și drumul până la una din cele mai apropiate ieșiri.

2. Cum ar trebui studiate optim materiile?

Se dă setul de mai jos de dependențe între materii (materia din stânga trebuie studiată într-un semestru anterior celei din dreapta). Să se precizeze o ordine de studiere a materiilor din curriculum.
Analiza Algoritmilor → Proiectarea Algoritmilor
Programarea Calculatoarelor → Proiectarea Algoritmilor
Structuri de Date → Proiectarea Algoritmilor
Proiectarea Algoritmilor → Algoritmi Paraleli și Distribuiti
Protocoale de Comunicatie → Algoritmi Paraleli și Distribuiti
Structuri de Date → Analiza Algoritmilor
Programarea Calculatoarelor → Protocoale de Comunicatie
Protocoale de Comunicatie → Rețele Locale
Algoritmi Paraleli și Distribuiți → Algoritmi și Prelucrări Paralele
Arhitectura Sistemelor de Calcul → Algoritmi și Prelucrări Paralele
Matematica 1 → Matematica 2
Matematica 2 → Matematici speciale
Matematica 1 → Fizică
Fizică → Electrotehnică
Fizică → Electronică Analogică
Electronică Analogică → Electronică DigitalăParcurgerea Grafurilor.

INDICATII REZOLVARE

1. Trebuie să evadez! Nu mai rezist!

Întrucât se dorește îndeplinirea criteriului de optimalitate se va aplica o parcurgere în lățime din poziția inițială, considerând totodată constrângerile impuse de celulele adiacente indisponibile (marcate cu 1). Labirintul poate fi modelat ca un graf, daca considerăm că pentru orice celulă (i, j), se poate construi ușor funcția succesori(i, j) care să întoarcă celulele adiacente în care se poate trece.

2. Cum ar trebui studiate optim materiile?

Se aplică o sortare topologică urmată de o alocare greedy pe semestre. Preferabil, se poate opta pentru includerea materiilor cu cele mai multe dependențe ulterioare în primele semestre. Pentru această evaluare se vor realiza parcurgeri suplimentare în adâncime pornind din fiecare materie și se vor prioritiza materiile în ordinea descrescătoare a adâncimii maxime.
Opțional, se poate realiza o combinație de Kahn și DFS pentru identificarea alocării optime.

7) APLICATII DFS

PROBLEME PROPUSE

Into the Matrix

Tocmai s-a lansat o noua rețea socială, dedicată exclusiv gamerilor. Întrucât toată lumea dorea să discute cu prietenii lor din comunitate într-un mod cât mai privat, comunicarea este unidirecțională prin canale dedicate. Astfel, atunci când un jucator intră prima dată pe un server, indică toate persoanele pe care le consideră prieteni și stabilește relațiile outbound. Ulterior, terțe persoane pot crea tuneluri permanente în sens invers pentru a permite comunicația bidirecțională între 2 noduri.

1) Identificați toate clanurile de pe un server. (Mesajele între oricare 2 membri dintr-un clan pot fi rutate prin canalele deschise)

2) Descoperiți ce relații există între clanuri. Care ar fi cel mai numeros clan dacă s-ar adauga o legatură artificială între oricare doi jucatori? Ulterior, pentru a spori ușurința utilizării, canalele de comunicație au devenit bi-direcționale și, pentru a evalua relațiile dintre jucători, dorim să identificăm următoarele elemente cheie din fiecare clan:

3) Care sunt jucătorii critici a căror eliminare ar duce la distrugerea comunicației în cadrul clanului?

4) Care canale între 2 jucători sunt critice la nivelul clanului?

INDICATII REZOLVARE

1) Graful este orientat și se aplica algoritmul lui Kosaraju sau Tarjan pentru identificarea componentelor tare conexe.

2) Varianta cea mai elegantă presupune contracția muchiilor (toate nodurile dintr-un CTC și muchiile aferente sunt contractate într-un singur nod), urmând ca muchia adăugată să creeze un ciclu în graful anterior și care să maximizeze numărul total de membri din CTC-uri incluse în ciclul nou creat. Soluția finală presupune adăugarea unei muchii înapoi între oricare dintre membrii celor două sub-comunități anterioare unificate prin muchia dintre CTC-uri.

3, 4). Jucătorii critici reprezintă punctele de articulație din graful neorientat, iar canalele critice sunt identificate sub formă de punți.

8) DRUMURI MINIME

PROBLEME PROPUSE

Zburând prin Europa

1. Andrei primește o hartă a Europei pe care sunt marcate toate zborurile posibile de la o localitate la alta, respectiv costurile acestora. Ca primă evaluare, Andrei își dorește să afle dacă două localități sunt conectate, indiferent de numărul de escale sau de linia aeriană.
2. Ulterior, Andrei își dorește să afle toate localitățile aflate pe drumul optim de la un oras la altul.
3. Întrucât Andrei se simte depășit de numărul foarte mare de localități de pe hartă, vă rugăm să identificați costul unei rute optime de la o singură localitate (cea în care se află Andrei) la oricare alta localitate.
4. Discutând cu un prieten, Andrei descoperă că pe anumite rute companiile oferă reduceri semnificative, clienții fideli primind chiar recompense financiare dacă aleg anumite rute. Determinați în aceste circumstanțe costul optim de la localitatea curentă la oricare alta localitate. În final, lui Andrei îi vine o idee ingenioasă și vă rugăm îl ajutați să descopere dacă exista posibilitatea de a specula recompensele liniilor aeriere pentru a circula la infinit între două localități, fără a plăti. În cazul identificării uneiastfel de rute, vă rugăm să o afișați.

INDICATII REZOLVARE

1) Pentru implementarea închiderii tranzitive se aplică algoritmul Floyd-Warshall puțin modificat: d[i, j] = (d[i, j] OR (d[i, k] AND d[k, j])).
2) Se aplică algoritmul Floyd-Warshall cu păstrarea părinților.
3) Se aplică algoritmul Dijkstra întrucât costurile muchiilor sunt pozitive.
4) În locul algoritmului Dijkstra trebuie aplicat algoritmul Bellman-Ford datorită costurilor negative, algoritm care permite totodată identificarea ciclurilor de cost negativ.

9) ARBORI MINIMI DE ACOPERIRE

PROBLEME PROPUSE

Cablare optimă în rețele de date

Prietenul nostru Andrei a fost asignat drept noul șef al departamentul de rețelistică al companiei Nuke Cola. Sediul companiei are arondate N sucursale, iar Andrei trebuie să conecteze asigure conectivitate între toate locațiile folosind o lungime minimă de fibră optică, lucru care duce implicit la reducerea costurilor totale. În cazul în care directorul general nu este de acord cu planul său, Andrei vrea să aibă un plan de backup, acesta reprezentând cea de-a doua cea mai bună soluție din punctul de vedere al lungimii totale de fibră optică utilizată. Vă rugăm să îl ajutați pe Andrei să găsească configurația optimă având drept intrare distanțele între locațiile care pot fi conectate direct una cu cealaltă.

INDICATII REZOLVARE

Cablare optimă în rețele de date

Inițial trebuie identificat arborele minim de acolerire (AMA) fosind fie algoritmul Prim, fie Kruskal. Ulterior, trebuie identificat cel de-al doilea arbore minim de acoperire (2AMA) utilizând următoarea construcție optimizată. Pentru fiecare muchie (u, v) $\notin$ AMA, se determină ciclul care

s-ar obține prin adăugarea muchiei (u,v) la AMA, urmând ca un candidat pentru 2AMA să se obțină prin eliminarea muchiei de cost maxim, diferită de (u, v), din acest ciclu nou creat. Astfel, în continuare este asigurată conectivitatea tuturor nodurilor (se elimină o muchie de cost maxim dintr-un ciclu), iar 2AMA este costul minim al tuturor arborilor construiți anterior pornind de la fiecare muchie care nu este în AMA-ul inițial. Pentru a optimiza construcția ciclurilor conținând muchia (u, v), se poate considera muchia de cost maxim din drumul de la u la v în AMA, cost care poate fi determinat cu ușurință folosind o parcurgere în lățime (BFS) în AMA.

10) FLUX MAXIM

PROBLEME PROPUSE

1. Networking

Pornind de la un graf orientat conex și două noduri u și v se cere să se determine următorele elemente:
a) Tăietura minimală a grafului afișată sub forma unei mulțimi de cardinal minim de muchii ce trebuie eliminate pentru a deconecta u și v.
b) Pentru topologia dată, afișați numărul maxim de drumuri disjuncte între u și v, dar și drumurile în sine. Două drumuri sunt considerate disjuncte dacă nu au nicio muchie în comun.

2. Generarea de grafuri

Se dă o listă de N noduri, pentru fiecare cunoscându-se gradul de intrare si de ieșire al acestora (In-degree și Out-degree). Implementați un algoritm care să construiască un graf orientat G cu N noduri, care să satisfacă constrângerile de grade (de intrare și de ieșire) date. Se poate construi maxim o muchie între oricare două noduri.

INDICATII REZOLVARE

1. Networking

a) Se aplică algoritmul Ford-Fulkerson, urmat de încă o parcurgere (BFS sau DFS) din nodul u pentru a detecta tăietura minimală. Această parcurgere va separa nodurile în 2 sub-mulțimi: o mulțime S1 cu noduri accesibile de u, respectiv mulțimea S2=V\S1 cu noduri inaccesibile care sunt blocate de muchiile saturate. Astfel, muchiile care trebuie eliminate sunt toate muchiile din graful G care au capătul stânga în S1 și capătul dreapta în S2.
b) Se aplică algoritmul Ford-Fulkerson adăugând fiecărei muchii o capacitate egală cu 1 și se afișează toate drumurile generate.

2. Generarea de grafuri

Se construiește un graf auxiliar pe care se aplică algoritmul Ford-Fulkerson. Acest graf are o sursă și o drenă virtuală și 2 straturi de N noduri (în total 2*N+2 noduri), iar arcele se construiesc astfel:
 Capacitățile muchiilor de la sursa virtuală la fiecare nod din primul strat de noduri este egal cu In-Degree; Flux Maxim 101
 Între cele 2 straturi de noduri se realizează un mesh/graf complet de muchii între nodurile primului strat cu capătul dreapta în al doilea strat (excluzând arcele de la nodul i din primul strat la nodul i din al doilea strat), toate de capacitate 1;
 Între fiecare nod din al doilea strat și drena virtuală se trasează arce de capacitate egal cu Out-Degree. Raportat la modelarea anterioare, se poate opta și pentru inversarea In-Degree cu Out-Degree, urmând ca muchiile saturat din mijloc să reflecte graful solicitat.

11) Algoritmi euristici. Algoritmul A*

PROBLEME PROPUSE

Problema comis voiajorului (Ciclu hamiltonian de cost minim)

Se dau n oraşe și rutele rutiere de interconectare dintre acestea. Se cere să se găsească un tur complet de lungime minimă (orașul de plecare și cel de întorcere sunt unul și același, iar toate celelalte orașe sunt străbătute o singură dată). Pentru fiecare dintre variantele propuse de rezolvare se va evalua numărul de stări explorate și se va interpreta evoluția acestora raportat la soluția și euristica propusă.

- Să se rezolve problema folosind o abordare de tip Best-First.
- Să se rezolve problema folosind o euristică simplă A*, admisibilă, definită în raport cu o singură localitate neexplorată în cadrul stării curente.
- Rafinați soluția de la punctul b), propunând o euristică mai informată prin evaluarea întregului orizont neexplorat (variabilele = localitățile care nu au fost deja parcurse în starea curentă).

INDICATII REZOLVARE

În prima etapă se consideră minimul distanței deja acoperite de localitățile selectate. Ulterior, pe lângă funcția g(x) definită anterior, se va considera h(x) = distanța către cel mai apropiat oraș neparcurs. Euristica cea mai bună presupune utilizarea, în locul funcției h(x) definită anterior, a costul arborelui minim de acoperire (AMA) aferent orașelor neparcurse.

12) ALGORITMI ALEATORI

PROBLEME PROPUSE

k-Means

Fie n puncte într-un spațiu bi-dimensional. Se dorește o grupare a acestora în k clustere – un cluster este un grup de puncte situate într-o vecinătate spațială care să maximizeze distanța internă (coeziune intra-cluster) și să asigure o cuplare slabă inter-clustere.

De exemplu, pentru setul de intrare:

X	2	3	2	3	3	3	4	5	8	8	9	10	10	10
Y	9	10	5	4	3	2	1	1	2	3	4	5	6	7

Se poate observa o grupare naturală în 3 clustere:

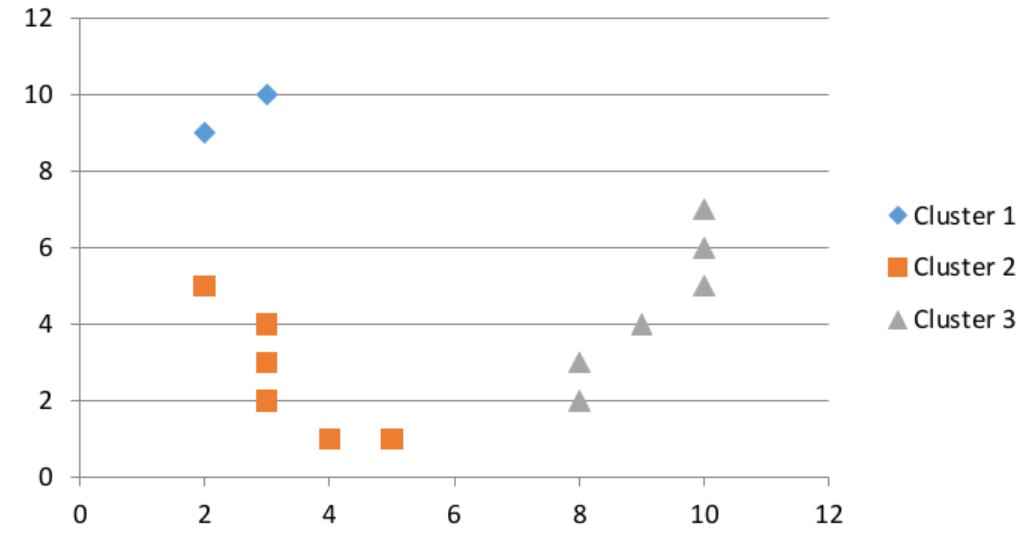


Figura 12.1 – Exemplu de clustering pentru setul de intrare format din 14 puncte

Pașii algoritmului sunt următorii (pentru un n și un k predefinite):
 Se selectează k puncte random din spațiu care vor fi centroizii inițiali ai clusterelor.
 Se efectuează iterativ următorii pași, până când atribuirile fiecărui punct la un cluster rămân neschimbate între doi pași consecutivi:
 Asignarea fiecărui punct unui cluster în mod greedy pentru fiecare punct în parte (acel centroid care minimizează distanța euclidiană către centroizii din pasul curent este minimă);
 Recalcularea centroidului drept media aritmetică a coordonatelor punctelor asignate (se face acest lucru pentru fiecare coordonată). Pentru un set de date și un k stabilit se vor determina clusterele aferente. Se va implementa și o optimizare a selecției inițiale de puncte pornind de la principiul că acestea ar trebui să fie geometric cât mai dispersate (se dorește maximizarea distanței între centroizii inițiali).

INDICATII REZOLVARE

Se implementează cele 2 etape specifice algoritmilor de tip Expectation Maximization:
 Asignarea fiecărui nod la unul dintre cele k clustere (distanța minimă euclidiană către centroizii din pasul curent este minimă);

 Recalcularea centroidului drept media aritmetică a coordonatelor
punctelor asignate fiecărui cluster. Pentru o asignare inițială mai eficientă, se pornește cu un punct random n 0. Ulterior, fiecare punct n i , cu $1 \leq i \leq k$, se alege pe considerentul că suma distanțelor până la toate punctele anterioare este maximă. Astfel, pentru n 1 se urmărește maximizarea distanței la n 0 , pentru n 2 suma distanțelor la n 1 și n 0, etc. În final, punctele n 1...k (n 0 este exclus) reprezintă alocările inițiale care asigură o dispersie cât mai mare a centroizilor.